

Prácticas de Cálculo

con W_X Maxima
con M_X Wxmaxima



ugr

Universidad
de Granada

Jerónimo Alaminos Prats
Camilo Aparicio del Prado
José Extremera Lizana
Pilar Muñoz Rivas
Armando R. Villena Muñoz

21 julio 2016



Reconocimiento-No
comercial 3.0 España

Usted es libre de:

-  copiar, distribuir y comunicar públicamente la obra
-  hacer obras derivadas

Bajo las condiciones siguientes:

-  **Reconocimiento.** Debe reconocer los créditos de la obra de la manera especificada por el autor o el licenciador (pero no de una manera que sugiera que tiene su apoyo o apoyan el uso que hace de su obra).
-  **No comercial.** No puede utilizar esta obra para fines comerciales.
 - a) Al reutilizar o distribuir la obra, tiene que dejar bien claro los términos de la licencia de esta obra.
 - b) Alguna de estas condiciones puede no aplicarse si se obtiene el permiso del titular de los derechos de autor
 - c) Nada en esta licencia menoscaba o restringe los derechos morales del autor.

Advertencia

Este resumen no es una licencia. Es simplemente una referencia práctica para entender la licencia completa que puede consultarse en

<http://creativecommons.org/licenses/by-nc/3.0/es/legalcode.es>

Índice

Índice iii

Introducción v

Aspectos básicos 7

1 Primeros pasos 9

1.1 Introducción 9 1.2 Resultados exactos y aproximación decimal 13 1.3 Funciones usuales 15 1.4 Operadores lógicos y relacionales 19 1.5 Variables 20 1.6 Expresiones simbólicas 22 1.7 La ayuda de *Maxima* 29 1.8 Ejercicios 31

2 Gráficos 33

2.1 Funciones 33 2.2 Gráficos en el plano con `plot2d` 37 2.3 Gráficos en el espacio con `plot3d` 47 2.4 Gráficos con `draw` 50 2.5 Animaciones gráficas 67 2.6 Ejercicios 71

3 Números y errores 75

3.1 Error absoluto y error relativo 75 3.2 Números y precisión 75 3.3 Aritmética de ordenador 77 3.4 Cancelación de cifras significativas 78

4 Listas y matrices 81

4.1 Listas 81 4.2 Matrices 87 4.3 Ejercicios 95

Resolución de ecuaciones 97

5 Resolución de ecuaciones con *Maxima* 99

5.1 Ecuaciones y operaciones con ecuaciones 99 5.2 Resolución de ecuaciones 100 5.3 Ecuaciones algebraicas 107 5.4 El teorema de los ceros de Bolzano y `find_root` 111

6 Breves conceptos de programación 113

6.1 Funciones y bloques 113 6.2 Bucles 118 6.3 Condicionales 121 6.4 Ejercicios 122

7 Métodos cerrados 123

7.1 Métodos cerrados y abiertos 123 7.2 El método de bisección 123 7.3 Un ejemplo concreto 124 7.4 De nuevo bisección 127 7.5 Regula-falsi 129 7.6 Ejercicios 129

8 Otros métodos 131

8.1 Métodos de iteración funcional 131 8.2 El método de Newton-Raphson 135

Cálculo 139

9 Límites y continuidad 141

9.1 Límites 141 9.2 Sucesiones 143 9.3 Continuidad 145 9.4 Ejercicios 146

10 Derivación 149

10.1 Cálculo de derivadas 149 10.2 Rectas secante y tangente a una función 152 10.3 Extremos relativos 155 10.4 Ejercicios 160

11 Integración 163

11.1 Cálculo de integrales 163 11.2 Sumas de Riemann 169 11.3 Aplicaciones 172 11.4 Ejercicios 179

12 Series numéricas y series de Taylor 183

12.1 Series numéricas 183 12.2 Desarrollo de Taylor 185 12.3 Ejercicios 186

Interpolación 189

13 Interpolación polinómica 191

13.1 Interpolación lineal 191 13.2 Interpolación de Lagrange 192 13.3 Polinomio de Taylor 196

14 Derivación e integración numérica 203

14.1 Derivación numérica 203 14.2 Integración numérica 204 14.3 Métodos simples 205 14.4 Métodos de aproximación compuestos 207

Ecuaciones diferenciales 211

15 Ecuaciones diferenciales ordinarias 213

15.1 Resolución de edos de primer y segundo orden 213 15.2 Resolución de sistemas de ecuaciones lineales 216 15.3 Campos de direcciones y curvas integrales 217 15.4 Ejercicios 220

Cálculo en varias variables 223

16 Diferenciación 225

16.1 Derivadas parciales de una función 225 16.2 Curvas y vectores tangentes 228 16.3 Funciones definidas implícitamente 230 16.4 Extremos relativos 235 16.5 Extremos condicionados 237 16.6 Extremos absolutos 239 16.7 Ejercicios 240

17 Integrales múltiples 243

17.1 Integrales iteradas 243 17.2 Integrales dobles 243 17.3 Ejercicios 246

A Números complejos 249

B Avisos y mensajes de error 253

C Bibliografía 257

Glosario 259

Introducción

Maxima es un programa que realiza cálculos matemáticos de forma tanto numérica como simbólica, esto es, sabe tanto manipular números como calcular la derivada de una función. Sus capacidades cubren sobradamente las necesidades de un alumno de un curso de Cálculo en unos estudios de Ingeniería. Se encuentra disponible bajo licencia GNU GPL tanto el programa como los manuales del programa.

Lo que presentamos aquí son unas notas sobre el uso de *Maxima* para impartir la parte correspondiente a unas prácticas de ordenador en una asignatura de Cálculo que incluya derivadas e integrales en una y varias variables y una breve introducción a ecuaciones diferenciales ordinarias. Además de eso, hemos añadido unos capítulos iniciales donde se explican con algo de detalle algunos conceptos más o menos generales que se utilizan en la resolución de problemas con *Maxima*. Hemos pensado que es mejor introducir, por ejemplo, la gestión de gráficos en un capítulo separado que ir comentando cada orden en el momento que se use por primera vez. Esto no quiere decir que todo lo que se cuenta en los cuatro primeros capítulos sea necesario para el desarrollo del resto de estas notas. De hecho, posiblemente es demasiado. En cualquier caso pensamos que puede ser útil en algún momento.

Por qué

Hay muchos programas que cumplen en mayor o menor medida los requisitos que se necesitan para enseñar y aprender Cálculo. Sólo por mencionar algunos, y sin ningún orden particular, casi todos conocemos *Mathematica* (©Wolfram Research) o *Maple* (©Maplesoft). También hay una larga lista de programas englobados en el mundo del software libre que se pueden adaptar a este trabajo.

Siempre hay que intentar escoger la herramienta que mejor se adapte al problema que se presenta y, en nuestro caso, *Maxima* cumple con creces las necesidades de un curso de Cálculo. Es evidente que *Mathematica* o *Maple* también pero creemos que el uso de programas de software libre permite al alumno y al profesor estudiar cómo está hecho, ayudar en su mejora y, si fuera necesario y posible, adaptarlo a sus propias necesidades.

Además pensamos que el programa tiene la suficiente capacidad como para que el alumno le pueda seguir sacando provecho durante largo tiempo. Estamos todos de acuerdo en que esto sólo es un primer paso y que *Maxima* se puede utilizar para problemas más complejos que los que aparecen en estas notas. Esto no es un callejón sin salida sino el comienzo de un camino.

Dónde y cómo

No es nuestra intención hacer una historia de *Maxima*, ni explicar cómo se puede conseguir o instalar, tampoco aquí encontrarás ayuda ni preguntas frecuentes ni nada parecido. Cualquiera de estas informaciones se encuentra respondida de manera detallada en la página web del programa:

<http://maxima.sourceforge.net/>

En esta página puedes descargarte el programa y encontrar abundante documentación sobre cómo instalarlo. Al momento de escribir estas notas, en dicha página puedes encontrar versiones listas para funcionar disponibles para entornos Windows y Linux e instrucciones detalladas para ponerlo en marcha en Mac OS X.

wxMaxima

En estas notas no estamos usando *Maxima* directamente sino un entorno gráfico que utiliza *Maxima* como motor para realizar los cálculos. Este entorno (programa) es *wxMaxima*. Nos va a permitir que la curva de aprendizaje sea mucho más suave. Desde el primer día el alumno será capaz de realizar la mayoría de las operaciones básicas. A pesar de ello, en todos los ejemplos seguimos utilizando la notación de *Maxima* que es la que le aparece al lector en pantalla y que nunca está de más conocer. *wxMaxima* se puede descargar de su página web

<http://maxima.sourceforge.net/>

Suele venir incluido con *Maxima* en su versión para entorno Windows y en Mac OS X. Sobre la instalación en alguna distribución Linux es mejor consultar la ayuda sobre su correspondiente programa para gestionar software.

Granada a 10 de septiembre de 2008

Aspectos básicos

Primeros pasos

1

1.1 Introducción	9	1.2 Resultados exactos y aproximación decimal	13	1.3 Funciones usuales	15
1.4 Operadores lógicos y relacionales	19	1.5 Variables	20	1.6 Expresiones simbólicas	22
ayuda de <i>Maxima</i>	29	1.8 Ejercicios	31		

1.1 Introducción

Vamos a comenzar familiarizándonos con *Maxima* y con el entorno de trabajo *wxMaxima*. Cuando iniciamos el programa se nos presenta una ventana como la de la Figura 1.1. En la parte superior tienes el menú con las opciones usuales (abrir, cerrar, guardar) y otras relacionadas con las posibilidades más “matemáticas” de *Maxima*. En segundo lugar aparecen algunos iconos que sirven de atajo a algunas operaciones y la ventana de trabajo. Por defecto, la ventana de *wxMaxima* aparece en blanco.

Ya iremos comentando con mayor profundidad los distintos menús y opciones que se nos presentan pero antes de ir más lejos, ¿podemos escribir algo? Sí, sitúa el cursor dentro de la ventana, pulsa y escribe $2+3$. Luego pulsa las teclas **Shift**+**Return**. Obtendrás algo similar a esto:

```
(%i1) 2+3;
(%o1) 5
```

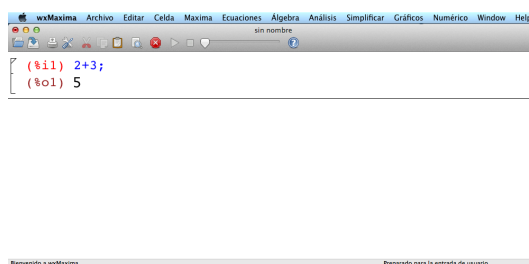


Figura 1.1 Ventana inicial de *wxMaxima*

Como puedes ver *Maxima* da la respuesta correcta: 5. Bueno, no parece mucho. Seguro que tienes una calculadora que hace eso. De acuerdo. Es sólo el principio.

Observación 1.1. Conviene hacer algunos comentarios sobre lo que acabamos de hacer:

- No intentes escribir los símbolos “**(%i1)**” y “**(%o1)**”, ya que éstos los escribe el programa para llevar un control sobre las operaciones que va efectuando. “**(%i1)**” se refiere a la primera entrada (input) y “**(%o1)**” a la primera respuesta (output).
- Las entradas terminan en punto y coma. *wxMaxima* lo añade si tú te has olvidado de escribirlo. Justamente lo que nos había pasado.

Puedes volver a editar cualquiera de las entradas anteriores, pero aunque borres y escribas una nueva operación, *Maxima* recuerda todo lo que se ha hecho.

Operaciones básicas

El producto se indica con “*****”:

```
(%i2) 3*5;
(%o2) 15
```

Para multiplicar números es necesario escribir el símbolo de la multiplicación. Si sólo dejamos un espacio entre los factores el resultado es un error:

```
(%i3) 5 4;
      Incorrect syntax: 4 is not an infix operator
(%o3) 5Space4;
      ^
```

+	suma
*	producto
/	división
^ o **	potencia
sqrt()	raíz cuadrada

También podemos dividir

```
(%i4) 5+(2*4+6)/7;
(%o4) 7
(%i5) 5+2*4+6/7;
(%o5) 97
      7
```

eso sí, teniendo cuidado con la precedencia de las operaciones. En estos casos el uso de paréntesis es obligado.

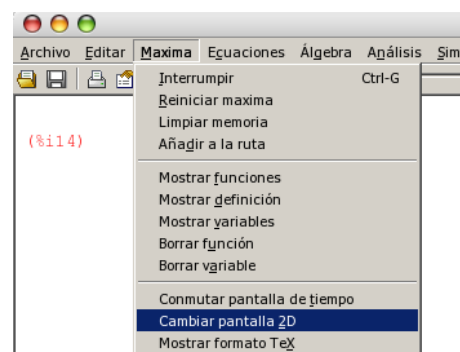
Podemos escribir potencias

```
(%i6) 3^57;
(%o6) 1570042899082081611640534563
```

Fíjate en el número de dígitos que tiene el resultado. Es un primer ejemplo de que la potencia de cálculo de *Maxima* es mayor que la de una calculadora que no suele tener más allá de 10 o 12. Ya sé lo que estarás pensando en este momento: en lugar de elevar a 57, elevemos a un número más grande. De acuerdo.

```
(%i7) 3^1000;
(%o7) 132207081948080663689045525[424 digits]366132173102768902855220001
```

Como puedes ver, *Maxima* realiza la operación pero no muestra el resultado completo. Nos dice que, en este caso, hay 418 dígitos que no está mostrando. ¿Se puede saber cuáles son? Sí. Nos vamos al menú **Maxima**→**Cambiar pantalla 2D** y escogemos **ascii**. Por último, repetimos la operación.



wxMaxima muestra una línea de dígitos y elimina el resto, por tanto, la cantidad eliminada depende del tamaño de la ventana, el tamaño de la fuente,...

```
(%i8) set_display('ascii)$
(%i9) 3^1000;
132207081948080663689045525975214436596542203275214816766492036
822682859734670489954077831385060806196390977769687258235595095
458210061891186534272525795367402762022519832080387801477422896
(%o9) 484127439040011758861804112894781562309443806156617305408667449
050617812548034440554705439703889581746536825491613622083026856
377858229022841639830788789691855640408489893760937324217184635
993869551676501894058810906042608967143886410281435038564874716
5832010614366132173102768902855220001
```

La salida en formato `ascii` es la que tiene por defecto *Maxima*. La salida con formato `xml` es una mejora de *wxMaxima*. Siempre puedes cambiar entre una y otra vía el menú o volviendo a escribir

```
(%i10) set_display('xml)$
(%i11) 3^1000;
(%o11) 13220708194808066368[438 digits]73102768902855220001
```

Observación 1.2. Antes de seguir, ¿por qué hemos escrito `$` y no punto y coma al final de la salida anterior? El punto y coma sirve para terminar un comando o separar varios de ellos. El dólar, `$`, también termina un comando o separa varios de ellos, pero, a diferencia del punto y coma, *no muestra el resultado en pantalla*. 

Si trabajamos con fracciones, *Maxima* dará por defecto el resultado en forma de fracción

```
(%i12) 2+5/11;
(%o12) 
$$\frac{27}{11}$$

```

simplificando cuando sea posible

```
(%i13) 128/234;
(%o13) 
$$\frac{64}{117}$$

```

Cálculo simbólico

Cuando hablamos de que *Maxima* es un programa de cálculo simbólico, nos referimos a que no necesitamos trabajar con valores concretos. Fíjate en el siguiente ejemplo:

```
(%i14) a/2+3*a/5;
(%o14) 
$$\frac{11a}{10}$$

```

Bueno, hasta ahora sabemos sumar, restar, multiplicar, dividir y poco más. *Maxima* tiene predefinidas la mayoría de las funciones usuales. Por ejemplo, para obtener la raíz de un número se usa el comando `sqrt`

```
(%i15) sqrt(5);
(%o15)  $\sqrt{5}$ 
```

lo cuál no parece muy buena respuesta. En realidad es la mejor posible: *Maxima* es un programa de cálculo simbólico y siempre intentará dar el resultado en la forma más exacta.

Obviamente, también puedes hacer la raíz cuadrada de un número, elevando dicho número al exponente $\frac{1}{2}$

```
(%i16) 5^(1/2);
(%o16)  $\sqrt{5}$ 
```

float Si queremos obtener la expresión decimal, utilizamos la orden `float`.

```
(%i17) float(sqrt(5));
(%o17) 2.23606797749979
```

Constantes

Además de las funciones usuales (ya iremos viendo más), *Maxima* también conoce el valor de algunas de las constantes típicas.

<code>%pi</code>	el número π
<code>%e</code>	el número e
<code>%i</code>	la unidad imaginaria
<code>%phi</code>	la razón áurea, $\frac{1+\sqrt{5}}{2}$

Podemos operar con ellas como con cualquier otro número.

```
(%i18) (2+3*i)*(5+3*i);
(%o18) (3*i+2)*(3*i+5)
```

Evidentemente necesitamos alguna manera de indicar a *Maxima* que debe desarrollar los productos, pero eso lo dejaremos para más tarde.

¿Cuál era el resultado anterior?

Con *Maxima* podemos usar el resultado de una operación anterior sin necesidad de teclearlo. Esto se consigue con la orden `%`. No sólo podemos referirnos a la última respuesta sino a cualquier entrada o salida anterior. Para ello

```
(%i19)  %o15
(%o19)   $\sqrt{5}$ 
```

además podemos usar esa información como cualquier otro dato.

```
(%i20)  %o4+%o5;
(%o20)   $\frac{146}{7}$ 
```

%	último resultado
%i <i>número</i>	entrada <i>número</i>
%o <i>número</i>	resultado <i>número</i>

1.2 Resultados exactos y aproximación decimal

Hay una diferencia básica entre el concepto abstracto de número real y cómo trabajamos con ellos mediante un ordenador: la memoria y la capacidad de proceso de un ordenador son finitos. La precisión de un ordenador es el número de dígitos con los que hace los cálculos. En un hipotético ordenador que únicamente tuviera capacidad para almacenar el primer decimal, el número π sería representado como 3.1. Esto puede dar lugar a errores si, por ejemplo, restamos números similares. *Maxima* realiza los cálculos de forma simbólica o numérica. En principio, la primera forma es mejor, pero hay ocasiones en las que no es posible.

Maxima tiene dos tipos de “números”: exactos y aproximados. La diferencia entre ambos es la esperable. $\frac{1}{3}$ es un número exacto y 0.333 es una aproximación del anterior. En una calculadora normal todos los números son aproximados y la precisión (el número de dígitos con el que trabaja la calculadora) es limitada, usualmente 10 o 12 dígitos. *Maxima* puede manejar los números de forma exacta, por ejemplo

```
(%i21)  1/2+1/3;
(%o21)   $\frac{5}{6}$ 
```

Mientras estemos utilizando únicamente números exactos, *Maxima* intenta dar la respuesta de la misma forma. Ahora bien, en cuanto algún término sea aproximado el resultado final será siempre aproximado. Por ejemplo

```
(%i22)  1.0/2+1/3;
(%o22)  0.833333333333333
```

Este comportamiento de *Maxima* viene determinado por la variable `numer` que tiene el valor `false` por defecto. En caso de que cambiemos su valor a `true`, la respuesta de *Maxima* será aproximada.

```
(%i23)  numer;
```

```
(%o23) false
(%i24) numer:true$
(%i25) 1/2+1/3;
(%o25) 0.83333333333333
(%i26) numer:false$
```

Recuerda cambiar el valor de la variable `numer` a `false` para volver al comportamiento original de *Maxima*. En *wxMaxima*, podemos utilizar el menú **Numérico**—**conmutar salida numérica** para cambiar el valor de la variable `numer`.

<i>número</i> , <code>numer</code>	expresión decimal de <i>número</i>
<code>float(número)</code>	expresión decimal de <i>número</i>
<code>bfloat(número)</code>	expresión decimal larga de <i>número</i>

Si sólo queremos conocer una aproximación decimal de un resultado exacto, tenemos a nuestra disposición las órdenes `float` y `bfloat`.

```
(%i27) float(sqrt(2));
(%o27) 1.414213562373095
```

En la ayuda de *Maxima* podemos leer

Valor por defecto: 16.

La variable ‘`fpprec`’ guarda el número de dígitos significativos en la aritmética con números decimales de punto flotante grandes (“`bigfloats`”). La variable ‘`fpprec`’ no afecta a los cálculos con números decimales de punto flotante ordinarios.

Maxima puede trabajar con cualquier precisión. Dicha precisión la podemos fijar asignando el valor que queramos a la variable `fpprec`. Por ejemplo, podemos calcular cuánto valen los 100 primeros decimales de π :

```
(%i28) fpprec:100;
(%o28) 100
(%i29) float(%pi);
(%o29) 3.141592653589793
```

No parece que tengamos 100 dígitos...de acuerdo, justo eso nos decía la ayuda de *máxima*: “La variable `fpprec` no afecta a los cálculos con números decimales de punto flotante ordinarios”. Necesitamos la orden `bfloat` para que *Maxima* nos muestre todos los decimales pedidos (y cambiar la pantalla a `asci i`):

```
(%i30) bfloat(%pi);
```



```
(%o30) 3.1415926535897932384626[55 digits]998628034825342117068b0
(%i31) set_display('ascii)$
(%i32) bfloat(%pi);
(%o32) 3.141592653589793238462643383279502884197169399375105820
974944592307816406286208998628034825342117068b0
```

Si te has fijado, en la salida anterior la expresión decimal del número π termina con “b0”. Los números en coma flotante grandes siempre terminan con “b” seguido de un número n para indicar que debemos multiplicar por 10^n . En el caso anterior, la expresión decimal de π deberíamos multiplicarla por $10^0 = 1$.

Por último, observa que, como se puede ver en la Figura 1.2, también se puede utilizar el menú **N**umérico→**A** real o **N**umérico→**A** real grande(**b**igfloat) para obtener la expresión decimal buscada.

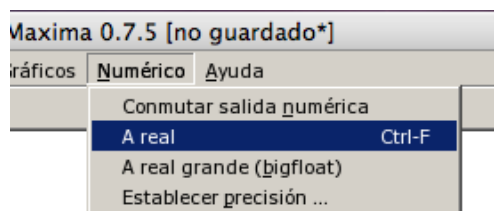


Figura 1.2 En el menú, la pestaña **N**umérico permite obtener la expresión decimal con la precisión que se desee

1.3 Funciones usuales

Además de las operaciones elementales que hemos visto, *Maxima* tiene definidas la mayor parte de las funciones elementales. Los nombres de estas funciones suelen ser su abreviatura en inglés, que algunas veces difiere bastante de su nombre en castellano. Por ejemplo, ya hemos visto raíces cuadradas

```
(%i33) sqrt(4);
(%o33) 2
```

Potencias, raíces y exponenciales

Hemos visto que podemos escribir potencias utilizando $\wedge$ o $**$. No importa que el exponente sea racional. En otras palabras: podemos calcular raíces de la misma forma.

```
(%i34) 625^(1/4);
(%o34) 5
(%i35) 625^(1/3)*2^(1/3):
(%o35) 21/3 54/3
```

En el caso particular de que la base sea el número e , podemos escribir

```
(%i36) %e^2;
(%o36) %e2
```

o, lo que es más cómodo especialmente si el exponente es una expresión más larga, utilizar la función exponencial `exp`

<code>sqrt(x)</code>	raíz cuadrada
<code>exp(x)</code>	exponencial de base e
<code>log(x)</code>	logaritmo neperiano de x
<code>sin(x)</code> , <code>cos(x)</code> , <code>tan(x)</code>	seno, coseno y tangente <i>en radianes</i>
<code>csc(x)</code> , <code>sec(x)</code> , <code>cot(x)</code>	cosecante, secante y cotangente <i>en radianes</i>
<code>asin(x)</code> , <code>acos(x)</code> , <code>atan(x)</code>	arcoseno, arcocoseno y arcotangente
<code>sinh(x)</code> , <code>cosh(x)</code> , <code>tanh(x)</code>	seno, coseno y tangente hiperbólicos
<code>asinh(x)</code> , <code>acosh(x)</code> , <code>atanh(x)</code>	arcoseno, arcocoseno y arcotangente hiperbólicos

```
(%i37) exp(2);
(%o37) %e^2
(%i38) exp(2),numer;
(%o38) 7.38905609893065
```

Logaritmos

Maxima sólo tiene la definición del logaritmo neperiano o natural que se consigue con la orden `log`:

```
(%i39) log(20);
(%o39) log(20)
```

y si lo que nos interesa es su expresión decimal

```
(%i40) log(20),numer;
(%o40) 2.995732273553991
```

 **Observación 1.3.** Mucho cuidado con utilizar `ln` para calcular logaritmos neperianos:

```
(%i41) ln(20);
(%o41) ln(20)
```

puede parecer que funciona igual que antes pero en realidad *Maxima* no tiene la más remota idea de lo que vale, sólo está repitiendo lo que le habéis escrito. Si no te lo crees, pídele que te diga el valor:

```
(%i42) ln(20),numer;
(%o42) ln(20)
```

¿Cómo podemos calcular $\log_2(64)$? Para calcular logaritmos en cualquier base podemos utilizar que

$$\log_b(x) = \frac{\log(x)}{\log(b)}.$$

Se puede definir una función que calcule los logaritmos en base 2 de la siguiente manera

```
(%i43) log2(x):=log(x)/log(2)$
(%i44) log2(64);
(%o44)  $\frac{\log(64)}{\log(2)}$ 
```

Te habrás dado cuenta de que *Maxima* no desarrolla ni simplifica la mayoría de las expresiones. En segundo lugar, la posibilidad de definir funciones a partir de funciones conocidas nos abre una amplia gama de posibilidades. En el segundo capítulo veremos con más detalle cómo trabajar con funciones.

Funciones trigonométricas e hiperbólicas

Maxima tiene predefinidas las funciones trigonométricas usuales seno, *sin*, coseno, *cos*, y tangente, *tan*, que devuelven, si es posible, el resultado exacto.

```
(%i45) sin(%pi/4);
(%o45)  $\frac{1}{\sqrt{2}}$ 
```

Por defecto, las funciones trigonométricas están expresadas en radianes.

También están predefinidas sus inversas, esto es, arcoseno, arcocoseno y arcotangente, que se escriben respectivamente *asin(x)*, *acos(x)* y *atan(x)*, así como las funciones recíprocas secante, *sec(x)*, cosecante, *csc(x)*, y cotangente, *cot(x)*

```
(%i46) atan(1);
(%o46)  $\frac{\pi}{4}$ 
(%i47) sec(0);
(%o47) 1
```

De forma análoga, puedes utilizar las correspondientes funciones hiperbólicas.

Otras funciones

Además de las anteriores, hay muchas más funciones de las que *Maxima* conoce la definición. Podemos, por ejemplo, calcular factoriales

```
(%i48) 32!;
(%o48) 263130836933693530167218012160000000
```

o números binómicos

```
(%i49) binomial(10,4);
(%o49) 210
```

¿Recuerdas cuál es la definición de $\binom{m}{n}$?

$$\binom{m}{n} = \frac{m(m-1)(m-2) \cdots (m-(n-1))}{n!}$$

En el desarrollo de Taylor de una función veremos que estos números nos simplifican bastante la notación.

	n!	factorial de n
entier(x), fix(x)		parte entera de x
abs(x)		valor absoluto o módulo de x
random(x)		devuelve un número aleatorio
signum(x)		signo de x
max($x_1, x_2, \dots$)		máximo de $x_1, x_2, \dots$
min($x_1, x_2, \dots$)		mínimo de $x_1, x_2, \dots$

random Una de las funciones que usaremos más adelante es `random`. Conviene comentar que su comportamiento es distinto dependiendo de si se aplica a un número entero o a un número decimal, siempre positivo, eso sí. Si el número x es natural, `random(x)` devuelve un natural menor o igual que $x - 1$.

```
(%i50) random(100);
(%o50) 7
```

Obviamente no creo que tú también obtengas un 7, aunque hay un caso en que sí puedes saber cuál es el número “aleatorio” que vas a obtener:

```
(%i51) random(1);
(%o51) 0
```

efectivamente, el único entero no negativo menor o igual que $1 - 1$ es el cero. En el caso de que utilicemos números decimales `random(x)` nos devuelve un número decimal menor que x . Por ejemplo,

```
(%i52) random(1.0);
(%o52) 0.9138095996129
```

nos da un número (decimal) entre 0 y 1.

La lista de funciones es mucho mayor de lo que aquí hemos comentado y es fácil que cualquier función que necesites esté predefinida en *Maxima*. En la ayuda del programa puedes encontrar la lista completa.

1.4 Operadores lógicos y relacionales

Maxima puede comprobar si se da una igualdad (o desigualdad). Sólo tenemos que escribirla y nos dirá qué le parece:

```
(%i53) is(3<5);
(%o53) true
```

<code>is(<i>expresión</i>)</code>	decide si la expresión es cierta o falsa
<code>assume(<i>expresión</i>)</code>	supone que la expresión es cierta
<code>forget(<i>expresión</i>)</code>	olvida la expresión
<code>and</code>	y
<code>or</code>	o

No se pueden encadenar varias condiciones. No se admiten expresiones del tipo $3 < 4 < 5$. Las desigualdades sólo se aplican a parejas de expresiones. Lo que sí podemos hacer es combinar varias cuestiones como, por ejemplo,

```
(%i54) is(3<2 or 3<4);
(%o54) true
```

En cualquier caso tampoco esperes de *Maxima* la respuesta al sentido de la vida:

```
(%i55) is((x+1)^2=x^2+2*x+1);
(%o55) false
```

<code>=</code>	igual
<code>notequal</code>	distinto
<code>x>y</code>	mayor
<code>x<y</code>	menor
<code>x>=y</code>	mayor o igual
<code>x<=y</code>	menor o igual

Pues no parecía tan difícil de responder. Lo cierto es que *Maxima* no ha desarrollado la expresión. Vamos con otra pregunta fácil:

```
(%i56) is((x+1)^2>0);
(%o56) unknown
```

Pero, ¿no era positivo un número al cuadrado? Hay que tener en cuenta que x podría valer -1 . ¿Te parece tan mala la respuesta ahora? Si nosotros disponemos de información adicional, siempre podemos “ayudar”. Por ejemplo, si sabemos que x es distinto de -1 la situación cambia:

```
(%i57)  assume(notequal(x,-1));
(%o57)  [notequal(x,-1)]
(%i58)  is((x+1)^2>0);
(%o58)  true
```

Eso sí, en este caso *Maxima* presupone que x es distinto de -1 en lo que resta de sesión. Esto puede dar lugar a errores si volvemos a utilizar la variable x en un ambiente distinto más adelante.

forget El comando `forget` nos permite “hacer olvidar” a *Maxima*.

```
(%i59)  forget(notequal(x,-1));
(%o59)  [notequal(x,-1)]
(%i60)  is(notequal(x,-1));
(%o60)  unknown
```

1.5 Variables

El uso de variables es muy fácil y cómodo en *Maxima*. Uno de los motivos de esto es que no hay que declarar tipos previamente. Para asignar un valor a una variable utilizamos los dos puntos

```
(%i61)  a:2
(%o61)  2
(%i62)  a^2
(%o62)  4
```

<code>var: expr</code>	la variable <i>var</i> vale <i>expr</i>
<code>kill(a1,a2,...)</code>	elimina los valores
<code>remvalue(var1,var2,...)</code>	borra los valores de las variables
<code>values</code>	muestra las variables con valor asignado

Cuando una variable tenga asignado un valor concreto, a veces diremos que es una constante, para distinguir del caso en que no tiene ningún valor asignado.

Observación 1.4. El nombre de una variable puede ser cualquier cosa que no empiece por un número. Puede ser una palabra, una letra o una mezcla de ambas cosas.

```
(%i63)  largo:10;
(%o63)  10
(%i64)  ancho:7;
(%o64)  7
(%i65)  largo*ancho;
(%o65)  70
```

Podemos asociar una variable con prácticamente cualquier cosa que se nos ocurra: un valor numérico, una cadena de texto, las soluciones de una ecuación, etc.

```
(%i66)  solucion:solve(x^2-1=0,x);
(%o66)  [x=-1,x=1]
```

para luego poder usarlas.

Los valores que asignamos a una variable no se borran por sí solos. Siguen en activo mientras no los cambiemos o comencemos una nueva sesión de *Maxima*. Quizá por costumbre, todos tendemos a usar como nombre de variables x , y , z , t , igual que los primeros nombres que se nos vienen a la cabeza de funciones son f o g . Después de trabajar un rato con *Maxima* es fácil que usemos una variable que ya hemos definido antes. Es posible que dar un valor a una variable haga que una operación posterior nos de un resultado inesperado o un error. Por ejemplo, damos un valor a x

```
(%i67)  x:3;
(%o67)  3
```

y después intentamos derivar una función de x , olvidando que le hemos asignado un valor. ¿Cuál es el resultado?

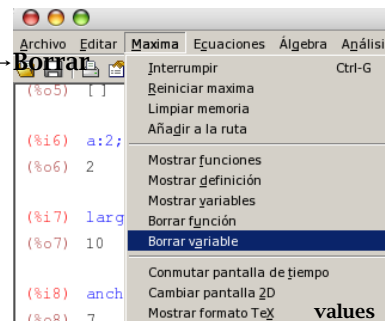
```
(%i68)  diff(sin(x),x);
Non-variable 2nd argument to diff:
3
- an error. To debug this try debugmode(true);
```

Efectivamente, un error. Hay dos maneras de evitar esto. La primera es utilizar el operador comilla, `'`, que evita que se evalúe la variable:

```
(%i69)  diff(sin('x),'x);
(%o69)  cos(x)
```

La segunda es borrar el valor de x . Esto lo podemos hacer con la orden `kill` o con la orden `remvalue`. También puedes ir al menú **Maxima** → **Borrar variable** y escribir las variables que quieres borrar. Por defecto se borrarán todas.

Si te fijas, dentro del menú **Maxima** también hay varios ítems interesantes: se pueden borrar funciones, se pueden mostrar aquellas variables (y funciones) que tengamos definidas y se puede pedir la expresión de una función que tengamos previamente definida. Por ejemplo, las variables que hemos usado son las siguientes: `values`.



```
(%i70)  values;
(%o70)  [a,largo,ancho,x,solucion]
```

remvalue Una vez que sabemos cuáles son, podemos borrar algunas de ellas

```
(%i71)  remvalue(a,x);
(%o71)  [a,x]
```

o todas.

```
(%i72)  remvalue(all);
(%o72)  [largo, ancho, solucion]
```

La orden **remvalue** sólo permite borrar valores de variables. Existen versiones similares para borrar funciones, reglas, etc. En cambio, la orden **kill** es la versión genérica de borrar valores de cualquier cosa.

```
(%i73)  ancho:10$
(%i74)  kill(ancho);
(%o74)  done
(%i75)  remvalue(ancho);
(%o75)  [false]
```

Una de las pequeñas diferencias entre **kill** y **remvalue** es que la primera no comprueba si la variable, o lo que sea, estaba previamente definida y siempre responde *done*. Existe también la posibilidad de borrar *todo*:

```
(%i76)  kill(all);
(%o0)   done
```

y, si te fijas, *Maxima* se reinicia: es como si empezáramos de nuevo. Hemos borrado cualquier valor que tuviésemos previamente definido.

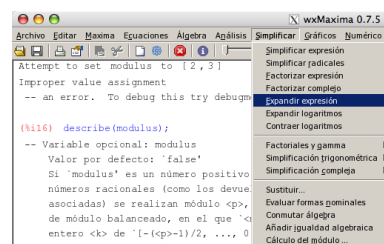
1.6 Expresiones simbólicas

Hasta ahora sólo hemos usado el *Maxima* como una calculadora muy potente, pero prácticamente todo lo que hemos aprendido puede hacerse sin dificultad con una calculadora convencional. Entonces, ¿qué puede hacer *Maxima* que sea imposible con una calculadora? Bueno, entre otras muchas cosas que veremos posteriormente, la principal utilidad de *Maxima* es el cálculo simbólico, es decir, el trabajar con expresiones algebraicas (expresiones donde intervienen variables, constantes... y no tienen por qué tener un valor numérico concreto) en vez de con números. Por ejemplo, el programa sabe que la función logaritmo y la función exponencial son inversas una de otra, con lo que si ponemos

```
(%i1)  exp(log(x));
(%o1)  x
```


es decir, sin saber el valor de la variable x el programa es capaz de trabajar simbólicamente con ella. Más ejemplos

```
(%i2) exp(x)*exp(y);
(%o2) %ey+x
```



Aunque parece que no siempre obtenemos el resultado esperado

```
(%i3) log(x*y);
(%o3) log(x y)
(%i4) log(x)+log(y);
(%o4) log(y)+log(x)
```

Vamos a practicar con comandos de *Maxima* para manejar expresiones algebraicas: polinomios, funciones racionales, trigonométricas, etc.

Casi todas las órdenes de esta sección, ya sea expandir o simplificar expresiones, se encuentran en el menú **Simplificar** y, opcionalmente, en los paneles de *wxMaxima*.

1.6.1 Desarrollo de expresiones simbólicas

La capacidad de *Maxima* para trabajar con expresiones es notable. Comencemos con funciones sencillas. Consideremos el polinomio

```
(%i5) p: (x+2)*(x-1);
(%o5) (x-1)(x+2)
```

lo único que hace *Maxima* es reescribirlo. ¿Y las potencias?

```
(%i6) q: (x-3)^2;
(%o6) (x-3)^2
```

Vale, tampoco desarrolla el cuadrado. Probemos ahora a restar las dos expresiones:

```
(%i7) p-q;
(%o7) (x-1)(x+2)-(x-3)^2
```

Si no había desarrollado las expresiones anteriores, no era lógico esperar que desarrollara ahora la diferencia. *Maxima* no factoriza ni desarrolla automáticamente: debemos decirle que lo haga. ¿Cómo lo hacemos?

<code>expand(expr)</code>	realiza productos y potencias
<code>partfrac(frac, var)</code>	descompone en fracciones simples
<code>num(frac)</code>	numerador
<code>denom(frac)</code>	denominador

expand La orden `expand` desarrollo productos, potencias,

(%i8) `expand(p);`

(%o8) x^2+x-2

y cocientes.

(%i9) `expand(p/q);`

(%o9) $\frac{x^2}{x^2-6x+9} + \frac{x}{x^2-6x+9} - \frac{2}{x^2-6x+9}$

Como puedes ver, `expand` sólo divide la fracción teniendo en cuenta el numerador. Si queremos dividir en fracciones simples tenemos que usar `partfrac`.

(%i10) `partfrac(p/q,x);`

(%o10) $\frac{7}{x-3} + \frac{10}{x-3^2}+1$

num Por cierto, también podemos recuperar el numerador y el denominador de una fracción con las órdenes `num` y `denom`:

(%i11) `denom(p/q);`

(%o11) $(x-3)^2$

(%i12) `num(p/q);`

(%o12) $(x-1)(x+2)$

Comportamiento de `expand`

El comportamiento de la orden `expand` viene determinado por el valor de algunas variables. No vamos a comentar todas, ni mucho menos, pero mencionar algunas de ellas nos puede dar una idea del grado de control al que tenemos acceso.

<code>expand(expr,n,m)</code>	desarrolla potencias con grado entre $-m$ y n
<code>logexpand</code>	variable que controla el desarrollo de logaritmos
<code>radexpand</code>	variable que controla el desarrollo de radicales

Si quisiéramos desarrollar la función

$$(x+1)^{100} + (x-3)^{32} + (x+2)^2 + x - 1 - \frac{1}{x} + \frac{2}{(x-1)^2} + \frac{1}{(x-7)^{15}}$$

posiblemente no estemos interesados en que *Maxima* escriba los desarrollos completos de los dos primeros sumandos o del último. Quedaría demasiado largo en pantalla. La orden `expand` permite acotar qué potencias desarrollamos. Por ejemplo, `expand(expr,3,5)` sólo desarrolla aquellas potencias que estén entre 3 y -5.

```
(%i13) expand((x+1)^100+(x-3)^32+(x+2)^2+x-1-1/x+2/((x-1)^2)
+1/((x-7)^15),3,4);
```

```
(%o13)  $\frac{2}{x^2-2x+1}+(x+1)^{100}+x^2+5x-\frac{1}{x}+(x-3)^{32}+\frac{1}{(x-7)^{15}}+3$ 
```

Las variables `logexpand` y `radexpand` controlan si se simplifican logaritmos de productos o radicales con productos. Por defecto su valor es `true` y esto se traduce en que `expand` no desarrolla estos productos:

```
(%i14) log(a*b);
```

```
(%o14) log(a b)
```

```
(%i15) sqrt(x*y);
```

```
(%o15)  $\sqrt{x y}$ 
```

Cuando cambiamos su valor a `all`,

```
(%i16) radexpand:all$ logexpand:all$
```

```
(%i17) log(a*b);
```

```
(%o17) log(a)+log(b)
```

```
(%i18) sqrt(x*y);
```

```
(%o18)  $\sqrt{x}\sqrt{y}$ 
```

Dependiendo del valor de `logexpand`, la respuesta de *Maxima* varía cuando calculamos $\log(a^b)$ o $\log(a/b)$.

Compara tú cuál es el resultado de $\sqrt{x^2}$ cuando `radexpand` toma los valores `true` y `all`.

Factorización

<code>factor(expr)</code>	escribe la expresión como producto de factores más sencillos
---------------------------	--

La orden `factor` realiza la operación inversa a `expand`. La podemos utilizar tanto en números **factor**

```
(%i19) factor(100);
```

```
(%o19)  $2^2 5^2$ 
```

como con expresiones polinómicas como las anteriores

```
(%i20) factor(x^2-1);
```

```
(%o20) (x-1)(x+1)
```

El número de variables que aparecen tampoco es un problema:

```
(%i21) (x-y)*(x*y-3*x^2);
(%o21) (x-y)(xy-3x^2)
(%i22) expand(%);
(%o22) -xy^2+4x^2y-3x^3
(%i23) factor(%);
(%o23) -x(y-3x)(y-x)
```

Evaluación de valores en expresiones

`ev(expr, arg1, arg2, ...)` evalúa la expresión según los argumentos

Ahora que hemos estado trabajando con expresiones polinómicas, para evaluar en un punto podemos utilizar la orden `ev`. En su versión más simple, esta orden nos permite dar un valor en una expresión:

```
(%i24) ev(p, x=7);
(%o24) 54
```

que puede escribirse también de la forma

```
(%i25) p, x=7;
(%o25) 54
```

También se puede aplicar `ev` a una parte de la expresión:

```
(%i26) x^2+ev(2*x, x=3);
(%o26) x^2+6
```

Este tipo de sustituciones se pueden hacer de forma un poco más general y sustituir expresiones enteras

```
(%i27) ev(x+(x+y)^2-3*(x+y)^3, x+y=t);
(%o27) x-3*t^3+t^2
```

En la ayuda de *Maxima* puedes ver con más detalle todos los argumentos que admite la orden `ev`, que son muchos.

1.6.2 Simplificación de expresiones

Es discutible qué queremos decir cuando afirmamos que una expresión es más simple o más sencilla que otra. Por ejemplo, ¿cuál de las dos siguientes expresiones te parece más sencilla?

(%i28) `radcan(p/q);`

(%o28) $\frac{x^2+x-2}{x^2-6x+9}$

(%i29) `partfrac(p/q,x);`

(%o29) $\frac{7}{x-3} + \frac{10}{x-3^2} + 1$

`radcan(expr)` simplifica expresiones con radicales

`ratsimp(expr)` simplifica expresiones racionales

`fullratsimp(expr)` simplifica expresiones racionales

Maxima tiene algunas órdenes que permiten simplificar expresiones pero muchas veces no hay nada como un poco de ayuda y hay que indicarle si queremos desarrollar radicales o no, logaritmos, etc como hemos visto antes.

Para simplificar expresiones racionales, `ratsimp` funciona bastante bien aunque hay veces que es necesario aplicarlo más de una vez. La orden `fullratsimp` simplifica algo mejor a costa de algo más de tiempo y proceso.

(%i30) `fullratsimp((x+a)*(x-b)^2*(x^2-a^2)/(x-a));`

(%o30) $x^4 + (2a-2b)x^3 + (b^2-4ab+a^2)x^2 + (2ab^2-2a^2b)x + a^2b^2$

Para simplificar expresiones que contienen radicales, exponenciales o logaritmos es más útil la orden `radcan`

(%i31) `radcan((%e^(2*x)-1)/(%e^x+1));`

(%o31) $e^x - 1$

1.6.3 Expresiones trigonométricas

Maxima conoce las identidades trigonométricas y puede usarlas para simplificar expresiones en las que aparezcan dichas funciones. En lugar de `expand` y `factor`, utilizaremos las órdenes `trigexpand`, `trigsimp` y `trigreduce`.

`trigexpand(expresion)` desarrolla funciones trigonométricas e hipérbolicas

`trigsimp(expresion)` simplifica funciones trigonométricas e hipérbolicas

`trigreduce(expresion)` simplifica funciones trigonométricas e hipérbolicas

Por ejemplo,

(%i32) `trigexpand(cos(a+b));`

```
(%o32) cos(a)cos(b)-sin(a)sin(b);
(%i33) trigexpand(sin(2*atan(x)));
(%o33)  $\frac{2x}{x^2+1}$ 
(%i34) trigexpand(sin(x+3*y)+cos(2*z)*sin(x-y));
(%o34) -(cos(x)sin(y)-sin(x)cos(y))(cos(z)^2-sin(z)^2)+cos(x)sin(3y)+
sin(x)cos(3y)
(%i35) trigexpand(8*sin(2*x)^2*cos(x)^3);
(%o35) 32 cos(x)^5 sin(x)^2
```

Compara el resultado del comando `trigexpand` con el comando `trigreduce` en la última expresión:

```
(%i36) trigreduce(8*sin(2*x)^2*cos(x)^3);
(%o36) 8  $\left( -\frac{\frac{\cos(7x)}{2} + \frac{\cos(x)}{2}}{8} - \frac{3 \left( \frac{\cos(5x)}{2} + \frac{\cos(3x)}{2} \right)}{8} + \frac{\cos(3x)}{8} + \frac{3 \cos(x)}{8} \right)$ 
```

Quizás es complicado ver qué está ocurriendo con estas expresiones tan largas. Vamos a ver cómo se comportan en una un poco más sencilla:

```
(%i37) eq:cos(2*x)+cos(x)^2$
(%i38) trigexpand(eq);
(%o38) 2cos(x)^2-sin(x)^2
(%i39) trigreduce(eq);
(%o39)  $\frac{\cos(2x)+1}{2} + \cos(2x)$ 
(%i40) trigsimp(eq);
(%o40) cos(2x)+cos(x)^2
```

Como puedes ver, `trigsimp` intenta escribir la expresión de manera simple, `trigexpand` y `trigreduce` desarrollan y agrupan en términos similares pero mientras una prefiere usar potencias, la otra utiliza múltiplos de la variable. Estos es muy a grosso modo.

Cualquiera de estas órdenes opera de manera similar con funciones hiperbólicas:

```
(%i41) trigexpand(sinh(2*x)^3);
(%o41) 8 cosh(x)^3 sinh(x)^3
(%i42) trigreduce(cosh(x+y)+sinh(x)^2);
(%o42) cosh(y+x)+ $\frac{\cosh(2x)-1}{2}$ 
```

Observación 1.5. Al igual que con `expand` o `ratsimp`, se puede ajustar el comportamiento de estas órdenes mediante el valor de algunas variables como `trigexpand`, `trigexpandplus` o `trigexpandtimes`. Consulta la ayuda de *Maxima* si estás interesado.

1.7 La ayuda de *Maxima*

El entorno *wxMaxima* permite acceder a la amplia ayuda incluida con *Maxima* de una manera gráfica. En el mismo menú tenemos algunos comandos que nos pueden ser útiles.

<code>describe(expr)</code>	ayuda sobre <i>expr</i>
<code>example(expr)</code>	ejemplo de <i>expr</i>
<code>apropos("expr")</code>	comandos relacionados con <i>expr</i>
<code>? expr</code>	ayuda sobre <i>expr</i>
<code>?? expr</code>	comandos que contienen <i>expr</i>

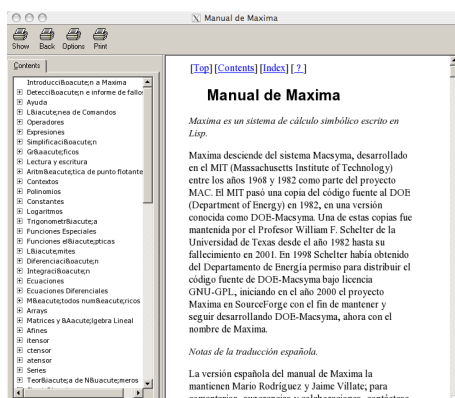


Figura 1.3 Ayuda de *wxMaxima*

En el caso de que conozcamos el nombre del comando sobre el que estamos buscando ayuda, la orden `describe`¹ nos da una breve, a veces no tan breve, explicación sobre la variable, comando `describe` o lo que sea que hayamos preguntado.

```
(%i43) describe(dependencies); - Variable del sistema: dependencies
Valor por defecto: '[]'
La variable 'dependencies' es la lista de átomos que tienen algún
tipo de dependencia funcional, asignada por 'depends' o 'gradeof'.
La lista 'dependencies' es acumulativa: cada llamada a 'depends' o
'gradeof' añade elementos adicionales.
Véanse 'depends' y 'gradeof'.

(%o43) true
```

Claro que a veces nos equivocamos y no nos acordamos exactamente del nombre del comando

```
(%i44) describe(plot); No exact match found for topic 'plot'.
Try '?? plot' (inexact match) instead.

(%o44) false
```

¹ El comando `describe(expr)` produce el mismo resultado que `?expr`. Es obligatorio el espacio en blanco entre la interrogación y la expresión.

La solución la tenemos escrita justo en la salida anterior: ?? busca en la ayuda comandos, variables, etc. que contengan la cadena “plot”.

```
(%i45) ?? plot; 0: Funciones y variables para plotdf
1: Introducción a plotdf
2: barsplot (Funciones y variables para gráficos estadísticos)
3: boxplot (Funciones y variables para gráficos estadísticos)
4: contour_plot (Funciones y variables para gráficos)
5: gnuplot_close (Funciones y variables para gráficos)
6: gnuplot_replot (Funciones y variables para gráficos)
7: gnuplot_reset (Funciones y variables para gráficos)
8: gnuplot_restart (Funciones y variables para gráficos)
9: gnuplot_start (Funciones y variables para gráficos)
10: plot2d (Funciones y variables para gráficos)
11: plot3d (Funciones y variables para gráficos)
12: plotdf (Funciones y variables para plotdf)
13: plot_options (Funciones y variables para gráficos)
14: scatterplot (Funciones y variables para gráficos estadísticos)
15: set_plot_option (Funciones y variables para gráficos)
Enter space-separated numbers, 'all' or 'none':none;

(%o45) true
```

Si, como en este caso, hay varias posibles elecciones, *Maxima* se queda esperando hasta que escribamos el número que corresponde al ítem en que estamos interesados, o `all` o `none` si estamos interesados en todos o en ninguno respectivamente. Mientras no respondamos a esto no podemos realizar ninguna otra operación.

apropos Si has mirado en el menú de *wxMaxima*, seguramente habrás visto **Ayuda→A propósito**. Su propósito es similar a las dos interrogaciones, ??, que acabamos de ver pero el resultado es levemente distinto:

```
(%i46) apropos("plot");

(%o46) [plot,plot2d,plot3d,plotheight,plotmode,plotting,
plot_format,plot_options,plot_realpart]
```

nos da la lista de comandos en los que aparece la cadena `plot` sin incluir nada más. Si ya tenemos una idea de lo que estamos buscando, muchas veces será suficiente con esto.

Muchas veces es mejor un ejemplo sobre cómo se utiliza una orden que una explicación “teórica”. **example** Esto lo podemos conseguir con la orden `example`.

```
(%i47) example(limit);
(%i48) limit(x*log(x),x,0,plus);
(%o48) 0
(%i49) limit((x+1)^(1/x),x,0);
(%o49) %e
```



```
(%i50) limit(%e^x/x,x,inf);
(%o50)  ∞
(%i51) limit(sin(1/x),x,0);
(%o51)  ind
(%o51)  done
```

Por último, la ayuda completa de *Maxima* está disponible en la página web de *Maxima*

<http://maxima.sourceforge.net/>

en formato PDF y como página web. Son más de 800 páginas que explican prácticamente cualquier detalle que se te pueda ocurrir.

1.8 Ejercicios

Ejercicio 1.1. Calcula

- a) Los 100 primeros decimales del número e ,
- b) el logaritmo en base 3 de 16423203268260658146231467800709255289.
- c) el arcocoseno hiperbólico de 1,
- d) el seno y el coseno de i , y
- e) el logaritmo de -2.

Ejercicio 1.2.

- a) ¿Qué número es mayor 1000^{999} o 999^{1000} ?
- b) Ordena de mayor a menor los números π , $\frac{73231844868435875}{37631844868435563}$ y $\cosh(3)/3$.

Ejercicio 1.3. Descompón la fracción $\frac{x^2-4}{x^5+x^4-2x^3-2x^2+x+1}$ en fracciones simples.

Ejercicio 1.4. Escribe $\sin(5x) \cos(3x)$ en función de $\sin(x)$ y $\cos(x)$.

Ejercicio 1.5. Comprueba si las funciones hiperbólicas y las correspondientes “arco”-versiones son inversas.

Gráficos

2

2.1 Funciones 33 2.2 Gráficos en el plano con `plot2d` 37 2.3 Gráficos en el espacio con `plot3d` 47
 2.4 Gráficos con `draw` 50 2.5 Animaciones gráficas 67 2.6 Ejercicios 71

El objetivo de este capítulo es aprender a representar gráficos en dos y tres dimensiones. Lo haremos, tanto para gráficos en coordenadas cartesianas como para gráficos en coordenadas paramétricas y polares. *wxMaxima* permite hacer esto fácilmente aunque también veremos cómo utilizar el módulo `draw` que nos da algunas posibilidades más sin complicar excesivamente la escritura.

2.1 Funciones

<code>func(var1,var2,...):=[expr1,expr2,...]</code>	define la función
<code>define(func,expr)</code>	la función vale <i>expr</i>
<code>fundef(func)</code>	devuelve la definición de la función
<code>functions</code>	lista de funciones definidas por el usuario
<code>remfunction(func1,func2,...)</code>	borra las funciones

Para definir una función en *Maxima* se utiliza el operador `:=`. Se pueden definir funciones de una o varias variables, con valores escalares o vectoriales,

```
(%i1) f(x):=sin(x);
(%o1) f(x):=sin(x)
```

que se pueden utilizar como cualquier otra función.

```
(%i2) f(%pi/4);
(%o2) 1/√2
```

Si la función tiene valores vectoriales o varias variables tampoco hay problema:

```
(%i3) g(x,y,z):=[2*x,3*cos(x+y)];
(%o3) g(x,y,z):=[2x,3cos(x+y)]
(%i4) g(1,%pi,0);
(%o4) [2,-3cos(1)]
```

define También se puede utilizar el comando `define` para definir una función. Por ejemplo, podemos utilizar la función g para definir una nueva función y , de hecho veremos que ésta es la manera correcta de hacerlo cuando la definición involucra funciones previamente definidas, derivadas de funciones, etc. El motivo es que la orden `define` evalúa los comandos que pongamos en la definición.

```
(%i5) define(h(x,y,z),g(x,y,z)^2);
(%o5) h(x,y,z):=[4x^2,9cos(y+x)^2]
```

Eso sí, aunque hemos definido las funciones f , g y h , para utilizarlas debemos añadirles variables:

```
(%i6) g;
(%o6) g
```

Si queremos saber cuál es la definición de la función g , tenemos que preguntar

```
(%i7) g(x,y);
Too few arguments supplied to g(x,y,z):
[x,y]
- an error. To debug this try debugmode(true);
```

pero teniendo cuidado de escribir el número correcto de variables

```
(%i8) g(x,y,z);
(%o8) [2x,3cos(y+x)]
```

Esto plantea varias cuestiones muy relacionadas entre sí: cuando llevamos un rato trabajando y hemos definido varias funciones, ¿cómo sabemos cuáles eran? y ¿cuál era su definición?. La lista de funciones que hemos definido se guarda en la variable `functions` a la que también puedes acceder desde el menú **Maxima**→**Mostrar funciones** de manera similar a como accedemos a la lista de variables. En el mismo menú, **Maxima**→**Borrar función** tenemos la solución a cómo borrar una función (o todas). También podemos hacer esto con la orden `remfunction`.

```
(%i9) functions;
(%o9) [f(x),g(x,y,z),h(x,y,z)]
```

Ya sabemos preguntar cuál es la definición de cada una de ellas. Más cómodo es, quizás, utilizar `fundef` la orden `fundef` que nos evita escribir las variables

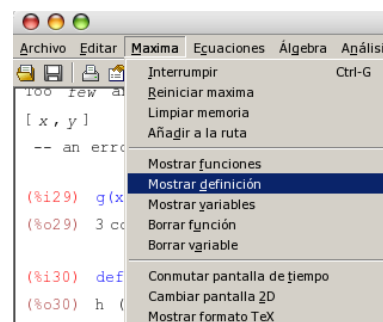


Figura 2.1 Desde el menú podemos consultar las funciones que tenemos definidas, cuál es su definición y borrar algunas o todas ellas

```
(%i10)  fundef(f);
(%o10)  f(x):=sin(x)
```

que, si nos interesa, podemos borrar

```
(%i11)  remfunction(f);
(%o11)  [f]
```

o, simplemente, borrar todas las que tengamos definidas

```
(%i12)  remfunction(all);
(%o12)  [g,h]
```

Funciones definidas a trozos

Las funciones definidas a trozos plantean algunos problemas de difícil solución para *Maxima*. Esencialmente hay dos formas de definir y trabajar con funciones a trozos:

- definir una función para cada trozo con lo que tendremos que ocuparnos nosotros de ir eligiendo de elegir la función adecuada, o
- utilizar una estructura `if-then-else` para definirla.

Cada uno de los métodos tiene sus ventajas e inconvenientes. El primero de ellos nos hace aumentar el número de funciones que definimos, usamos y tenemos que nombrar y recordar. Además de esto, cualquier cosa que queramos hacer, ya sea representar gráficamente o calcular una integral tenemos que plantearlo nosotros. *Maxima* no se encarga de esto. La principal limitación del segundo método es que las funciones definidas de esta manera no nos sirven para derivarlas o integrarlas, aunque sí podremos dibujar su gráfica. Por ejemplo, la función

$$f(x) = \begin{cases} x^2, & \text{si } x < 0, \\ x^3, & \text{en otro caso,} \end{cases}$$

la podemos definir de la siguiente forma utilizando el segundo método

```
(%i13)  f(x):=if x< 0 then x^2 else x^3;
(%o13)  f(x):=if x< 0 then x^2 else x^3
```

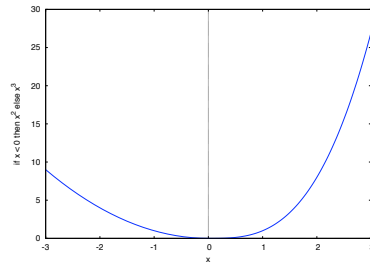
y podemos evaluarla en un punto

```
(%i14)  f(-2);
(%o14)  4
(%i15)  f(2);
(%o15)  8
```

o dibujarla

```
(%i16) plot2d(f(x),[x,-3,3]);
```

```
(%o16)
```



pero no podemos calcular $\int_{-3}^3 f(x) dx$:

```
(%i17) integrate(f(x),x,-3,3);
```

```
(%o17)  $\int_{-3}^3 \text{if } x < 0 \text{ then } x^2 \text{ else } x^3 dx$ 
```

La otra posibilidad es mucho más de andar por casa, pero muy práctica. Podemos definir las funciones

```
(%i18) f1(x):=x^2$
```

```
(%i19) f2(x):=x^3$
```

y decidir nosotros cuál es la que tenemos que utilizar:

```
(%i20) integrate(f1(x),x,-3,0)+integrate(f2(x),x,0,3);
```

```
(%o20)  $\frac{117}{4}$ 
```

Evidentemente, si la función tiene “muchos” trozos, la definición se alarga; no cabe otra posibilidad. En este caso tenemos que anidar varias estructuras if-then-else o definir tantas funciones como trozos. Por ejemplo, la función

$$g(x) = \begin{cases} x^2, & \text{si } x \leq 1, \\ \sin(x), & \text{si } 1 \leq x \leq \pi, \\ -x + 1, & \text{si } x > \pi \end{cases}$$

la podemos escribir como sigue anidando dos condicionales

```
(%i21) g(x):=if x<=1 then x^2 else
         if x <= %pi then sin(x) else -x+1$
```

Comprobamos que la definición se comporta correctamente en un valor de cada intervalo

```
(%i22) [g(-3),g(2),g(5)];
```

```
(%o22) [9,sin(2),-4]
```

2.2 Gráficos en el plano con `plot2d`

2.2.1 Coordenadas cartesianas

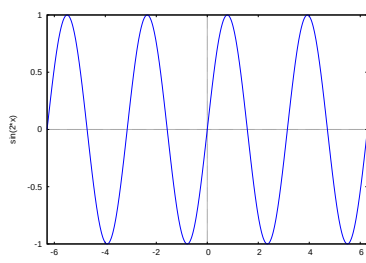
El comando que se utiliza para representar la gráfica de una función de una variable real es `plot2d` que actúa, como mínimo, con dos parámetros: la función (o lista de funciones a representar), `plot2d` y el intervalo de valores para la variable x . Al comando `plot2d` se puede acceder también a través del menú **Gráficos**→**Gráficos 2D**.

<code>plot2d(f(x), [x,a,b])</code>	gráfica de $f(x)$ en $[a,b]$
<code>plot2d([f1(x), f2(x), ...], [x,a,b])</code>	gráfica de una lista de funciones en $[a,b]$

Podemos dibujar la gráfica de una función

(%i23) `plot2d(sin(2*x), [x,-2*pi,2*pi]);`

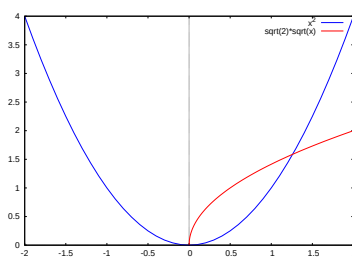
(%o23)



o de varias

(%i24) `plot2d([x^2, sqrt(2*x)], [x,-2,2]);`

(%o24)



Observa en esta última salida cómo el programa asigna a cada gráfica un color distinto para diferenciarlas mejor y añade la correspondiente explicación de qué color representa a cada función.

Cuando accedemos a través del menú, aparece una ventana de diálogo con varios campos que podemos completar o modificar:

- Expresión(es). La función o funciones que queramos dibujar. Por defecto, *wxMaxima* rellena este espacio con % para referirse a la salida anterior.
- Variable x . Aquí establecemos el intervalo de la variable x donde queramos representar la función.

c) Variable y . Ídem para acotar el recorrido de los valores de la imagen.

d) Graduaciones. Nos permite regular el número de puntos en los que el programa evalúa una función para su representación.

e) Formato. *Maxima* realiza por defecto la gráfica con un programa auxiliar. Si seleccionamos en `línea`, obtendremos la gráfica en una ventana alineada con la salida correspondiente. Hay dos opciones más y ambas abren una ventana externa para dibujar la gráfica requerida: `gnuplot` es la opción por defecto que utiliza el programa *Gnuplot* para realizar la representación; también está disponible la opción `openmath` que utiliza el programa *XMaxima*. Prueba las diferentes opciones y decide cuál te gusta más.

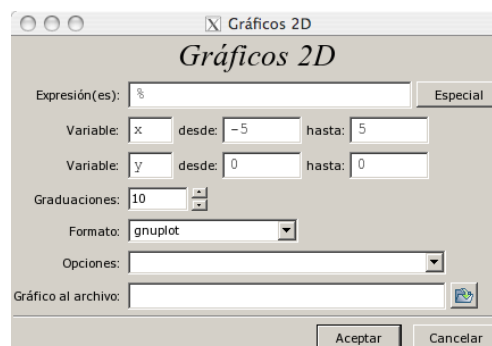


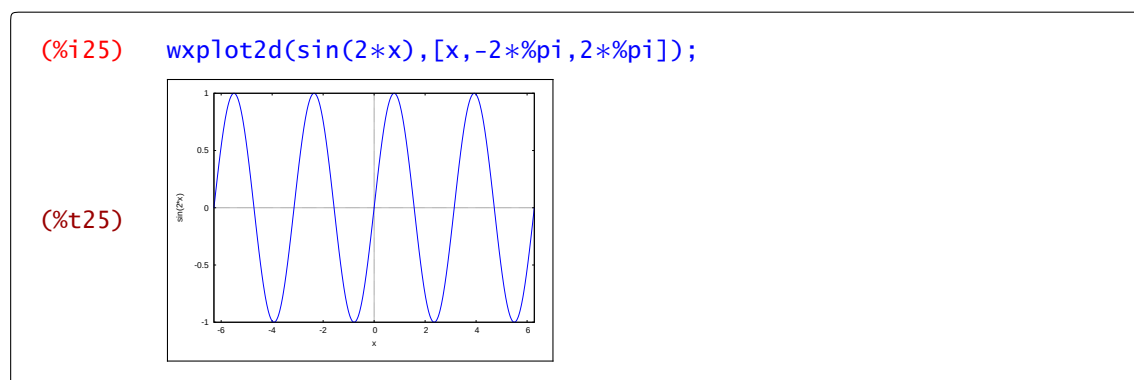
Figura 2.2 Gráficos en 2D

f) Opciones. Aquí podemos seleccionar algunas opciones para que, por ejemplo, dibuje los ejes de coordenadas ("`set zeroaxis;`"); dibuje los ejes de coordenadas, de forma que cada unidad en el eje Y sea igual que el eje X ("`set size ratio 1; set zeroaxis;`"); dibuje una cuadrícula ("`set grid;`") o dibuje una gráfica en coordenadas polares ("`set polar; set zeroaxis;`"). Esta última opción la comentamos más adelante.

g) Gráfico al archivo. Guarda el gráfico en un archivo con formato Postscript.

Evidentemente, estas no son todas las posibles opciones. La cantidad de posibilidades que tiene *Gnuplot* es inmensa.

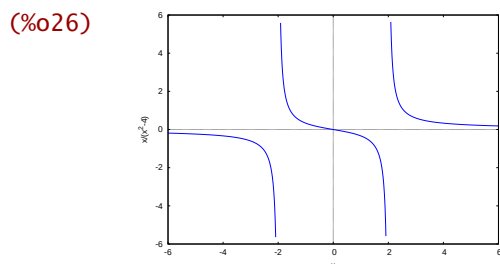
⚠ Observación 2.1. El prefijo "`wx`" añadido a `plot2d` o a cualquiera del resto de las órdenes que veremos en este capítulo hace que *wxMaxima* pase automáticamente a mostrar los gráficos en la misma ventana y no en una ventana separada. Es lo mismo que seleccionar en `línea`. Por ejemplo,



Es complicado representar una ventana separada en unas notas escritas así que, aunque no utilizamos `wxplot2d`, sí hemos representado todas las gráficas a continuación de la correspondiente orden.

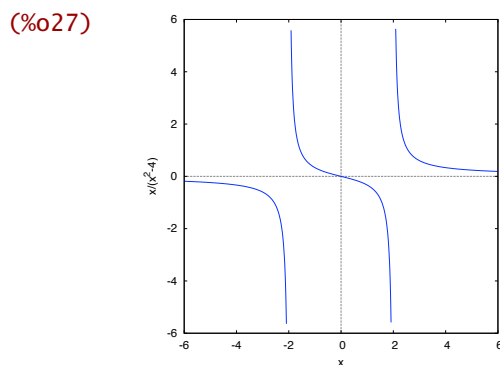
Veamos algunos ejemplos de las opciones que hemos comentado. Podemos añadir ejes,


```
(%i26) plot2d(x/(x^2-4), [x,-6,6], [y,-6,6],
             [gnuplot_preamble, "set zeroaxis;"])$
```



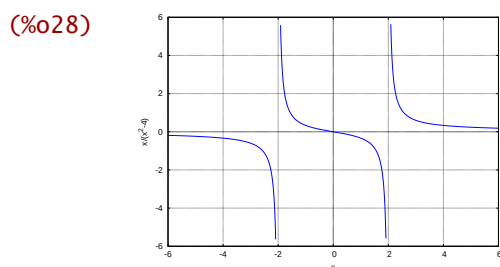
podemos cambiar la proporción entre ejes.

```
(%i27) plot2d(x/(x^2-4), [x,-6,6], [y,-6,6],
             [gnuplot_preamble, "set size ratio 1; set zeroaxis;"])$
```



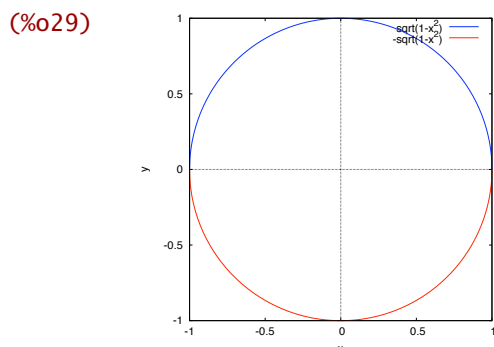
`set size ratio 1` dibuja ambos ejes con el mismo tamaño en pantalla, `set size ratio 2` o `set size ratio 0.5` dibuja el eje X el doble o la mitad de grande que el eje Y. O podemos añadir una malla que nos facilite la lectura de los valores de la función.

```
(%i28) plot2d(x/(x^2-4), [x,-6,6], [y,-6,6],
             [gnuplot_preamble, "set grid;"])$
```



Con el siguiente ejemplo vamos a ver la utilidad de la opción `"set size ratio 1; set zeroaxis;".` En primer lugar dibujamos las funciones $\sqrt{1-x^2}$ y $-\sqrt{1-x^2}$, con $x \in [-1, 1]$. El resultado debería ser la circunferencia unidad. Sin embargo, aparentemente es una elipse. Lo arreglamos de la siguiente forma:

```
(%i29) plot2d([sqrt(1-x^2),-sqrt(1-x^2)],[x,-1,1],[y,-1,1],
             [gnuplot_preamble, "set size ratio 1; set zeroaxis;"])$
```



También podemos dibujar gráficas de funciones a trozos. Antes, tenemos que recordar cómo se definen estas funciones. Lo hacemos con un ejemplo. Consideremos la función $f: \mathbb{R} \rightarrow \mathbb{R}$ definida como

$$f(x) = \begin{cases} \sqrt{-x} & \text{si } x < 0 \\ x^3 & \text{si } x \geq 0. \end{cases}$$

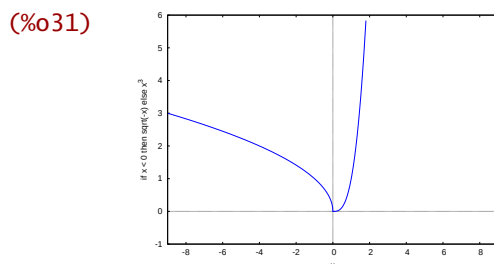
Vamos, en primer lugar, a definirla:

```
(%i30) f(x):= if x<0 then sqrt(-x) else x^3;
```

```
(%o30) f(x):= if x<0 then sqrt(-x) else x^3
```

y luego la representamos

```
(%i31) plot2d(f(x),[x,-9,9],[y,-1,6],
             [gnuplot_preamble,"set zeroaxis;"])$
```



2.2.2 Gráficos en coordenadas polares

Para dar un punto del plano, tenemos que indicar los valores de las proyecciones sobre los ejes X e Y (esto es a lo que llamamos coordenadas cartesianas) o podemos indicar la distancia al origen y el ángulo que forma con una dirección fija (en nuestro caso la parte positiva del eje X).

Al representar una curva en coordenadas polares estamos escribiendo la longitud del vector como una función que depende del ángulo. En otras palabras, para cada ángulo fijo decimos cuál es el módulo del vector. El ejemplo más sencillo de función que a cualquiera se nos viene a la cabeza son las funciones constantes. La función $f: [0, 2\pi] \rightarrow \mathbb{R}$, $f(\vartheta) = 1$ tiene como imagen aquellos vectores que tienen módulo 1 y argumento entre 0 y 2π . Para ello, tenemos que seleccionar "`set polar; set zeroaxis;`" en el campo **Opciones** de Gráficos 2D y obtenemos su gráfica, una circunferencia. La segunda opción muestra los ejes cruzando el origen como suele ser habitual.

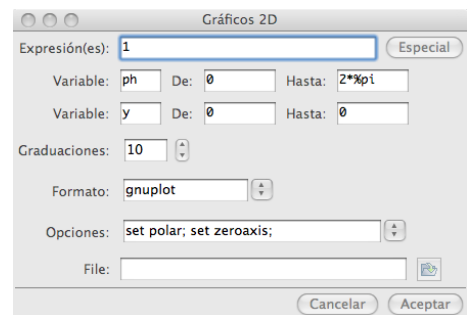
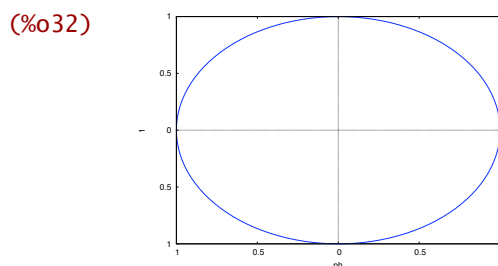


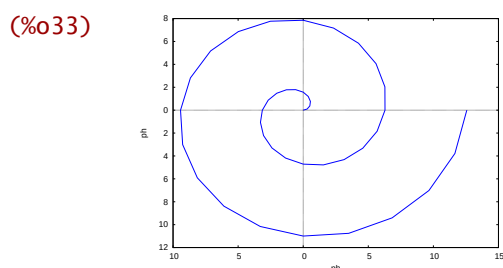
Figura 2.3 Gráfico en coordenadas polares

```
(%i32) plot2d(
    [1], [ph,0,2*pi],
    [plot_format, gnuplot],
    [gnuplot_preamble, "set polar; set zeroaxis;"]
)$
```



Si la función es creciente, la identidad por ejemplo, el módulo aumenta con el argumento. En este caso obtenemos una espiral.²

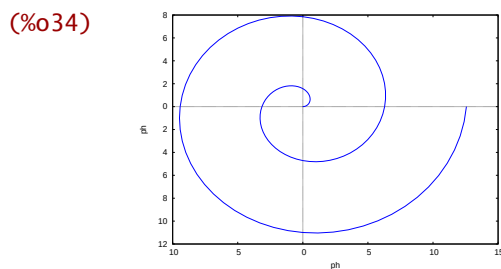
```
(%i33) plot2d(ph,[ph,0,4*pi],
    [gnuplot_preamble, "set polar; set zeroaxis;"])$
```



Observamos que la hélice resultante no es nada "suave". Para conseguir el efecto visual de una línea curva como es esta hélice, añadimos el parámetro `nticks`. Por defecto, para dibujar una gráfica en paramétricas el programa evalúa en 10 puntos. Para aumentar este número de puntos, aumentamos dicho parámetro, por ejemplo `nticks=30`, o bien, podemos regularlo desde el campo **Graduaciones** dentro de de Gráficos 2D.

² Como puedes observar, la variable por defecto para indicar el argumento es `ph` y no la "`x`" usual cuando trabajamos en coordenadas cartesianas.

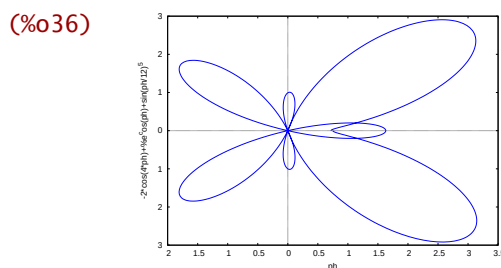
```
(%i34) plot2d(ph,[ph,0,4*%pi],
             [gnuplot_preamble, "set polar; set zeroaxis;"], [nticks,30])$
```



Si representamos la función: $r(\theta) = e^{\cos(\theta)} - 2\cos(4\theta) + \sin\left(\frac{\theta}{12}\right)^5$ obtenemos algo parecido a una mariposa.

```
(%i35) r(ph):=exp(cos(ph))-2*cos(4*ph)+sin(ph/12)**5$
```

```
(%i36) plot2d(r(ph),[ph,0,2*%pi],
             [gnuplot_preamble, "set polar; set zeroaxis;"])$
```



2.2.3 Gráficos en coordenadas paramétricas

El programa *wxMaxima* nos permite también representar curvas en forma paramétrica, es decir, curvas definidas como $(x(t), y(t))$ donde el parámetro t varía en un determinado intervalo compacto $[a, b]$. Para ello, dentro del comando `plot2d` añadimos “parametric” de la forma siguiente:

```
plot2d([parametric,x(t),y(t),[t,a,b]])
```

gráfica de la curva $(x(t), y(t))$ en $[a, b]$

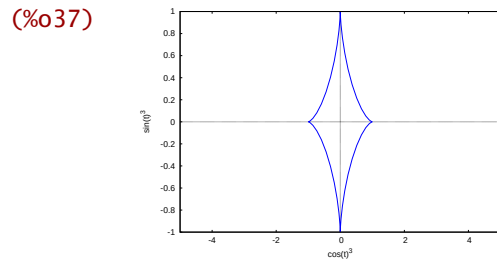
Para acceder a esta opción de la función `plot2d` podemos hacerlo a través del botón **Especial** que aparece en la parte superior derecha de la ventana de diálogo **Gráficos 2D**.

Algunos ejemplos

Para terminar, aquí tienes algunas curvas planas interesantes.

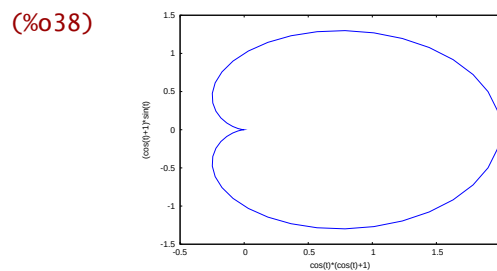
Astroide: Es la curva trazada por un punto fijo de un círculo de radio r que rueda sin deslizar dentro de otro círculo fijo de radio $4r$. Sus ecuaciones paramétricas son:

```
(%i37) plot2d([parametric,
               cos(t)^3,sin(t)^3,
               [t,0,2*%pi],[nticks,50]]);
```



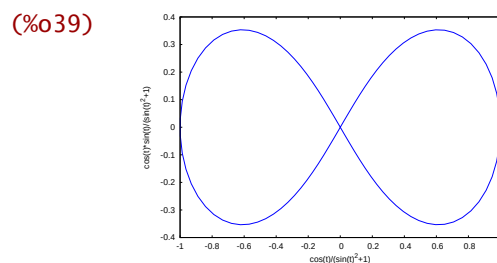
Cardioide: Es la curva trazada por un punto fijo de un círculo de radio r que rueda sin deslizar alrededor de otro círculo fijo del mismo radio. Sus ecuaciones paramétricas son.

```
(%i38) plot2d([parametric,
               (1+cos(t))*cos(t),(1+cos(t))*sin(t)),
               [t,0,2*%pi],[nticks,50]])
```



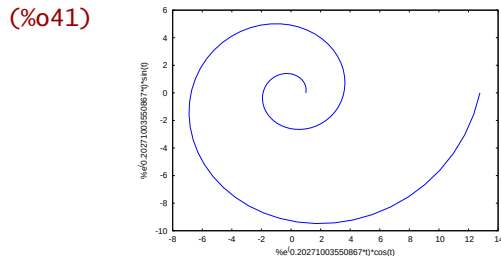
Lemniscata de Bernoulli: Es el lugar geométrico de los puntos P del plano, cuyo producto de distancias a dos puntos fijos F_1 y F_2 , llamados focos, verifica la igualdad $|P - F_1| \cdot |P - F_2| = \frac{1}{4} |F_1 - F_2|^2$. En coordenadas cartesianas esta curva viene dada por la ecuación $(x^2 + y^2)^2 = x^2 - y^2$. Aquí tienes sus ecuaciones paramétricas.

```
(%i39) plot2d([parametric,
               cos(t)/(1+sin(t)^2), cos(t)*sin(t)/(1+sin(t)^2),
               [t,0,2*%pi],[nticks,70]])
```



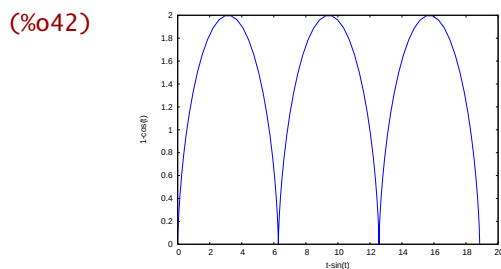
Espiral equiangular: También llamada espiral logarítmica. Es aquella espiral en la que el radio vector corta a la curva en un ángulo constante α . Sus ecuaciones paramétricas son:

```
(%i40) %alpha:%pi/2-0.2$
(%i41) plot2d([parametric,
               exp(t*cot(%alpha))*cos(t), exp(t*cot(%alpha))*sin(t),
               [t,0,4*pi],[nticks,90]])
```



Cicloide: También conocida como tautocrona o braquistocrona. Es la curva que describiría un punto de una circunferencia que avanza girando sin deslizar. Sus ecuaciones paramétricas son:

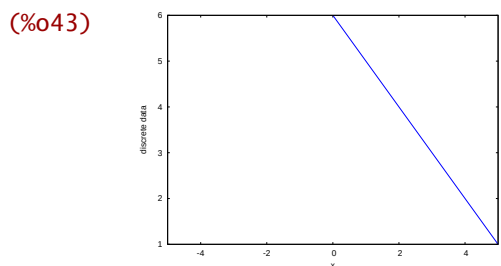
```
(%i42) plot2d([parametric,
               t-sin(t),1-cos(t),
               [t,0,6*pi],[nticks,90]])
```



2.2.4 Gráficas de curvas poligonales

Además de dibujar curvas definidas de forma paramétrica, podemos dibujar líneas poligonales, haciendo uso de una opción más dentro de **Especial** (concretamente, Gráfico discreto) en la ventana de diálogo de **Gráficos 2D**. Cuando elegimos esta opción, aparece una nueva ventana en la que tendremos que escribir las coordenadas, separadas por comas, de los puntos que van a ser los vértices de la curva poligonal que queremos dibujar. Por ejemplo, para dibujar la recta que une los puntos (0,6) y (5,1), escribimos:

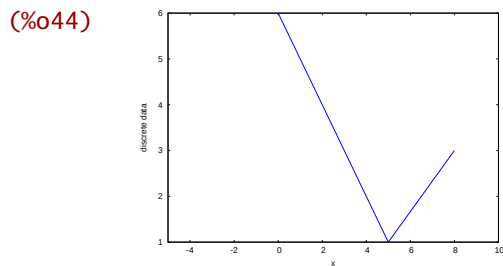
```
(%i43) plot2d([discrete,[0,5],[6,1]],
               [x,-5,5]);
```



<code>plot2d([discrete,[x1,x2,...],[y1,y2,...]],opc)</code>	poligonal que une los puntos $(x_1, y_1), (x_2, y_2), \dots$
<code>plot2d([discrete,[[x1,y1],[x2,y2],...]],opc)</code>	poligonal que une los puntos $(x_1, y_1), (x_2, y_2), \dots$

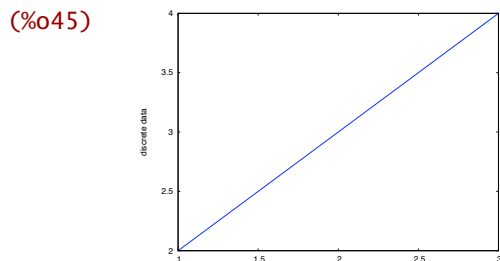
Si lo que queremos es pintar una línea poligonal sólo tenemos que pasar la lista de las primeras y de las segundas coordenadas:

```
(%i44) plot2d([discrete,[0,5,8],[6,1,3]],
[x,-5,10]);
```



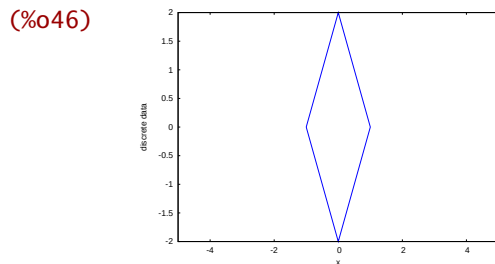
o, agrupar los puntos en una lista,

```
(%i45) plot2d([discrete,[[1,2],[2,3],[3,4]]]);
```



Si el primer y el último punto coinciden se obtiene lo esperable: una línea poligonal cerrada. Por ejemplo, el rombo de vértices $(-1, 0)$, $(0, -2)$, $(1, 0)$ y $(0, 2)$:

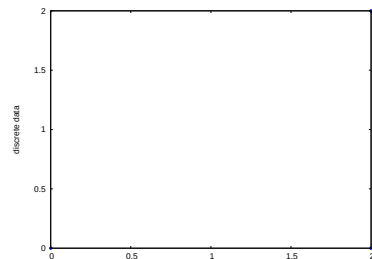
```
(%i46) plot2d([discrete,[-1,0,1,0,-1],[0,-2,0,2,0]],
[x,-5,5]);
```



Puntos y segmentos

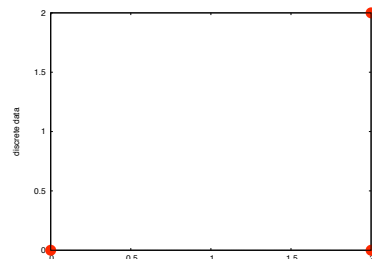
¿Y si lo que queremos es pintar puntos en el plano? Para ello definimos la lista de puntos que queramos pintar y a continuación, dentro del `plot2d` añadimos la opción “`style,points`” que dibuja los puntos y no los segmentos que los unen.

```
(%i47) xy:[[0,0],[2,0],[2,2]]$
(%i48) plot2d([discrete,xy],[style,points]);
(%o48)
```



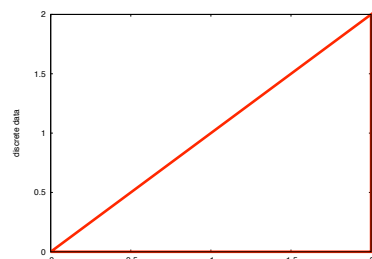
Si queremos modificar el aspecto de los puntos a pintar, escribiremos tres números (`[points,a,b,c]`) después de la opción `points` cuyo significado es el siguiente: `a`, radio de los puntos; `b`, índice del color (1 para azul, 2 para rojo,...); `c`, tipo de objeto utilizado (puntos, circunferencias, cruces, asteriscos, cuadrados,...). De la misma forma, si queremos modificar el aspecto de los segmentos: dentro de `style` elegimos la opción `lines` seguida de dos cifras (`[lines, $\alpha$ , $\beta$ ]`) que hacen referencia al ancho de la línea (α) y al color (β).

```
(%i49) plot2d([discrete,xy],[style,[points,10,2]]);
(%o49)
```



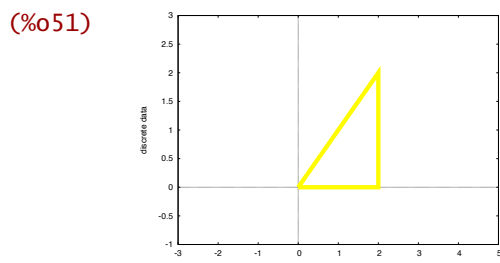
Dibujemos ahora un triángulo rectángulo de vértices: (0,0), (2,0) y (2,2) con los lados en rojo.

```
(%i50) plot2d([discrete,[0,2,2,0],[0,0,2,0]],[style,[lines,15,2]]);
(%o50)
```



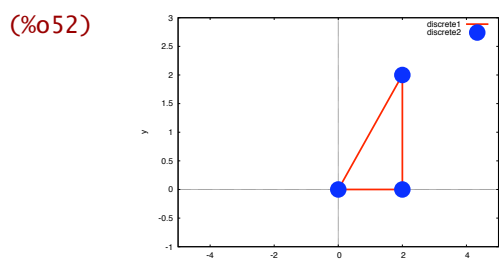
Observa que si no indicamos ningún rango, *Maxima* ajusta el dominio a los valores que estemos representando. Podemos ampliar el rango de las variables x e y y obtenemos algo así.


```
(%i51) plot2d([discrete, [0,2,2,0],[0,0,2,0]],
               [x,-3,5],[y,-1,3],
               [style,[lines,25,5]]);
```



Y si ahora queremos dibujar el triángulo anterior (en rojo) junto con los vértices (en azul):

```
(%i52) plot2d([[discrete, [0,2,2,0],[0,0,2,0]],[discrete,xy]],
               [x,-5,5],[y,-1,3],
               [style,[lines,10,2],[points,15,1,1]]);
```

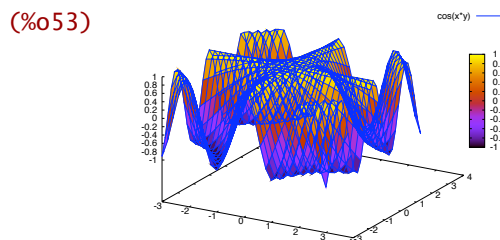


2.3 Gráficos en el espacio con `plot3d`

Con *Maxima* se pueden representar funciones de dos variables de forma similar a como hemos representado funciones de una. La principal diferencia es que vamos a utilizar el comando `plot3d` en lugar de `plot2d`, pero igual que en el caso anterior, son obligatorios la función o funciones y el dominio que tiene que ser de la forma $[a, b] \times [c, d]$.

`plot3d(f(x,y), [x,a,b], [y,c,d])` gráfica de $f(x,y)$ en $[a,b] \times [c,d]$

```
(%i53) plot3d(cos(x*y), [x,-3,3], [y,-3,3]);
```



Observación 2.2. La gráfica que acabas de obtener se puede girar sin más que pinchar con el ratón sobre ella y deslizar el cursor.



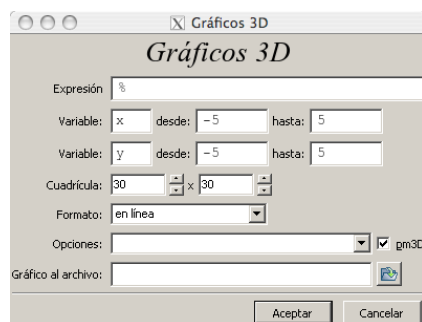


Figura 2.4 Gráficos 3D

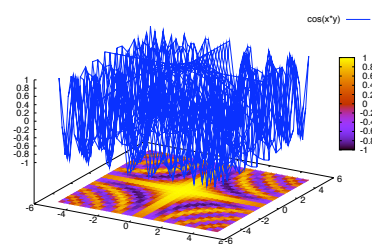
Al comando `plot3d` se accede a través del menú **Gráficos—Gráficos 3D** o del botón **Gráficos 3D**. Después de esto aparece la ventana de la Figura 2.4 con varios campos para rellenar:

- a) Expresión. La función o funciones que vayamos a dibujar.
- b) Variable. Hay dos campos para indicar el dominio de las dos variables.
- c) Cuadrícula. Indica cuántas valores se toman de cada variable para representar la función. Cuanto mayor sea, más suave será la representación a costa de aumentar la cantidad de cálculos.
- d) Formato. Igual que en `plot2d`, permite escoger qué programa se utiliza para representar la función. Se puede girar la gráfica en todos ellos salvo si escoges “en línea”.
- e) Opciones. Las comentamos a continuación.
- f) Gráfico al archivo. Permite elegir un fichero donde se guardará la gráfica.

Quizá la mejor manera de ver el efecto de las opciones es repetir el dibujo anterior. La primera de ellas es “`set pm3d at b`” que dibuja la superficie usando una malla y en la base añade curvas de nivel (como si estuviéramos mirando la gráfica desde arriba):

```
(%i54) plot3d(cos(x*y), [x,-5,5], [y,-5,5], [plot_format,gnuplot],
[gnuplot_preamble, "set pm3d at b"])$
```

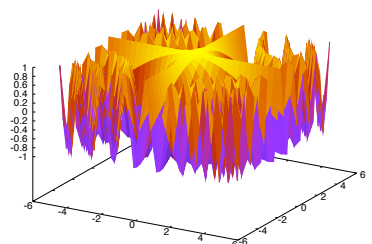
```
(%o54)
```



La segunda hace varias cosas, “`set pm3d at s`” nos dibuja la superficie coloreada, “`unset surf`” elimina la malla y “`unset colorbox`” elimina la barra que teníamos en la derecha con la explicación de los colores y su relación con la altura (el valor de la función):

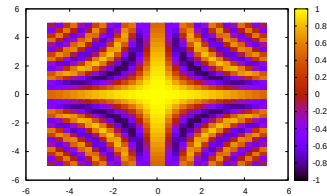
```
(%i55) plot3d(cos(x*y), [x,-5,5], [y,-5,5],
[plot_format,gnuplot],
[gnuplot_preamble, "set pm3d at s; unset surf;
unset colorbox"])$
```

```
(%o55)
```



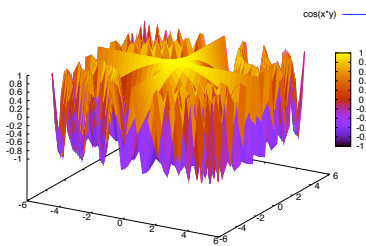
La tercera, "`set pm3d map`", nos dibuja un mapa de curvas de nivel con alguna ayuda adicional dada por el color:

```
(%i56) plot3d(cos(x*y), [x,-5,5], [y,-5,5], [plot_format,gnuplot],
        [gnuplot_preamble, "set pm3d map; unset surf"]$
(%o56)
```



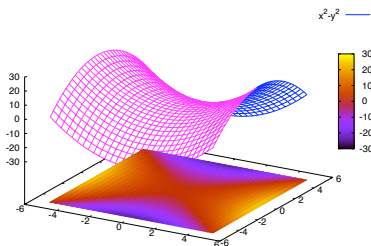
La cuarta, "`set hidden3d`", sólo muestra la parte de la superficie que sería visible desde nuestro punto de vista. En otras palabras, hace la superficie sólida y no transparente.

```
(%i57) plot3d(cos(x*y), [x,-5,5], [y,-5,5], [plot_format,gnuplot],
        [gnuplot_preamble, "set hidden3d"]$
(%o57)
```



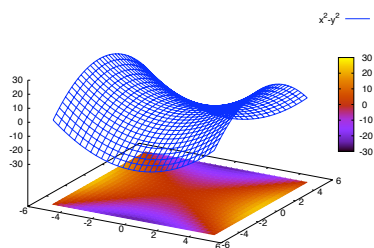
En el dibujo anterior (en el papel) es posible que no aprecie bien. A simple vista parece el mismo dibujo que teníamos dos salidas antes. Observa bien: hay una pequeña diferencia. El uso de `pm3d` hace que se coloree el dibujo, pero cuando decimos que no se muestra la parte no visible de la figura nos estamos refiriendo a la malla. Quizá es mejor dibujar la malla y el manto de colores por separado para que se vea la diferencia. Esta opción no viene disponible por defecto en *wxMaxima*. Ten en cuenta que las opciones que tiene *Gnuplot* son casi infinitas y sólo estamos comentando algunas.

```
(%i58) plot3d(x^2-y^2, [x,-5,5], [y,-5,5],
        [plot_format,gnuplot],
        [gnuplot_preamble, "set pm3d at b; set hidden3d"])$
(%o58)
```



```
(%i59) plot3d(x^2-y^2, [x,-5,5], [y,-5,5], [plot_format,gnuplot],
          [gnuplot_preamble, "set pm3d at b"])$
```

```
(%o59)
```



La quinta y la sexta opciones nos permiten dibujar en coordenadas esféricas o cilíndricas. Ya veremos ejemplos más adelante.

2.4 Gráficos con draw

El módulo “draw” es reciente en la historia de *Maxima* y permite dibujar gráficos en 2 y 3 dimensiones con relativa comodidad. Se trata de un módulo adicional que hay que cargar previamente. Este se hace de la siguiente forma

```
(%i60) load(draw)$
```

<code>gr2d(opciones, objeto gráfico,...)</code>	gráfico dos dimensional
<code>gr3d(opciones, objeto gráfico,...)</code>	gráfico tres dimensional
<code>draw(opciones, objeto gráfico,...)</code>	dibuja un gráfico
<code>draw2d(opciones, objeto gráfico,...)</code>	dibuja gráfico dos dimensional
<code>draw3d(opciones, objeto gráfico,...)</code>	dibuja gráfico tres dimensional

Una vez cargado el módulo draw, podemos utilizar las órdenes `draw2d` y `draw3d` para dibujar gráficos en 2 y 3 dimensiones o `draw`. Un gráfico está compuesto por varias opciones y el objeto gráfico que queremos dibujar. Por ejemplo, en dos dimensiones tendríamos algo así:

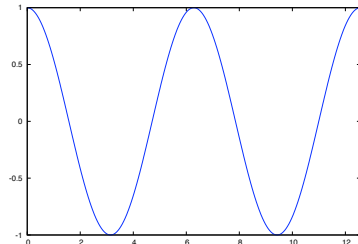
```
objeto:gr2d(
  color=blue,
  nticks=60,
  explicit(cos(t),t,0,2*$*$\%pi)
)
```

Las opciones son numerosas y permiten controlar prácticamente cualquier aspecto imaginable. Aquí comentaremos algunas de ellas pero la ayuda del programa es insustituible. En segundo lugar aparece el objeto gráfico. En este caso “`explicit(cos(t),t,0,2*$*$\%pi)`”. Estos pueden ser de varios tipos aunque los que más usaremos son quizás `explicit` y `parametric`. Para dibujar un gráfico tenemos dos posibilidades

- `draw` a) Si tenemos previamente definido el objeto, `draw(objeto)`, o bien,
- `draw2d` b) `draw2d(definición del objeto)` si lo definimos en ese momento para dibujarlo.

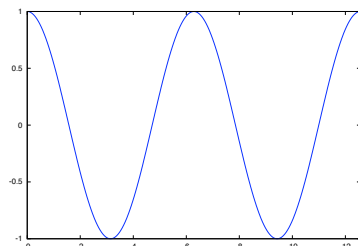
Por ejemplo,

```
(%i61) coseno:gr2d(
        color=blue,
        explicit(cos(x),x,0,4*%pi))$
(%i62) draw(coseno);
(%o62)
```



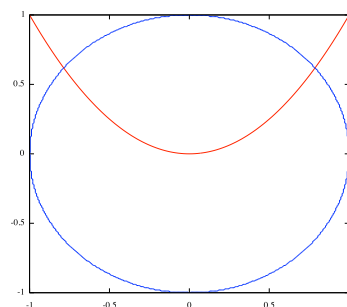
da el mismo resultado que

```
(%i63) draw2d(
        color=blue,
        explicit(cos(x),x,0,4*%pi)
        );
(%o63)
```



También podemos representar más de un objeto en un mismo gráfico. Simplemente escribimos uno tras otro separados por comas. En el siguiente ejemplo estamos mezclando una función dada explícitamente y una curva en coordenadas paramétricas.

```
(%i64) draw2d(
        color=red, explicit(x^2,x,-1,1),
        color=blue, nticks=60, implicit(x^2+y^2=1,x,-1,1,y,-1,1)
        );
(%o64)
```



draw3d En tres dimensiones, la construcción es similar.

2.4.1 Objetos

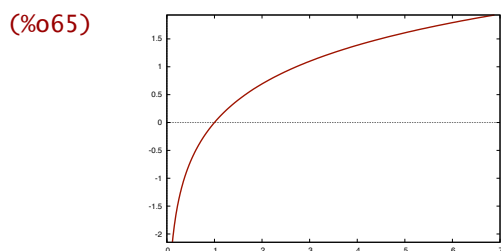
Hay muchos tipos de objetos que podemos representar de forma sencilla con el paquete **draw**. En la versión 5.38 de Maxima, existen los siguientes:

- a) en 2D: bars, ellipse, errors, explicit, image, implicit, label, parametric, points, polar, polygon, quadrilateral, rectangle, region, triangle y vector;
- b) en 3D: cylindrical, elevation_grid, explicit, implicit, label, mesh, parametric, parametric_surface, points, quadrilateral, spherical, triangle, tube y vector.

A continuación vemos ejemplos de algunos de los objetos que podemos representar y, en la siguiente sección, comentamos algunas de las opciones.

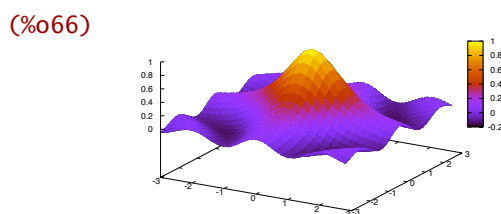
explicit: nos permite dibujar una función de una o dos variables. Para funciones de una variable usaremos `explicit( $f(x)$ ,  $x$ ,  $a$ ,  $b$ )` para dibujar $f(x)$ en $[a, b]$.

```
(%i65) draw2d(
        color=dark-red,line_width=5,
        xaxis=true,yaxis=true,
        explicit(log(x),x,0,7) );
```



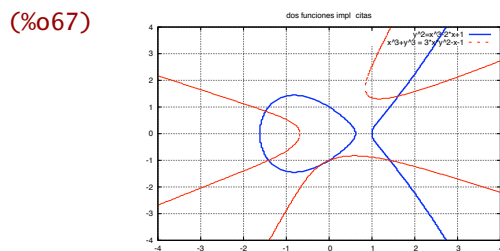
Con funciones de dos variables escribiremos `explicit( $f(x, y)$ ,  $x$ ,  $a$ ,  $b$ ,  $y$ ,  $c$ ,  $d$ )`.

```
(%i66) draw3d(enhanced3d=true,
        surface_hide=true,
        explicit((1+x^2+y^2)^(-1)*cos(x*y),x,-3,3,y,-3,3));
```



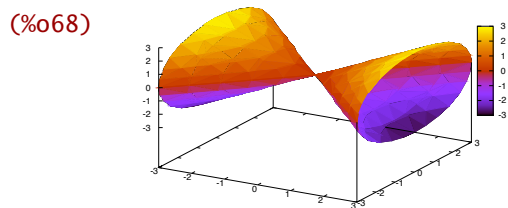
implicit: nos permite dibujar el lugar de los puntos que verifican una ecuación en el plano

```
(%i67) draw2d(
    grid=true,
    line_type=solid, color=blue, key="y^2=x^3-2x+1",
    implicit(y^2=x^3-2*x+1, x, -4,4, y, -4,4),
    line_type=dots, color=red, key="x^3+y^3 = 3xy^2-x-1",
    implicit(x^3+y^3 = 3*x*y^2-x-1, x,-4,4, y,-4,4),
    title="dos funciones implícitas");
```



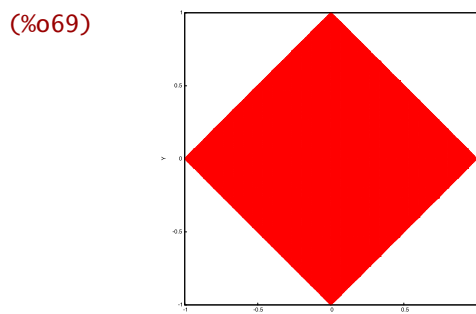
o en tres dimensiones. Además de la ecuación debemos indicar los intervalos dónde pueden tomar valores las variables.

```
(%i68) draw3d(
    surface_hide=true, enhanced3d=true,
    implicit(x^2-y^2=z^2,x,-3,3,y,-3,3,z,-3,3));
```



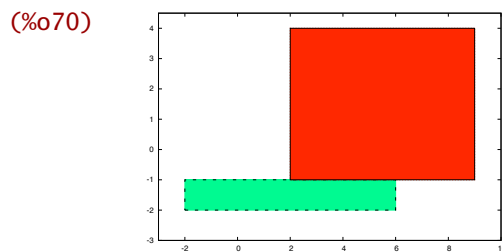
region: colorea la zona del plano en la que una desigualdad se cumple. Como entradas debemos indicar la desigualdad o desigualdades y la parte del plano donde miramos si se cumple dicha desigualdad.

```
(%i69) wxdraw2d(x_voxel=30,y_voxel=30,
    proportional_axes=xy,
    region((abs(x)+abs(y)<=1),x,-1,1,y,-1,1);
```



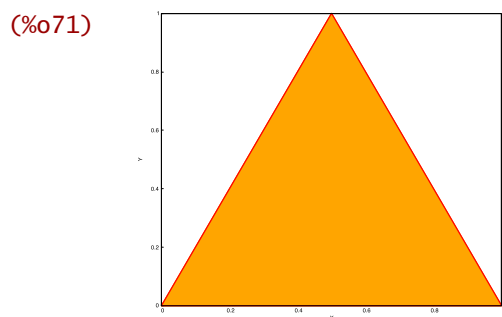
rectangle: para dibujar un rectángulo sólo tenemos que indicar el vértice inferior izquierdo y el superior derecho.

```
(%i70) draw2d(line_width=6,
             line_type=dots, transparent=false,
             fill_color=spring-green,
             rectangle([-2,-2],[6,-1]),
             fill_color=red, line_type=solid,
             line_width=2,
             rectangle([9,4],[2,-1]),
             xrange=[-3,10], yrange=[-3,4.5]);
```



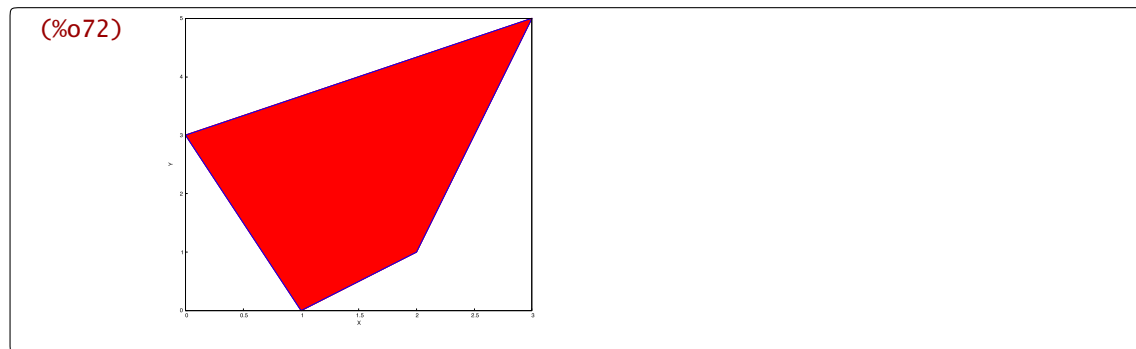
triangle: dibuja un triángulo indicando sus vértices.

```
(%i71) wxdraw2d( line_width=2, color=red,
                 fill_color=orange,
                 triangle([0,0],[0.5,1],[1,0]));
```

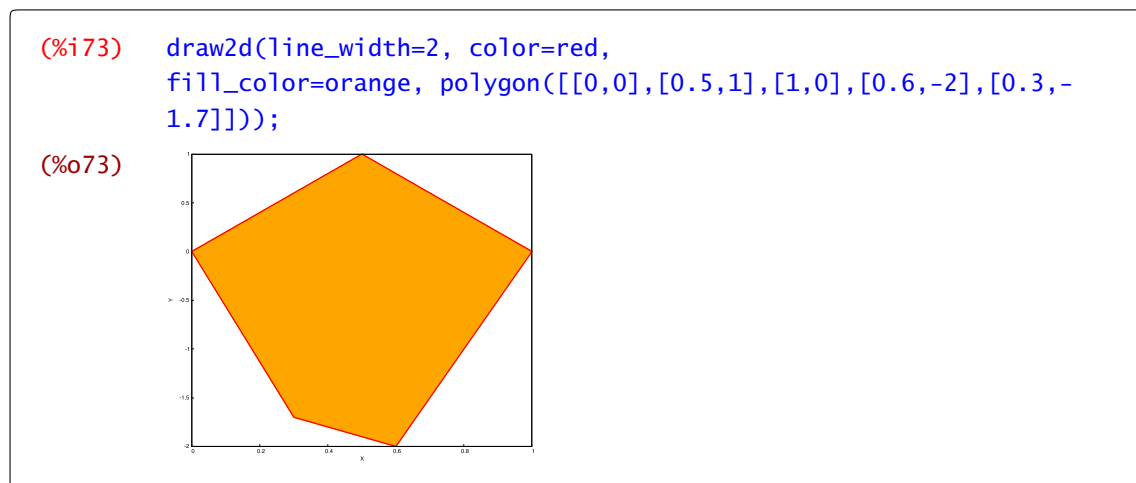


quadrilateral: la orden quadrilateral dibuja un cuadrilátero indicando los cuatros vértices, ya sea en el plano o en 3D.

```
(%i72) wxdraw2d(
             quadrilateral([0,3],[1,0],[2,1],[3,5]);
```

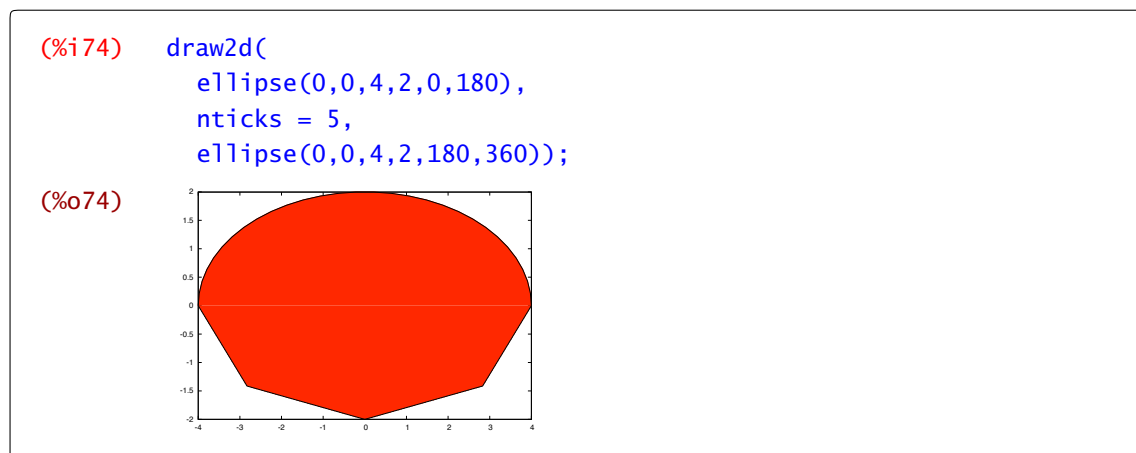



polygon: dibuja el polígono dado por una *lista* de vértices.



ellipse: la orden `ellipse` permite dibujar elipses indicando 3 pares de números: los dos primeros son las coordenadas del centro, los dos segundos indican la longitud de los semiejes y los últimos son los ángulos inicial y final.

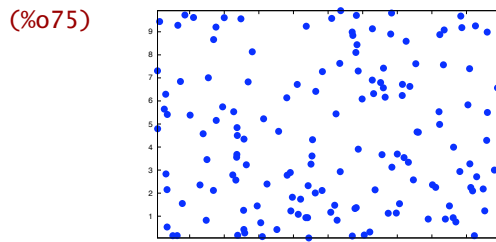
En el dibujo siguiente puedes comprobar cómo la opción `nticks` permite mejorar, aquí empeorar, un gráfico aumentando o, como en este caso, disminuyendo el número de puntos que se utilizan para dibujarlo.



La parte superior de la elipse se ha dibujado utilizando 30 puntos y la inferior únicamente 5.

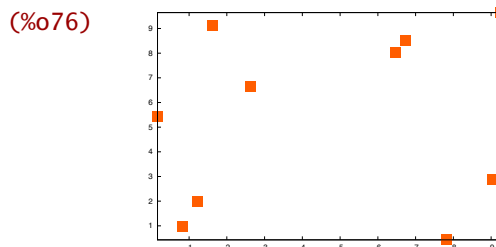
points: para representar una lista de puntos en el plano o en el espacio tenemos dos posibilidades. Podemos dar los vectores de la forma³ $[[x_1, y_1], [x_2, y_2], \dots]$, como por ejemplo

```
(%i75) draw2d(
        color=blue,
        point_type=filled_circle,
        point_size=2,
        points(makelist([random(10.0), random(10.0)], k, 1, 150)));
```



o podemos agrupar por coordenadas de la forma $[[x_1, x_2, x_3, \dots], [y_1, y_2, y_3, \dots]]$ como aquí.

```
(%i76) draw2d(
        color=orange-red,
        point_type=5,
        point_size=3,
        points(makelist(random(10.0), k, 1, 10),
               makelist(random(10.0), k, 1, 10)));
```

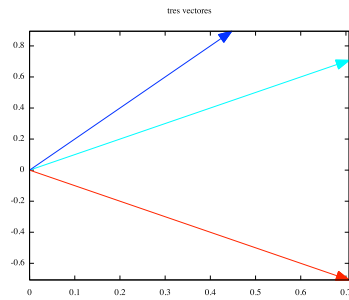


vector: dibuja vectores tanto en dos como en tres dimensiones. Para dar un vector hay que fijar el origen y la dirección.

```
(%i77) draw3d(color = cyan,
               vector([0,0,0], [1,1,1]/sqrt(3)),
               color=red, vector([0,0,0], [1,-1,0]/sqrt(2)),
               color=blue, vector([0,0,0], [1,1,-2]/sqrt(6)),
               title="tres vectores");
```

³ En el ejemplo usaremos la orden `makelist` que genera una lista de acuerdo a la regla que aparece como primera entrada con tantos elementos como indique el contador que le sigue. En el próximo capítulo lo comentaremos con más detalle.

(%o77)



En la ayuda puedes encontrar varias opciones sobre el aspecto como se representan los vectores.

parametric: nos permite dibujar una curva en dos o en tres dimensiones. En coordenadas paramétricas, tenemos que dar las dos o tres coordenadas de la forma

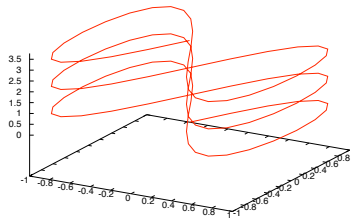
$$\text{parametric}(f_1(x), f_2(x), x, a, b)$$

$$\text{parametric}(f_1(x), f_2(x), f_3(x), x, a, b)$$

Por ejemplo, en tres dimensiones la curva $t \mapsto (\sin(t), \sin(2t), t/5)$ se representa de la siguiente forma.

```
(%i78) draw3d(color=red,
           nticks=100,
           parametric(sin(t), sin(2*t), t/5, t, 0, 6*%pi));
```

(%o78)

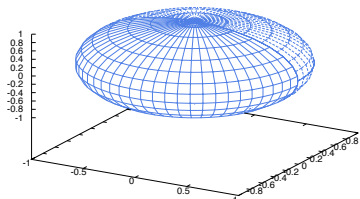


parametric_surface: nos permite dibujar una superficie en paramétricas. Tenemos pues que dar las tres coordenadas así que escribimos

$$\text{parametric_surface}(f_1(x, y), f_2(x, y), f_3(x, y), x, a, b, y, c, d)$$

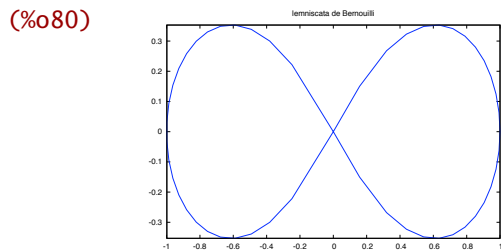
```
(%i79) draw3d(
           color=royalblue,
           surface_hide=true,
           parametric_surface(cos(u)*cos(v), cos(v)*sin(u), sin(v),
                               u, 0, %pi, v, 0, 2*%pi));
```

(%o79)



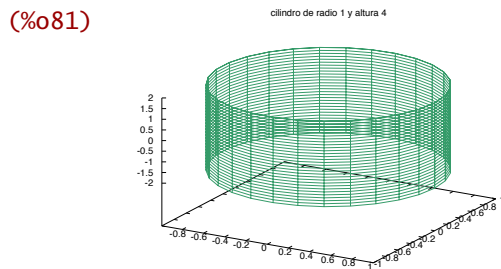
polar: dibuja el módulo del vector en función del ángulo t

```
(%i80) draw2d(
        color=blue,
        nticks=100,
        title="lemniscata de Bernouilli",
        polar(sqrt(cos(2*t)),t,0,2*%pi));
```



cylindrical: para dar un vector del espacio en coordenadas cilíndricas lo que hacemos es cambiar a coordenadas polares en las dos primeras variables (aunque tendría perfecto sentido hacerlo con cualquier par de ellas). La figura cuya representación es más sencilla en coordenadas cilíndricas te la puedes imaginar.

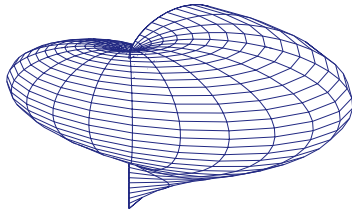
```
(%i81) draw3d(
        color=sea-green,
        title="cilindro de radio 1 y altura 4",
        cylindrical(1,z,-2,2,t,0,2*%pi));
```



spherical: en coordenadas esféricas la construcción es muy parecida. Representamos el módulo y damos dónde varían los dos ángulos.

```
(%i82) draw3d(
        color=midnight-blue,
        surface_hide = true,
        axis_3d=false,
        xtics=None,
        ytics=None,
        ztics=None,
        spherical(a*z,a,0,2.5*%pi,z,0,%pi));
```

(%o82)



2.4.2 Opciones

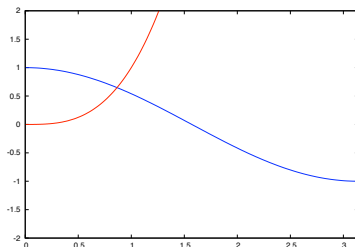
Es importante destacar que hay dos tipos de opciones: locales y globales. Las locales sólo afectan al objeto que les sigue y, obligatoriamente, tienen que precederlo. En cambio las globales afectan a todos los objetos dentro de la orden draw y da igual su posición (aunque solemos escribirlas todas juntas al final).

Opciones globales

xrange, yrange, zrange: rango de las variables x , y , z . Por defecto se ajusta automáticamente al objeto que se esté representando pero hay ocasiones en que es preferible fijar un rango común.

```
(%i83) draw2d(color=blue,
             explicit(cos(x),x,0,4*%pi),
             color=red,
             explicit(x^3,x,-5,5),
             xrange=[0,%pi],yrange=[-2,2])$
```

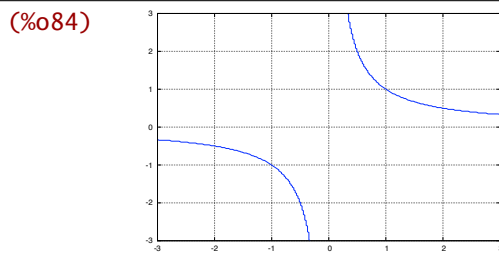
(%o83)



Si en el ejemplo anterior no limitamos el rango a representar, al menos en la coordenada y , es difícil poder ver a la vez la función coseno que toma valores entre 1 y -1 y la función x^3 que en 5 vale bastante más.

grid: dibuja una malla sobre el plano XY si vale true.

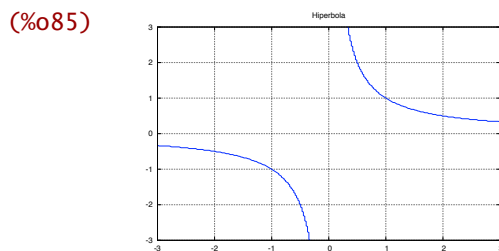
```
(%i84) draw2d(
             color=blue,nticks=100,
             implicit(x*y=1,x,-3,3,y,-3,3),
             grid=true)$
```



Acabamos de dibujar la hipérbola definida implícitamente por la ecuación $xy = 1$. La opción `grid` nos ayuda a hacernos una idea de los valores que estamos representando.

title = "título de la ventana" nos permite poner un título a la ventana donde aparece el resultado final. Es una opción global.

```
(%i85) draw2d(
      color=blue,
      nticks=100,
      implicit(x*y=1,x,-3,3,y,-3,3),
      grid=true,
      title="Hiperbola"
    )$
```



columns: indica el número de columnas que se usan cuando se dibujan varios gráficos.

proportional_axes: puede tomar los valores `xy` o `xyz`. En ese caso, la relación entre los ejes mencionados es 1:1.

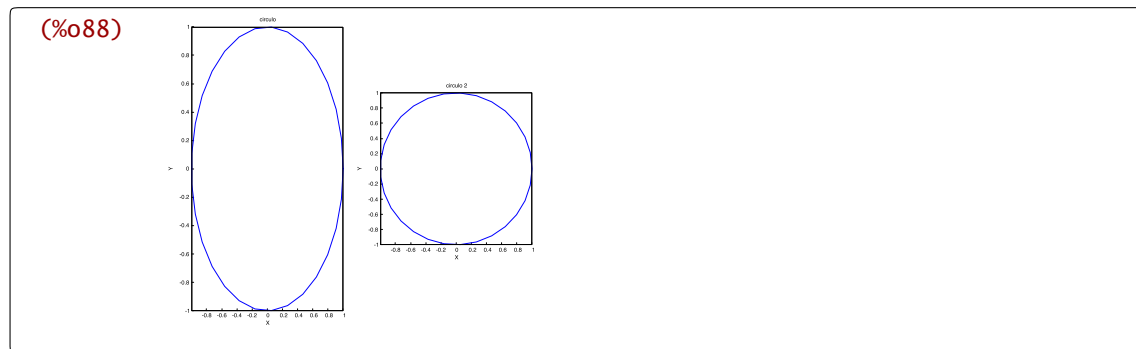
```
(%i86) escena1:gr2d(title="circulo", nticks=30,
      parametric(cos(t),sin(t),t,0,2*pi));

(%o86) (escena1)
gr2d(title="circulo",nticks=30,parametric(cos(t),sin(t),t,0,2*pi))

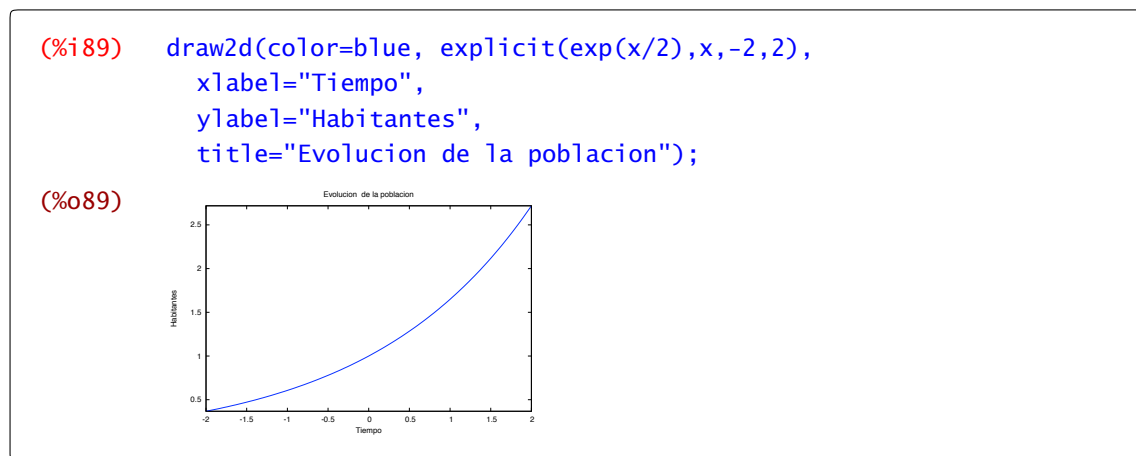
(%i87) escena2:gr2d( nticks=30, title="circulo 2", proportional_axes=xy,
      parametric(cos(t),sin(t),t,0,2*pi) );

(%o87) (escena2) gr2d(nticks=30,title="circulo 2",proportional_axes=xy,
      parametric(cos(t),sin(t),t,0,2*pi))

(%i88) draw(escena1,escena2,columns=2);
```

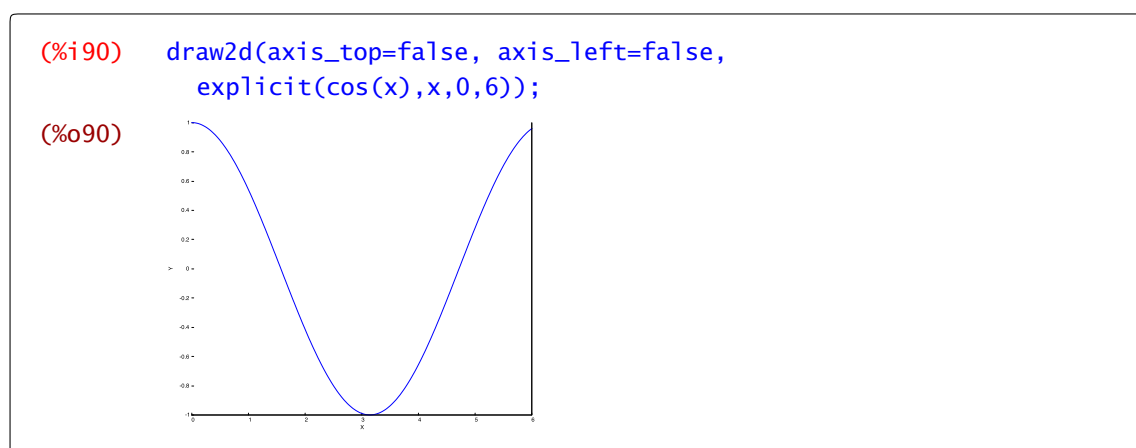


xlabel, ylabel, zlabel: indica la etiqueta de cada eje. Es una opción global.



axis, yaxis, zaxis: si vale true se dibuja el correspondiente eje. Es una opción global y su valor por defecto es true.

axis_bottom, axis_left, axis_right, axis_top: pueden tomar el valor true o false y dependiendo de esto dibuja el correspondiente lado o no. Su valor por defecto es true.

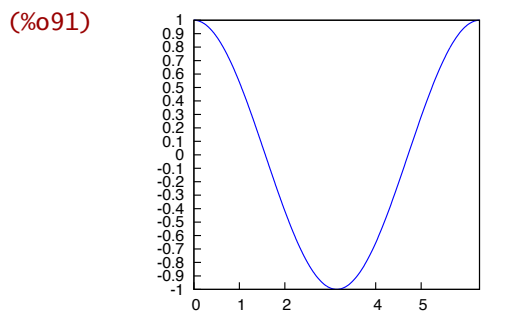


enhanced3d: si vale true se colorean las superficies en gráficos tridimensionales.

xtics, ytics, ztics: permite elegir el formato de las etiquetas que aparecen en los ejes. Hay varias posibilidades:

- a) `xticks=false` no se dibuja ninguna marca.
- b) `xticks=num` dibuja marcas cada `num`, donde `num` es un número positivo.
- c) `xticks=[1,3,5,9]` dibuja marcas únicamente en 1, 3, 5 y 9.
- d) `xticks=[["Origen",0],["uno",1]]` permite poner la etiqueta que se quiere en la marca indicada.

```
(%i91) draw2d(
        ytics=0.1, xtics={0,1,2,4,5},
        explicit(cos(x),x,0,2*pi)
    );
```



Opciones locales

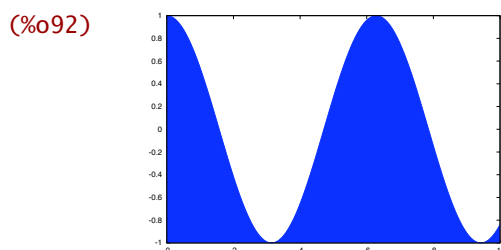
point_size: tamaño al que se dibujan los puntos. Su valor por defecto es 1. Afecta a los objetos de tipo `point`.

point_type: indica cómo se van a dibujar los puntos. El valor para esta opción puede ser un nombre o un número: `none` (-1), `dot` (0), `plus` (1), `multiply` (2), `asterisk` (3), `square` (4), `filled_square` (5), `circle` (6), `filled_circle` (7), `up_triangle` (8), `filled_up_triangle` (9), `down_triangle` (10), `filled_down_triangle` (11), `diamant` (12) y `filled_diamant` (13). Afecta a los objetos de tipo `point`.

points_joined: cuando vale `true`, los puntos se unen mediante segmentos. Sólo afecta a los objetos de tipo `point`.

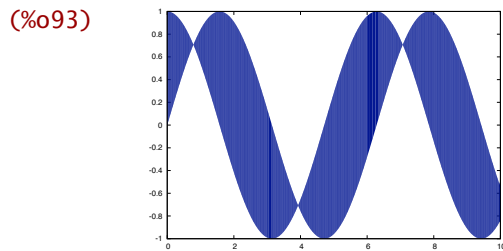
filled_func: esta orden nos permite rellenar con un color la gráfica de una función. Existen dos posibilidades: si `filled_func` vale `true` se rellena la gráfica de la función hasta la parte inferior de la ventana con el color establecido en `fill_color`

```
(%i92) draw2d(fill_color=blue,
        filled_func=true,
        explicit(cos(x),x,0,10)
    );
```



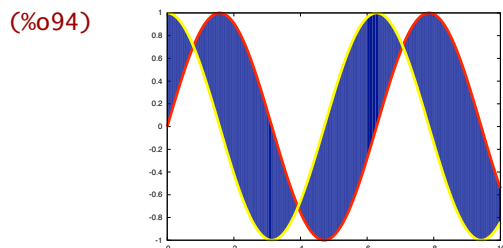
en cambio, si `filled_func` es una función, entonces se colorea el espacio entre dicha función y la gráfica que estemos dibujando

```
(%i93) draw2d(
    filled_func=sin(x),
    fill_color=navy,
    explicit(cos(x),x,0,10));
```



En este caso, tenemos sombreada el área entre las funciones seno y coseno. Podemos dibujar éstas también pero es necesario suprimir el sombreado si queremos que no tape a las funciones:

```
(%i94) draw2d(
    filled_func=sin(x),fill_color=navy,
    explicit(cos(x),x,0,10),
    filled_func=false,color=red,line_width=10,
    explicit(sin(x),x,0,10),
    color=yellow,line_width=10,
    explicit(cos(x),x,0,10));
```



fill_color: ver el apartado anterior `filled_func`.

color: especifica el color en el que se dibujan líneas, puntos y bordes de polígonos. Directamente de la ayuda de *Maxima*:

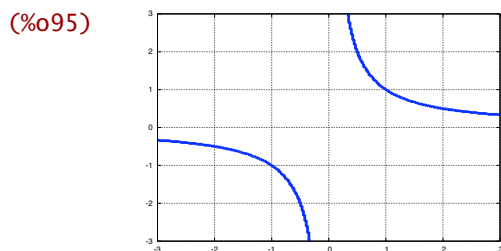
Los nombres de colores disponibles son: "white", "black", "gray0", "grey0", "gray10", "grey10", "gray20", "grey20", "gray30", "grey30", "gray40", "grey40", "gray50", "grey50", "gray60", "grey60", "gray70", "grey70", "gray80", "grey80", "gray90", "grey90", "gray100", "grey100", "gray", "grey", "light-gray", "light-grey", "dark-gray", "dark-grey", "red", "light-red", "dark-red", "yellow", "light-yellow", "dark-yellow", "green", "light-green", "dark-green",

```
"spring-green", "forest-green", "sea-green", "blue",
"light-blue", "dark-blue", "midnight-blue", "navy",
"medium-blue", "royalblue", "skyblue", "cyan",
"light-cyan", "dark-cyan", "magenta", "light-magenta",
"dark-magenta", "turquoise", "light-turquoise",
"dark-turquoise", "pink", "light-pink", "dark-pink",
"coral", "light-coral", "orange-red", "salmon",
"light-salmon", "dark-salmon", "aquamarine", "khaki",
"dark-khaki", "goldenrod", "light-goldenrod",
"dark-goldenrod", "gold", "beige", "brown", "orange",
"dark-orange", "violet", "dark-violet", "plum" y
"purple".
```

Ya lo hemos usado en casi todos los ejemplos anteriores.

line_width: grosor con el que se dibujan las líneas. Su valor por defecto es 1.

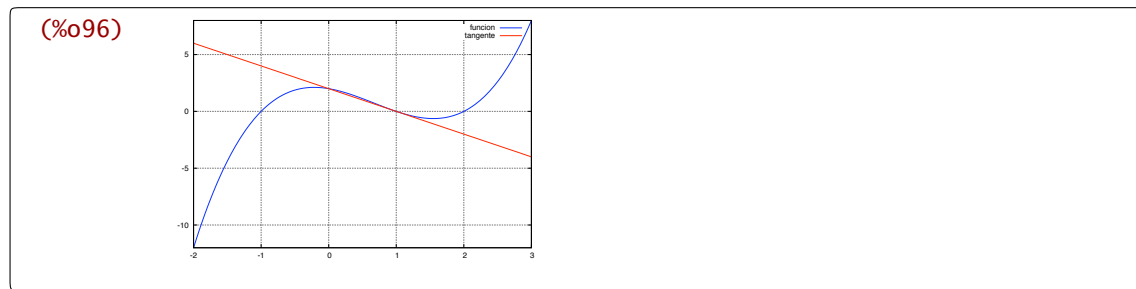
```
(%i95) draw2d(
    color=blue,line_width=10,nticks=100,
    implicit(x*y=1,x,-3,3,y,-3,3),
    grid=true,
    )$
```



nticks: número de puntos que se utilizan para calcular los dibujos. Por defecto es 30. Un número mayor aumenta el detalle del dibujo aunque a costa de un mayor tiempo de cálculo y tamaño del fichero (si se guarda). Sólo afecta a los objetos de tipo ellipse, explicit, parametric, polar y parametric.

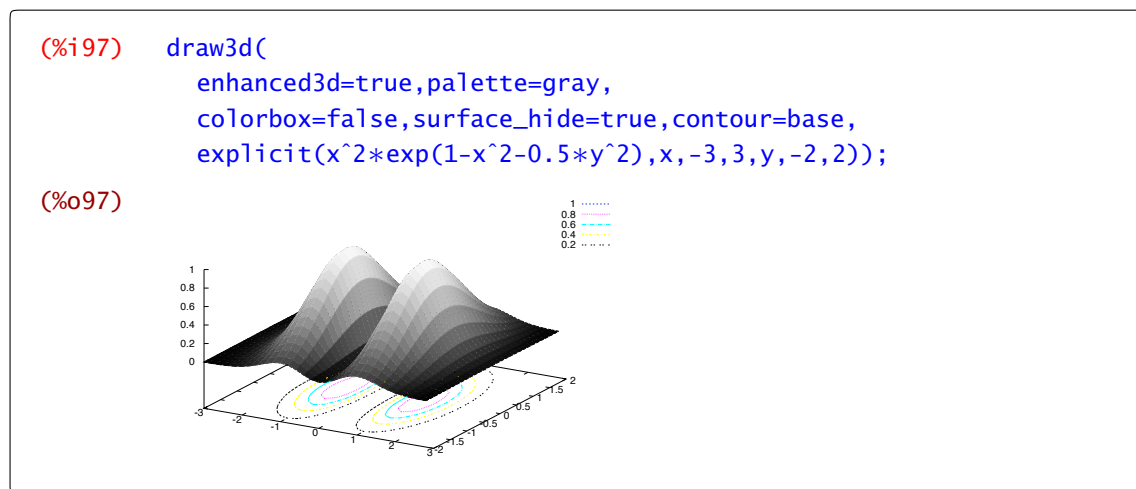
key: indica la leyenda con la que se identifica la función.

```
(%i96) draw2d(color=blue,
    key="función",
    explicit(f(x),x,-2,3),
    color=red,key="tangente",
    explicit(tangente(x,1),x,-2,3),
    grid=true);
```

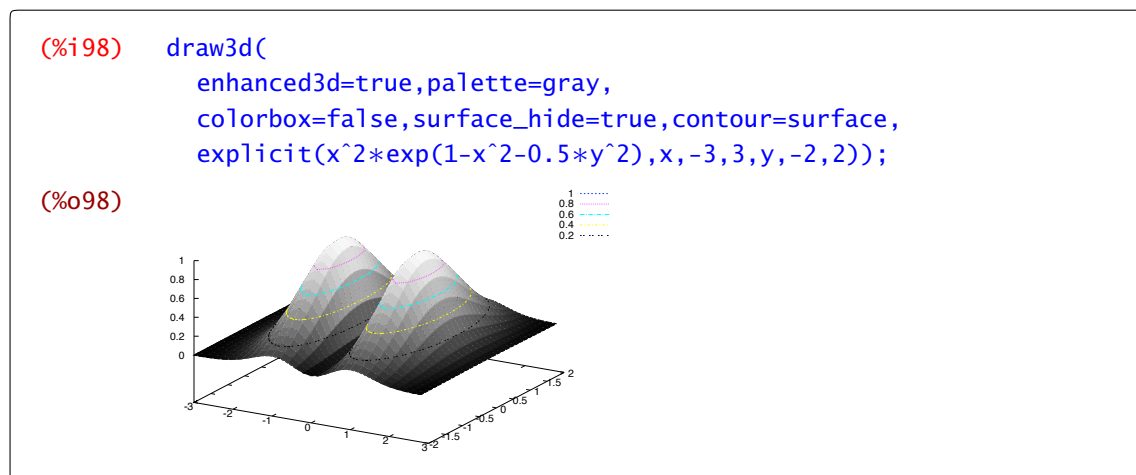


surface_hide: cuando vale `true` no se dibuja la parte no visible de las superficies.

contour nos permite dibujar o no las curvas de nivel de una superficie. Por defecto no se muestran (`none`) pero también podemos dibujarlas sobre el plano XY con `base`



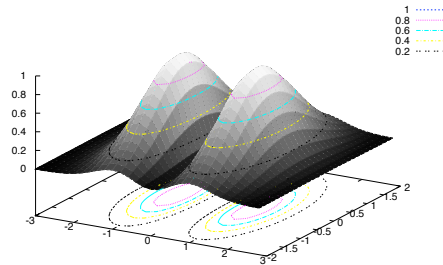
sobre la propia superficie con `surface`



las dos posibilidades anteriores se pueden usar a la vez con `both`

```
(%i99) draw3d(
    enhanced3d=true,
    palette=gray, colorbox=false,
    surface_hide=true, contour=both,
    explicit(x^2*exp(1-x^2-0.5*y^2),x,-3,3,y,-2,2));
```

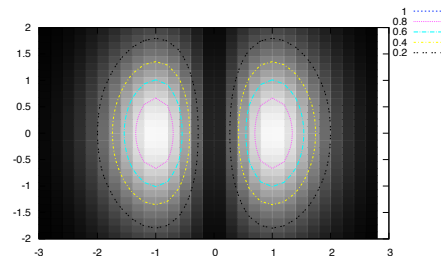
(%o99)



o, simplemente, podemos dibujar las curvas de nivel vistas desde arriba con map

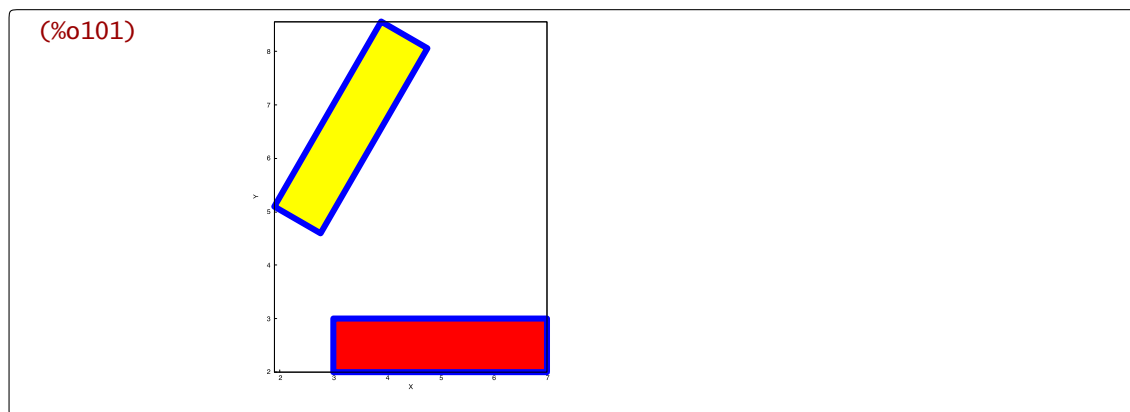
```
(%i100) draw3d(
    enhanced3d=true,
    palette=gray,
    colorbox=false,
    surface_hide=true,
    contour=map,
    explicit(x^2*exp(1-x^2-0.5*y^2),x,-3,3,y,-2,2));
```

(%o100)



transform: permite usar una transformación de las variables, en el plano o en el espacio, y “mover” el objeto que se desee. Por ejemplo, en el siguiente ejemplo rotamos 60° y trasladamos (3 unidades en el eje OX y 1 unidad en el eje OY) un rectángulo.

```
(%i101) giro:%pi/3$
draw2d(proportional_axes=xy,
    line_width=8,
    rectangle([3,2],[7,3]),
    fill_color=yellow,
    transform=[cos(giro)*x-sin(giro)*y+3,sin(giro)*x+cos(giro)*y+1,
    x,y],
    rectangle([3,2],[7,3]));
```



Hay muchas más opciones que se pueden consultar en la ayuda de *Maxima*.

2.5 Animaciones gráficas

Con *wxMaxima* es muy fácil hacer animaciones gráficas que dependen de un parámetro. Por ejemplo, la función $\sin(x + n)$ depende del parámetro n . Podemos representar su gráfica para distintos valores de n y con ello logramos una buena visualización de su evolución (que en este caso será una onda que se desplaza). Para que una animación tenga calidad es necesario que todos los gráficos individuales tengan el mismo tamaño y que no “den saltos” para lo que elegimos un intervalo del eje de ordenadas común.

Para ver la animación, cuando se hayan representado las gráficas, haz clic con el ratón sobre ella y desplaza la barra (slider) que tienes bajo el menú. De esta forma tú mismo puedes controlar el sentido de la animación, así como la velocidad.

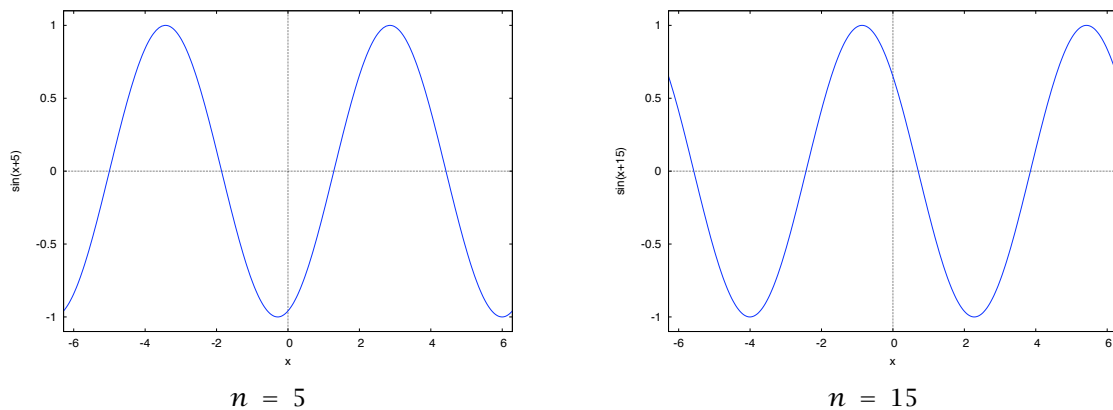
<code>with_slider</code>	animación de <code>plot2d</code>
<code>with_slider_draw</code>	animación de <code>draw2d</code>
<code>with_slider_draw3d</code>	animación de <code>draw3d</code>

Tenemos dos posibilidades para construir animaciones dependiendo de si queremos que *Maxima* utilice `plot2d` o `draw2d`. En cualquier caso, en primer lugar siempre empezamos con el parámetro, una lista de valores del parámetro y el resto debe ser algo aceptable por el correspondiente comando con el que vayamos a dibujar.

Por ejemplo, vamos a crear una animación con la orden `with_slider` de la función $\sin(x + n)$, `with_slider` donde el parámetro n va a tomar los valores desde 1 a 20. La orden `makelist(i,i,1,20)` nos da todos los números naturales comprendidos entre 1 y 20. Ya veremos con más detalle en el Capítulo 4 cómo podemos manejar listas.

```
(%i102) with_slider(n,
          makelist(i,i,1,20),
          sin(x+n),
          [x,-2*%pi,2*%pi],
          [y,-1.1,1.1]);
```

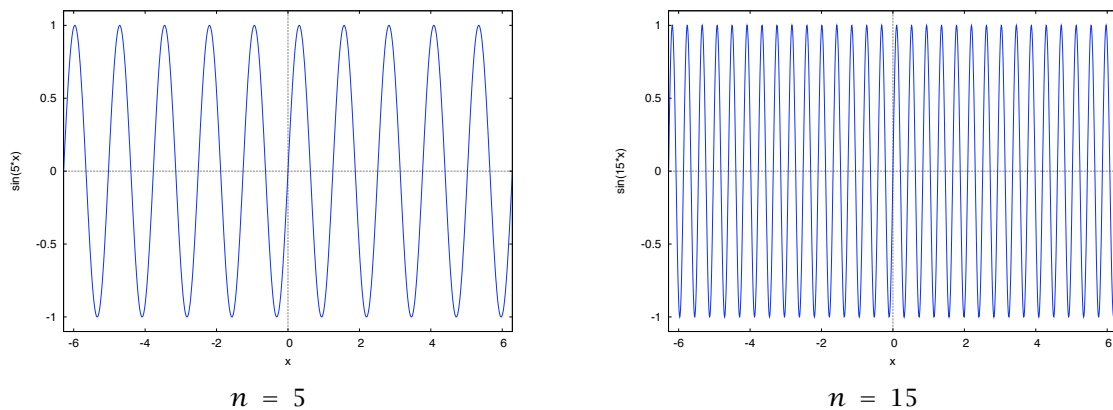
En la Figura 2.5 tienes la representación de algunos valores.

Figura 2.5 $\sin(x + n)$

Si en lugar de sumar el parámetro a la variable (que traslada la función), multiplicamos el parámetro y la variable conseguimos cambiar la frecuencia de la onda que estamos dibujando.

```
(%i103) with_slider(n,makelist(i,i,1,20),sin(x*n),
        [x,-2*%pi,2*%pi],[y,-1.1,1.1]);
```

Puedes ver en la Figura 2.6 puedes ver cómo aumenta la frecuencia con n .

Figura 2.6 $\sin(nx)$

Si en lugar de `plot2d`, utilizamos el módulo `draw` para diseñar los dibujos, tenemos que usar `with_slider_draw` o `with_slider_draw3d`. De nuevo, en primer lugar va el parámetro, después, una lista que indica los valores que tomará el parámetro y el resto debe ser algo aceptable por la orden `draw` o `draw3d`, respectivamente. Un detalle importante en este caso es que el parámetro no sólo puede afectar a la función sino que podemos utilizarlo en cualquier otra parte de la expresión. Por ejemplo, podemos utilizar esto para ir dibujando poco a poco una circunferencia en coordenadas paramétricas de la siguiente forma

```
(%i104) with_slider_draw(
          t,makelist( $\pi*i/10,i,1,20$ ),
          parametric(cos(x),sin(x),x,0,t),
          xrange=[-1,1],
          yrange=[-1,1],
          user_preamble="set size ratio 1")$
```

En la Figura 2.7 tenemos representados algunos pasos intermedios

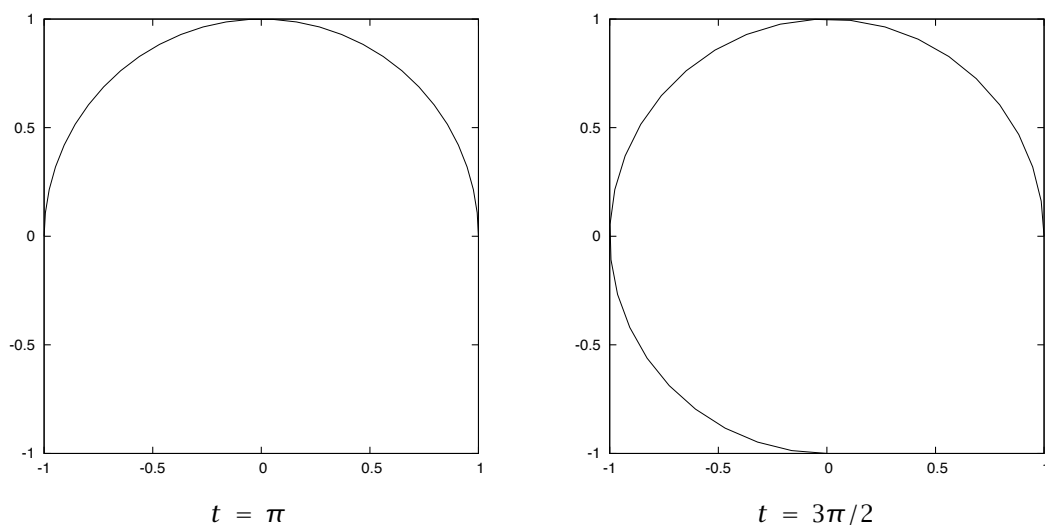


Figura 2.7 Construcción de una circunferencia en paramétricas

El tipo de objeto “parametric” no lo hemos comentado en los apartados anteriores. Nos permite representar la gráfica de una curva en el plano. En la ayuda de *Maxima* puedes encontrar más detalles.

Por último, veamos algunos ejemplos de las posibilidades en tres dimensiones. Te recuerdo que el parámetro puede aparecer en cualquier posición. Podemos utilizarlo para indicar el ángulo de rotación y conseguir “dar la vuelta” a la superficie.

```
(%i105) with_slider_draw3d(
          k,makelist(i*36,i,1,10),
          parametric_surface(cos(u)+.5*cos(u)*cos(v),
                              sin(u)+.5*sin(u)*cos(v),.5*sin(v),
                              u, -%pi, %pi, v, -%pi, %pi),
          parametric_surface(1+cos(u)+.5*cos(u)*cos(v),
                              .5*sin(v), sin(u)+.5*sin(u)*cos(v),
                              u, -%pi, %pi, v, -%pi, %pi),
          surface_hide=true,rot_horizontal=k)$
```

De nuevo, aquí representamos en la Figura 2.8 algunos pasos intermedios

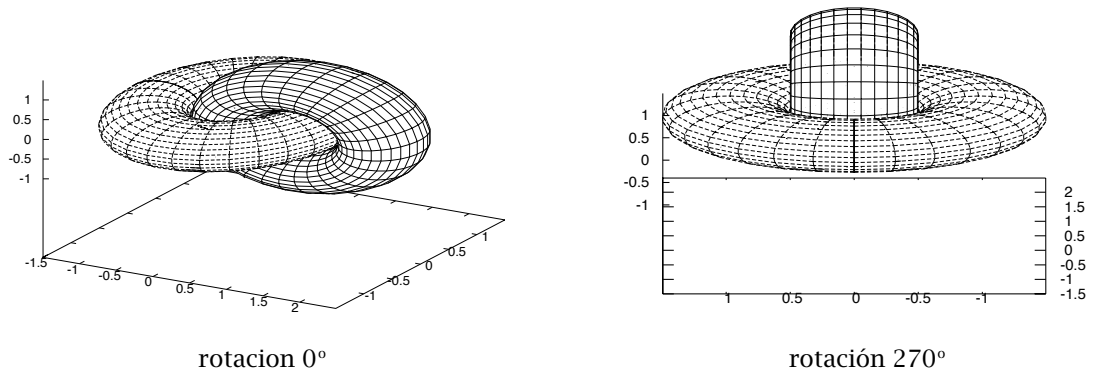


Figura 2.8 Giro alrededor de una superficie en paramétricas

También se puede utilizar de la forma “clásica” para representar una función que depende de un parámetro.

```
(%i106) with_slider_draw3d(
        k,makelist(i,i,-4,4),
        explicit((x^2-k*y^2)*exp(1-x^2-y^2),x,-2,2,y,-2,2),
        surface_hide=true
    )$
```

como puedes ver en la Figura 2.9.

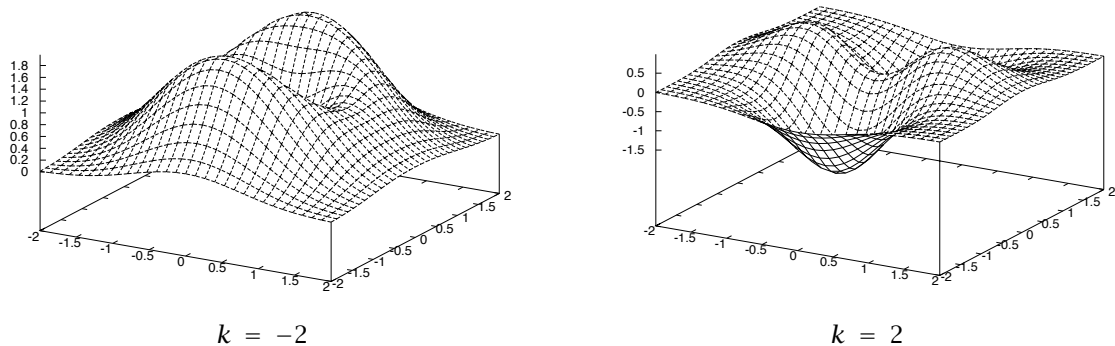


Figura 2.9 Función $f(x, y) = (x^2 - ky^2)e^{1-x^2-y^2}$

En el último ejemplo podemos ver cómo se pueden combinar funciones definidas explícita e implícitamente juntos con vectores para obtener una representación de las funciones seno y coseno.


```
(%i107) with_slider_draw(t,makelist(2*pi*i/39,i,1,40),
        line_width=3, color=blue,
        parametric(cos(x),sin(x),x,0,t),
        color=light-red, key="seno",
        explicit(sin(x),x,0,t),
        color=dark-red, key="coseno",
        explicit(cos(x),x,0,t),
        line_type=dots, head_length=0.1,
        color=dark-red, key="",
        vector([0,0],[cos(t),0]),
        color=light-red, line_type=dots,
        head_length=0.1, key="",
        vector([0,0],[0,sin(t)]),
        line_type=dots, head_length=0.1, key="",
        vector([0,0],[cos(t),sin(t)]),
        xaxis=true,yaxis=true,
        title="Funciones seno y coseno",
        xrange=[-1,2*pi],yrange=[-1,1]);
```

Para $t = 5$, el resultado lo puedes ver en la Figura 2.10

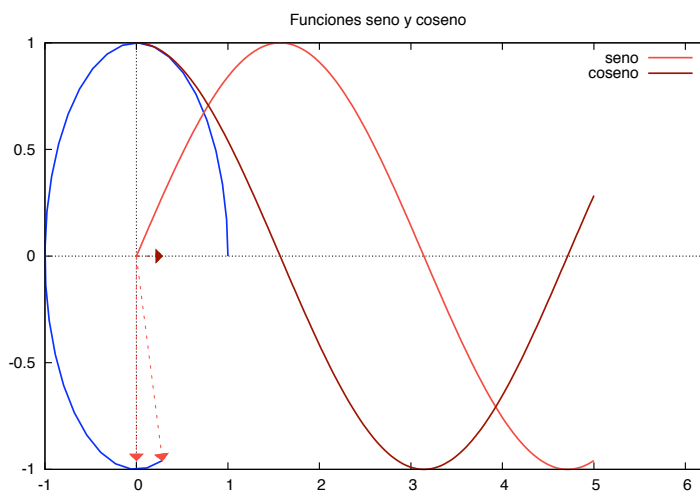


Figura 2.10 Las funciones seno y coseno

2.6 Ejercicios

Ejercicio 2.1. Representa en una misma gráfica las funciones seno y coseno en el intervalo $[-2\pi, 2\pi]$. Utiliza las opciones adecuadas para que una de las funciones se represente en azul y otra en rojo y, además, tengan grosores distintos.

Ejercicio 2.2. Compara las gráficas de las funciones $\cos(x)$ y $\cos(-x)$. ¿A qué conclusión llegas sobre la paridad o imparidad de la función coseno? Haz lo mismo con las funciones $\sin(x)$ y $\sin(-x)$.

Ejercicio 2.3. Representa las funciones logaritmo neperiano, exponencial y $f(x) = x^2$ con colores diferentes. Compara el crecimiento de estas funciones cerca de cero y lejos de cero. ¿Qué ocurre si la base de la exponencial y del logaritmo es menor que 1?

Ejercicio 2.4. Dibuja las gráficas de las funciones coseno hiperbólico, seno hiperbólico, argumento seno hiperbólico y argumento coseno hiperbólico. ¿Alguna de ellas es par o impar? ¿Son positivas?

Ejercicio 2.5. Representa la función $\cos(x)^2 - x \sin(x)^2$ en el intervalo $[-\pi, \pi]$ y sobre ella 5 puntos cuyo tamaño y color debes elegir tú. ¿Sabrías hacer lo mismo con 8 puntos elegidos aleatoriamente?⁴

Ejercicio 2.6. Representa la gráfica de la función $f: \mathbb{R}_0^+ \rightarrow \mathbb{R}$ definida como

$$f(x) = \begin{cases} e^{3x+1}, & \text{si } 0 \leq x < 10, \\ \log(x^2 + 1), & \text{si } x \geq 10. \end{cases}$$

Ejercicio 2.7. Dibuja un triángulo y colorea los vértices en rojo, verde y azul. Une con segmentos los puntos medios de cada lado del triángulo para dibujar otro triángulo.

Ejercicio 2.8. Representar las siguientes curvas y superficies dadas en forma polar o paramétrica.

a) $\gamma(\theta) = 2 + \cos(5\theta), \forall \theta \in [0, 2\pi]$.

b) $\gamma(\theta) = 8 \sin(5/2\theta), \forall \theta \in [0, 2\pi]$.

c) $\gamma(t) = (\sin(t), \sin(2t), t/5), t \in [0, 5]$,

d) $\gamma(t) = 2(\cos(3t), \sin(5t)), t \in [0, 2\pi]$.

e) $\gamma(u, v) = (\sin(u), \sin(v), v)$ con $u \in [-\pi, \pi], v \in [0, 5]$.

f) $\gamma(u, v) = (u \cos(v) \sin(u), u \cos(u) \cos(v), -u \sin(v))$, con $u, v \in [0, 2\pi]$.

Ejercicio 2.9. Representa una elipse de semiejes 2 y 4. Inscribe en ella un rectángulo y, dentro del rectángulo, una circunferencia. Dibuja la elipse en azul, el rectángulo en verde y la circunferencia en rojo.

Ejercicio 2.10. Realiza una animación gráfica para representar la curva

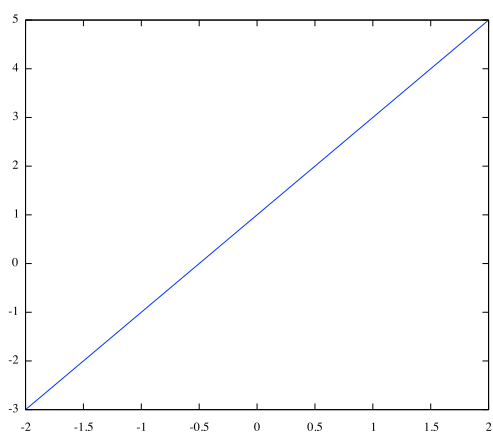
$$t \mapsto (\sin(t), \cos(4t) + 0.2)(1 - s) + s(\cos(t), \sin(t)),$$

con $t \in [0, 2\pi]$ y donde el parámetro s toma los valores de 0 a 1 con incrementos de 0.1.

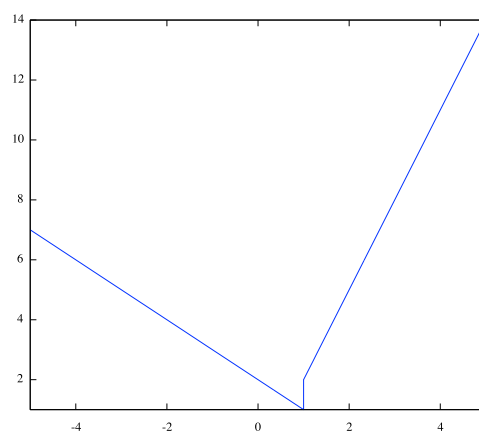
Ejercicio 2.11. Realiza una animación gráfica que represente la cicloide.

Ejercicio 2.12. Encuentra las funciones cuyas gráficas corresponden a las siguientes curvas:

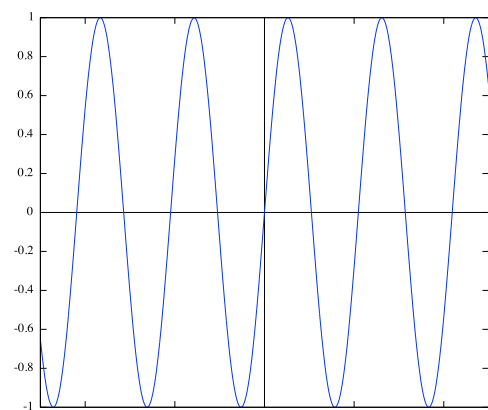
⁴ En el siguiente capítulo puedes encontrar una explicación más detallada sobre como definir y operar con listas.



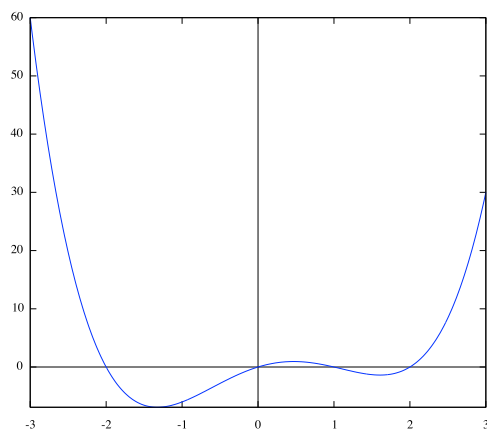
(a)



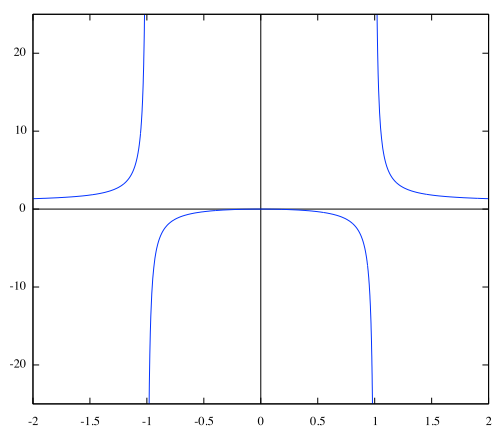
(b)



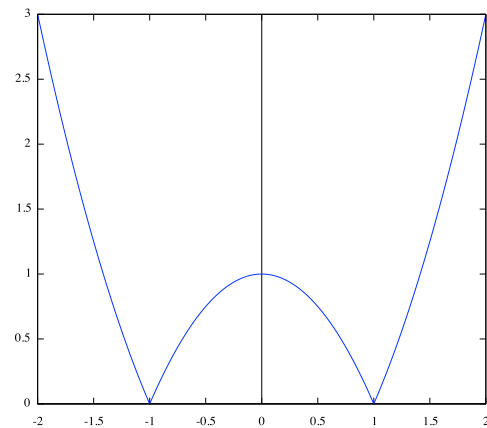
(c)



(d)



(e)



(f)

Números y errores

3

3.1 Error absoluto y error relativo	75	3.2 Números y precisión	75	3.3 Aritmética de ordenador	77
3.4 Cancelación de cifras significativas	78				

En este capítulo vamos a ver cómo encontrar soluciones aproximadas a ecuaciones que no podemos resolver de forma exacta. En la primera parte, presentamos algunos de los comandos incluidos en *Maxima* para este fin. En la segunda parte, mostramos algunos métodos para el cálculo de soluciones como el método de bisección o el de Newton-Raphson.

Comenzamos la primera sección hablando sobre las ventajas e inconvenientes de trabajar en modo numérico.

Los ordenadores tienen una capacidad limitada para almacenar cada número real por lo que en un ordenador únicamente pueden representarse un número finito de números reales. Nos referiremos a ellos como números máquina. Si un número real no coincide con uno de estos números máquina, entonces se aproxima al más próximo. En este proceso se pueden producir, y de hecho se producen, errores de redondeo al eliminar decimales. También se pueden introducir errores en la conversión entre sistema decimal y sistema binario: puede ocurrir que un número que en sistema decimal presente un número finito de dígitos, en sistema binario presente un número infinito de los mismos.

Como consecuencia de esto, algunas propiedades aritméticas dejan de ser ciertas cuando utilizamos un ordenador.

La precisión de un número máquina depende del número de bits utilizados para ser almacenados.

Puede producirse una severa reducción en la precisión si al realizar los cálculos se restan dos números similares. A este fenómeno se le conoce como cancelación de cifras significativas. Lo que haremos para evitar este fenómeno será reorganizar los cálculos en un determinado desarrollo.

3.1 Error absoluto y error relativo

Cuando decimos que π vale aproximadamente 3.14, que la distancia de Granada a Madrid son cuatrocientos kilómetros “más o menos” o que hemos pescado una trucha de 3 kilos cuando en realidad pesaba uno estamos cometiendo un error. El *error absoluto* es la diferencia entre el valor exacto y el valor aproximado. Por ejemplo, la distancia entre Granada y Madrid son 428 kilómetros y, en el caso anterior, el error absoluto sería 28 kilómetros. El *error relativo* es el cociente entre el error absoluto y el valor exacto. En el caso anterior sería 28/428. No es lo mismo equivocarnos en un metro al medir una mesa que equivocarnos en un metro al medir un campo de fútbol. El error absoluto en ambos casos es el mismo, pero el relativo es muy distinto.

3.2 Números y precisión

Cuando decimos que un número en *Maxima* tiene precisión arbitraria queremos decir que podemos calcular tantos decimales como queramos. Si es posible, *Maxima* trabaja de forma exacta

```
(%i1) sqrt(2);
```

```
(%o1)   $\sqrt{2}$ 
```

o con la precisión por defecto

```
(%i2)  sqrt(2), numer;  
(%o2)  1.414213562373095
```

Cuando decimos que $\sqrt{2}$ es un número de precisión arbitraria no queremos decir que podamos escribir su expresión decimal completa (ya sabes que es un número irracional) sino que podemos elegir el número de dígitos que deseemos y calcular su expresión decimal con esa precisión.

```
(%i3)  fpprec:20;  
(%o3)  20  
(%i4)  bfloat(sqrt(2));  
(%o4)  1.4142135623730950488b0
```

Vamos a comentar un par de detalles que tenemos que tener en cuenta en este proceso.

3.2.1 Errores de redondeo

Si sólo tenemos 5 dígitos de precisión, ¿cómo escribimos el número 7.12345? Hay dos métodos usuales: podemos truncar o podemos redondear. Por truncar se entiende desechar los dígitos sobrantes. El redondeo consiste en truncar si los últimos dígitos están entre 0 y 4 y aumentar un dígito si estamos entre 5 y 9. Por ejemplo, 7.46 se convertiría en 7.4 si truncamos y en 7.5 si redondeamos. El error es siempre menor. ¿Cuál de las dos formas usa Maxima? Puedes comprobarlo tu mismo.

```
(%i5)  fpprec:5;  
(%o5)  5  
(%i6)  bfloat(7.12345);  
(%o6)  7.1234b0
```

¿Qué pasa si aumentamos la precisión en lugar de disminuirla?

```
(%i7)  fpprec:20;  
(%o7)  20  
(%i8)  bfloat(0.1);  
(%o8)  1.0000000000000000555b-1
```

¿Qué ha pasado? 0.1 es un número exacto. ¿Porqué la respuesta no ha sido 0.1 de nuevo? Fíjate en la siguiente respuesta

```
(%i9)  bfloat(1/10);
```

```
(%o9) 1.0b-1
```

¿Cuál es la diferencia entre una otra? ¿Porqué una es exacta y la otra no? La diferencia es el error que se puede añadir (y acabamos de ver que se añade) cuando pasamos de representar un número en el sistema decimal a binario y viceversa.

3.3 Aritmética de ordenador

Sabemos que el ordenador puede trabajar con números muy grandes o muy pequeños; pero, por debajo de cierto valor, un número pequeño puede hacerse cero debido al error de redondeo. Por eso hay que tener cuidado y recordar que propiedades usuales en la aritmética real (asociatividad, elemento neutro) no son ciertas en la aritmética de ordenador.

3.3.1 Elemento neutro

Tomamos un número muy pequeño, pero distinto de cero y vamos a ver cómo *Maxima* interpreta que es cero:

```
(%i10) h:2.22045*10^(-17);
(%o10) 2.22045 10^-17
```

Y si nos cuestionamos si h funciona como elemento neutro:

```
(%i11) is(h+1.0=1.0);
(%o11) true
```

la respuesta es que sí que es cierto que $h+1.0=1.0$, luego h sería cero.

Por encima, con números muy grandes puede hacer cosas raras.

```
(%i12) g:15.0+10^(20);
(%o12) 1.10^20
(%i13) is(g-10^(20)=0);
(%o13) false
(%i14) g-10^(20);
(%o14) 0.0
```

Aquí no sale igual, pero si los restáis cree que la diferencia es cero.

3.3.2 Propiedad asociativa de la suma

Con aritmética de ordenador vamos a ver que no siempre se cumple que: $(a + b) + c = a + (b + c)$

```
(%i15) is((11.3+10^(14))+(-(10)^14)=11.3+(10^(14)+(-(10)^14)));
```



```
(%i24)  b:1+a$  
(%i25)  c:(a^2-1)/(a-1)$  
(%i26)  is(b=c);  
(%o26)  false
```

No las reconoce como iguales. Este es el resultado del efecto de cancelación de cifras significativas que tiene lugar cuando se restan dos cantidades muy parecidas. En este caso es claro cuál de ambas formas de realizar el cálculo es mejor.

Listas y matrices

4

4.1 Listas 81 4.2 Matrices 87 4.3 Ejercicios 95

4.1 Listas

Maxima tiene una manera fácil de agrupar objetos, ya sean números, funciones, cadenas de texto, etc. y poder operar con ellos. Una lista se escribe agrupando entre corchetes los objetos que queramos separados por comas. Por ejemplo,

```
(%i1) [0,1,-3];
(%o1) [0,1,-3]
```

es una lista de números. También podemos escribir listas de funciones

```
(%i2) [x,x^2,x^3];
(%o2) [x,x^2,x^3]
```

o mezclar números, variables y texto

```
(%i3) [0,1,-3,a,"hola"];
(%o3) [0,1,-3,a,hola]
```

first, second,...,tenth	primera, segunda,...,décima entrada de una lista
lista[i]	entrada <i>i</i> -ésima de la lista
last	último elemento de una lista
part	busca un elemento dando su posición en la lista
reverse	invertir lista
sort	ordenar lista
flatten	unifica las sublistas en una lista
length	longitud de la lista
unique	elementos que sólo aparecen una vez en la lista

Los elementos que forman la lista pueden ser, a su vez, listas (aunque no es exactamente lo mismo, piensa en matrices como “listas de vectores”):

```
(%i4) lista:[1,2],1,[3,a,1]
(%o4) [[1,2],1,[3,a,1]]
```

Podemos referirnos a una entrada concreta de una lista. De hecho *Maxima* tiene puesto nombre a las diez primeras: `first`, `second`, ..., `tenth`

```
(%i5) first(lista);
(%o5) [1,2]
(%i6) second(lista);
(%o6) 1
```

`last` o podemos referirnos directamente al último término.

```
(%i7) last(lista);
(%o7) [3,a,1]
```

`part` Si sabemos la posición que ocupa, podemos referirnos a un elemento de la lista utilizando `part`. Por ejemplo,

```
(%i8) part(lista,1);
(%o8) [1,2]
```

nos da el primer elemento de la lista anterior. Obtenemos el mismo resultado indicando la posición entre corchetes. Por ejemplo,

```
(%i9) lista[3];
(%o9) [3,a,1]
```

y también podemos anidar esta operación para obtener elementos de una sublista

```
(%i10) lista[3][1];
(%o10) 3
```

Con `part` podemos extraer varios elementos de la lista enumerando sus posiciones. Por ejemplo, el primer y el tercer elemento de la lista son

```
(%i11) part(lista,[1,3]);
(%o11) [[1,2],[3,a,1]]
```

o el segundo término del tercero que era a su vez una lista:

```
(%i12)  part(lista,3,2);
(%o12)  a
```

El comando `flatten` construye una única lista con todos los elementos, sean estos listas o no. `flatten`
 Mejor un ejemplo:

```
(%i13)  flatten([[1,2],1,[3,a,1]]);
(%o13)  [1,2,1,3,a,1]
```

La lista que hemos obtenido contiene todos los anteriores. Podemos eliminar los repetidos con `unique`

```
(%i14)  unique(%);
(%o14)  [1,2,3,a]
```

Vectores

En el caso de vectores, listas de números, tenemos algunas posibilidades más. Podemos sumarlos

```
(%i15)  v1:[1,0,-1];v2:[-2,1,3];
(%o15)  [1,0,-1]
(%o16)  [-2,1,3]
(%i17)  v1+v2;
(%o17)  [-1,1,2]
```

o multiplicarlos.

```
(%i18)  v1*v2;
(%o18)  [-2,0,-3]
```

Un momento, ¿cómo los hemos multiplicado? Término a término. Esto no tiene nada que ver con el producto escalar o con el producto vectorial. El producto escalar, por ejemplo, se indica con “.”

```
(%i19)  v1.v2;
(%o19)  -5
```

Podemos ordenar los elementos de la lista (del vector en este caso)

```
(%i20)  sort(v1);
(%o20)  [-1,0,1]
```

`length` o saber cuántos elementos tiene

```
(%i21) length(v1);
(%o21) 3
```

4.1.1 Construir y operar con listas

<code>makelist</code>	genera lista
<code>apply</code>	aplicar un operador a una lista
<code>map</code>	aplicar una función a una lista
<code>listp(expr)</code>	devuelve true si la expresión es una lista

Los ejemplos que hemos visto de listas hasta ahora son mezcla de números y letras de forma bastante aleatoria. En la práctica, muchas de las listas que aparecen están definidas por alguna regla. Por ejemplo, queremos dibujar las funciones $\sin(x)$, $\sin(2x)$, ..., $\sin(20x)$. Seguro que no tienes ganas de escribir la lista completa. Este es el papel de la orden `makelist`. Para escribir esa lista necesitamos la regla, la fórmula que la define, un parámetro y entre qué dos valores se mueve dicho parámetro:

```
(%i22) makelist(sin(t*x),t,1,20);
[ sin(x), sin(2 x), sin(3 x), sin(4 x), sin(5 x), sin(6 x),
(%o22) sin(7 x), sin(8 x), sin(9 x), sin(10 x), sin(11 x),
sin(12 x), sin(13 x), sin(14 x), sin(15 x),
sin(16 x), sin(17 x), sin(18 x), sin(19 x), sin(20 x)]
```

Las listas también se pueden utilizar como contadores. El caso que suele ser más útil es una lista cuyas entradas sean un rango de enteros. Por ejemplo, los primeros cincuenta naturales (empezamos en uno) son

```
(%i23) makelist(i,i,1,50);
[1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,
(%o23) 23,24,25,26,27,28,29,30,31,32,33,34,35,36,37,38,39,40,41,
42,43,44,45,46,47,48,49,50]
```

o si sólo queremos los pares:

```
(%i24) makelist(2*i,i,1,25);
[2,4,6,8,10,12,14,16,18,20,22,24,26,28,30,32,34,36,38,40,
(%o24) 42,44,46,48,50]
```

Hay algunas formas más de usar la orden `makelist`

```

makeList()      genera una lista vacía
makeList(expr,n) genera n elementos usando expr
makeList(expr,i,i0,imax,step) genera lista usando expr
                        donde i va de i0 a imax
                        con paso step
makeList(expr,i,lista) genera lista usando
                        los valores de lista

```

Si se elimina i_0 o $step$, por defecto *Maxima* usa uno. Veamos algunos ejemplos: la lista de los cuadrados de los primeros cinco naturales es

```

(%i25) makeList(i^2,i,5);
(%o25) [1,4,9,16,25]

```

que es el mismo resultado que se obtiene si hacemos

```

(%i26) makeList(i^2,i,1,5);
(%o26) [1,4,9,16,25]

```

o, incluso,

```

(%i27) makeList(i^2,i,[1,2,3,4,5]);
(%o27) [1,4,9,16,25]

```

Para generar 5 números aleatorios, enteros, 0 y 7 podemos hacer lo siguiente.

```

(%i28) makeList(random(7),5);
(%o28) [1,0,1,1,2]

```

Ya que tenemos una lista, ¿cómo podemos “jugar” con sus elementos? Por ejemplo, ¿se puede calcular el cuadrado de los 100 primeros naturales? ¿Y su media aritmética o su media geométrica? Las órdenes `map` y `apply` nos ayudan a resolver este problema. La orden `map` permite aplicar una función a cada uno de los elementos de una lista. Por ejemplo, para calcular $\sin(1)$, $\sin(2)$, ..., $\sin(10)$, hacemos lo siguiente

```

(%i29) map(sin,makeList(i,i,1,10));
(%o29) [sin(1),sin(2),sin(3),sin(4),sin(5),sin(6),sin(7),sin(8),
sin(9),sin(10)]

```

o si queremos la expresión decimal

```

(%i30) %,numer;

```

```
(%o30) [0.8414709848079,0.90929742682568,0.14112000805987,
-0.75680249530793,-0.95892427466314,-0.27941549819893,
0.65698659871879,0.98935824662338,0.41211848524176,
-0.54402111088937]
```

apply La orden `apply`, en cambio, pasa todos los valores de la lista a un operador que, evidentemente, debe saber qué hacer con la lista. Ejemplos típicos son el operador suma o multiplicación. Por ejemplo

```
(%i31) apply("+",makelist(i,i,1,100));
(%o31) 5050
```

nos da la suma de los primeros 100 naturales.

Ejemplo 4.1. Vamos a calcular la media aritmética y la media geométrica de los 100 primeros naturales. ¿Cuál será mayor? ¿Recuerdas la desigualdad entre ambas medias? La media aritmética es la suma de todos los elementos dividido por la cantidad de elementos que sumemos:

```
(%i32) apply("+",makelist(i,i,1,100))/100;
(%o32)  $\frac{101}{2}$ 
```

La media geométrica es la raíz n -ésima del producto de los n elementos:

```
(%i33) apply("*",makelist(i,i,1,100))^(1/100);
(%o33) 17 $\frac{1}{20}$  19 $\frac{1}{20}$  23 $\frac{1}{25}$  37 $\frac{1}{50}$  41 $\frac{1}{50}$  43 $\frac{1}{50}$  47 $\frac{1}{50}$  2401 $\frac{1}{25}$  15625 $\frac{1}{25}$  531441 $\frac{1}{25}$ 
638714741182055044530[30digits]99766363898994157944832 $\frac{1}{100}$ 
(%i34) float(%);
(%o34) 37.9926893448343
```

Parece que la media geométrica es menor.

Ejemplo 4.2. ¿Cuál es el módulo del vector $(1, 3, -7, 8, 1)$? Tenemos que calcular la raíz cuadrada de la suma de sus coordenadas al cuadrado:

```
(%i35) vector:[1,3,-7,8,1];
(%o35) [1,3,-7,8,1]
(%i36) sqrt(apply("+",vector^2));
(%o36)  $2\sqrt{31}$ 
```

También es posible calcular el módulo como la raíz cuadrada del producto escalar de un vector consigo mismo.

```
(%i37) sqrt(vector.vector);
```



```
(%o37) 2√3i
```

A la vista de estos ejemplos, ¿cómo podríamos definir una función que nos devuelva la media aritmética, la media geométrica de una lista o el módulo de un vector?

4.1.2 Operaciones con listas

FALTA

4.2 Matrices

Las matrices se escriben de forma parecida a las listas y, de hecho, sólo tenemos que agrupar las filas de la matriz escritas como listas bajo la orden `matrix`. Vamos a definir un par de matrices `matrix` y un par de vectores que van a servir en los ejemplos en lo que sigue.

```
(%i38) A:matrix([1,2,3],[-1,0,3],[2,1,-1]);
        B:matrix([-1,1,1],[1,0,0],[-3,7,2]);
        a:[1,2,1];
        b:[0,1,-1];
```

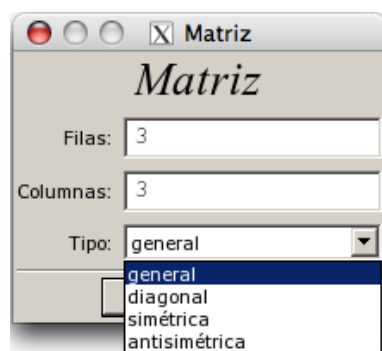
```
(%o39) [ 1  2  3
        -1  0  3
        2  1 -1]
```

```
(%o40) [-1  1  1
        1  0  0
        -3  7  2]
```

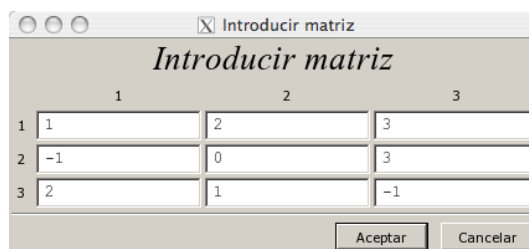
```
(%o41) [1,2,1]
```

```
(%o41) [0,-1,1]
```

En *wxMaxima* también podemos escribir una matriz usando el menú **Álgebra**→**Introducir matriz**. Nos aparece una ventana como las de la Figura 4.1 donde podemos rellenar los valores.



Seleccionar tipo de matriz



Introducir valores

Figura 4.1 Introducir matriz

<code>matrix(filal, fila2, ...)</code>	matriz
<code>matrix_size(matriz)</code>	número de filas y columnas
<code>matrixp(expresión)</code>	devuelve true si <i>expresión</i> es una matriz

matrix_size Las dimensiones de una matriz se pueden recuperar mediante la orden `matrix_size` que devuelve una lista con el número de filas y columnas.

```
(%i42) matrix_size(A);
(%o42) [3,3]
```

 **Observación 4.3.** Aunque muy similares, *Maxima* distingue entre listas y matrices. La orden `matrixp(expresión)` devuelve true o false dependiendo de si la expresión es o no una matriz. Por ejemplo, los vectores *a* y *b* que hemos definido antes, ¿son o no son matrices?

```
(%i43) matrixp(a);
(%o43) false
```

Aunque pueda parecer lo contrario, no son matrices, son listas.

```
(%i44) listp(a);
(%o44) true
```

Sólo es aceptado como matriz aquello que hallamos definido como matriz mediante la orden `matrix` o alguna de sus variantes. Al menos en *wxMaxima*, hay un pequeño truco para ver si algo es o no una matriz. ¿Cuál es la diferencia entre las dos siguientes salidas?

```
(%i45) [1,2,3];
(%o45) [1,2,3]
(%i46) matrix([1,2,3]);
(%o46) [1 2 3]
```

wxMaxima respeta algunas de las diferencias usuales entre vectores y matrices: no pone comas separando las entradas de las matrices y, además, dibuja los corchetes un poco más grandes en el caso de matrices.

4.2.1 Operaciones elementales con matrices

La suma y resta de matrices se indica de la misma forma que la suma y la resta de números, como es usual,

```
(%i47) A+B;
```

```
(%o47) 
$$\begin{bmatrix} 0 & 3 & 4 \\ 0 & 0 & 3 \\ -1 & 8 & 1 \end{bmatrix}$$

```

```
(%i48) A-B;
```

```
(%o48) 
$$\begin{bmatrix} 2 & 1 & 2 \\ -2 & 0 & 3 \\ 5 & -6 & -3 \end{bmatrix}$$

```

en cambio el producto de matrices se indica con un punto, “.”, como ya vimos con vectores. El operador * multiplica los elementos de la matriz entrada a entrada.

```
(%i49) A.B;
```

```
(%o49) 
$$\begin{bmatrix} -8 & 22 & 7 \\ -8 & 20 & 5 \\ 2 & -5 & 0 \end{bmatrix}$$

```

```
(%i50) A*B;
```

```
(%o50) 
$$\begin{bmatrix} -1 & 2 & 3 \\ -1 & 0 & 0 \\ -6 & 7 & -2 \end{bmatrix}$$

```

Con las potencias ocurre algo parecido: “^n” eleva toda la matriz a n , esto es, multiplica la matriz consigo misma n veces,

```
(%i51) A^2;
```

```
(%o51) 
$$\begin{bmatrix} 5 & 5 & 6 \\ 5 & 1 & -6 \\ -1 & 3 & 10 \end{bmatrix}$$

```

y “^n” eleva cada entrada de la matriz a n .

```
(%i52) A^2
```

```
(%o52) 
$$\begin{bmatrix} 1 & 4 & 9 \\ 1 & 0 & 9 \\ 4 & 1 & 1 \end{bmatrix}$$

```

Para el producto de una matriz por un vector sólo tenemos que tener cuidado con utilizar el punto.

```
(%i53) A.a;
```

```
(%o53) 
$$\begin{bmatrix} 8 \\ 2 \\ 3 \end{bmatrix}$$

```

y no tenemos que preocuparnos de si el vector es un vector “fila” o “columna”

```
(%i54)  a.A;
(%o54)  [ 1  3  8 ]
```

El único caso en que `*` tiene el resultado esperado es el producto de una matriz o un vector por un escalar.

```
(%i55)  2*A;
(%o55)  [ 2  4  6 ]
        [-2  0  6 ]
        [ 4  2 -2 ]
```

4.2.2 Otras operaciones usuales

<code>rank(matriz)</code>	rango de la matriz
<code>col(matriz,i)</code>	columna i de la <i>matriz</i>
<code>row(matriz,j)</code>	fila j de la <i>matriz</i>
<code>minor(matriz,i,j)</code>	menor de la matriz obtenido al eliminar la fila i y la columna j
<code>submatrix(fila1,fila2,..,matriz,col1,..)</code>	matriz obtenida al eliminar las filas y columnas mencionadas
<code>triangularize(matriz)</code>	forma triangular superior de la matriz
<code>determinant(matriz)</code>	determinante
<code>invert(matriz)</code>	matriz inversa
<code>transpose(matriz)</code>	matriz transpuesta
<code>nullspace(matriz)</code>	núcleo de la matriz

Existen órdenes para la mayoría de las operaciones comunes. Podemos calcular la matriz transpuesta con `transpose`,

```
(%i56)  transpose(A);
(%o56)  [ 1 -1  2 ]
        [ 2  0  1 ]
        [ 3  3 -1 ]
```

`determinant` calcular el determinante,

```
(%i57)  determinant(A);
(%o57)  4
```

`invert` o, ya que sabemos que el determinante no es cero, su inversa:

```
(%i58) invert(A);
```

```
(%o58)  $\begin{bmatrix} -\frac{3}{4} & \frac{5}{4} & \frac{3}{2} \\ \frac{5}{4} & -\frac{7}{4} & -\frac{3}{2} \\ -\frac{1}{4} & \frac{3}{4} & \frac{1}{2} \end{bmatrix}$ 
```

Como $\det(A) \neq 0$, la matriz A tiene rango 3. En general, podemos calcular el rango de una matriz cualquiera $n \times m$ con la orden `rank`

`rank`

```
(%i59) m:matrix([1,3,0,-1],[3,-1,0,6],[5,-3,1,1])$
```

```
(%i60) rank(m);
```

```
(%o60) 3
```

El rango es fácil de averiguar si escribimos la matriz en forma triangular superior utilizando el método de Gauss con la orden `triangularize` y le echamos un vistazo a la diagonal:

`triangularize`

```
(%i61) triangularize(m);
```

```
(%o61)  $\begin{bmatrix} 1 & 3 & 0 & -1 \\ 0 & -10 & 0 & 9 \\ 0 & 0 & -10 & 102 \end{bmatrix}$ 
```

Cualquiera de estos métodos es más rápido que ir menor a menor buscando alguno que no se anule. Por ejemplo, el menor de la matriz A que se obtiene cuando se eliminan la segunda fila y la primera columna es

`minor`

```
(%i62) minor(A,2,1);
```

```
(%o62)  $\begin{bmatrix} 2 & 3 \\ 1 & -1 \end{bmatrix}$ 
```

Caso de que no fuera suficiente con eliminar una única fila y columna podemos eliminar tantas filas y columnas como queramos con la orden `submatrix`. Esta orden elimina todas las filas que escribamos antes de una matriz y todas las columnas que escribamos después. Por ejemplo, para eliminar la primera y última columnas junto con la segunda fila de la matriz m escribimos:

`submatrix`

```
(%i63) submatrix(2,m,1,4);
```

```
(%o63)  $\begin{bmatrix} 3 & 0 \\ -3 & 1 \end{bmatrix}$ 
```

En el extremo opuesto, si sólo queremos una fila o una columna de la matriz, podemos usar el comando `col` para extraer una columna

`col`

```
(%i64) col(m,2);
```

```
(%o64)  $\begin{bmatrix} 3 \\ -1 \\ -3 \end{bmatrix}$ 
```

`row` y el comando `row` para extraer una fila. El resultado de ambas órdenes es una matriz.

```
(%i65) row(m,1);
(%o65)  $\begin{bmatrix} 1 & 3 & 0 & -1 \end{bmatrix}$ 
(%i66) matrixp(%);
(%o66) true
```

Para acabar con esta lista de operaciones, conviene mencionar cómo se calcula el núcleo de una matriz. Ya sabes que el núcleo de una matriz $A = (a_{ij})$ de orden $n \times m$ es el subespacio

$$\ker(A) = \{x; A.x = 0\}$$

`nullspace` y es muy útil, por ejemplo, en la resolución de sistemas lineales de ecuaciones. La orden `nullspace` nos da una base del núcleo de la matriz:

```
(%i67) nullspace(matrix([1,2,4],[-1,0,2]));
(%o67) span  $\left( \begin{bmatrix} -4 \\ 6 \\ -2 \end{bmatrix} \right)$ 
```

4.2.3 Más sobre escribir matrices

Si has utilizado el menú **Álgebra**→**Introducir matriz** para escribir matrices ya has visto que tienes atajos para escribir matrices diagonales, simétricas y antisimétricas.

<code>diagmatrix(n,x)</code>	matriz diagonal $n \times n$ con x en la diagonal
<code>entermatrix(m,n)</code>	definir matriz $m \times n$
<code>genmatrix</code>	genera una matriz mediante una regla
<code>matrix[i,j]</code>	elemento de la fila i , columna j de la matriz

Existen otras formas de dar una matriz en *Maxima*. La primera de ellas tiene más interés si estás utilizando *Maxima* y no *wxMaxima*. Se trata de la orden `entermatrix`. Por ejemplo, para definir una matriz con dos filas y tres columnas, utilizamos `entermatrix(2,3)` y *Maxima* nos va pidiendo que escribamos entrada a entrada de la matriz:

```
(%i68) c:entermatrix(2,3);
      Row 1 Column 1: 1;
      Row 1 Column 2: 2;
      Row 1 Column 3: 4;
      Row 2 Column 1: -1;
      Row 2 Column 2: 0;
      Row 2 Column 3: 2;
      Matrix entered.
```

```
(%o68)  $\begin{bmatrix} 1 & 2 & 4 \\ -1 & 0 & 2 \end{bmatrix}$ 
```

También es fácil de escribir la matriz diagonal que tiene un mismo valor en todas las entradas de la diagonal: sólo hay que indicar el orden y el elemento que ocupa la diagonal. Por ejemplo, la `diagmatrix` matriz identidad de orden 4 se puede escribir como sigue.

```
(%i69) diagmatrix(4,1);
```

```
(%o69)  $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$ 
```

Por último, también podemos escribir una matriz si sabemos una regla que nos diga cuál es el valor de la entrada (i, j) de la matriz. Por ejemplo, para escribir la matriz que tiene como entrada $a_{ij} = i * j$, escribimos en primer lugar dicha regla⁵

```
(%i70) a[i,j]:=i*j;
```

```
(%o70) a_ij:=i*j
```

y luego utilizamos `genmatrix` para construir la matriz (3×3 en este caso):

`genmatrix`

```
(%i71) genmatrix(a,3,3);
```

```
(%o71)  $\begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 3 & 6 & 9 \end{bmatrix}$ 
```

Observa que hemos utilizado corchetes y no paréntesis para definir la regla a_{ij} . Bueno, que ya  hemos definido la matriz `a`...un momento, ¿seguro?

```
(%i72) matrixp(a);
```

```
(%o72) false
```

⁵ Si no has borrado el vector `a` que definimos hace algunas páginas, *Maxima* te dará un error.

¿Pero no acabábamos de definirla? En realidad, no. Lo que hemos hecho es definir la regla que permite construir los elementos de la matriz pero no le hemos puesto nombre:

```
(%i73) c:genmatrix(a,4,5);
```

```
(%o73) 
$$\begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 2 & 4 & 6 & 8 & 10 \\ 3 & 6 & 9 & 12 & 15 \\ 4 & 8 & 12 & 16 & 20 \end{bmatrix}$$

```

Podemos utilizar la misma notación para referirnos a los elementos de la matriz. Por ejemplo, al elemento de la fila i y la columna j , nos referimos como $c[i,j]$ (de nuevo, observa que estamos utilizando corchetes):

```
(%i74) c[2,3];
```

```
(%o74) 6
```

4.2.4 Valores propios

<code>charpoly(matriz,variable)</code>	polinomio característico
<code>eigenvalues(matriz)</code>	valores propios de la matriz
<code>eigenvectors(matriz)</code>	valores y vectores propios de la matriz

Los valores propios de una matriz cuadrada, A , son las raíces del polinomio característico $\det(A - \text{charpoly } xI)$, siendo I la matriz identidad. La orden `charpoly` nos da dicho polinomio.

```
(%i75) S:matrix([-11/15,-2/15,-4/3],[-17/15,16/15,-1/3],
                [-8/15,4/15,5/3]);
```

```
(%o75) 
$$\begin{bmatrix} -\frac{11}{15} & -\frac{2}{15} & -\frac{4}{3} \\ -\frac{17}{15} & \frac{16}{15} & -\frac{1}{3} \\ -\frac{8}{15} & \frac{4}{15} & \frac{5}{3} \end{bmatrix}$$

```

```
(%i76) charpoly(S,x);
```

```
(%o76) 
$$\left( \left( \frac{16}{15} - x \right) \left( \frac{5}{3} - x \right) + \frac{4}{45} \right) \left( -x - \frac{11}{15} \right) + \frac{2 \left( -\frac{17}{15} \left( \frac{5}{3} - x \right) - \frac{8}{45} \right)}{15} - \frac{4 \left( \frac{8}{15} \left( \frac{16}{15} - x \right) - \frac{68}{225} \right)}{3}$$

```

```
(%i77) expand(%);
```

```
(%o77) -x^3+2x^2+x-2
```

Por tanto, sus valores propios son

```
(%i78) solve(%,x);
```

```
(%o78) [x=2,x=-1,x=1]
```

`eigenvalues` Todo este desarrollo nos lo podemos ahorrar: la orden `eigenvalues` nos da los valores propios junto con su multiplicidad.


```
(%i79)  eigenvalues(S);
(%o79)  [[2, -1, 1], [1, 1, 1]]
```

En otras palabras, los valores propios son 2, -1 y 1 todos con multiplicidad 1. Aunque no lo vamos a utilizar, también se pueden calcular los correspondientes vectores propios con la orden **eigenvectors**:

```
(%i80)  eigenvectors(S);
(%o80)  [[[2, -1, 1], [1, 1, 1]], [1, -1/2, -2], [1, 4/7, 1/7], [1, 7, -2]]
```

La respuesta es, en este caso, cinco listas. Las dos primeras las hemos visto antes: son los valores propios y sus multiplicidades. Las tres siguientes son los tres vectores propios asociados a dichos valores propios.

4.3 Ejercicios

Ejercicio 4.1. Consideremos los vectores $a = (1, 2, -1)$, $b = (0, 2, 3/4)$, $c = (e, 1, 0)$, y $d = (0, 0, 1)$. Realiza las siguientes operaciones

- a) $a + b$, c) $c.d$, y
- b) $3c + 2b$, d) $b.d + 3a.c$.

Ejercicio 4.2. Consideremos las matrices

$$A = \begin{pmatrix} 1 & -2 & 0 \\ 2 & 5 & 3 \\ -3 & 1 & -4 \end{pmatrix} \quad B = \begin{pmatrix} 0 & -2 & 6 \\ 12 & 2 & 0 \\ -1 & -1 & 3 \end{pmatrix}$$

$$C = \begin{pmatrix} 1 & 2 & 0 & -5 \\ -4 & -2 & 1 & 0 \\ 3 & 2 & -1 & 3 \\ 5 & 4 & -1 & -5 \end{pmatrix} \quad D = \begin{pmatrix} -1 & 2 & 3 & 0 \\ 12 & -5 & 0 & 3 \\ -6 & 0 & 0 & 1 \end{pmatrix}$$

- a) Calcular $A.B$, $A + B$, $D.C$.
- b) Extraer la segunda fila de A , la tercera columna de C y el elemento $(3, 3)$ de D .
- c) Calcular $\det(A)$, $\det(B)$ y $\det(C)$. Para las matrices cuyo determinante sea no nulo, calcular su inversa. Calcular sus valores propios.
- d) Calcular el rango de las matrices A , B , C , D , $D.C$ y $A + B$.
- e) Construye una matriz del orden 3×3 , de forma que el elemento (i, j) sea $i * j + j - i$. Calcula el determinante, su inversa si la tiene, y su rango. ¿Cuáles son sus valores propios?

Ejercicio 4.3. Calcula el rango de la matriz

$$A = \begin{pmatrix} 2 & 7 & -4 & 3 & 0 & 1 \\ 0 & 0 & 5 & -4 & 1 & 0 \\ 2 & 1 & 0 & -2 & 1 & 3 \\ 0 & 6 & 1 & 1 & 0 & -2 \end{pmatrix}$$

Ejercicio 4.4. Calcula los valores y vectores propios de las siguientes matrices:

$$A = \begin{pmatrix} 0 & 4 \\ 4 & -4 \end{pmatrix}, \quad B = \begin{pmatrix} 3 & 0 & 4 \\ 0 & 3 & 1 \\ 4 & 1 & -4 \end{pmatrix} \quad \text{y} \quad C = \begin{pmatrix} 0 & 3 & 9 \\ -4 & 8 & 10 \\ 8 & -4 & -2 \end{pmatrix}$$

Ejercicio 4.5.

a) Genera una lista de 10 números aleatorios entre 5 y 25 y reordénala en orden decreciente.

Ejercicio 4.6. Define `listauno:makeList(i,i,2,21)`, `listados:makeList(i,i,22,31)`. Realiza las siguientes operaciones usando algunos de los comandos antes vistos.

- a) Multiplica cada elemento de “listauno” por todos los elementos de “listados”. El resultado será una lista con 20 elementos (que a su vez serán listas de 10 elementos), a la que llamarás “productos”.
- b) Calcula la suma de cada una de las listas que forman la lista “productos” (no te equivoques, comprueba el resultado). Obtendrás una lista con 20 números.
- c) Calcula el producto de los elementos de la lista obtenida en el apartado anterior.

Ejercicio 4.7. Genera una lista de 30 elementos cuyos elementos sean listas de dos números que no sean valores exactos.

Ejercicio 4.8.

- a) Calcula la suma de los números de la forma $\frac{(-1)^{k+1}}{\sqrt{k}}$ desde $k = 1$ hasta $k = 1000$.
- b) Calcula el producto de los números de la forma $\left(1 + \frac{1}{k^2}\right)$ desde $k = 1$ hasta $k = 1000$.

Resolución de ecuaciones

Resolución de ecuaciones con *Maxima*

5

5.1 Ecuaciones y operaciones con ecuaciones 99 5.2 Resolución de ecuaciones 100 5.3 Ecuaciones algebraicas 107 5.4 El teorema de los ceros de Bolzano y `find_root` 111

Maxima nos va a ser de gran ayuda en la resolución de ecuaciones, ya sean sistemas de ecuaciones lineales con un número grande de incógnitas (y parámetros) o ecuaciones no lineales. Un ejemplo típico es encontrar las soluciones de un polinomio. En este caso es fácil que alguna de las soluciones sea compleja. No importa. *Maxima* se maneja bien con números complejos. De hecho, *siempre* trabaja con números complejos. Si tienes alguna duda de cómo operar con números complejos, en el Apéndice A tienes una breve introducción sobre su uso.

5.1 Ecuaciones y operaciones con ecuaciones

En *Maxima*, una ecuación es una igualdad entre dos expresiones algebraicas escrita con el símbolo `=`.

$expresión1=expresión2$	ecuación
<code>lhs(expresión1=expresión2)</code>	expresión1
<code>rhs(expresión1=expresión2)</code>	expresión2

Si escribimos una ecuación, *Maxima* devuelve la misma ecuación.

```
(%i1) 3*x^2+2*x+x^3-x^2=4*x^2;
(%o1) x^3+2x^2+2x=4x^2
```

además podemos asignarle un nombre para poder referirnos a ella

```
(%i2) eq: 3*x^2+2*x+x^3-a*x^2=4*x^2;
(%o2) x^3-ax^2+3x^2+2x=4x^2
```

y operar como con cualquier otra expresión

```
(%i3) eq-4*x^2;
(%o3) x^3-ax^2-x^2+2x=0
```

Podemos seleccionar la expresión a la izquierda o la derecha de la ecuación con las órdenes `lhs` y `rhs` respectivamente.

```
(%i4) lhs(eq);
```

```
(%o4) x^3-ax^2+3x^2+2x
```

5.2 Resolución de ecuaciones

Maxima puede resolver los tipos más comunes de ecuaciones y sistemas de ecuaciones algebraicas de forma exacta. Por ejemplo, sabe encontrar las raíces de polinomios de grado bajo (2,3 y 4). En cuanto a polinomios de grado más alto o ecuaciones más complicadas, no siempre será posible encontrar la solución exacta. En este caso, podemos intentar encontrar una solución aproximada en lugar de la solución exacta.

5.2.1 La orden `solve`

La primera orden que aparece en el menú dentro de *Maxima* para resolver ecuaciones es `solve`. Esta orden intenta dar todas las soluciones, ya sean reales o complejas de una ecuación o sistema de ecuaciones. Se puede acceder a esta orden desde el menú **Ecuaciones**→**Resolver** o escribiendo directamente en la ventana de *Maxima* la ecuación a resolver.

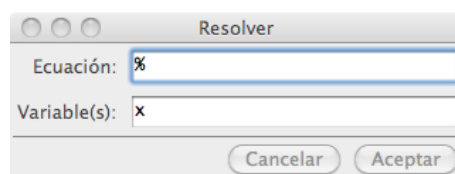


Figura 5.1 Resolver una ecuación

```
(%i5) solve(x^2-3*x+1=0,x);
```

```
(%o5) [x=-\frac{\sqrt{5}-3}{2}, x=\frac{\sqrt{5}+3}{2}]
```

<code>solve(ecuación, incógnita)</code>	resuelve la ecuación
<code>solve(expr, incógnita)</code>	resuelve la expresión igualada a cero
<code>solve([ecuaciones], [variables])</code>	resuelve el sistema
<code>multiplicities</code>	guarda la multiplicidad de las soluciones

También podemos resolver ecuaciones que dependan de algún parámetro. Consideremos la ecuación “eq1”:

```
(%i6) eq1:x^3-a*x^2-x^2+2*x=0;
```

```
(%o6) x^3-ax^2-x^2+2x=0
```

```
(%i7) solve(eq1,x);
```

```
(%o7) [x=-\frac{\sqrt{a^2+2a-7}-a-1}{2}, x=\frac{\sqrt{a^2+2a-7}+a+1}{2}, x=0]
```

Sólo en el caso de ecuaciones con una única variable podemos ahorrarnos escribirla

```
(%i8) solve(x^2+2*x=3);
```

```
(%o8) [x=-3, x=1]
```

También podemos no escribir el segundo miembro de una ecuación cuando éste sea cero

```
(%i9) solve(x^2+2*x);
(%o9) [x=-2, x=0]
(%i10) solve(x^2+2*x=0);
(%o10) [x=-2, x=0]
```

Cuando buscamos las raíces de un polinomio hay veces que es conveniente tener en cuenta la multiplicidad de las raíces. Ésta se guarda automáticamente en la variable `multiplicities`. Por ejemplo, el polinomio $x^7 - 2x^6 + 2x^5 - 2x^4 + x^3$ tiene las raíces 0, 1, i , $-i$, `multiplicities`

```
(%i11) solve(x^7-2*x^6+2*x^5-2*x^4+x^3);
(%o11) [x=-%i, x=%i, x=1, x=0]
```

pero estas raíces no pueden ser simples: es un polinomio de grado 7 y sólo tenemos 4 raíces. ¿Cuál es su multiplicidad?

```
(%i12) multiplicities;
(%o12) [1, 1, 2, 3]
```

O sea que i y $-i$ son raíces simples, 1 tiene multiplicidad 2 y 0 es una raíz triple.

Observación 5.1. Mucho cuidado con olvidar escribir cuáles son las incógnitas.



```
(%i13) solve(eq1);
More unknowns than equations - 'solve'
Unknowns given:
[a, x]
Equations given:
[x^3-ax^2+3x^2+2x=4x^2]
- an error. To debug this try debugmode(true);
```

Hay dos variables y *Maxima* no sabe con cuál de ellas quedarse como incógnita. Aunque nosotros estemos acostumbrados a utilizar las letras x , y , z como incógnitas, para *Maxima* tanto a como x tienen perfecto sentido como incógnitas y la respuesta en uno de ellos no nos interesa:

```
(%i14) solve(eq1, a);
(%o14) [a = (x^2 - x + 2) / 2]
```

La orden `solve` no sólo puede resolver ecuaciones algebraicas.

```
(%i15) solve(sin(x)*cos(x)=0, x);
solve: is using arc-trig functions to get a solution.
Some solutions will be lost.
```

```
(%o15) [x=0, x= $\frac{\pi}{2}$ ]
```

¿Qué ocurre aquí? La expresión $\sin(x) \cos(x)$ vale cero cuando el seno o el coseno se anulen. Para calcular la solución de $\sin(x) = 0$ aplicamos la función arcoseno a ambos lados de la ecuación. La función arcoseno vale cero en cero pero la función seno se anula en muchos más puntos. Nos estamos dejando todas esas soluciones y eso es lo que nos está avisando *Maxima*.

Como cualquiera puede imaginarse, *Maxima* no resuelve todo. Incluso en las ecuaciones más “sencillas”, las polinómicas, se presenta el primer problema: no hay una fórmula en términos algebraicos para obtener las raíces de un polinomio de grado 5 o más. Pero no hay que ir tan lejos. Cuando añadimos raíces, logaritmos, exponenciales, etc., la resolución de ecuaciones se complica mucho. En esas ocasiones lo más que podemos hacer es ayudar a *Maxima* a resolverlas.

```
(%i16) eq:x+3=sqrt(x+1);
(%o16) x+3=sqrt(x+1)
(%i17) solve(eq,x);
(%o17) [x= $\sqrt{x+1}-3$ ]
(%i18) solve(eq^2);
(%o18) [x= $-\frac{\sqrt{7}i+5}{2}$ , x= $\frac{\sqrt{7}i-5}{2}$ ]
```

Cómo hacer referencia a las soluciones

Uno de los ejemplos usuales en los que utilizaremos las soluciones de una ecuación es en el estudio de una función. Necesitaremos calcular puntos críticos, esto es, ceros de la derivada. El resultado de la orden `solve` no es una lista de puntos, es una lista de ecuaciones.

Una primera solución consiste en usar la orden `rhs` e ir recorriendo uno a uno las soluciones:

```
(%i19) sol:solve(x^2-4*x+3);
(%o19) [x=3, x=1]
```

La primera solución es

```
(%i20) rhs(part(sol,1));
(%o20) 3
```

y la segunda

```
(%i21) rhs(part(sol,2));
(%o21) 1
```

Este método no es práctico en cuanto tengamos un número un poco más alto de soluciones. Tenemos que encontrar una manera de aplicar la orden `rhs` a toda la lista de soluciones. Eso es justamente para lo que habíamos presentado la orden `map`:


```
(%i22) sol:map(rhs,solve(x^2-4*x+3));
(%o22) [3,1]
```

Sistemas de ecuaciones

También podemos resolver sistemas de ecuaciones. Sólo tenemos que escribir la lista de ecuaciones y de incógnitas. Por ejemplo, para resolver el sistema

$$\left. \begin{array}{l} x^2 + y^2 = 1 \\ (x - 2)^2 + (y - 1)^2 = 4 \end{array} \right\}$$

escribimos

```
(%i23) solve([x^2+y^2=1,(x-2)^2+(y-1)^2=4],[x,y]);
(%o23) [[x=4/5,y=-3/5],[x=0,y=1]]
```

Siempre hay que tener en cuenta que, por defecto, *Maxima* da todas las soluciones incluyendo las complejas aunque muchas veces no pensemos en ellas. Por ejemplo, la recta $x + y = 5$ no corta a la circunferencia $x^2 + y^2 = 1$:

```
(%i24) solve([x^2+y^2=1,x+y=5],[x,y]);
(%o24) [[x=-sqrt(23)i-5/2,y=sqrt(23)i+5/2],[x=sqrt(23)i+5/2,y=-sqrt(23)i-5/2]]
```

Si la solución depende de un parámetro o varios, *Maxima* utilizará %r1, %r2,... para referirse a estos. Por ejemplo,

```
(%i25) solve([x+y+z=3,x-y=z],[x,y,z]);
(%o25) [[x=3/2,y=-2%r1-3/2,z=%r1]]
```

¿Qué pasa si el sistema de ecuaciones no tiene solución? Veamos un ejemplo (de acuerdo, no es muy difícil)

```
(%i26) solve([x+y=0,x+y=1],[x,y]);
[]
```

¿Y si todos los valores de x cumplen la ecuación?

```
(%i27) solve((x+1)^2=x^2+2x+1,x);
(%o27) [x=x]
```

Maxima nos dice que el sistema se reduce a $x = x$ que claramente es cierto para todo x . El siguiente caso es similar. Obviamente $(x + y)^2 = x^2 + 2xy + y^2$. ¿Qué dice al respecto *Maxima*?

```
(%i28) solve((x+y)^2=x^2+2*x*y+y^2,[x,y]);
Dependent equations eliminated: (1)
(%o28) [[x=%r3,y=%r2]]
```

En otras palabras, x puede tomar cualquier valor e y lo mismo.

Cómo hacer referencia a las soluciones (II)

Si resolvemos un sistema de ecuaciones, la lista de soluciones está formada a su vez por listas.

```
(%i29) lista:solve([x*y=1,x-y=1],[x,y]);
(%o29) [[x=-\frac{\sqrt{5}-1}{2},y=-\frac{\sqrt{5}+1}{2}], [x=\frac{\sqrt{5}+1}{2},y=\frac{\sqrt{5}-1}{2}]]
```

Usar `map` no da el resultado esperado:

```
(%i30) map(rhs,lista);
(%o30) [0,0]
```

pero podemos usar la orden `makelist` para recorrer las sublistas y obtener lo deseado

```
(%i31) makelist(map(rhs,i),i,lista);
(%o31) [[-\frac{\sqrt{5}-1}{2},-\frac{\sqrt{5}+1}{2}], [\frac{\sqrt{5}+1}{2},\frac{\sqrt{5}-1}{2}]]
```

Operaciones con ecuaciones (II)

Podemos operar con expresiones o ecuaciones y esto puede ayudarnos a resolver ecuaciones. Por ejemplo, la orden `solve` puede tener dificultades para resolver una ecuación en la que aparecen raíces o valores absolutos. Veamos un ejemplo.

```
(%i32) eq:abs(x-1)=3*x+7;
(%o32) x-1=3x+7
(%i33) solve(eq,x);
(%o33) [x=\frac{|x-1|-7}{3}]
```

Elevando al cuadrado, eliminamos el valor absoluto: $\left| (x - 1)^2 \right| = (x - 1)^2$, con lo que

```
(%i34) solve(eq^2,x);
```

```
(%o34) [x=-3/2,x=-4]
```

Pero, ¿hemos añadido alguna solución? Es muy fácil dar un ejemplo de una ecuación que tiene más soluciones si la elevamos al cuadrado: $x = 2$ cuando la elevamos al cuadrado se transforma en $x^2 = 4$. ¿Estamos en la misma situación? Sería conveniente comprobar que las dos soluciones obtenidas son soluciones de la ecuación original.

```
subst[a, b, c]  Sustituye a por b en c
```

La orden `subst` permite dar un valor a la incógnita: el primer candidato a solución era -1.5 .

```
(%i35) subst(x,-1.5,eq);
```

```
(%o35) 2.5=2.5
```

Se cumple la ecuación y, por tanto, es solución. La segunda posible solución era -4 .

```
(%i36) subst(x,-4,eq);
```

```
(%o36) 5=-5
```

Como se ve, no es una solución de ecuación original.

5.2.2 La orden `to_poly_solve`

Hay una segunda forma de resolver ecuaciones en *Maxima*. Puedes acceder a ella desde el menú **Ecuaciones** → **Resolver (to_poly)**. Sin entrar en detalles, algunas ecuaciones las resuelve mejor. Por ejemplo, cuando hay radicales por medio la orden `solve` no siempre funciona bien:

```
(%i37) solve(3*x=sqrt(x^2+1),x);
```

```
(%o37) [x =  $\frac{\sqrt{x^2+1}}{3}$ ]
```

En cambio, tiene un poco más de éxito

```
(%i38) to_poly_solve(3*x=sqrt(x^2+1),x);
```

```
(%o38) %union([x =  $\frac{1}{2^{(3/2)}}$ ])
```

Como puedes ver, la respuesta es el *conjunto* de soluciones que verifica la ecuación. De ahí la palabra `union` delante de la respuesta.

```
to_poly_solve(ecuación, variable)  resuelve la ecuación
```

```
to_poly_solve(expr, variable)  resuelve la expresión igualada a cero
```

Además de este ejemplo, hay otras ocasiones en las que la respuesta de `to_poly_solve` es mejor o más completa. Por ejemplo, con funciones trigonométricas ya hemos visto que `solve` no da la lista completa de soluciones

```
(%i39) solve(x*cos(x));
solve: using arc-trig functions to get a solution.
Some solutions will be lost.

(%o39) [x=0]
```

En cambio, con `to_poly_solve` la respuesta es un poco más amplia

```
(%i40) to_poly_solve(x*cos(x),x);
(%o40) %union([x=0],[x=2π%z101 - π/2],[x=2π%z103 + π/2])
```

Los parámetros “z101” y “z103” indican un número entero arbitrario y la numeración depende del número de operaciones que hayas realizado. No es quizá la forma más elemental de escribir la solución, pero sí que tenemos todas las soluciones de la ecuación. Observa también que en este caso no hemos escrito una ecuación sino una expresión y `to_poly_solve` ha resuelto dicha expresión igualada a cero lo mismo que ocurriría con la orden `solve`.

5.2.3 Inecuaciones

El paquete “fourier_elim” permite resolver o, al menos, escribir de forma más sencilla ecuaciones y/o inecuaciones.

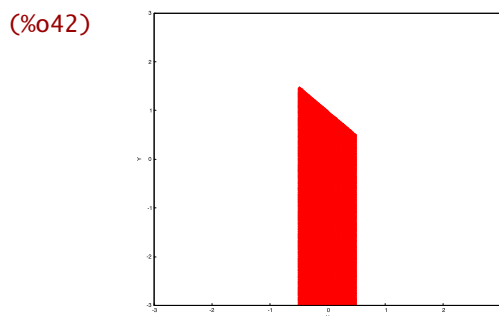
```
fourier_elim([eq1,eq2,...],[var1,var2,...]) simplifica las desigualdades
```

La orden `fourier_elim` tiene como entradas una lista de ecuaciones o inecuaciones y una lista de variables.

```
(%i41) fourier_elim([x+y<1,abs(x)<1/2],[x,y]);
(%o41) [-1/2<x,x<min(1/2,1-y),y<3/2]
```

Podemos dibujar dicho conjunto para hacernos una idea de su aspecto.

```
(%i42) draw2d(
      x_voxel=50, y_voxel=50,
      region((-1/2<x and x<min(1/2,1-y) and y<3/2),x,-3,3,y,-3,3)
);
```



En clase hemos visto ejemplos como el siguiente

```
(%i43) fourier_elim(x*(x+1)*(x+2)<0,[x]);
(%o43) [-1<x,x<0] ∨ [x<-2]
```

esto es, la solución es el conjunto $] - \infty, -2[\cup] - 1, 0[$.

5.3 Ecuaciones algebraicas

5.3.1 Ecuaciones polinómicas

Las ecuaciones polinómicas se pueden resolver de manera aproximada. Los comandos `allroots` y `realroots` están especializados en encontrar soluciones racionales aproximadas de polinomios en una variable.

<code>allroots(<i>polinomio</i>)</code>	soluciones aproximadas del polinomio
<code>bfallroots(<i>polinomio</i>)</code>	soluciones aproximadas del polinomio con precisión arbitraria
<code>realroots(<i>polinomio</i>)</code>	soluciones aproximadas reales del polinomio
<code>realroots(<i>polinomio</i>, <i>error</i>)</code>	soluciones aproximadas reales del polinomio con cota del error
<code>nroots(<i>polinomio</i>, <i>a</i>, <i>b</i>)</code>	número de soluciones reales del polinomio entre <i>a</i> y <i>b</i>
<code>algsys([<i>ecuaciones</i>], [<i>variables</i>])</code>	resuelve la ecuación o ecuaciones

Estos órdenes nos dan todas las soluciones reales y complejas de un polinomio en una variable y son útiles en polinomios de grado alto cuando falla la orden `solve`. La primera de ellas, `allroots`, `allroots` nos da las soluciones con la precisión por defecto

```
(%i44) eq:x^9+x^7-x^4+x$
(%i45) allroots(eq);
[x=0.0,x=0.30190507748312%i+0.8440677798278,
x=0.8440677798278-0.30190507748312%i,
x=0.8923132916888%i-0.32846441923834,
x=-0.8923132916888%i-0.32846441923834,
(%o45) x=0.51104079208431%i-0.80986929589487,
x=-0.51104079208431%i-0.80986929589487,
x=1.189238256723466%i+0.29426593530541,
x=0.29426593530541-1.189238256723466%i]
```

Si queremos una precisión determinada, usamos la orden `bfallroots`.

bfallroots

```
(%i46) fpprec:6$
bfallroots(eq);
```

```
(%o46) [x=0.0b0,x=3.0191b-1%i+8.44063b-1,
x=8.44063b-1-3.0191b-1%i,x=8.92279b-1%i-3.28481b-1,
x=-8.92279b-1%i-3.28481b-1,x=5.11037b-1%i-8.09838b-1,
x=-5.11037b-1%i-8.09838b-1,x=1.18924b0%i+2.94256b-1,
x=2.94256b-1-1.18924b0%i]
```

realroots Si sólo nos interesan las soluciones reales, la orden `realroots` calcula soluciones racionales aproximadas del polinomio.

```
(%i47) eq1:x^4-3*x^3+x^2-4*x+12$
(%i48) realroots(eq1);
(%o48) [x=2,x= $\frac{81497599}{33554432}$ ]
```

Si comparas con la salida de `allroots`, comprobarás que 2 es solución, pero que $\frac{81497599}{33554432}$ sólo es una solución aproximada. La precisión con la que se realiza la aproximación se puede controlar con un segundo parámetro. Por ejemplo, si queremos que el error sea menor que 10^{-5} , escribimos lo siguiente.

```
(%i49) realroots(eq1,10^(-5));
(%o49) [x=2,x= $\frac{636699}{262144}$ ]
```

Recuerda que la variable `multiplicities` guarda la multiplicidad de cada una de las raíces de la última ecuación que has resuelto.

```
(%i50) realroots((x-2)^2*eq1,10^(-5));
(%o50) [x=2,x= $\frac{636699}{262144}$ ]
(%i51) multiplicities;
(%o51) [3,1]
```

Por último, comentar que es posible saber el número de raíces de un polinomio en una variable en un intervalo concreto. Incluso se admiten $\pm\infty$ como posibles extremos del intervalo.

```
(%i52) nroots(eq1,0,2);
(%o52) 1
```

eso sí, ten cuidado porque se cuentan raíces con su multiplicidad

```
(%i53) nroots((x-2)^2*eq1,0,2);
(%o53) 3
```

5.3.2 La orden `algsys`

<code>algsys([ecuaciones],[variables])</code>	resuelve la ecuación o ecuaciones
<code>realonly</code>	si vale <code>true</code> , <code>algsys</code> muestra sólo soluciones reales

La orden `algsys` resuelve ecuaciones o sistemas de ecuaciones algebraicas. La primera diferencia con la orden `solve` es pequeña: `algsys` siempre tiene como entrada listas, en otras palabras, tenemos que agrupar la ecuación o ecuaciones entre corchetes igual que las incógnitas.

```
(%i54) eq:x^2-4*x+3;
(%o54) x^2-4*x+3
(%i55) algsys([eq],[x]);
(%o55) [[x=3],[x=1]]
```

La segunda diferencia es que `algsys` intenta resolver numéricamente la ecuación si no es capaz de encontrar la solución exacta.

```
(%i56) solve(eq:x^6+x+1);
(%o56) [0=x^6+x+1]
(%i57) algsys([eq],[x]);
(%o57) [[x=-1.038380754458461%i-0.15473514449684],
[x=1.038380754458461%i-0.15473514449684],
[x=-0.30050692030955%i-0.79066718881442],
[x=0.30050692030955%i-0.79066718881442],
[x=0.94540233331126-0.61183669378101%i],
[x=0.61183669378101%i+0.94540233331126]]
```

En general, para ecuaciones polinómicas `algsys` nos permite algo más de flexibilidad ya que funciona bien con polinomios de grado alto y, además, permite seleccionar las raíces reales. El comportamiento de `algsys` está determinado por la variable `realonly`. Su valor por defecto es `false`. Esto significa que `algsys` muestra todas las raíces. Si su valor es `true` sólo muestra las raíces reales.

```
(%i58) eq:x^4-1=0$
(%i59) realonly;
(%o59) false
(%i60) algsys([eq],[x]);
(%o60) [[x=1],[x=-1],[x=%i],[x=-%i]]
(%i61) realonly:true$
(%i62) algsys([eq],[x]);
(%o62) [[x=1],[x=-1]]
```

5.3.3 Ejercicios

Ejercicio 5.1. Calcula los puntos donde se cortan las parábolas $y = x^2$, $y = 2x^2 + ax + b$. Discute todos los casos posibles dependiendo de los valores de a y b .

Ejercicio 5.2. Dibuja, en un mismo gráfico, la elipse de semieje horizontal $a = 3$ y de semieje vertical $b = 5$ y la bisectriz del primer cuadrante. Calcula los puntos donde se cortan ambas curvas.

Ejercicio 5.3. Considera el triángulo de vértices $A = (12, 21)$, $B = (321, 457)$ y $C = (723, -289)$. Si D es el punto medio del segmento $\overline{AB}$, encuentra el punto E de forma que el segmento $\overline{DE}$ divide al triángulo $\triangle ABC$ en dos trozos con igual área.

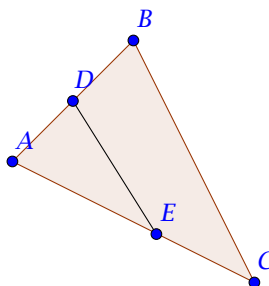


Figura 5.2

E **Ejercicio 5.4.**

- Comprueba que los puntos $A = (-\sqrt{7}, -1)$ y $B = (\sqrt{7}, 1)$ pertenecen a las gráficas de las dos curvas siguientes: la circunferencia de centro $(0, 0)$ y radio $\sqrt{8}$ y la hipérbola de ecuación $x^2 - y^2 = 6$.
- Dibuja, en un mismo gráfico, las dos curvas anteriores, así como los puntos A y B y el centro de la circunferencia dada. Utiliza opciones de color y grosor de línea para diferenciar los distintos objetos representados.
- Añade al gráfico anterior la recta que une los puntos representados.

E **Ejercicio 5.5.** Dibuja la elipse centrada en el punto $(1, 1)$ de semiejes 4 y 2 y los rectángulos de área $\sqrt{20}$ inscritos en ella.

Ejercicio 5.6. Consideremos la circunferencia de centro $(0, 0)$ y radio 2. Dibújala. Ahora consideremos un rectángulo centrado en el origen e inscrito en ella. Determina el rectángulo así construido cuya área sea 1.

Ejercicio 5.7. Representa gráficamente y determina los puntos de corte de las siguientes curvas:

- la recta $x - y = 5$ y la parábola $(x - 1)^2 + y = 4$;
- la hipérbola equilátera y la circunferencia de centro $(-1, 1)$ y radio 1;
- las circunferencias de centro $(0, 0)$ y radio 2 y la de centro $(-1, 3)$ y radio 3;
- la parábola $y = x^2 + 3x$ y la hipérbola $x^2 - y^2 = 7$.

Ejercicio 5.8. Resuelve la ecuación logarítmica $\log(x) + \log(x + 1) = 3$.

5.4 El teorema de los ceros de Bolzano y *find_root*

Uno de los primeros resultados que aprendemos sobre funciones continuas es que si cambian de signo en un intervalo tienen que valer cero en algún momento. Este resultado se conoce como teorema de los ceros de Bolzano y es una variante del teorema del valor intermedio.

Teorema 5.2. Sea $f: [a, b] \rightarrow \mathbb{R}$ una función continua verificando que $f(a)f(b) < 0$, entonces existe $c \in]a, b[$ tal que $f(c) = 0$.

Teorema de los ceros de Bolzano

<code>find_root(f(x), x, a, b)</code>	solución de f en $[a, b]$
<code>bf_find_root(f(x), x, a, b)</code>	solución de f en $[a, b]$ usando números con precisión grande

El comando *find_root* encuentra una solución de una función continua que cambia de signo o de una ecuación. El método que usa *Maxima* es algo una versión más elaborada del método de bisección.

find_root

Ejemplo 5.3. Una de las utilidades más importantes del Teorema de los ceros de Bolzano es garantizar que una ecuación tiene solución. Por ejemplo, para comprobar que la ecuación $e^x + \log(x) = 0$ tiene solución, estudiamos la función $f(x) = e^x + \log(x)$: es continua en $\mathbb{R}^+$ y se puede comprobar que $f(e^{-10}) < 0$ y $0 < f(e^{10})$. Por tanto, la ecuación $e^x + \log(x) = 0$ tiene al menos una solución entre e^{-10} y e^{10} . En particular, tiene solución en $\mathbb{R}^+$.

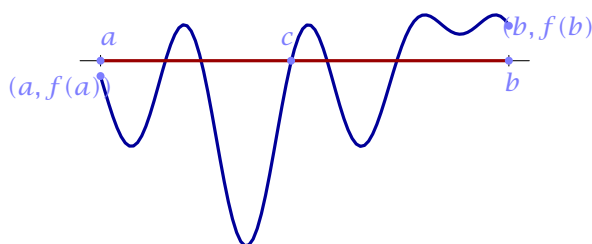


Figura 5.3 Teorema de los ceros de Bolzano

Si queremos encontrar la solución con *Maxima*, calculamos un par de puntos donde la función cambie de signo, cuanto más cercanos en principio mejor, ya que tendremos más control sobre donde se encuentra el cero de la función.

```
(%i63) f(x):=exp(x)+log(x);
(%o63) f(x):=exp(x)+log(x)
(%i64) f(1);
(%o64) %e
(%i65) f(exp(-3));
(%o65) %e%e-3+3
(%i66) f(exp(-3)),numer;
(%o66) -1.948952728663784
```

Vale, ya que tenemos dos puntos donde cambia de signo podemos usar *find_root*:

```
(%i67) find_root(f(x), x, exp(-3), 1);
```

```
(%o67) 0.26987413757345
```

Si necesitamos mayor precisión, podemos usar números con precisión grande. Recuerda que el valor de la variable `fpprec` establece la precisión que *Maxima* va a usar al trabajar con números de precisión grande.

```
(%i68) fpprec:20;
(%o68) 20
(%i69) bf_find_root(f(x),x,exp(-3),1);
(%o69) 2.6987413757344922388b-1
```



Observación 5.4.

Este método encuentra *una* solución pero no nos dice cuántas soluciones hay. Para eso tendremos que echar mano de otras herramientas adicionales como, por ejemplo, el estudio de la monotonía de la función.

5.4.1 Ejercicios

Ejercicio 5.9. Calcula las soluciones de $8 \sin(x) + 1 - \frac{x^2}{3} = 0$.

Ejercicio 5.10. Encuentra una solución de la ecuación $\tan(x) = \frac{1}{x}$ en el intervalo $]0, \frac{\pi}{2}[$.

Ejercicio 5.11. ¿Cuántas soluciones tiene la ecuación $\frac{e^x}{2} - 2 \sin(x) = 1$ en el intervalo $[-3, 3]$?

Ejercicio 5.12. Calcula las soluciones de las siguientes ecuaciones:

a) $\log(x) - \arctan(x^2) + x^3 - x^4 + 3 = 0$.

b) $\exp(-x^2) + 3x(\sin(x)^2 + 1) - 5 = 0$.

Breves conceptos de programación

6

6.1 Funciones y bloques 113 6.2 Bucles 118 6.3 Condicionales 121 6.4 Ejercicios 122

Hemos visto cómo resolver ecuaciones y sistemas de ecuaciones con *Maxima* mediante la orden `solve` o `algsys`. La resolución de ecuaciones y sistemas de ecuaciones de manera exacta está limitada a aquellas para las que es posible aplicar un método algebraico sencillo. En estas condiciones, nos damos cuenta de la necesidad de encontrar o aproximar soluciones para ecuaciones del tipo $f(x) = 0$, donde, en principio, podemos considerar como f cualquier función real de una variable. Nuestro siguiente objetivo es aprender a “programar” algoritmos con *Maxima* para aproximar la solución de estas ecuaciones.

Lo primero que tenemos que tener en cuenta es que *no existe ningún método general* para resolver todo este tipo de ecuaciones en un número finito de pasos. Lo que sí tendremos son condiciones para poder asegurar que un determinado valor es una aproximación de la solución de la ecuación con un error prefijado.

El principal resultado para asegurar la existencia de solución para la ecuación $f(x) = 0$ en un intervalo $[a, b]$, es el Teorema de los ceros de Bolzano que hemos recordado antes. Dicho teorema asegura que si f es una función continua en un intervalo $[a, b]$ y cambia de signo dicho intervalo, entonces existe al menos un punto donde la función se anula.

Vamos a ver varios métodos que se basan en este resultado. Dichos métodos nos proporcionan un algoritmo para calcular una sucesión de aproximaciones, y condiciones sobre la función f para poder asegurar que la sucesión que obtenemos converge a la solución del problema. Una vez asegurada esta convergencia, bastará tomar alguno de los términos de la sucesión que se aproxime a la sucesión con la exactitud que deseemos.

6.1 Funciones y bloques

La forma más elemental de “programa” en *Maxima* es la siguiente: entre paréntesis y separados por comas se incluyen comandos que se ejecutan sucesivamente y se devuelve como salida la respuesta de la última sentencia.

```
(%i1) (a:3,b:2,a+b);
(%o1) 5
```

6.1.1 Bloques

<code>(expr1, expr2, ...)</code>	evalúa $expr1, expr2, \dots$ y devuelve la última expresión evaluada
<code>block([var1, var2, ...], expr1, expr2, ...)</code>	evalúa $expr1, expr2, \dots$ y devuelve la última expresión evaluada
<code>local(funciones)</code>	declara funciones locales a un bloque

Es conveniente que las variables que se utilicen dentro de un programa que no sean necesarias fuera de él, no exporten sus valores fuera de dicho programa. En otras palabras, que si es posible las variables y, por el mismo motivo, las funciones sean locales a dicho programa y que no afecten al resto de la sesión. Esto se consigue agrupando estas órdenes en un bloque. A la vez que se declara una variable como local se le puede asignar su valor.

```
(%i2)  a:1;
(%o2)  1
(%i3)  block([a:2,b],b:3,a+b);
(%o3)  5
(%i4)  a;
(%o4)  1
```

Como puedes ver, la variable a global sigue valiendo uno y no cambia su valor a pesar de las asignaciones dentro del bloque. Esto no ocurre con las funciones que definamos dentro de un `local` bloque. Su valor es global a menos que lo declaremos local explícitamente con la sentencia `local`. Observa la diferencia entre las funciones f y g .

```
(%i5)  block([a,b],
            local(g),g(x):=x^3,
            f(x):=x^2,
            a:2,b:3,g(a+b));
(%o5)  125
```

Si preguntamos por el valor de f o de g fuera del bloque, f tiene un valor concreto y g no:

```
(%i6)  f(x);
(%o6)  x^2
(%i7)  g(x);
(%o7)  g(x)
```

Los bloques nos permiten definir funciones un poco más complejas. Por ejemplo, el factorial de un número natural n se define recursivamente como

$$1! = 1, \quad (n+1)! = (n+1) \cdot n!.$$

Podemos calcular el factorial de un natural usando un bucle: usaremos la variable f para ir acumulando los productos sucesivos y multiplicamos todos los naturales hasta llegar al pedido.

```
(%i8)  fact(n):=block([f:1], for k:1 thru n do f:f*k,f);
(%o8)  fact(n):=block([f:1],for k thru n do f:f*k,f)
(%i9)  fact(5);
```

```
(%o9) 120
```

6.1.2 Mostrar resultados intermedios

<code>print(expr1,expr2,...)</code>	evalúa y escribe las expresiones en pantalla una tras otra; devuelve como resultado la última expresión
<code>display(expr1,expr2,...)</code>	muestra como ecuaciones las expresiones centradas en la línea
<code>disp(expr1,expr2,...)</code>	muestra las expresiones centradas en la línea

Aunque un bloque devuelve por defecto el resultado de evaluar la última expresión, hay veces que es útil mostrar por pantalla algunos cálculos intermedios.

La orden `print` admite una lista, separada por comas, de literales y expresiones. Muestra una `print` tras otra las expresiones evaluadas *en la misma línea*.

```
(%i10) a:2;
        print("La variable a^2 vale ",a^2);

(%o10) 2
        La variable a^2 vale 4

(%o11) 4
```

Las órdenes `display` y `disp` también se pueden usar para mostrar un resultado en pantalla. A diferencia de `print` muestran cada una de las expresiones en un línea nueva. La diferencia entre ambas se ve en el siguiente ejemplo: si usamos `display`

```
(%i12) a:2$b:3$
        display(a,b)$
        a=2
        b=3
```

o si usamos `disp`

```
(%i15) a:2$b:3$
        disp(a,b)$
        2
        3
```

6.1.3 ¿Y si quiero acabar antes?

<code>return(expr)</code>	detiene la ejecución y devuelve <i>expr</i>
<code>return()</code>	detiene la ejecución
<code>error(expr)</code>	detiene la ejecución, devuelve un error y <i>expr</i>

Si queremos salir de un bloque y devolver un resultado antes de llegar a la última expresión, podemos usar la orden `return`. Por ejemplo, recuerda que podemos definir el logaritmo en base *a* de la siguiente forma

```
(%i18)  loga(x):=log(x)/log(a)$
```

Esto podemos mejorarlo algo utilizando dos variables:

```
(%i19)  loga(x,a):=log(x)/log(a)$
```

pero deberíamos tener en cuenta si *a* es una base válida. Sólo nos valen los números positivos distintos de 1. Vamos a utilizar un bloque y un condicional para filtrar los casos que no tienen sentido.

```
(%i20)  loga(x,a):=block(
        if a<0 then print("La base es negativa"),
        if a=1 then print("La base es 1"),
        log(x)/log(a)
    )$
```

Si probamos con números positivos

```
(%i21)  loga(3,4);
(%o21)   $\frac{\log(3)}{\log(4)}$ 
```

Funciona. ¿Y si la base no es válida?

```
(%i22)  log(3,-1);
        La base es negativa
(%o22)   $\frac{\log(3)}{\log(-1)}$ 
```

Fíjate que no hemos puesto ninguna condición de salida en el caso de que la base no sea válida. Por tanto, *Maxima* evalúa una tras otra cada una de las sentencias y devuelve la última. Vamos a arreglarlo usando la orden `error`.

```
(%i23) loga(x,a):=block(
    if a<0 then error("La base es negativa"),
    if a=1 then error("La base es 1"),
    log(x)/log(a)
)$
```

6.1.4 Funciones con variables opcionales

Se puede definir una función con un número variable de variables usando una lista como último argumento en la definición. Por ejemplo, la función

```
(%i24) f(a,[b]):=flatten([a,b])$
```

crea una lista con la primera entrada, a , y las opcionales, b .

```
(%i25) f(2);
(%o25) [2]
(%i26) f(1,2,3,4);
(%o26) [1,2,3,4]
```

Como puedes ver, “[b]” en este caso es una lista en la que incluimos todas las variables opcionales que necesitemos.

Podemos usar esta idea para redefinir la función “loga” de forma que calcule el logaritmo neperiano si sólo encuentra una variable y el logaritmo en base a si escribimos dos variables.

```
(%i27) loga(x,[a]):=block(
    if length(a)=0
    then return(log(x))
    else (
        if a[1]<0 then (print("La base es negativa"),return()),
        if a[1]=1 then (print("La base es 1"),return()),
        log(x)/log(a[1])
    )
)$
```

Observación 6.1. Se puede salir de un bloque usando la sentencia `error(mensaje)` en los casos en que la base no sea la adecuada.

```
(%i28)  loga(x,[a]):=block(
        if length(a)=0
        then return(log(x))
        else (
            if a[1]<0 then error("Cambia la base"),
            if a[1]=1 then error("La base es 1"),
            log(x)/log(a[1])
        )
    )$
```

6.1.5 Otras opciones dentro de un bloque

Aunque no vamos a usarlas ahora mismo, hay un par de cosas que podemos utilizar dentro de un bloque. La primera es la orden `go` que nos permite romper el orden usual, de izquierda a derecha, de evaluar las expresiones dentro de un bloque y volver a una etiqueta anterior.

```
(%i29)  block([a:1],display(a),hola,a:a+1,if a<3 then go(hola) else a);
a=1
(%o29)  3
```

La segunda orden que vamos a comentar, `read`, permite pedir al usuario que introduzca un valor.

```
(%i30)  a:3$
(%i31)  a:read(("a vale ",a," – escribe su nuevo valor ");
a vale 3 – escribe su nuevo valor 4;
(%o31)  4
(%i32)  a;
(%o32)  4
```

<code>go(etiqueta)</code>	vuelve a <i>etiqueta</i>
<code>read(expr1,expr2,...)</code>	muestra las expresiones y lee una expresión de la consola

Puedes usar esta orden para que los siguientes programas sean algo más interactivos.

6.2 Bucles

También vamos a necesitar algunas estructuras sencillas para programar los métodos de resolución de ecuaciones. Un bucle es un proceso repetitivo que se realiza un cierto número de veces. Un ejemplo de bucle puede ser el siguiente: supongamos que queremos obtener los múltiplos de siete comprendidos entre 7 y 70; para ello, multiplicamos 7 por cada uno de los números naturales comprendidos entre 1 y 10, es decir, repetimos 10 veces la misma operación: multiplicar por 7. La primera de las órdenes de este tipo que podemos usar es un bucle tipo `for`.

<code>for var:valor1 step valor2 thru valor3 do expr</code>	bucle for
<code>for var:valor1 step valor2 while cond do expr</code>	bucle for
<code>for var:valor1 step valor2 unless cond do expr</code>	bucle for

En un bucle for nos pueden aparecer los siguientes elementos (no necesariamente todos)

- a) `var:valor1` valor inicial del contador.
- b) `cond` dirá a *Maxima* valor final del contador.
- c) `step valor2` indica la forma de aumentar la condición inicial.
- d) `expr` dirá a *Maxima* lo que tiene que realizar en cada paso; `expr` puede estar compuesta de varias sentencias separadas mediante punto y coma.

En los casos en que el paso es 1, no es necesario indicarlo.

<code>for var:valor1 thru valor3 do expr</code>	bucle for con paso 1
<code>for var:valor1 while cond do expr</code>	bucle for con paso 1
<code>for var:valor1 unless cond do expr</code>	bucle for con paso 1

Para comprender mejor el funcionamiento de esta orden vamos a ver algunos ejemplos sencillos.

En primer lugar, generemos los múltiplos de 7 hasta 70:

```
(%i33) for i:1 step 1 thru 10 do print(7*i);
7
14
21
28
35
42
49
56
63
70
(%o33) done
```

Se puede conseguir el mismo resultado usando while o unless: tanto

```
(%i34) for i:1 while i<=10 do print(7*i);
```

como

```
(%i35) for i:1 unless i>10 do print(7*i);
```

muestran en pantalla la tabla del 7. Observa que como el paso es 1, no hemos escrito “step 1”.

Se puede conseguir el mismo efecto sumando en lugar de multiplicando. Por ejemplo, los múltiplos de 5 hasta 25 son

```
(%i36) for i:5 step 5 thru 25 do print(i);
      5
      10
      15
      20
      25
(%o36) done
```

Ejemplo 6.2. Podemos utilizar un bucle para sumar una lista de números pero nos hace falta una variable adicional en la que ir guardando las sumas parciales que vamos obteniendo. Por ejemplo, el siguiente código suma los cuadrados de los 100 primeros naturales.

```
(%i37) suma:0$
      for i:1 thru 100 do suma:suma+i^2$
      print("la suma de los cuadrados de los 100 primeros
            naturales vale ",suma)$
      la suma de los cuadrados de los 100 primeros
      naturales vale 338350
```

Por último, comentar que no es necesario utilizar una variable como contador. Podemos estar ejecutando una serie de expresiones mientras una condición sea cierta (bucle `while`) o mientras sea falsa (bucle `unless`). Incluso podemos comenzar un bucle infinito con la orden `do`, sin ninguna condición previa, aunque, claro está, en algún momento tendremos que ocuparnos nosotros de salir (recuerda el comando `return`).

<pre>while cond do expr</pre>	bucle while
<pre>unless cond do expr</pre>	bucle unless

Este tipo de construcciones son útiles cuando no sabemos cuántos pasos hemos de dar pero tenemos clara cuál es la condición de salida. Veamos un ejemplo bastante simple: queremos calcular $\cos(x)$ comenzando en $x = 0$ e ir aumentando de 0.3 en 0.3 hasta que el coseno deje de ser positivo.

```
(%i38) i:0;
(%o38) 0
(%i39) while cos(i)>0 do (
      display(cos(i)),
      i:i+0.3
    );
cos(0)=1
cos(0.3)=0.955336489125606
cos(0.6)=0.8253356149096783
cos(0.8999999999999999)=0.6216099682706646
cos(1.2)=0.3623577544766736
cos(1.5)=0.0707372016677029
```

```
(%o39) done
```

6.3 Condicionales

La segunda sentencia es la orden condicional `if`. Esta sentencia comprueba si se verifica una condición, después, si la condición es verdadera *Maxima* ejecutará una *expresión1*, y si es falsa ejecutará otra *expresión2*.

```
if condición then expr1 else expr2  condicional if-then-else
if condición then expr              condicional if-then
```

Las expresiones 1 y 2 pueden estar formadas por varias órdenes separadas por comas. Como siempre en estos casos, quizá un ejemplo es la mejor explicación:

```
(%i40) if log(2)<0 then x:5 else 3;
(%o40) 3
```

Observa que la estructura `if-then-else` devuelve la expresión correspondiente y que esta expresión puede ser una asignación, algo más complicado o algo tan simple como "3".

La última sentencia de programación que vamos a ver es la orden `return(var)` cuya única finalidad es la de interrumpir un bucle en el momento que se ejecuta y devolver un valor. En el siguiente ejemplo se puede comprender rápidamente el uso de esta orden.

```
(%i41) for i:1 thru 10 do
(
  if log(i)<2 then print("el logaritmo de ",i," es menor
    que 2") else return(x:i)
)$
print("el logaritmo de ",x," es mayor que 2")$
el logaritmo de 1 es menor que 2
el logaritmo de 2 es menor que 2
el logaritmo de 3 es menor que 2
el logaritmo de 4 es menor que 2
el logaritmo de 5 es menor que 2
el logaritmo de 6 es menor que 2
el logaritmo de 7 es menor que 2
el logaritmo de 8 es mayor que 2
```

Observación 6.3. La variable que se utiliza como contador, `i` en el caso anterior, es siempre local al bucle. No tiene ningún valor asignado fuera de él. Es por esto que hemos guardado su valor en una variable auxiliar, `x`, para poder usarla fuera del bucle.

6.4 Ejercicios

Ejercicio 6.1. Usa el comando `for` en los siguientes ejemplos:

- a) Sumar los números naturales entre 400 y 450.
- b) Calcula la media aritmética de los cuadrados de los primeros 1000 naturales.

Ejercicio 6.2. Dado un número positivo x , se puede conseguir que la suma

$$1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}$$

sea mayor que x tomando un número n suficientemente grande. Encuentra la forma de calcular dicho número de manera general. ¿Cuál es el valor para $x = 7, 8$ y 10 ?

Ejercicio 6.3. Calcula las medias geométricas de los n primeros naturales y averigua cuál es el primer natural para el que dicha media es mayor que 20.

Ejercicio 6.4. Calcula la lista de los divisores de un número natural n .

Ejercicio 6.5. Calcula la suma de los números naturales entre 100 y 1000 que no son divisibles por 7.

Métodos cerrados

7

7.1 Métodos cerrados y abiertos	123	7.2 El método de bisección	123	7.3 Un ejemplo concreto	124
7.4 De nuevo bisección	127	7.5 Regula-falsi	129	7.6 Ejercicios	129

7.1 Métodos cerrados y abiertos

La búsqueda de las soluciones de una ecuación es un problema que se presenta con mucha frecuencia. Usando el ordenador se pueden encontrar soluciones aproximadas en el caso de que no sea posible encontrar la solución exacta. Veremos como, partiendo de una aproximación inicial, se pueden construir aproximaciones cada vez mejores. La aproximación inicial puede ser de dos tipos:

- Métodos cerrados*: conocemos un par de valores entre los que sabemos que se encuentra una solución y vamos construyendo cada vez mejores aproximaciones que mantienen encajonada a la solución.
- Métodos abiertos*: conocemos uno o varios valores y, a partir de ellos, construimos una sucesión de aproximaciones que, en las condiciones adecuadas convergen a la solución

Cada uno tiene sus ventajas e inconvenientes. En general,

- Los métodos cerrados garantizan la convergencia de las sucesión de aproximaciones a la solución al mantener en cada paso acotada la solución.
- Los métodos abiertos, si convergen, suelen ser más rápidos que los cerrados.
- Es posible que no se pueda encontrar un intervalo inicial. Por ejemplo, no podemos encontrar un intervalo donde la función x^2 cambie de signo aunque sabemos que se anula en cero.

7.2 El método de bisección

El método de bisección es una de las formas más elementales de buscar una solución de una ecuación. Ya sabemos que si una función continua cambia de signo en un intervalo, entonces se anula en algún punto. ¿Cómo buscamos dicho punto?

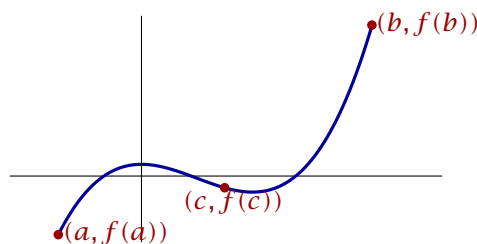


Figura 7.1 Método de bisección

Comencemos con una función $f: [a, b] \rightarrow \mathbb{R}$ continua y verificando que tiene signos distintos en los extremos del intervalo. La idea básica es ir dividiendo el intervalo por la mitad en dos subintervalos, $[a, \frac{1}{2}(a+b)]$ y $[\frac{1}{2}(a+b), b]$, y elegir aquel en el que la función siga cambiando de signo. Si repetimos este proceso, obtenemos un intervalo cada vez más pequeño donde se encuentra la raíz.

Más concretamente el proceso sería,

Datos iniciales: función f , números a, b

Se verifica que: $f(a)f(b) < 0$

bucle

- ▷ Calculamos el punto medio $c = (a + b)/2$,
- ▷ Comparamos los signos de $f(a)$, $f(c)$ y $f(b)$
- ▷ y elegimos aquél donde la función cambie de signo

final bucle

7.3 Un ejemplo concreto

Vamos a aplicar este método a la función $f(x) = x^6 + x - 5$ en el intervalo $[0, 2]$.

Definiremos en primer lugar la función y el intervalo y luego un bucle que nos calcula el intervalo donde, después de los pasos que digamos, se encuentra localizado el cero de la función.

```
(%i1) f(x):=x^6+x-5;
      a:0.0;
      b:2.0;

(%o2) x^6+x-5
(%o3) 0.0
(%o4) 2.0
```

Observa que hemos declarado a y b como valores numéricos. Comprobamos que la función cambia de signo.

```
(%i5) f(a)*f(b);
(%o5) -305.0
```

Ahora el bucle. Recuerda que los símbolos $/*$ y $*/$ indican el principio y el fin de un comentario y que lo que encierran no se ejecuta.

```
(%i6) for i:1 thru 10 do
(
  c:(a+b)/2, /* calculamos el punto medio */
  if f(a)*f(c)<0 /* ¿cambia de signo en [a,c]? */
    then b:c /* elegimos [a,c] */
    else a:c, /* elegimos [c,b] */
  print(a,b) /* escribimos los resultados por pantalla */
);
1.0 2.0
1.0 1.5
1.0 1.25
1.125 1.25
1.1875 1.25
1.21875 1.25
1.234375 1.25
1.2421875 1.25
1.24609375 1.25
1.24609375 1.248046875
(%o6) done
```

Fíjate que ya sabemos que la solución es aproximadamente 1.24. No podemos estar seguros todavía del tercer decimal. Quizá sería mejor repetir el bucle más de diez veces, pero, ¿cuántas? Podemos establecer como control que la distancia entre los extremos sea pequeña. Eso sí, habría que añadir un tope al número de pasos para asegurarnos de que el bucle termina.

Observación 7.1. Si no se quiere ralentizar mucho la ejecución de este bucle y del resto de programas en el resto del tema, es conveniente trabajar en modo numérico. Recuerda que este comportamiento se controla con la variable `numer`. Puedes cambiarlo en el menú **Numérico**→**Conmutar salida numérica** o directamente estableciendo el valor de `numer` en verdadero. 

```
(%i7) numer:true;
(%o7) true
```

7.3.1 Control del error

En este método es fácil acotar el error que estamos cometiendo. Sabemos que la función f se anula entre a y b . ¿Cuál es la mejor elección sin tener más datos? Si elegimos a la solución, teóricamente, podría ser b . El error sería en este caso $b - a$. ¿Hay alguna elección mejor? Sí, el punto medio $c = (a + b)/2$. ¿Cuál es el error ahora? Lo peor que podría pasar sería que la solución fuera alguno de los extremos del intervalo. Por tanto, el error sería como mucho $\frac{b-a}{2}$. En cada paso que damos dividimos el intervalo por la mitad y al mismo tiempo también el error cometido que en el paso n -ésimo es menor o igual que $\frac{b-a}{2^n}$.

A partir de aquí, podemos deducir el número de iteraciones necesarias para obtener una aproximación con un error o exactitud prefijados. Si notamos por “`err`” a la exactitud prefijada, entonces para conseguir dicha precisión, el número “ n ” de iteraciones necesarias deberá satisfacer

$$\frac{b-a}{2^n} \leq err$$

así,

$$n \geq \log_2 \left(\frac{b-a}{err} \right) = \frac{\log \left(\frac{b-a}{err} \right)}{\log(2)}.$$

<code>ceiling(a)</code>	menor entero mayor o igual que a
<code>floor(a)</code>	mayor entero menor o igual que a

ceiling La orden `ceiling(x)` nos da el menor entero mayor o igual que x . Bueno, ya sabemos cuántos pasos tenemos que dar. Reescribimos nuestro algoritmo con esta nueva información:

Datos iniciales: función f , números a , b , error err , contador i

Se verifica que: $f(a)f(b) < 0$

▷ Calculamos el número de pasos

para $i:1$ hasta número de pasos **hacer**

▷ Calculamos el punto medio $c = (a + b)/2$,

▷ Comparamos los signos de $f(a)$, $f(c)$ y $f(b)$

▷ y elegimos aquél donde la función cambie de signo

final bucle

Volviendo a nuestro ejemplo, nos quedaría algo así.

```
(%i8) f(x):=x^6+x-5;
      a:0$
      b:2$
      err:10^(-6)$
      log2(x):=log(x)/log(2)$ /* hay que definir el logaritmo en base 2 */
      pasos:ceiling(log2((b-a)/err))$
      for i:1 thru pasos do
      (
        c:(a+b)/2,
        if f(a)*f(c)<0
          then b:c
          else a:c,
        print(a,b) /* escribimos los resultados por pantalla */
      )$
```

7.3.2 ¿Y si hay suerte?

Si encontramos la solución en un paso intermedio no habría que hacer más iteraciones. Deberíamos parar y presentar la solución encontrada. En cada paso, tenemos que ir comprobando que $f(c)$ vale o no vale cero. Podríamos comprobarlo con una orden del tipo `is(f(c)=0)`, pero recuerda que con valores numéricos esto puede dar problemas. Mejor comprobemos que es “suficientemente” pequeño.

Datos iniciales: función f , números a , b , error err , contador i , precisión pr

Se verifica que: $f(a)f(b) < 0$

▷ Calculamos el número de pasos

para $i:1$ hasta número de pasos **hacer**


```

    ▷ Calculamos el punto medio  $c = (a + b)/2$ ,
si  $f(c) < pr$  entonces
    ▷ La solución es  $c$ 
en otro caso
    ▷ Comparamos los signos de  $f(a)$ ,  $f(c)$  y  $f(b)$ 
    ▷ y elegimos aquél donde la función cambie de signo
final si
    ▷ La solución aproximada es  $c$ 
final del bucle

```

En nuestro ejemplo, tendríamos lo siguiente

```

(%i9)  f(x):=x^6+x-5;
        a:0$
        b:2$
        err:10^(-6)$
        pr:10^(-5)$
        log2(x):=log(x)/log(2)$
        pasos:ceiling(log2((b-a)/err))$
        for i:1 thru pasos do
        (
          c:(a+b)/2,
          if abs(f(c))<pr
            then (print("La solucion es exacta"), return(c))
          else if f(a)*f(c)<0
            then b:c
            else a:c
        )$
        print("la solucion es ",c)$ /* aproximada o exacta, es la solución
        */

```

¿Se te ocurren algunas mejoras del algoritmo? Algunas ideas más:

- el cálculo de $f(a)f(c)$ en cada paso no es necesario: si sabemos el signo de $f(a)$, sólo necesitamos saber el signo de $f(c)$ y no el signo del producto,
- habría que comprobar que $f(a)$ y $f(b)$ no son cero (eso ya lo hemos hecho) ni están cerca de cero como hemos hecho con c .
- Si queremos trabajar con una precisión mayor de 16 dígitos, sería conveniente utilizar números en coma flotante grandes.

7.4 De nuevo bisección

Si unimos todo lo que hemos aprendido, podemos definir una función que utilice el método de bisección. Comentamos algunos de los cambios

- Usamos las variables fa , fb y fc para conservar los valores de la función y no recalcularlos en cada paso.

- b) Los extremos se pueden escribir en cualquier orden.
- c) Usamos números reales grandes y 24 dígitos.
- d) La precisión y tolerancia son variables opcionales.
- e) Hemos usado la sentencia `subst` para definir la función a la que aplicamos bisección dentro del bloque. La orden `subst(a,b,c)` sustituye `a` por `b` en `c`.
- f) Devolvemos como respuesta el número de pasos que se han dado, el valor de la solución y el valor de la función en dicho punto.

```

biseccion(expr,var,ext_inf,ext_sup,[t]):=
  block(
    [a,b,c,pasos,fa,fb,fc,j,err:10^(-18),prec:10^(-20),fpprec:24],

    /* comprobamos que t tiene dos entradas err y prec */
    if length(t)=2 then (err:t[1],prec:t[2]),
    if length(t)=1 then error("Numero erróneo de datos"),
    if length(t)>2 then error("Numero erróneo de datos"),

    /* los extremos pueden estar escritos desordenados */
    a:bfloat(min(ext_inf,ext_sup)),
    b:bfloat(max(ext_inf,ext_sup)),
    if a=b then error("Los extremos deben ser distintos"),

    /* número de pasos */
    local(log2,f),
    define(log2(x),log(x)/log(2)),
    pasos:ceiling(log2((b-a)/err)),

    /* definimos la función f cambiando la variable si es necesario */
    define(f(x),subst(x,var,expr)),

    /* usamos fa, fb y fc para los valores de la función */
    fa:f(a),
    fb:f(b),

    /* ¿se alcanza la solución en los extremos? */
    if abs(f(a)) < prec then return(a),
    if abs(f(b)) < prec then return(b),

    /* comprobamos las condiciones iniciales */
    if sign(f(a))=sign(f(b)) then error("Error: no hay cambio de signo"),

    /* hacemos el bucle for */
    for k:1 thru pasos do(
      j:k, /* paso por el que vamos */
      c:(a+b)/2,
      fc:f(c),

```

```

        if abs(fc)< prec then return(),
        if sign(fa) # sign(fc) then (b:c,fb:fc) else (a:c,fa:fc)
    ),
    /* devolvemos número de pasos, solución, valor de la función */
    [j,c,fc]
);

```

A partir de este momento, podemos utilizarlo usando

```

(%i10)  biseccion(x^2-2,x,0.0,3.0);
(%o10)  [62,1.41421356237309504941431b0,1.73275313175770578249038b-18]

```

Observa que las cotas del error y la precisión la hemos fijado dentro del bloque. Prueba a añadirlo como valores opcionales.

7.5 Regula-falsi

El método de regula falsi o de la falsa posición es muy parecido al método de bisección. La única diferencia es que se cambia el punto medio por el punto de corte del segmento que une los puntos $(a, f(a))$ y $(b, f(b))$ con el eje de abscisas.

Escribe un programa que utilice este método. Para la función $f(x) = x^2 - 5$ en el intervalo $[0, 4]$, compara los resultados obtenidos. ¿Cuál es mejor?

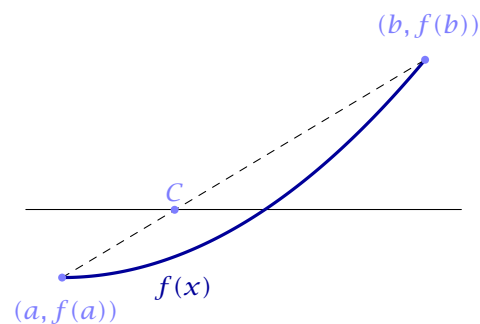


Figura 7.2 Método de regula falsi

7.6 Ejercicios

Ejercicio 7.1. Prueba a cambiar la función, los extremos del intervalo (en los cuales dicha función cambia de signo), así como la exactitud exigida. Intenta también buscar un caso simple en el que se encuentre la solución exacta en unos pocos pasos. Por último, intenta usar el algoritmo anterior para calcular $\sqrt[3]{5}$ con una exactitud de 10^{-10} .

- E** **Ejercicio 7.2.** El método de trisección para el cálculo de ceros de funciones difiere del método de bisección en que los intervalos se dividen en tres trozos de igual longitud en vez de hacerlo en dos.

Escribe un programa que desarrolle el método de trisección y que tenga como entradas: la función, el intervalo y el error permitido.

- E** **Ejercicio 7.3.** En el método de bisección siempre se elige el punto medio del intervalo como pivote para dividir el intervalo en dos partes iguales. Escribe un programa que elija como pivote un número aleatorio del intervalo. Compara los resultados que se obtienen para la función $f(x) = x^3 - 5$ en el intervalo $[-10, 10]$.

Otros métodos

8

8.1 Métodos de iteración funcional 131 8.2 El método de Newton-Raphson 135

8.1 Métodos de iteración funcional

En esta sección tratamos de encontrar una solución aproximada de una ecuación de la forma $x = f(x)$. Es usual referirse a dichas soluciones como *puntos fijos* de la función f . Los puntos fijos de una función f no son más los puntos de intersección de las gráficas de la función f y de la identidad. Por ejemplo, la función de la Figura 8.1 tiene tres puntos fijos.

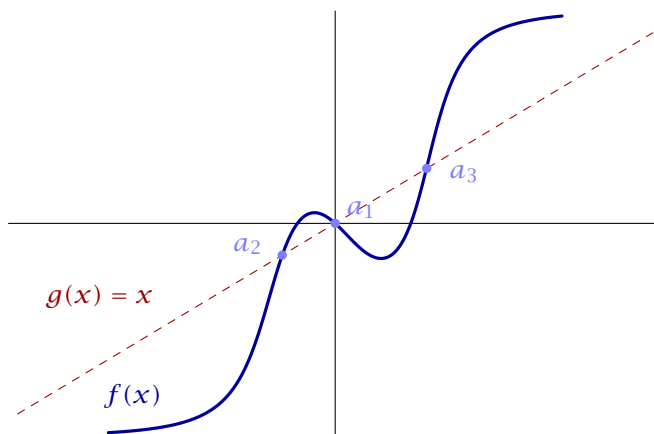


Figura 8.1 Puntos fijos de una función

Hay algunas condiciones sencillas que nos garantizan que una función tiene un único punto fijo.

Teorema 8.1. Sea $f: [a, b] \rightarrow [a, b]$ una función continua.

a) La función f tiene al menos un punto fijo.

b) Si f es derivable y $|f'(x)| \leq L < 1$, $\forall x \in [a, b]$, entonces tiene un único punto fijo.

Iteración funcional

Para buscar un punto fijo de una función, se elige un punto inicial $x_1 \in [a, b]$ cualquiera y aplicamos la función repetidamente. En otras palabras, consideramos la sucesión definida por recurrencia como

$$x_{n+1} = f(x_n)$$

para $n \geq 1$. Si la sucesión es convergente y llamamos s a su límite, entonces

$$s = \lim_{n \rightarrow \infty} x_{n+1} = \lim_{n \rightarrow \infty} f(x_n) = f(s),$$

o, lo que es lo mismo, s es un punto fijo de la función f . Más aún, en algunos casos es posible controlar el error.

Teorema 8.2. Sea $f: [a, b] \rightarrow [a, b]$ derivable verificando que $|f'(x)| \leq L < 1, \forall x \in [a, b]$. Sea s el único punto fijo de la función f . Sea $x_1 \in [a, b]$ cualquiera y $x_{n+1} = f(x_n)$, entonces

$$|x_n - s| \leq \frac{L}{1-L} |x_n - x_{n-1}| \leq \frac{L^{n-1}}{1-L} |x_2 - x_1|.$$

El método de construcción de la sucesión es lo que se conoce como un método de iteración funcional.

Ejemplo 8.3. Consideremos la función $f(x) = \frac{1}{4}(\cos(x) + x^2)$ con $x \in [0, 1]$. Acotemos la derivada,

$$|f'(x)| = \left| \frac{-\sin(x) + 2x}{4} \right| \leq \left| \frac{\sin(x)}{4} \right| + \left| \frac{2x}{4} \right| \leq \frac{1}{4} + \frac{2}{4} = \frac{3}{4} < 1.$$

Por tanto, la función f tiene un único punto fijo en el intervalo $[0, 1]$. Podemos encontrar el punto fijo resolviendo la correspondiente ecuación.

```
(%i1) find_root((cos(x)+x^2)/4-x,x,0,1);
(%o1) .2583921443715997
```

También podemos calcular las iteraciones comenzando en un punto inicial dado de manera sencilla utilizando un bucle

```
(%i2) define(f(x),(cos(x)+x^2)/4)$
(%i3) x0:0;
      for i:1 thru 10 do(
          x1:f(x0),
          print("Iteración ",i," vale ", x1),
          x0:x1
      );
(%o3) 0
      Iteración 1 vale 0.25
      Iteración 2 vale .2578531054276612
      Iteración 3 vale .2583569748525884
      Iteración 4 vale .2583898474528139
      Iteración 5 vale .2583919943502456
(%o4) Iteración 6 vale .2583921345730372
      Iteración 7 vale .2583921437316118
      Iteración 8 vale .2583921443297992
      Iteración 9 vale .2583921443688695
      Iteración 10 vale .2583921443714213
      done
```

Observación 8.4. Existen muchas formas de cambiar una ecuación de la forma $f(x) = 0$ en un problema de puntos fijos de la forma $g(x) = x$. Por ejemplo, consideremos la ecuación $x^2 - 5x + 2 = 0$.

a) Sumando x en los dos miembros

$$x^2 - 5x + 2 = 0 \iff x^2 - 4x + 2 = x,$$

y las soluciones de f son los puntos fijos de $g_1(x) = x^2 - 4x + 2$ (si los tiene).

b) Si despejamos x ,

$$x^2 - 5x + 2 = 0 \iff x = \frac{x^2 + 2}{5}$$

y, en este caso, los puntos fijos de la función $g_2(x) = \frac{x^2 + 2}{5}$ son las soluciones buscadas.

c) También podemos despejar x^2 y extraer raíces cuadradas

$$x^2 - 5x + 2 = 0 \iff x^2 = 5x - 2 \iff x = \sqrt{5x - 2}.$$

En este caso, nos interesan los puntos fijos de la función $g_3(x) = \sqrt{5x - 2}$.

Como puedes ver, la transformación en un problema de puntos fijos no es única. Evidentemente, algunas de las transformaciones mencionadas antes dependen de que x sea distinto de cero, mayor o menor que $2/5$, etc. Además de eso las funciones g_i pueden tener mejores o peores propiedades, algunas verificarán las condiciones del teorema anterior y otras no.

8.1.1 Ejercicios

Ejercicio 8.1. Escribe un programa que dada una función, un punto inicial y un número de iteraciones, devuelva la última de ellas.

Ejercicio 8.2. Utiliza el método de iteración con las 3 funciones anteriores empezando en cada uno de los puntos 0.5, 1.5 y 6. ¿En cuáles obtienes convergencia a un punto fijo? ¿Es siempre el mismo?

8.1.2 Representación gráfica con el paquete *dynamics*

Para representar gráficamente los puntos de la sucesión, comenzamos con el primer punto de la sucesión $(x_1, f(x_1))$ y, a partir de ese momento, nos vamos moviendo horizontalmente hasta cruzar la bisectriz y verticalmente hasta encontrar de nuevo la gráfica de la función. Más concretamente,

- comenzamos con $(x_1, f(x_1))$;
- nos movemos horizontalmente hasta cortar la bisectriz. El punto de corte será $(f(x_1), f(x_1))$;
- nos movemos verticalmente hasta cortar a la gráfica de f o, lo que es lo mismo, tomamos $x_2 = f(x_1)$ y le calculamos su imagen. El punto de corte será esta vez $(x_2, f(x_2))$.
- Repetimos.

El paquete *dynamics* permite hacer estas representaciones de forma muy sencilla. Primero lo cargamos

```
(%i5) load(dynamics)$
```

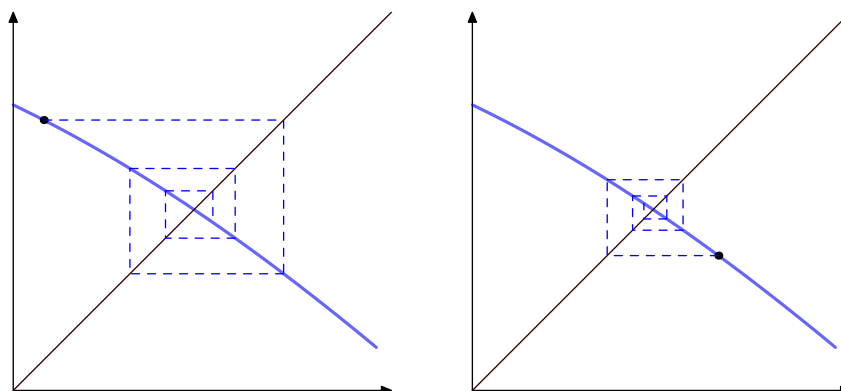
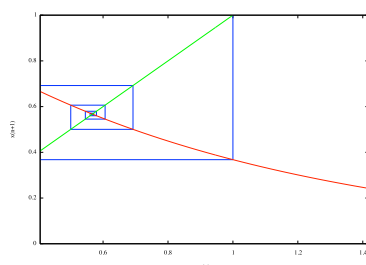


Figura 8.2 Método de iteración funcional

y luego podemos usar los comandos `evolution` y `staircase` indicando el punto inicial y el número de iteraciones. Por ejemplo, el diagrama de escalera para la función e^{-x} tomando como punto inicial 1 y 10 pasos

```
(%i6) staircase(exp(-x),1,10,[y,0,1]);
```

```
(%o6)
```



Observa que hemos añadido `[y,0,1]` para indicar un rango más apropiado que el se dibuja por defecto.

```
evolution(func,ptol,pasos,opciones)
```

Gráfico de las iteraciones de *func*

```
staircase(func,ptol,pasos,opciones)
```

Gráfico de escalera
de las iteraciones de *func*

8.1.3 Criterios de parada

Suele ocurrir que, cuando trabajamos con métodos iterativos, tenemos una sucesión que sabemos que es convergente, pero no conocemos cuál es el valor exacto de su límite. En estos casos no podemos poner como condición de parada que la diferencia entre un término de la sucesión, a_n , y el límite l sea pequeña. El mismo problema lo tenemos con el error relativo: dado que no conocemos el valor de l , no podemos calcular $|a_n - l|$ ni $\left| \frac{a_n - l}{l} \right|$.

Lo que se suele hacer es usar el error estimado. Sustituimos el valor desconocido del límite por uno de los términos de la sucesión que haría el papel de una aproximación de dicho límite. Una forma de establecer un criterio de parada es considerar un número pequeño, al que llamaremos *tolerancia* y denotaremos por T , y parar el cálculo de términos de la sucesión cuando se de una de las dos circunstancias siguientes:

Tolerancia

a) $|a_n - a_{n-1}| < T$ (error absoluto estimado) o

b) $\frac{|a_n - a_{n-1}|}{|a_n|} < T$ (error relativo estimado).

Suele ser mejor utilizar esta última, lo mismo que suele ser mejor usar el error relativo que el absoluto.

8.1.4 Ejercicios

Ejercicio 8.3. Añade una condición de parada al método de iteración.

8.2 El método de Newton-Raphson

El método de Newton-Raphson nos proporciona un algoritmo para obtener una sucesión de puntos que aproxima un cero de una función dada.

La forma de construir los términos de la sucesión de aproximaciones es sencilla. Una vez fijado un valor inicial x_1 , el término x_2 se obtiene como el punto de corte de la recta tangente a f en x_1 con el eje OX . De la misma forma, obtenemos x_{n+1} como el punto de corte de la recta tangente a f en el punto x_n con el eje OX . De lo dicho hasta aquí se deduce:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

Como observarás se trata de un método de iteración funcional. Para comprender el algoritmo observa la Figura 8.3 donde se ve cómo se generan los valores de las aproximaciones.

Para asegurar la convergencia de la sucesión (hacia la solución de la ecuación) usaremos el siguiente resultado.

Teorema 8.5. Sea f una función de clase dos en el intervalo $[a, b]$ que verifica:

- a) $f(a)f(b) < 0$,
- b) $f'(x) \neq 0$, para todo $x \in [a, b]$,
- c) $f''(x)$ no cambia de signo en $[a, b]$.

Entonces, tomando como primera aproximación el extremo del intervalo $[a, b]$ donde f y f'' tienen el mismo signo, la sucesión de valores x_n del método de Newton-Raphson es convergente hacia la única solución de la ecuación $f(x) = 0$ en $[a, b]$.

Teorema de
Newton-Raphson

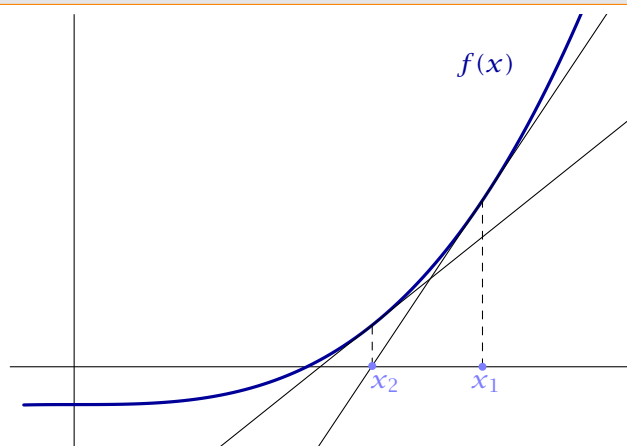


Figura 8.3 Método de Newton-Raphson

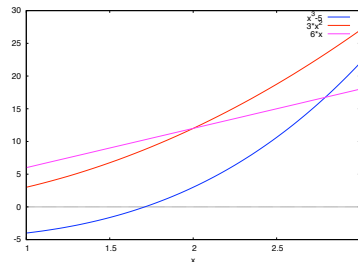
Una vez que tenemos asegurada la convergencia de la sucesión hacia la solución de la ecuación, deberíamos decidir la precisión. Sin embargo, veremos que el método es tan rápido en su convergencia que por defecto haremos siempre 10 iteraciones. Otra posibilidad sería detener el cálculo

cuando el valor absoluto de la diferencia entre x_n y x_{n+1} sea menor que la precisión buscada (lo cual no implica necesariamente que el error cometido sea menor que la precisión).

Utilizaremos ahora *Maxima* para generar la sucesión de aproximaciones. Resolvamos de nuevo el ejemplo de $x^3 - 5 = 0$ en el intervalo $[1, 3]$.

Podemos comprobar, dibujando las gráficas de $f(x) = x^3 - 5$, $f'(x)$ y $f''(x)$ en el intervalo $[1, 3]$, que estamos en las condiciones bajo las cuales el Teorema de Newton-Raphson nos asegura convergencia.

```
(%i7) f(x):=x^3-5$
(%i8) define(df(x),diff(f(x),x))$
(%i9) define(df2(x),diff(f(x),x,2))$
(%i10) plot2d([f(x),df(x),df2(x)], [x,1,3]);
(%o10)
```



A continuación, generaremos los términos de la sucesión de aproximaciones mediante el siguiente algoritmo. Comenzaremos por definir la función f y el valor de la primera aproximación. Inmediatamente después definimos el algoritmo del método de Newton-Raphson, e iremos visualizando las sucesivas aproximaciones. Como dijimos, pondremos un límite de 10 iteraciones, aunque usando mayor precisión decimal puedes probar con un número mayor de iteraciones.

```
(%i11) y:3.0$
for i:1 thru 10 do
  (y1:y-f(y)/df(y),
  print(i,"- aproximación",y1),
  y:y1
  );
1 - aproximación 2.185185185185185
2 - aproximación 1.80582775632091
3 - aproximación 1.714973662124988
4 - aproximación 1.709990496694424
5 - aproximación 1.7099759468005
6 - aproximación 1.709975946676697
7 - aproximación 1.709975946676697
8 - aproximación 1.709975946676697
9 - aproximación 1.709975946676697
10 - aproximación 1.709975946676697
```

Observarás al ejecutar este grupo de comandos que ya en la séptima iteración se han “estabilizado” diez cifras decimales. Como puedes ver, la velocidad de convergencia de este método es muy alta.

El módulo mnewton

El método que acabamos de ver se encuentra implementado en *Maxima* en el módulo `mnewton` de forma mucho más completa. Esta versión se puede aplicar tanto a funciones de varias variables, en otras palabras, también sirve para resolver sistemas de ecuaciones.

Primero cargamos el módulo

```
(%i12) load(mnewton)$
```

y luego podemos buscar una solución indicando función, variable y punto inicial

```
(%i13) mnewton(x^3-5,x,3);
(%o13) [[x=1.709975946676697]]
```

8.2.1 Ejercicios

Ejercicio 8.4. Reescribe el método de Newton-Raphson añadiendo una condición de salida (cuando el error relativo o absoluto sea menor que una cierta cantidad) y que compruebe que la primera derivada está “lejos” de cero en cada paso.

Ejercicio 8.5. El método de la secante evita calcular la derivada de una función utilizando recta secantes. Partiendo de dos puntos iniciales x_0 y x_1 , el siguiente es el punto de corte de la recta que pasa por $(x_0, f(x_0))$ y $(x_1, f(x_1))$ y el eje de abscisas. Se repite el proceso tomando ahora los puntos x_1 y x_2 y así sucesivamente.

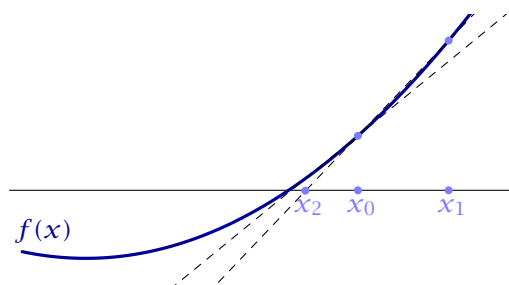


Figura 8.4 Método de la secante

La convergencia de este método no está garantizada, pero si los dos puntos iniciales están próximos a la raíz no suele haber problemas. Su convergencia es más lenta que el método de Newton-Raphson aunque a cambio los cálculos son más simples.

Escribe un programa que utilice este método. Para la función $f(x) = x^2 - 5$, compara los resultados obtenidos con el método de Newton-Raphson. ¿Cuál es mejor?

Ejercicio 8.6. Calcula alguna solución de la ecuación $\log(x) - \cos(x) = 10^{-8}$.

Ejercicio 8.7. Resuelve las ecuaciones

a) $e^{-x} + x^2 - 3 \sin(x) = 0$,

b) $e^{-|x|} = \arctan(x)$, y

c) $x^{15} - 2 = 0$,

utilizando los métodos que hemos estudiado. Compara cómo se comportan y decide en cuál la convergencia es más rápida.

Ejercicio 8.8.

- Considérese la ecuación $e^{(x^2+x+1)} - e^{x^3} - 2 = 0$. Calcular programando los métodos de bisección y de Newton-Raphson, la solución de dicha ecuación en el intervalo $[-0.3, 1]$ con exactitud 10^{-10} .
- Buscar la solución que la ecuación $\tan(x) = \frac{1}{x}$ posee en el intervalo $[0, \frac{\pi}{2}]$ usando los métodos estudiados.

Ejercicio 8.9. Consideremos la función $f(x) = x^3 - x - 1$.

- Usa el método de bisección tomando como intervalo inicial $[0, 2]$ para aproximar la solución. ¿Cuáles son los cinco primeros pasos?
- Usa el método de Newton-Raphson tomando como punto inicial el origen. ¿Cuáles son los cinco primeros pasos?
- La sucesión definida por recurrencia como $x_1 = 1$, $x_{n+1} = \sqrt[3]{1 + x_n}$ para cualquier número natural n también tiene como límite a la única solución de f . Calcula el primer término de dicha sucesión que cumple que $|x_{n+1} - x_n| < 10^{-4}$.

Ejercicio 8.10. Encuentra una solución positiva de la ecuación

$$\exp(-x^2) = \frac{1}{1 + b^2 x^2} \quad (8.1)$$

para $b = 2$ y $b = 4$ usando el método de bisección y el método de regula-falsi.

Ejercicio 8.11. El *método de Steffensen* es un método iterativo para encontrar los ceros de una función. Dada una función f y un punto inicial x_0 , las iteraciones se construyen mediante la expresión

$$x_{n+1} = x_n - \frac{f(x_n)^2}{f(x_n + f(x_n)) - f(x_n)}, \quad \forall n \in \mathbb{N}.$$

Escribe un programa que desarrolle el método de Steffensen y que tenga como entradas: la función, un punto inicial y una condición de parada. Úsalo para resolver la ecuación

$$3x^4 - 4.5x^3 + 23.1x^2 - 14.4x - 8.2 = 0.$$

Cálculo

Límites y continuidad

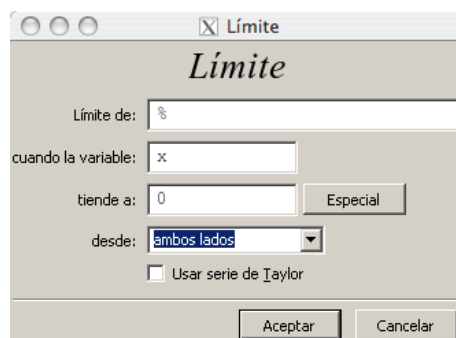
9

9.1 Límites 141 9.2 Sucesiones 143 9.3 Continuidad 145 9.4 Ejercicios 146

Uno de los primeros conceptos que se presentan en un curso de Cálculo es el de continuidad. Este concepto está íntimamente ligado al concepto de límite. En clase hemos utilizado sucesiones para definir límite funcional. En este capítulo veremos cómo usar *Maxima* para resolver algunos problemas relacionados con todos esto.

9.1 Límites

El cálculo de límites se realiza con la orden `limit`. Con ella podemos calcular límites de funciones o de sucesiones en un número, en $+\infty$ o en $-\infty$. Podemos acceder a esta orden usando la entrada del menú **Análisis** → **Calcular límite**. Ahí podemos escoger, además de a qué función le estamos calculando el límite, a qué tiende la variable incluyendo los valores “especiales” como π , e o infinito. Además de esto, también podemos marcar si queremos calcular únicamente el límite por la derecha o por la izquierda en lugar de la opción por defecto que es por ambos lados.



<code>limit (expr,x,a)</code>	$\lim_{x \rightarrow a} expr$
<code>limit (expr,x,a,plus)</code>	$\lim_{x \rightarrow a^+} expr$
<code>limit (expr,x,a,minu)</code>	$\lim_{x \rightarrow a^-} expr$
<code>inf</code>	$+\infty$
<code>minf</code>	$-\infty$
<code>und</code>	indefinido
<code>ind</code>	indefinido pero acotado

El cálculo de límites con *Maxima*, como puedes ver, es sencillo. Sabe calcular límites de cocientes de polinomios en infinito

```
(%i1) limit(n/(n+1),n,inf);
(%o1) 1
```

o en $-\infty$,

```
(%i2) limit((x^2+3*x+1)/(2*x+3),x,minf);
(%o2) -inf
```

aplicar las reglas de L'Hôpital,

```
(%i3) limit(sin(x)/x,x,0);
(%o3) 1
```

Incluso es capaz de dar alguna información en el caso de que no exista el límite. Por ejemplo, sabemos que las funciones periódicas, salvo las constantes, no tienen límite en ∞ . La respuesta de *Maxima* cuando calculamos el límite de la función coseno en ∞ es

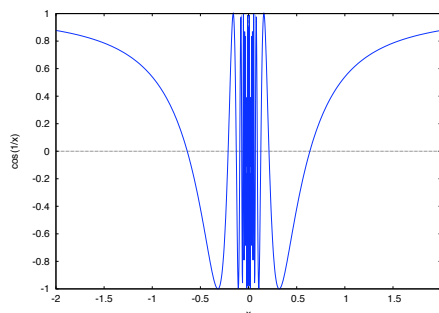
```
(%i4) limit(cos(x),x,inf);
(%o4) ind
```

Indeterminado. Este límite es equivalente a

```
(%i5) limit(cos(1/x),x,0);
(%o5) ind
```

La función $\cos\left(\frac{1}{x}\right)$ oscila cada vez más rápidamente cuando nos acercamos al origen. Observa su gráfica.

```
(%i6) plot2d([cos(1/x)], [x,-2,2]);
(%o6)
```



ind *Maxima* tiene dos formas de indicar indeterminación. Una es **ind**, para indicar que está acotado, **und** y la otra es **und**, para indicar indeterminación a secas. Ahora bien, mucho cuidado con pensar que si la respuesta es **und** entonces la función es no acotada. Esto puede ser cierto o no

```
(%i7) limit(abs(x)/x,x,0);
(%o7) und
```

! Observación 9.1. La acotación que incluye **ind** es una información adicional que da *Maxima*. Si no sabe si es cierta la acotación o, directamente, no es cierta, entonces responde **und** pero esto no quiere decir que la función a la que le estamos calculando el límite no esté acotada: solamente quiere decir que no sabe si lo está o no.

En este último límite lo que ocurre es que tenemos que estudiar los límites laterales


```
(%i8)  limit(abs(x)/x,x,0,plus);
(%o8)  1
(%i9)  limit(abs(x)/x,x,0,minus);
(%o9)  -1
```

Por tanto, no existe el límite puesto que los límites laterales no coinciden. Si recuerdas la definición de función derivable, acabamos de comprobar que la función valor absoluto no es derivable en el origen.

¿Infinito o infinitos?

Maxima diferencia entre “infinitos reales” e “infinitos complejos”. ¿Qué quiere decir esto? Veamos un ejemplo. Si calculamos el límite de la función $1/x$ en 0 inmediatamente pensamos que el resultado depende de si calculamos el límite por la izquierda o por la derecha. En efecto,

```
(%i10) limit(1/x,x,0,plus);
(%o10)  ∞
(%i11) limit(1/x,x,0,minus);
(%o11)  -∞
```

Pero, ¿qué ocurre si no estudiamos límites laterales?

```
(%i12) limit(1/x,x,0);
(%o12)  infinity
```

La constante *infinity* representa “infinito complejo”. Esto quiere decir que en módulo el límite *infinity* es infinito.

9.2 Sucesiones

En clase hemos visto cómo calcular límites de sucesiones, pero ¿cómo podemos calcular esos límites con *Maxima*? Bueno, en la práctica hemos visto dos tipos de sucesiones dependiendo de cómo estaba definidas. Por un lado tenemos aquellas definidas mediante una fórmula que nos vale para todos los términos. Por ejemplo, la sucesión que tiene como término general $x_n = \left(1 + \frac{1}{n}\right)^n$. En este caso no hay ningún problema en definir

```
(%i13) f(n):=(1+1/n)^n;
(%o13) f(n):=⎛1+⎞n
```

y se puede calcular el límite en $+\infty$ sin ninguna dificultad

```
(%i14) limit(f(n), n, inf);
(%o14) %e
```

La situación es diferente cuando no tenemos una fórmula para el término general como, por ejemplo, cuando la sucesión está definida por recurrencia. Veamos un ejemplo. Consideremos la sucesión que tiene como término general $c_1 = 1$ y $c_{n+1} = \frac{c_n}{1+c_n}$ para cualquier natural n . Podemos definirla utilizando una lista definida, como no, por recurrencia:

```
(%i15)  c[1]:1;
(%o15)  1
(%i16)  c[n]:=c[n-1]/(1+c[n-1]);
(%o16)   $c_n := \frac{c_{n-1}}{1+c_{n-1}}$ 
```

Si somos capaces de encontrar una fórmula para el término general, podemos calcular el límite. Con lo que tenemos hasta ahora no vamos muy lejos:

```
(%i17)  limit(c[n],n,inf);
Maxima encountered a Lisp error:
  Error in PROGN [or a callee]: Bind stack overflow.
  Automatically continuing.
  To reenale the Lisp debugger set *debugger-hook* to nil.
```

Podemos demostrar por inducción que la sucesión es, en este caso, decreciente y acotada inferiormente. Una vez hecho, la sucesión es convergente y el límite L debe verificar que $L = \frac{L}{1+L}$. Esta ecuación sí nos la resuelve *Maxima*

```
(%i18)  solve(L/(1+L)=L,L);
(%o18)  [L=0]
```

con lo que tendríamos demostrado que el límite es 0.

Observación 9.2. Para esta sucesión en concreto, sí se puede encontrar una fórmula para el término general. De hecho existe un módulo, `solve_rec`, que resuelve justo este tipo de problemas.

```
(%i19)  kill(all);
(%o0)   done
(%i1)   load(solve_rec)$
(%i2)   solve_rec(c[n]=c[n-1]/(1+c[n-1]),c[n]);
(%o2)    $c_n = \frac{n+\%k_1+1}{n+\%k_1} - 1$ 
```

que, simplificando, nos queda

```
(%i3)   ratsimp(%);
(%o3)    $c_n = \frac{1}{n+\%k_1}$ 
```

si a esto le añadimos que $c_1 = 1$ obtenemos que $c_n = \frac{1}{1+n}$.

De todas formas no hay que ilusionarse demasiado. Encontrar una fórmula para el término general es *difícil* y lo normal es no poder hacerlo. Es por ello que no vamos a entrar en más detalles con `solve_rec`. Lo único que podemos hacer con *Maxima* es calcular términos. Por ejemplo, `solve_rec` no es capaz de encontrar el término general de la sucesión $x_1 = 1$, $x_n = \sqrt{1 + x_{n-1}}$, $\forall n \in \mathbb{N}$. En cambio, no tiene ninguna dificultad en calcular tanto términos como se quiera,

```
(%i4) x[1]:1;
(%o4) 1
(%i5) x[n]:=sqrt(1+x[n-1]);
(%o5)  $x_n = \sqrt{1+x_{n-1}}$ 
(%i6) x[10];
(%o6)  $\sqrt{\sqrt{\sqrt{\sqrt{\sqrt{\sqrt{\sqrt{\sqrt{\sqrt{\sqrt{2}}+1}}+1}}+1}}+1}}+1}}+1}}+1}}+1}}+1}}+1}}+1}}+1}}$ 
(%i7) %,numer;
(%o7) 1.618016542231488
```

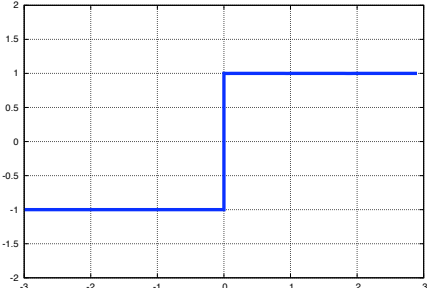
pero seremos nosotros los que tendremos que demostrar la convergencia estudiando la monotonía y la acotación de la sucesión.

9.3 Continuidad

El estudio de la continuidad de una función es inmediato una vez que sabemos calcular límites. Una función $f: A \subset \mathbb{R} \rightarrow \mathbb{R}$ es continua en $a \in A$ si $\lim_{x \rightarrow a} f(x) = f(a)$. Conocido el valor de la función en el punto, la única dificultad es, por tanto, saber si coincide o no con el valor del límite.

Con respecto a las funciones discontinuas, la gráfica puede darnos una idea del tipo de discontinuidad. Si la discontinuidad es evitable, es difícil apreciar un único pixel en la gráfica. Una discontinuidad de salto es fácilmente apreciable. Por ejemplo, la función signo, esto es, $\frac{|x|}{x}$, tiene un salto en el origen que *Maxima* une con una línea vertical.

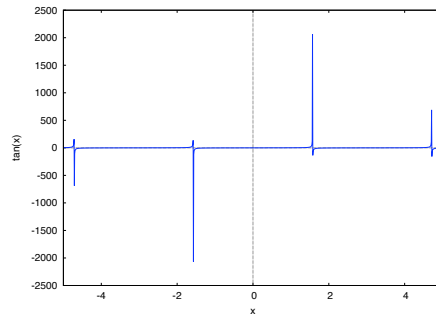
```
(%i8) load(draw)$
(%i9) draw2d(color=blue,
    explicit(abs(x)/x,x,-3,3),
    xrange=[-2,2],
    grid=true);
(%o9)
```



Cuando el salto es infinito o, lo que es lo mismo, cuando la función tiene una asíntota vertical, la primera dificultad que se encuentra *Maxima* es escoger un rango adecuado para representarla:

```
(%i10) plot2d(tan(x), [x, -5, 5]);
```

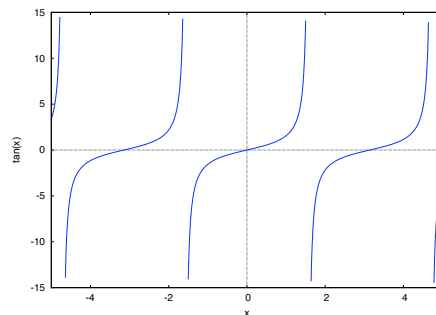
```
(%o10)
```



En estos casos tenemos que ayudar nosotros a *Maxima* restringiendo el rango donde representamos la función

```
(%i11) plot2d(tan(x), [x, -5, 5], [y, -15, 15]);
```

```
(%o11)
```



9.4 Ejercicios

Ejercicio 9.1. La sucesión de Fibonacci está definida por recurrencia como

$$x_n = \begin{cases} 0, & \text{si } n = 0, \\ 1, & \text{si } n = 1, \\ x_{n-1} + x_{n-2}, & \text{si } n > 1. \end{cases}$$

Calcula el término 200 de dicha sucesión.

Ejercicio 9.2. Se define una sucesión por recurrencia como

$$x_1 = 1, \quad x_{n+1} = \sqrt{1 + x_n}, \quad \forall n \in \mathbb{N}.$$

Calcula x_{10} y x_{25} . ¿Es convergente? De serlo, calcula su límite. Por último, encuentra un natural n de forma que la diferencia entre x_n y el límite sea menor que 10^{-3} .

Ejercicio 9.3. Estudia la continuidad de la función $f: \mathbb{R} \rightarrow \mathbb{R}$ definida como $f(x) = x \log |x|$ si $x \neq 0$ y $f(0) = 0$.

Ejercicio 9.4. Sean a y b dos números reales verificando $b < 0 < a$; estudia el comportamiento en cero de la función

$$f(x) = \arctan\left(\frac{a}{x}\right) - \arctan\left(\frac{b}{x}\right), \quad \forall x \in \mathbb{R}^*.$$

Ejercicio 9.5. Estudia la continuidad de la función $f(x) = \arctan\left(\frac{1+x}{1-x}\right)$ con $x \neq 1$, así como su comportamiento en 1 , $+\infty$ y $-\infty$.

Ejercicio 9.6.

- a) Dibuja una función continua cuya imagen no sea un intervalo.
- b) Dibuja una función definida en un intervalo cuya imagen sea un intervalo y que no sea continua.
- c) Dibuja una función continua en todo $\mathbb{R}$, no constante y cuya imagen sea un conjunto (obligatoriamente un intervalo) acotado.
- d) Dibuja una función continua en $[0, 1[$ tal que $f([0, 1[)$ no sea acotado.
- e) Dibuja una función continua definida en un intervalo abierto acotado y cuya imagen sea un intervalo cerrado y acotado.

Ejercicio 9.7. Consideremos la función $f: [0, 1] \rightarrow \mathbb{R}$ definida como $f(x) = \frac{1}{2}(\cos(x) + \sin(x))$.

- a) Utiliza que $f([0, 1]) \subset [0, 1]$ para probar que existe $x \in [0, 1]$ tal que $f(x) = x$ (sin utilizar *Maxima*). A dicho punto se le suele llamar un *punto fijo* de la función f .
- b) Se puede demostrar que la sucesión $x_1 = 1$, $x_{n+1} = f(x_n)$, para cualquier natural n tiende a un punto fijo. Usa un bucle para encontrar un punto fijo con una exactitud menor que 10^{-5} .

Derivación

10

10.1 Cálculo de derivadas	149	10.2 Rectas secante y tangente a una función	152	10.3 Extremos relativos	155
10.4 Ejercicios	160				

En este capítulo vamos a aprender a calcular y evaluar derivadas de cualquier orden de una función; representar gráficamente rectas tangentes y normales a la gráfica de una función; calcular extremos de funciones reales de una variable y, por último, calcular polinomios de Taylor y representarlos gráficamente para aproximar una función.

10.1 Cálculo de derivadas

Para calcular la derivada de una función real de variable real, una vez definida, por ejemplo, como $f(x)$, se utiliza el comando `diff` que toma como argumentos la función a derivar, la variable con respecto a la cual hacerlo y, opcionalmente, el orden de derivación.

<code>diff(expr, variable)</code>	derivada de <i>expr</i>
<code>diff(expr, variable, n)</code>	derivada <i>n</i> -ésima de <i>expr</i>

A este comando también podemos acceder a través del menú **Análisis→Derivar**. Aparece una ventana de diálogo con varios datos a rellenar; a saber:

- Expresión. Por defecto, *wxMaxima* rellena este espacio con % para referirse a la salida anterior. Si no es la que nos interesa, la escribimos directamente nosotros.
- respecto la variable. Se refiere a la variable respecto a la cual vamos a derivar.
- veces. Se refiere al orden de derivación.

Comencemos con un ejemplo,

```
(%i1) diff(tan(x), x);
(%o1) sec(x)^2
```

La orden `diff` considera como constantes cualquier otra variable que aparezca en la expresión a derivar, salvo que explícitamente manifestemos que están relacionadas.

```
(%i2) diff(x*y*sin(x+y), x);
(%o2) ysin(y+x)+xycos(y+x)
```

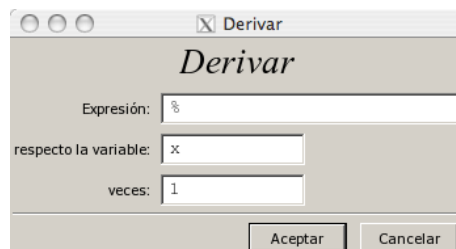


Figura 10.1 Introducir derivada

También podemos trabajar con funciones que previamente hayamos definido.

```
(%i3) f(x):= x^4+sin(x^2);
(%o3) f(x):=x^4+sin(x^2)
(%i4) diff(f(x),x);
(%o4) 2x cos(x^2)+4x^3
```

La tercera entrada de la orden `diff` nos permite calcular derivadas de orden superior. Por ejemplo, la cuarta derivada de f sería la siguiente.

```
(%i5) diff(f(x),x,4);
(%o5) 16x^4sin(x^2) - 12 sin(x^2) - 48x^2cos(x^2) + 24
```

10.1.1 Reutilizar la derivada

Las derivadas sucesivas de una función nos dan mucha información sobre la función original y con frecuencia nos hace falta utilizarlas de nuevo, ya sea para calcular puntos críticos, evaluar para estudiar monotonía o extremos relativos, etc. Es por ello que es cómodo escribir la derivada como una función. Hay varias formas en las que podemos hacerlo. Podemos, por ejemplo, utilizar la orden `define`

```
(%i6) define(g(x),diff(f(x),x));
(%o6) g(x):=2xcos(x^2)+4x^3
```

o podemos aprovechar las dobles comillas

```
(%i7) df(x):="( %o4 );
(%o7) df(x):=2xcos(x^2)+4x^3
(%i8) df(1);
(%o8) 2cos(1)+4
```

 **Observación 10.1.** Recuerda que la comilla que utilizamos para asignar a la función `df(x)` la derivada primera de f es la que aparece en la tecla `(?)`, es decir, son dos apóstrofes, `'`+`'`, y no hay que confundirla con las dobles comillas de la tecla `(2)`.

También podemos evaluar la derivada en un determinado punto sin necesidad de definir una nueva función,

```
(%i9) "(diff(f(x),x)),x=1;
(%o9) 2cos(1)+4
```

aunque esto deja de ser práctico cuando tenemos que calcular el valor en varios puntos.

El operador comilla y dobles comillas

Hasta ahora no hemos utilizado demasiado, en realidad prácticamente nada, los operadores comilla y dobles comillas. Estos operadores tienen un comportamiento muy distinto: una comilla simple hace que no se evalúe, en cambio las dobles comillas obligan a una evaluación de la expresión que le sigue. Observa cuál es la diferencia cuando aplicamos ambos operadores a una misma expresión:

```
(%i10) 'diff(f(x),x)='diff(f(x),x);
(%o10)  $\frac{d}{dx}(\sin(x^2)+x^4)=2x\cos(x^2)+4x^3$ 
```

En la parte de la izquierda tenemos la derivada sin evaluar, la expresión que hemos escrito tal cual. En la derecha tenemos la derivada calculada de la función f .

Observación 10.2. El uso de las dobles comillas para definir la derivada de una función puede dar lugar a error. Observa la siguiente secuencia de comandos: definimos la función coseno y “su derivada”, 

```
(%i11) remfunction(all)$
(%i12) f(x):=cos(x);
(%o12) f(x):=cos(x)
(%i13) g(x):=diff(f(x),x);
(%o13) g(x):=diff(f(x),x)
(%i14) h(x):='diff(f(x),x);
(%o14) h(x):=diff(f(x),x)
```

bueno, no parece que haya mucha diferencia entre usar o no las comillas. Vamos a ver cuánto valen las funciones g y h :

```
(%i15) g(x);
(%o15) -sin(x)
(%i16) h(x);
(%o16) -sin(x)
```

Parece que no hay grandes diferencias. De hecho no se ve ninguna. Vamos a ver cuánto valen en algún punto.

```
(%i17) g(1);
Non-variable 2nd argument to diff:
1
#0: g(x=1)
- an error. To debug this try debugmode(true);
```

Por fin, un error. Habíamos dicho que necesitábamos las comillas para que se evaluara la derivada. Sólo lo ha hecho cuando lo hemos definido pero, sin las dobles comillas, no vuelve a hacerlo y, por tanto, no sabe evaluar en 1. En la práctica $g(x) := -\sin(x)$ es una simple cadena de texto y no una función. Vale. Entonces, vamos con la función h :

```
(%i18) h(1);
Non-variable 2nd argument to diff:
1
#0: g(x=1)
- an error. To debug this try debugmode(true);
```

¿Qué ha pasado aquí? ¡Pero si hemos puesto las comillas dobles! ¿Qué está mal? ¿Hemos escrito mal las comillas? Repásalo y verás que no. El problema es un poco más sutil: las dobles comillas afectan a lo que tienen directamente a su derecha. En la definición de la función h no hemos escrito entre paréntesis la derivada y las dobles comillas no afectan a todo; sólo afectan al operador `diff` pero no a la x . Es por eso que no la considera una variable y tampoco se puede evaluar h en un punto. Llegados a este punto entenderás porqué hemos recomendado que utilices el comando `define` en lugar de las comillas.

10.2 Rectas secante y tangente a una función

La definición de derivada de una función real de variable real en un punto a es, como conoces bien,

$$\lim_{x \rightarrow a} \frac{f(x) - f(a)}{x - a},$$

límite que denotamos $f'(a)$.

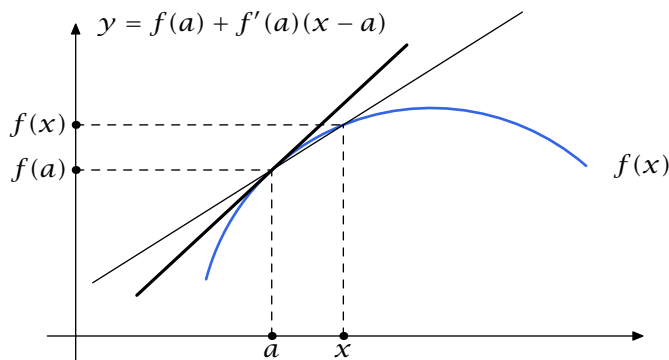


Figura 10.2 Recta tangente

En otras palabras, calculamos la recta que pasa por el punto $(a, f(a))$ y el punto $(x, f(x))$, hacemos tender x a a y, en el límite, la recta que obtenemos es la recta tangente. La pendiente de dicha recta es la derivada de f en a .

Vamos a aprovechar la orden `with_slider` para representar gráficamente este proceso en un ejemplo. Consideremos la función $f(x) = x^3 - 2x^2 - x + 2$ y su derivada, a la que notaremos df ,

```
(%i19) f(x):=x^3-2*x^2-x+2;
(%o19) f(x):=x^3-2x^2-x+2
```

```
(%i20) define(df(x),diff(f(x),x));
```

```
(%o20) df(x):=3x^2-4x-1
```

La recta tangente a una función f en un punto a es la recta $y = f(a) + f'(a)(x - a)$. La definimos.

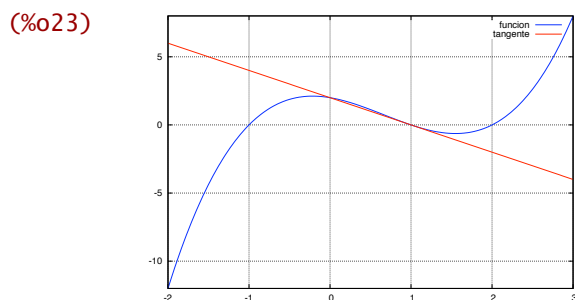
```
(%i21) tangente(x,a):=f(a)+df(a)*(x-a);
```

```
(%o21) tangente(x,a):=f(a)+df(a)*(x-a)
```

Ya podemos dibujar la función y su tangente en 1:

```
(%i22) load(draw)$
```

```
(%i23) draw2d(
    color=blue,key="función",explicit(f(x),x,-2,3),
    color=red,key="tangente",explicit(tangente(x,1),x,-2,3),
    grid=true);
```



Para dibujar la recta secante la primera cuestión es ¿cuál es la recta que pasa por $(1, f(1))$ y por $(x, f(x))$? Recordemos que la recta que pasa por un par de puntos (a, c) , (b, d) que no estén verticalmente alineados es la gráfica de la función

$$\text{recta}(x) = \frac{c - d}{a - b} x + \frac{ad - bc}{a - b}.$$

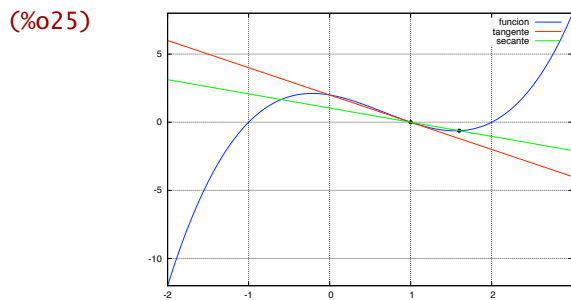
La definimos en *Maxima*:

```
(%i24) recta(x,a,c,b,d):=x*(c-d)/(a-b)+(a*d-b*c)/(a-b);
```

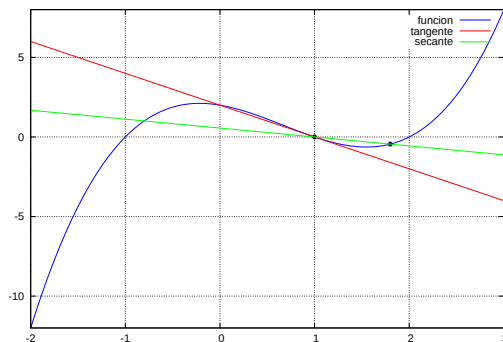
```
(%o24) recta(x,a,c,b,d):=x(c-d)/(a-b)+ad-bc/(a-b)
```

Ahora podemos ver qué ocurre con la rectas que pasan por los puntos $(1, f(1))$ y $(1+h, f(1+h))$ cuando h tiende a cero. Por ejemplo para $h = 0.6$, tendríamos

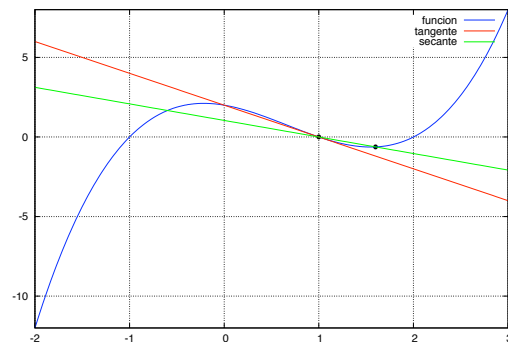
```
(%i25) draw2d(
    point_type=filled_circle,
    color=black,points([[1,f(1)]]),
    point_type=filled_circle,
    color=black,points([[1.6,f(1.6)]]),
    color=blue,key="funcion",explicit(f(x),x,-2,3),
    color=red,key="tangente",explicit(tangente(x,1),x,-2,3),
    color=green,key="secante",
    explicit(recta(x,1,f(1),1+0.6,f(1.6)),x,-2,3),
    grid=true)$
```



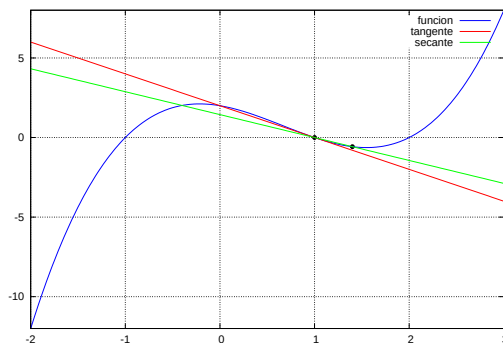
Si te fijas, en la gráfica anterior hemos añadido también el par de puntos que definen la recta secante.



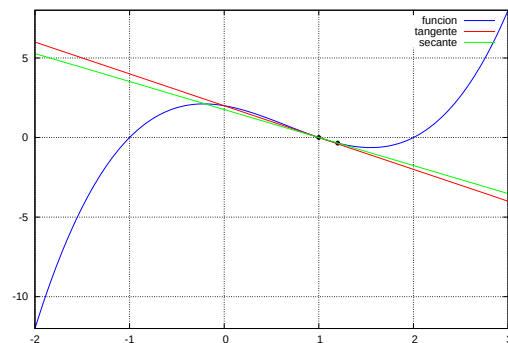
$h = 0.8$



$h = 0.6$



$h = 0.4$



$h = 0.2$

Figura 10.3 Rectas secantes y tangente

En la Figura 10.3 puedes ver el resultado para $h = 0.2, 0.4, 0.6$, y 0.8 .

Podemos unir todo lo que hemos hecho y usar la capacidad de *wxMaxima* para representar gráficos en función de un parámetro. Dibujemos la función, la recta tangente y las secantes para $h = 0.1, 0.2, \dots, 2$:

```
(%i26) with_slider(
      n,0.1*reverse(makelist(i,i,1,20)),
      [f(x),tangente(x,1),recta(x,1,f(1),1+n,f(1+n))],
      [x,-2,3]);
```

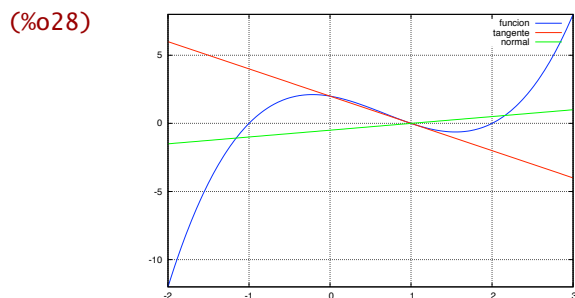
10.2.1 Recta normal

La recta normal es la recta perpendicular a la recta tangente. Su pendiente es $-1/f'(a)$ y, por tanto, tiene como ecuación

$$y = f(a) - \frac{1}{f'(a)}(x - a).$$

Con todo lo que ya tenemos hecho es muy fácil dibujarla.

```
(%i27) normal(x,a):=f(a)-df(a)^(-1)*(x-a)$
(%i28) draw2d(
      point_type=filled_circle,color=black,points([[1,f(1)]]),
      color=blue,key="funcion",explicit(f(x),x,-2,3),
      color=red,key="tangente",explicit(tangente(x,1),x,-2,3),
      color=green,key="normal",explicit(normal(x,1),x,-2,3),
      grid=true)$
```



En la figura anterior, la recta normal no parece perpendicular a la recta tangente. Eso es por que no hemos tenido en cuenta la escala a la que se dibujan los ejes. Prueba a cambiar la escala para que queden perpendiculares.

Prueba a cambiar la función f , el punto a . ¡Ahora es tu turno! Por ejemplo, haz esto mismo para una función que no sea derivable como $f(x) = \sqrt{|x|}$ en el origen.

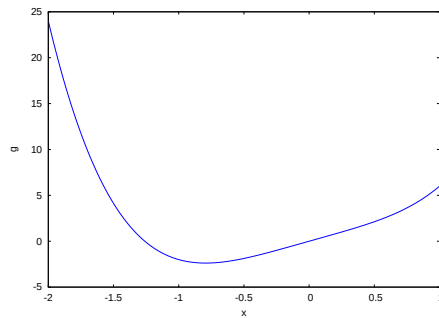
10.3 Extremos relativos

En esta sección vamos a aprender a localizar extremos relativos de una función f . Para ello encontraremos las soluciones de la ecuación de punto crítico: $f'(x) = 0$. Y para resolver dicha ecuación podemos usar el comando `solve`.

Ejemplo 10.3. Calculemos los extremos relativos de la función $f(x) = 2x^4 + 4x$, para cualquier $x \in \mathbb{R}$.

Comenzamos, entonces, presentándosela al programa (no olvides borrar de la memoria la anterior función f) y pintando su gráfica para hacernos una idea de dónde pueden estar sus extremos.

```
(%i29) f(x):=2*x^4+4*x$
(%i30) plot2d(f(x),[x,-2,1]);
(%o30)
```



Parece que hay un mínimo en las proximidades de -1. Para confirmarlo, calculamos los puntos críticos de f .

```
(%i31) define(d1f(x),diff(f(x),x))$
(%i32) puntosf:solve(d1f(x),x);
(%o32) [x=(-sqrt(3)*%i-1)/(2*2^(1/3)),x=(sqrt(3)*%i+1)/(2*2^(1/3)),
x=-1/2^(1/3)]
```

Observamos que hay sólo una raíz real que es la única que nos interesa.

```
(%i33) last(puntosf);
(%o33) x=-1/2^(1/3)
```

Llamamos a al único punto crítico que hemos obtenido, y evaluamos en él la segunda derivada:

```
(%i34) a:rhs(%)$
(%i35) define(d2f(x),diff(f(x),x,2))$
(%i36) d2f(a);
(%o36) 24/2^(2/3)
```

Por tanto la función f tiene un mínimo relativo en dicho punto a . ¿Puede haber otro extremo más? ¿Cómo podemos asegurarnos?

Ejemplo 10.4. Vamos a calcular los extremos relativos de la función:

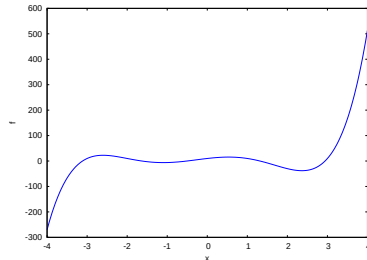
$$f(x) = x^5 + x^4 - 11x^3 - 9x^2 + 18x + 10 \text{ en el intervalo } [-4, 4]$$

Procedemos de la misma forma que en el ejemplo anterior.

```
(%i37) f(x):=x^5+x^4-11*x^3-9*x^2+18*x+10$
```

```
(%i38) plot2d(f(x),[x,-4,4])$
```

```
(%o38)
```



A simple vista observamos que hay:

- a) un máximo relativo entre -3 y -2,
- b) un mínimo relativo entre -2 y -1,
- c) un máximo relativo entre 0 y 1, y
- d) un mínimo relativo entre 2 y 3.

Vamos entonces a derivar la función e intentamos calcular los ceros de f' haciendo uso del comando `solve`.

```
(%i39) define(d1f(x),diff(f(x),x));
```

```
(%o39) d1f(x):=5x^4+4x^3-33x^2-18x+18
```

```
(%i40) solve(d1f(x),x);
```

Las soluciones que nos da el programa no son nada manejables, así que vamos a resolver la ecuación $f'(x) = 0$ de forma numérica, esto es, con el comando `find_root`. Para ello, nos apoyamos en la gráfica que hemos calculado más arriba, puesto que para resolver numéricamente esta ecuación hay que dar un intervalo en el que se puede encontrar la posible solución. Vamos, entonces, a obtener la lista de puntos críticos que tiene la función.

```
(%i41) puntosf:[find_root(d1f(x),x,-3,-2),
                 find_root(d1f(x),x,-2,-1), find_root(d1f(x),x,0,1),
                 find_root(d1f(x),x,2,3)];
```

```
(%o41) [-2.600821117505113,-1.096856508567827,0.53386135374308,
        2.363816272329865]
```

Ahora, para decidir si en estos puntos críticos se alcanza máximo o mínimo relativo, vamos a aplicar el test de la segunda derivada. Esto es:

```
(%i42) define(d2f(x),diff(f(x),x,2));
```

```
(%o42) d2f(x):=20x^3+12x^2-66x-18
(%i43) map(d2f,puntosf);
(%o43) [-117.0277108731797,42.43722588782084,-46.77165945967596,
157.2021444450349]
```

Lo que nos dice que en el primer y tercer puntos hay máximos relativos, mientras que en los otros dos tenemos mínimos relativos.

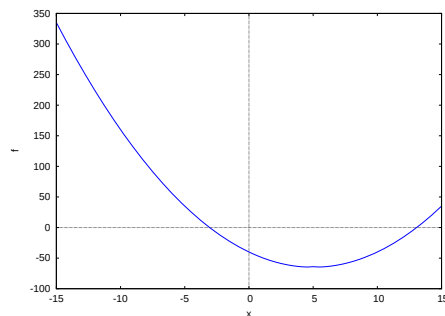
A continuación veremos un caso en el que la gráfica diseñada por el programa nos puede llevar a engaño. Descubriremos el error gracias al “test de la segunda derivada”.

Ejemplo 10.5. Vamos a estudiar los extremos relativos de la función

$$f(x) = x^2 - 10x - 40 + \frac{1}{10x^2 - 100x + 251}.$$

Comenzamos dibujando su gráfica en el intervalo $[-15, 15]$

```
(%i44) f(x):=x^2-10*x-40+1/(10*x^2-100*x+251)$
(%i45) plot2d(f(x),[x,-15,15]);
(%o45)
```



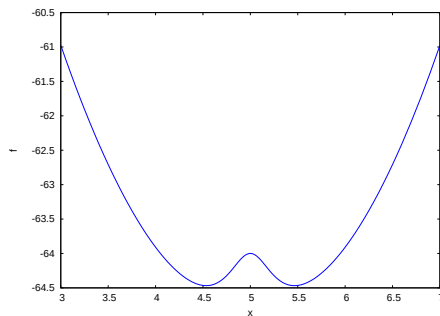
Aparentemente hay un mínimo cerca de 5. De hecho, si aplicamos el comando `solve` a la primera derivada de f obtenemos que precisamente la derivada de f se anula en $x = 5$. A la vista de la gráfica parece que efectivamente en $x = 5$ hay un mínimo de la función f . Pero nada más lejos de la realidad, porque si evaluamos la derivada segunda de f en 5 obtendremos que $f''(5) = -18 < 0$. La segunda derivada es negativa y por tanto en $x = 5$ tiene que haber un máximo. En efecto:

```
(%i46) define(d1f(x),diff(f(x),x))$
(%i47) d1f(5);
(%o47) 0
(%i48) define(d2f(x),diff(f(x),x,2))$
(%i49) d2f(5);
(%o49) -18
```

De hecho, si hubiéramos dibujado la gráfica en un intervalo más pequeño, sí se hubiera apreciado el máximo en el punto 5.


```
(%i50) plot2d(f(x),[x,3,7]);
```

```
(%o50)
```



En resumen, ten cuidado con los resultados que obtengas y, siempre, comprueba si se ajustan a los contenidos teóricos que has visto en clase.

En el siguiente ejemplo, resolvemos un problema de optimización.

Ejemplo 10.6. Se desea construir una ventana con forma de rectángulo coronado de un semicírculo de diámetro igual a la base del rectángulo. Pondremos cristal blanco en la parte rectangular y cristal de color en el semicírculo. Sabiendo que el cristal coloreado deja pasar la mitad de luz (por unidad de superficie) que el blanco, calcular las dimensiones para conseguir la máxima luminosidad si se ha de mantener el perímetro constante dado.

Llamemos x a la longitud de la base de la ventana y h a su altura. El perímetro es una cantidad dada A ; es decir, $x + 2h + \frac{\pi x}{2} = A$. Despejamos h en función de x :

```
(%i51) altura:solve(x+2*h+(%pi*x)/2=A,h)$
```

```
(%o51) [h= (2 A+(-pi-2) x)/4]
```

La luminosidad viene dada por

$$f(x) = 2xh + \pi \frac{x^2}{8} = x(A - x - \pi \frac{x}{2}) + \pi \frac{x^2}{8} = Ax - \frac{1}{8}(8 + 3\pi)x^2$$

Definimos entonces la función f y calculamos sus puntos críticos:

```
(%i52) f(x):=A*x-(1/8)*(8+3*%pi)*x^2$
```

```
(%i53) define(d1f(x),diff(f(x),x))$
```

```
(%i54) solve(diff(f(x),x),x);
```

```
(%o54) [x= 4A/(3pi+8)]
```

```
(%i55) %[1];
```

```
(%o55) x= 4A/(3pi+8)
```

Y ahora evaluaremos la segunda derivada en dicho punto crítico. Para ello:

```
(%i56) a:rhs(%)$
```

```
(%i57) define(d2f(x),diff(f(x),x,2))$
(%i58) d2f(a);
(%o58)  $-\frac{3\pi+8}{4}$ 
```

La segunda derivada es negativa: ya tenemos el máximo que estábamos buscando.

Por último, veamos cómo podemos verificar una desigualdad haciendo uso de técnicas de derivación.

Ejemplo 10.7. Se nos plantea demostrar la siguiente desigualdad: $\log(x+1) \geq \frac{x}{x+1}$, para cualquier $x \geq -1$. Estudiemos entonces la siguiente función: $f(x) = \log(x+1) - \frac{x}{x+1}$. Tendremos que comprobar que dicha función es siempre positiva. Para esto es suficiente comprobar que, si la función f tiene mínimo absoluto, su valor es mayor o igual que cero. En efecto:

```
(%i59) f(x):=log(x+1)-x/(x+1)$
(%i60) define(d1f(x),diff(f(x),x))$
(%i61) solve(d1f(x),x);
(%o61) [x=0]
```

Por tanto sólo tiene un punto crítico, y además:

```
(%i62) define(d2f(x),diff(f(x),x,2))$
(%i63) d2f(0);
(%o63) 1
```

Luego, en el punto 0 alcanza un mínimo relativo que, por ser único, es el mínimo absoluto. Como se verifica que $f(0) = 0$, la desigualdad es cierta.

10.4 Ejercicios

Ejercicio 10.1.

- Calcula las tangentes a la hipérbola $xy = 1$ que pasan por el punto $(-1, 1)$. Haz una representación gráfica.
- Calcula las tangentes a la gráfica de $y = 2x^3 + 13x^2 + 5x + 9$ que pasan por el origen. Haz una representación gráfica.

Ejercicio 10.2. Dibuja la semicircunferencia superior centrada en el origen y de radio 2. Dibuja el punto en el que la recta tangente a dicha circunferencia pasa por el punto $(3, 7)$, así como dicha recta tangente.

Ejercicio 10.3. Haz una animación gráfica en la que se represente la gráfica de la parábola $f(x) = 6 - (x - 3)^2$ en azul, la recta tangente en el punto $(1, f(1))$ en negro y las rectas secantes que pasan por los puntos $(1, f(1))$ y $(1 - h, f(1 - h))$ donde h varía de -3 a -0.2 con incrementos iguales a 0.2 , en rojo. Utiliza las opciones que consideres más apropiadas para hacer la animación.

Ejercicio 10.4. Representa en una misma gráfica la parábola $y = x^2/4$ y sus rectas tangentes en los puntos de abscisas $-3 + 6k/30$ para $1 \leq k \leq 29$. Utiliza las opciones que consideres más apropiadas.

Ejercicio 10.5. Sea $f: \mathbb{R} \rightarrow \mathbb{R}$ la función definida como

$$f(x) = \left(\frac{x+1}{x}\right) \exp\left(-\frac{x+1}{x^2}\right), \quad \text{si } x \neq 0, \quad f(0) = 0.$$

Calcula la imagen de la función f . Para ello se recomienda representar su gráfica, estudiar puntos críticos y extremos relativos y límites en infinito.

Ejercicio 10.6. Halla dos números no negativos tales que la suma de uno más el doble del otro sea 12 y su producto sea máximo.

Ejercicio 10.7. Un móvil tiene un movimiento rectilíneo con desplazamiento descrito por la función

$$s(t) = t^4 - 2t^3 - 12t^2 + 60t - 10.$$

¿En qué instante del intervalo $[0, 3]$ alcanza su máxima velocidad?

Integración

11

11.1 Cálculo de integrales 163 11.2 Sumas de Riemann 169 11.3 Aplicaciones 172 11.4 Ejercicios 179

En este capítulo vamos a aprender a calcular integrales en una variable, así como aplicar estas integrales al cálculo de áreas de recintos en el plano limitados por varias curvas, longitudes de curvas, volúmenes de cuerpos de revolución y áreas de superficies de revolución.

11.1 Cálculo de integrales

La principal orden de *Maxima* para calcular integrales es `integrate`. Nos va a permitir calcular integrales, tanto definidas como indefinidas, con mucha comodidad.

<code>integrate(f(x), x)</code>	primitiva de la función $f(x)$
<code>integrate(f(x), x, a, b)</code>	$\int_a^b f(x) dx$

Disponemos también de la siguiente opción en el menú para calcular integrales, sin necesidad de escribir el comando correspondiente en la ventana de entradas: **Análisis** → **Integrar**. Después sólo tenemos que rellenar los datos que nos interesen.



Figura 11.1 Cálculo de integrales

Comencemos por integrales indefinidas. *Maxima* calcula primitivas de funciones trigonométricas,

```
(%i1) integrate(x*sin(x), x);
(%o1) sin(x)-x*cos(x)
```

de funciones racionales,

```
(%i2) integrate(1/(x^4-1),x);
```

$$(\%o2) \quad -\frac{\log(x+1)}{4} - \frac{\operatorname{atan}(x)}{2} + \frac{\log(x-1)}{4}$$

irracionales,

```
(%i3) integrate(sqrt(x^2+1),x);
```

$$(\%o3) \quad \frac{\operatorname{asinh}(x)}{2} + \frac{x\sqrt{x^2+1}}{2}$$

ya sea aplicando integración por partes

```
(%i4) integrate(x^3*%e^x,x);
```

$$(\%o4) \quad (x^3 - 3x^2 + 6x - 6) e^x$$

o el método que considere necesario

```
(%i5) integrate(%e^(-x^2),x);
```

$$(\%o5) \quad \frac{\sqrt{\pi} \operatorname{erf}(x)}{2}$$

integral que no sabíamos hacer. Bueno, como puedes ver, *Maxima* se defiende bien con integrales. Eso sí, es posible que nos aparezcan funciones (como erf). El motivo es muy sencillo: la forma de saber calcular primitivas de muchas funciones es saberse muchas funciones. *Maxima* se sabe muchas y, cuando nos aparezca alguna nueva, siempre podemos preguntar cuál es.

```
(%i6) describe(erf); - Función: erf (<x>)
Es la función de error, cuya derivada
es '2*exp(-x^2)/sqrt(%pi)'.
There are also some inexact matches for 'erf'.
Try '?? erf' to see them.
```

$$(\%o6) \quad \text{true}$$

Para calcular integrales definidas sólo tenemos que añadir los extremos del intervalo de integración

```
(%i7) integrate(x*sin(x),x,%pi/2,%pi);
```

$$(\%o7) \quad \pi - 1$$

```
(%i8) integrate(1/(x^2-1),x,2,5);
```

$$(\%o8) \quad -\frac{\log(6)}{2} + \frac{\log(4)}{2} + \frac{\log(3)}{2}$$

En la integrales pueden aparecer parámetros y si *Maxima* tiene dudas acerca de su valor pregunta,

```
(%i9) integrate(x^n,x);
      Is n+1 zero or nonzero? nonzero;
(%o9)  $\frac{x^{n+1}}{n+1}$ 
```

pregunta todo lo que haga falta:

```
(%i10) integrate(1/x,x,a,b);
      Is b - a positive, negative, or zero? positive;
      Is b positive, negative, or zero? positive;
      Is a positive, negative, or zero? positive;
      Is x + b positive or negative? positive;
(%o10)  $\log(b) - \log(a)$ 
```

Evidentemente hay funciones a las que *Maxima* no sabe calcular una integral indefinida. En ese caso, da como respuesta la misma integral que le hemos preguntado.

```
(%i11) integrate(exp(x^3+x),x);
(%o11)  $\int e^{x^3+x} dx$ 
```

Veremos que este problema desaparece cuando pasamos a integrales definidas.

11.1.1 Integración impropia

Como recordarás de las clases teóricas, la integral que, en principio, se define para funciones continuas en intervalos compactos, puede extenderse a funciones continuas definidas en intervalos de longitud infinita y a funciones que no están acotadas en un intervalo de longitud finita. Es lo que se conoce como *integración impropia*. Para el programa *Maxima*, trabajar con integrales impropias no supone ningún problema ya que las trata exactamente igual que las integrales de funciones continuas en intervalos cerrados y acotados. De hecho, no hay forma de distinguir unas de otras: en ningún momento hemos indicado al comando `integrate` que la función era acotada ni hemos dicho si el intervalo era abierto, cerrado o semiabierto. Sólo hemos indicado los extremos y ya está. Por tanto, con la misma orden `integrate` se pueden calcular integrales impropias. Por ejemplo:

```
(%i12) integrate(1/sqrt(1-x^2),x,-1,1);
(%o12)  $\pi$ 
(%i13) integrate(%e(-x^2),x,0,inf);
(%o13)  $\frac{\sqrt{\pi}}{2}$ 
```

Intenta calcular una primitiva de los integrandos anteriores. Como verás, la primitiva de la función $f(x) = \frac{1}{\sqrt{1-x^2}}$ ya la conocías; en cambio, en la primitiva de la función $f(x) = \exp(-x^2)$ aparece una función que seguro que no conocías, pero que ha aparecido en el apartado dedicado al cálculo de integrales. Probamos a hacer las siguientes integrales impropias:

$$\int_1^{+\infty} \frac{1}{x^2} dx, \quad \int_1^{+\infty} \frac{1}{x} dx$$

```
(%i14) integrate(1/x^2,x,1,inf);
(%o14) 1
(%i15) integrate(1/x,x,0,inf);
Integral is divergent
- an error. To debug this try debugmode(true);
```

Observa cómo en la salida de la última integral el programa advierte de la “no convergencia” de la integral planteada. ¿Sabéis decir el por qué? Bueno, recordemos que la definición de dicha integral es

$$\int_1^{+\infty} \frac{1}{x} dx = \lim_{x \rightarrow +\infty} G(x) - \lim_{x \rightarrow 1} G(x),$$

donde G es una primitiva de $\frac{1}{x}$. En este caso, una primitiva es $\log(x)$ que no tiene límite en $+\infty$ (es $+\infty$).

Observación 11.1. Hay que tener cierto cuidado con la integración de funciones que no son continuas. Por ejemplo, la función $f(x) = \frac{1}{x}$ no es impropia integrable en $[-1, 1]$. Si así fuera, también lo sería en el intervalo $[0, 1]$ y esto es claramente falso. Dependiendo de las versiones, la respuesta de *Maxima* puede variar, pero siempre involucra el llamado *valor principal* de la integral. Por ejemplo, con la versión 5.17 de *Maxima* el resultado es el siguiente

```
(%i16) integrate(1/x,x,-1,1);
Principal value
(%o16) log(-1)+%i*pi
```

11.1.2 Integración numérica

<code>quad_quags(f(x),x,a,b)</code>	aproximación numérica de $\int_a^b f(x) dx$
<code>quad_qagi(f(x),x,a,b)</code>	aproximación numérica de $\int_a^b f(x) dx$
<code>romberg(f(x),x,a,b)</code>	aproximación numérica de $\int_a^b f(x) dx$

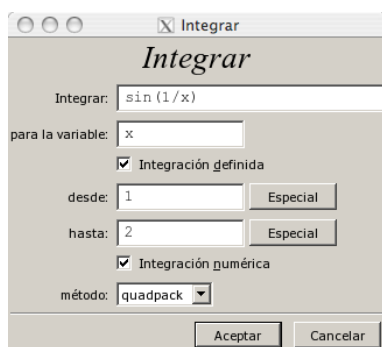


Figura 11.2

El cálculo de primitivas no es interesante por sí mismo, el motivo que lo hace interesante es la regla de Barrow: si conoces una primitiva de una función, entonces el cálculo de la integral en un intervalo se reduce a evaluar la primitiva en los extremos y restar.

En la práctica, no siempre es fácil calcular una primitiva, pero sí es fácil (para un ordenador y teóricamente) aproximar el valor de la integral por las áreas de los rectángulos que aparecen en la definición de integral. Este método no es el mejor, pero versiones mejoradas permiten aproximar el valor de casi cualquier integral.

Si en el menú **Análisis** → **Integrar** marcamos “Integración definida” e “Integración numérica” se nos da la opción de escoger entre dos métodos: quadpack y

romberg, cada uno referido a un tipo diferente de aproximación numérica de la integral. Vamos a calcular numéricamente la integral que hemos hecho más arriba que tenía como resultado $\pi - 1$

```
(%i17) quad_qags(x*sin(x),x,%pi/2,%pi);
(%o17) [2.141592653589794,2.3788064330872769 10-14,21,0]
(%i18) romberg(x*sin(x),x,%pi/2,%pi);
(%o18) 2.141591640806944
```

Observamos que hay una diferencia entre las salidas de ambos comandos. Mientras que en la última aparece el valor aproximado de la integral (fíjate que es $\pi - 1$), en la primera aparece como salida una lista de 4 valores:

- la aproximación de la integral
- el error absoluto estimado de la aproximación
- el número de evaluaciones del integrando
- el código de error (que puede ir desde 0 hasta 6) que, copiado de la ayuda de *Maxima*, significa

El código de error (el cuarto elemento del resultado) puede tener los siguientes valores:

- '0' si no ha habido problemas;
- '1' si se utilizaron demasiados intervalos;
- '2' si se encontró un número excesivo de errores de redondeo;
- '3' si el integrando ha tenido un comportamiento extraño frente a la integración;
- '4' fallo de convergencia;
- '5' la integral es probablemente divergente o de convergencia lenta;
- '6' si los argumentos de entrada no son válidos.

Qué duda cabe que este tipo de integración numérica es interesante para integrandos de los cuales no se conoce una primitiva. Vamos a intentar calcular una primitiva y posteriormente una integral definida de la función $f(x) = e^{-x^2}$.

```
(%i19) integrate(exp(-x^2),x);
(%o19)  $\frac{\sqrt{\pi} \operatorname{erf}(x)}{2}$ 
(%i20) integrate(exp(-x^2),x,0,1);
(%o20)  $\frac{\sqrt{\pi} \operatorname{erf}(1)}{2}$ 
```

Si ahora queremos, podemos calcular numéricamente este valor,

```
(%i21) float(%);
(%o21) 0.74682413281243
```

o también, podíamos haber usado los comandos de integración numérica directamente

```
(%i22) quad_qags(exp(-x^2),x,0,1);
(%o22) [0.74682413281243,8.2954620176770253 10-15,21,0]
(%i23) romberg(exp(-x^2),x,0,1);
(%o23) 0.7468241699099
```

quadpack En realidad, quadpack no es un método concreto sino una serie de métodos para aproximar numéricamente la integral. Los dos que hemos visto antes, quad_qags y romberg, se pueden utilizar en intervalos finitos. Comprueba tú mismo lo que ocurre cuando calculas $\int_1^\infty \cos(x)/x^2 dx$ utilizando el menú **Análisis**→**Integrar**: obtendrás algo así

```
(%i24) quad_qagi(cos(x)/x^2, x, 1, inf);
* * *MESSAGE FROM ROUTINE DQAGI IN LIBRARY SLATEC.
* * *INFORMATIVE MESSAGE, PROG CONTINUES, TRACEBACK REQUESTED
* ABNORMAL RETURN
* ERROR NUMBER = 1
*
* * *END OF MESSAGE
(%o24) [-0.084302101159614,1.5565565173632223 10-4,5985,1]
```

Maxima decide cuál es el método que mejor le parece y, en este caso utiliza quad_qagi. Evidentemente, depende del integrando el resultado puede dar uno u otro tipo de error. Por ejemplo, vamos ahora a calcular de forma numérica $\int_0^1 \frac{\sin(1/x)}{x}$.

```
(%i25) quad_qags((1/x)*sin(1/x),x,0,1);
* * *MESSAGE FROM ROUTINE DQAGS IN LIBRARY SLATEC.
* * *INFORMATIVE MESSAGE, PROG CONTINUES, TRACEBACK REQUESTED
* ABNORMAL RETURN
* ERROR NUMBER = 5
*
* * *END OF MESSAGE
(%o25) [-1.050233246377689,0.20398634967385,8379,5]
(%i26) romberg((1/x)*sin(1/x),x,0,1);
(%o26) romberg( $\frac{\sin(\frac{1}{x})}{x}$ ,x,0.0,1.0)
```

Se trata de una función que cerca del cero oscila mucho, lo que hace que el comando `quad-qags` nos dé aviso de error (aunque da una aproximación), mientras que el comando `romberg` no nos dé ninguna salida.

11.2 Sumas de Riemann

El proceso que hemos seguido en clase para la construcción de la integral de una función $f: [a, b] \rightarrow \mathbb{R}$ pasa por lo que hemos llamado sumas superiores y sumas inferiores. Lo que hacíamos era dividir el intervalo $[a, b]$ en n trozos usando una partición $P = \{a = x_0 < x_1 < \dots < x_n = b\}$ del intervalo. Luego calculábamos el supremo y el ínfimo de f en cada uno de los intervalos $I_i = [x_{i-1}, x_i]$ y podíamos aproximar la integral de dos formas: si escogíamos el área asociada a los rectángulos que tienen como altura el supremo obteníamos las sumas superiores. Las sumas inferiores se obtenían de forma análoga pero con el ínfimo.

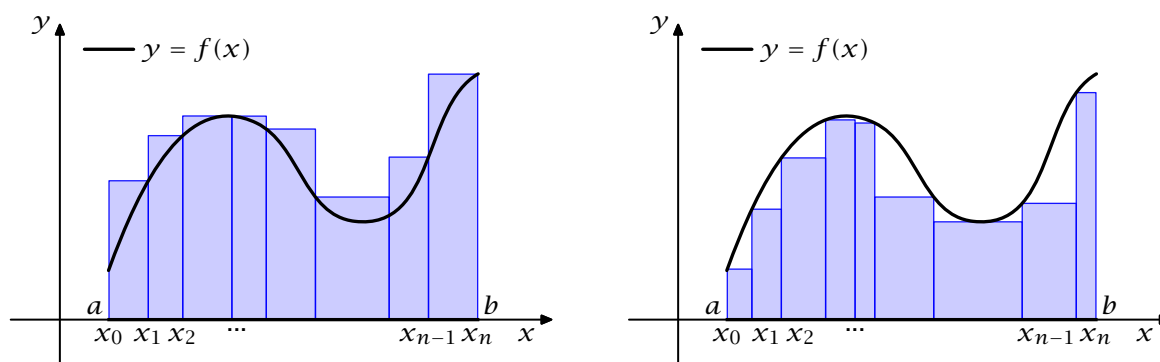


Figura 11.3 Sumas superiores e inferiores

Las dos áreas que tienes marcadas en la Figura 11.3 son

$$S(f, P) = \sum_{i=1}^n \sup (f(I_i)) \cdot \ell(I_i) \text{ y } I(f, P) = \sum_{i=1}^n \inf (f(I_i)) \cdot \ell(I_i),$$

donde $\ell(I_i)$ denota la longitud del intervalo I_i o, lo que es lo mismo, la base de cada uno de los rectángulos que tenemos en la figura.

Si coinciden las mejores aproximaciones obtenidas de esta manera o hablando un poco a la ligera, si la menor de las sumas superiores coincide con la mayor de las sumas inferiores entonces llamamos a ese valor integral de la función.

Este método tiene varios inconvenientes si pretendemos hacerlo con un ordenador. Por ejemplo, ¿cómo escogemos la partición? y, más importante, ¿cómo calculamos el supremo y el ínfimo de la función en cada uno de los intervalos? El primer problema se puede evitar tomando la partición de una forma particular: dividimos el intervalo $[a, b]$ en trozos iguales, en muchos trozos iguales. Esto nos permite dar una expresión concreta de la partición.

Si dividimos $[a, b]$ en dos trozos, estarás de acuerdo que los puntos serían a , b y el punto medio $\frac{a+b}{2}$. ¿Cuáles serían los puntos que nos dividan $[a, b]$ en tres trozos? Ahora no hay punto medio, ahora nos hacen falta dos puntos además de a y b . Piénsalo un poco. En lugar de fijarnos en cuáles son los extremos de los intervalos, fijémonos en cuánto miden. Efectivamente, todos miden lo mismo $\frac{b-a}{3}$ puesto que queremos dividir el intervalo en tres trozos iguales. Ahora la pregunta es: ¿qué intervalo tiene como extremos de la izquierda a y mide $\frac{b-a}{3}$? La respuesta es muy fácil: $[a, a + \frac{b-a}{3}]$. Con este método es sencillo seguir escribiendo intervalos: sólo tenemos que seguir sumando $\frac{b-a}{3}$ hasta llegar al extremo de la derecha, b .

En general, si n es un natural mayor o igual que 2,

$$a, a + \frac{b-a}{n}, a + 2 \cdot \frac{b-a}{n}, \dots, a + n \cdot \frac{b-a}{n}$$

son los $n + 1$ extremos que dividen a $[a, b]$ en n intervalos iguales. Vamos a seguir estos pasos con *Maxima*. Escojamos una función y un intervalo:

```
(%i27) a:-2;b:5;n:10;
(%o27) -2
(%o28) 5
(%o29) 10
(%i30) f(x):=(x-1)*x*(x-3);
(%o30) f(x):=(x-1)x(x-3)
```

La lista de extremos es fácil de escribir usando el comando `makelist`.

```
(%i31) makelist(a+i*(b-a)/n,i,0,n);
(%o31) [-2,-13/10,-3/5,1/10,4/5,3/2,11/5,29/10,18/5,43/10,5]
```

La segunda dificultad que hemos mencionado es la dificultad de calcular supremos e ínfimos. La solución es escoger un punto cualquiera de cada uno de los intervalos I_i : el extremo de la izquierda, el de la derecha, el punto medio o, simplemente, un punto elegido al azar. Esto tiene nombre: una suma de Riemann. Si dividimos el intervalo en muchos trocitos, no debería haber demasiada diferencia entre el supremo, el ínfimo o un punto intermedio.

Convergencia de las sumas de Riemann **Proposición 11.2.** Sea $f:[a, b] \rightarrow \mathbb{R}$ integrable. Sea P_n la partición

$$a, a + \frac{b-a}{n}, a + 2 \cdot \frac{b-a}{n}, \dots, a + n \cdot \frac{b-a}{n}$$

y sea $x_i \in \left[a + (i-1) \frac{b-a}{n}, a + i \frac{b-a}{n} \right]$. Entonces

$$\lim_{n \rightarrow \infty} \sum_{i=1}^n f(x_i) \frac{b-a}{n} = \int_a^b f(x) dx.$$

En la Figura 11.4 podemos ver cómo mejora cuando aumentamos el número de subdivisiones. Buenos, sigamos adelante. Elegimos puntos al azar en cada uno de los intervalos.

```
(%i32) puntos:makelist(a+(i-1)*(b-a)/n+random(1.0)*(b-a)/n,i,1,n);
[-1.974524678870055,-0.86229173716074,0.095612020508637,
(%o32) 0.12911247230811,1.242837772556621,1.915206527061883,
2.658260436280155,3.229497471823565,3.636346487565163,
4.30162951626272]
```

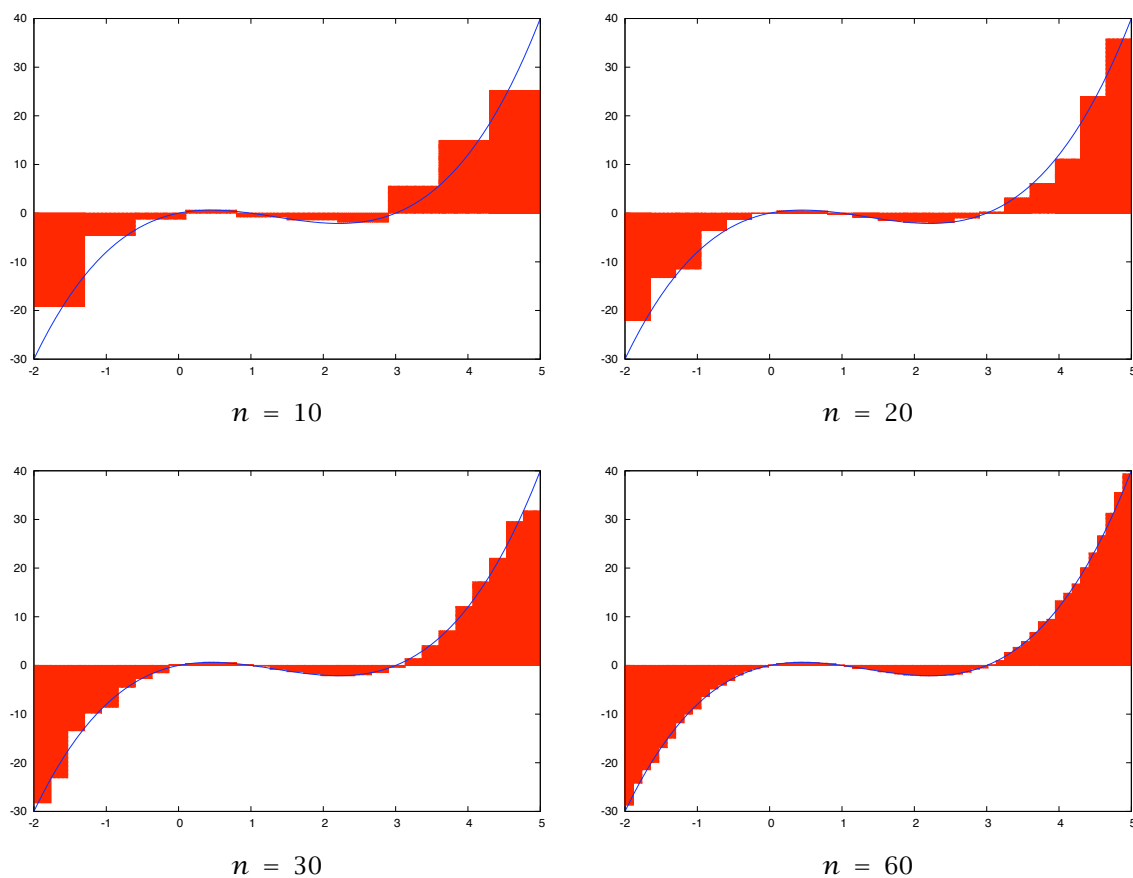


Figura 11.4 Convergencia de las sumas de Riemann

Ahora tenemos que sumar las áreas de los rectángulos de las sumas de Riemann, cosa que podemos hacer de la siguiente manera usando la orden `sum`:

```
(%i33) sum((b-a)/n*(f(a+(i-1)*(b-a)/n+random(1.0)*(b-a)/n)),i,1,n);
(%o33) 5.859818716400653
```

que si lo comparamos con el valor de dicha integral

```
(%i34) integrate(f(x),x,a,b);
(%o34) 77
12
(%i35) %,numer
(%o35) 6.416666666666667
```

...pues no se parece demasiado. Bueno, aumentemos el número de intervalos:

```
(%i36) n:100;
(%o36) 100
```

```
(%i37) sum((b-a)/n*(f(a+(i-1)*(b-a)/n+random(1.0)*(b-a)/n)),i,1,n);
(%o37) 6.538846969978081
```

Vale, esto es otra cosa. Ten en cuenta que, debido al uso de `random`, cada vez que ejecutes la orden obtendrás un resultado diferente y, por supuesto, que dependiendo de la función puede ser necesario dividir en una cantidad alta de intervalos.

11.3 Aplicaciones

11.3.1 Cálculo de áreas planas

Si f y g son funciones integrables definidas en un intervalo I , el área entre las dos funciones es

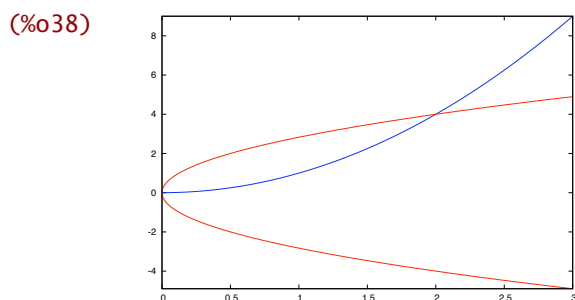
$$\int_I |f(x) - g(x)| dx$$

Evidentemente, calcular una integral con un valor absoluto no es fácil (¿cuál es la primitiva de la función $|x|$?). Como función definida a trozos, lo que tenemos que hacer es dividir el intervalo $[a, b]$ en subintervalos en los que sepamos cuál de las dos funciones f y g es la más grande.

Ejemplo 11.3. Calculemos el área entre las curvas $y = x^2$, $y^2 = 8x$.

Podemos dibujar las dos curvas y hacernos una idea del aspecto del área que queremos calcular.

```
(%i38) draw2d(
color=blue,
explicit(x^2,x,0,3),
color=red,
explicit(sqrt(8*x),x,0,3),
explicit(-sqrt(8*x),x,0,3)
)
```



En realidad, lo primero que nos hace falta averiguar son los puntos de corte:

```
(%i39) solve([y=x^2,y^2=8*x],[x,y]);
(%o39) [[x=2,y=4],[x=-sqrt(3)*i-1,y=2*sqrt(3)*i-2],[x=sqrt(3)*i-1,y=-2*sqrt(3)*i-2],
[x=0,y=0]]
```

De todas las soluciones, nos quedamos con las soluciones reales: $(0,0)$ y $(2,4)$. En el intervalo $[0, 2]$, ¿cuál de las dos funciones es mayor? Son dos funciones continuas en un intervalo que sólo

coinciden en 0 y en 2, por tanto el Teorema de los ceros de Bolzano nos dice que una de ellas es siempre mayor que la otra. Para averiguarlo sólo tenemos que evaluar en algún punto entre 0 y 2. En este caso es más fácil: se ve claramente que $y = x^2$ está por debajo. Por tanto el área es

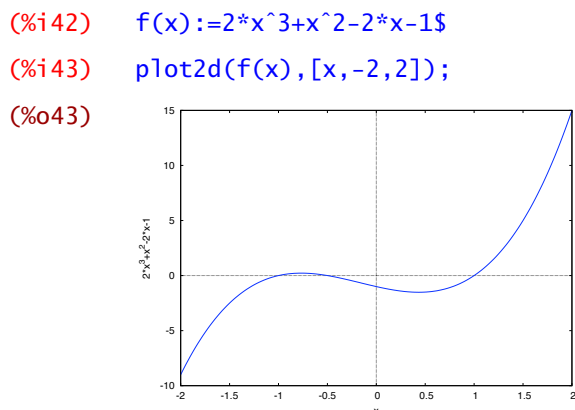
```
(%i40) integrate(sqrt(8*x)-x^2,x,0,2);
(%o40) 8/3
```

Observación 11.4. La fórmula que define el cálculo del área entre dos funciones tiene un valor absoluto que hace difícil calcular una primitiva directamente. Ese es el motivo por el que dividimos el intervalo en trozos: quitar ese valor absoluto. Si sólo pretendemos calcular el valor numéricamente, entonces el valor absoluto no es un impedimento y podemos calcular directamente la integral olvidándonos de puntos de corte o de qué función es mayor:

```
(%i41) quad_qags(abs(x^2-sqrt(8*x)),x,0,2);
(%o41) [2.666666666666668,2.9605947323337525 10^-15,189,0]
```

No siempre es tan evidente qué función es mayor. En esos casos la continuidad nos permite utilizar el Teorema de los ceros de Bolzano para averiguarlo.

Ejemplo 11.5. Calculemos el área entre la función $f(x) = 2x^3 + x^2 - 2x - 1$ y el eje OX. La función es un polinomio de grado 3 que puede, por tanto, tener hasta tres raíces reales. Si le echamos un vistazo a su gráfica



se ve que tiene tres raíces. Pero ¿cómo sabíamos que teníamos que dibujar la función entre -2 y 2? En realidad el camino correcto es, en primer lugar, encontrar las raíces del polinomio:

```
(%i44) solve(f(x)=0,x);
(%o44) [x=-1/2,x=-1,x=1]
```

Ahora que sabemos las raíces se entiende por qué hemos dibujado la gráfica de la función en ese intervalo particular y no en otro. Si las raíces son $-\frac{1}{2}$, -1 y 1 sabemos que f no se anula en el

intervalo en $] -1, -\frac{1}{2}[$ ni en $] -\frac{1}{2}, 1[$ pero, ¿cuál es su signo en cada uno de dichos intervalos? Aquí es donde entra en juego el Teorema de los ceros de Bolzano. Si f , una función continua, sí cambiase de signo en $] -\frac{1}{2}, 1[$ o en $] -1, -\frac{1}{2}[$ tendría que anularse, cosa que no ocurre. Por tanto, f siempre tiene el mismo signo en cada uno de esos intervalos. ¿Cuál? Sólo es cuestión de mirar el valor de f en un punto cualquiera. En $] -1, -\frac{1}{2}[$

```
(%i45) f(-0.75);
(%o45) 0.21874999813735
```

la función es positiva. Y en $] -\frac{1}{2}, 1[$

```
(%i46) f(0);
(%o46) -1
```

la función es negativa. Por tanto, la integral que queremos hacer es

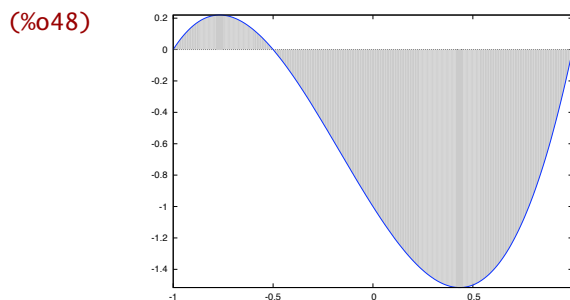
$$\int_{-1}^1 |f(x)| dx = \int_{-1}^{-\frac{1}{2}} f(x) dx - \int_{-\frac{1}{2}}^1 f(x) dx,$$

o, lo que es lo mismo,

```
(%i47) integrate(f(x),x,-1,-1/2)+integrate(-f(x),x,-1/2,1);
(%o47) 71
      48
```

Observación 11.6. Usando las opciones `filled_func` y `fill_color` podemos representar las funciones y el área, sombreada, que estamos calculando.

```
(%i48) draw2d(
        filled_func=0, fill_color=grey,
        explicit(f(x),x,-1,1),
        filled_func=false,
        color=blue, line_width=2,
        explicit(f(x),x,-1,1),
        xaxis=true);
```



El paquete `abs_integrate`

El cálculo de integrales o de primitivas de funciones en las que aparezcan valores absolutos puede ser una cuestión delicada. Como acabamos de ver es fácil que estemos lidiando con una función a trozos. El paquete `abs_integrate` añade algunas reglas más de cálculo a la orden `integrate` que, en algunos casos, nos puede resolver el problema directamente.

Por ejemplo, la primitiva de $x|x|$ no la sabe calcular *Maxima*, pero si cargamos el paquete las cosas cambian. Lo mismo ocurre con el área del ejemplo anterior:

```
(%i49) integrate(abs(x)*x,x);
(%o49)  $\int x|x|dx$ 
(%i50) integrate(abs(2*x^3+x^2-2*x+1),x);
(%o50)  $\int |2*x^3+x^2-2*x+1| dx$ 
```

Una vez que cargamos el paquete, la respuesta es distinta:

```
(%i51) load(abs_integrate)$
(%i52) integrate(abs(x)*x,x);
(%o52)  $\frac{x^2x}{2} - \frac{x^3 \operatorname{signum}(x)}{6}$ 
(%i53) integrate(abs(2*x^3+x^2-2*x+1),x,-1,1);
(%o53)  $\frac{71}{48}$ 
```

11.3.2 Longitud de curvas

Si f es una función definida en el intervalo $[a, b]$, la longitud del arco de curva entre $(a, f(a))$ y $(b, f(b))$ se puede calcular mediante la fórmula

$$\text{longitud} = \int_a^b \sqrt{1 + f'(x)^2} dx$$

Por ejemplo, la semicircunferencia superior de radio 1 centrada en el origen es la gráfica de la función $f(x) = \sqrt{1 - x^2}$, con x variando entre -1 y 1. Si aplicamos la fórmula anterior podemos calcular la longitud de una circunferencia.

```
(%i54) f(x):=sqrt(1-x^2);
(%o54) f(x):=sqrt(1-x^2)
(%i55) diff(f(x),x);
(%o55)  $-\frac{x}{\sqrt{1-x^2}}$ 
(%i56) integrate(sqrt(1+diff(f(x),x)^2),x,-1,1);
(%o56)  $\pi$ 
```

Observa que hemos calculado la longitud de media circunferencia, ya que la longitud de la circunferencia completa de radio 1 es 2π .

Esto que hemos visto es un caso particular de la longitud de una curva en el plano. Ya vimos curvas cuando hablamos de dibujar en coordenadas paramétricas. Una curva es una aplicación de la forma $x \mapsto (f(x), g(x))$, $a \leq x \leq b$. La longitud de dicha curva es

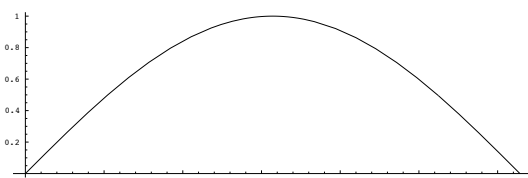
$$\text{longitud} = \int_a^b \sqrt{f'(x)^2 + g'(x)^2} dx$$

Por ejemplo, la circunferencia unidad la podemos parametrizar como $t \mapsto (\cos(t), \sin(t))$ con $0 \leq t \leq 2\pi$. Por tanto, la longitud de dicha circunferencia es

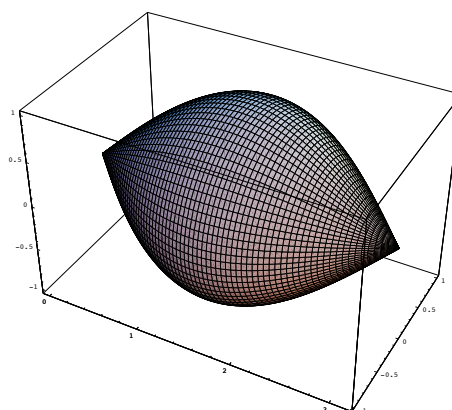
```
(%i57) integrate(sqrt(sin(t)^2+cos(t)^2),x,0,2*pi);
(%o57) 2*pi
```

11.3.3 Volúmenes de revolución

Los cuerpos de revolución o sólidos de revolución son regiones de $\mathbb{R}^3$ que se obtienen girando una región plana alrededor de una recta llamada eje de giro. En la Figura 11.5 tienes el sólido de revolución engendrado al girar alrededor del eje OX la gráfica de la función seno entre 0 y π .



Función seno



Función seno girada

Figura 11.5

Giro de una curva $y = f(x)$ respecto al eje X

Sea $f: [a, b] \rightarrow \mathbb{R}$ una función continua. Girando la región del plano comprendida entre la curva $y = f(x)$, el eje de abscisas con x entre a y b , alrededor del eje OX obtenemos un sólido de revolución Ω con volumen igual a

$$\text{Vol}(\Omega) = \pi \int_a^b f(x)^2 dx$$

Veamos algunos ejemplos.

Ejemplo 11.7. Calculemos el volumen de una esfera de radio R viéndola como superficie de revolución. La curva que consideramos es $f(x) = \sqrt{R^2 - x^2}$ con $x \in [-R, R]$ y, por tanto, el volumen lo calculamos aplicando la fórmula anterior :

```
(%i58) %pi*integrate(R^2-x^2,x,-R,R);
```

```
(%o58) 4*pi*R^3/3
```

Ejemplo 11.8. Vamos ahora a calcular el volumen de un cono circular recto. Un cono circular recto de altura h y radio de la base R se obtiene girando la recta $y = Rx/h$ entre 0 y h . Su volumen es igual a

```
(%i59) %pi*integrate(R^2*x^2/h^2,x,0,h);
```

```
(%o59) pi*h*R^2/3
```

Giro de una curva $y = f(x)$ respecto al eje Y

Consideremos la gráfica de una función positiva $y = f(x)$ en un intervalo $[a, b]$. Por ejemplo la función $1 - x^2$ en $[0, 1]$.

Si giramos respecto al eje OY obtenemos la Figura 11.6.

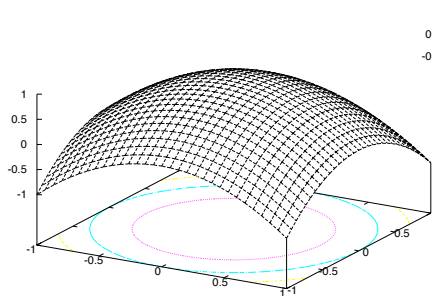
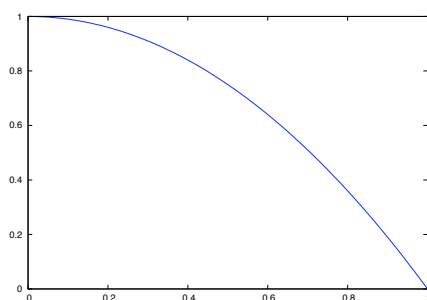


Figura 11.6 Función $1 - x^2$ girada respecto al eje OY

Pues bien, el volumen de la región así obtenida, Ω , viene dado por

$$\text{Volumen}(\Omega) = 2\pi \int_a^b x f(x) dx.$$

Ejemplo 11.9.

Consideremos el toro T obtenido al girar el disco de centro $(a, 0)$, $a > 0$, y radio R alrededor del eje OY . Puedes verlo para $a = 2$ y $R = 1$ en la Figura 11.7.

Por simetría, su volumen es el doble del volumen del sólido obtenido al girar la semicircunferencia

$$y = \sqrt{R^2 - (x - a)^2},$$

con $a - R \leq x \leq a + R$ alrededor del eje OY . Por tanto

$$\text{volumen}(T) = 4\pi \int_{a-R}^{a+R} x \sqrt{R^2 - (x - a)^2} dx$$

integral que calculamos con *wxMaxima*:

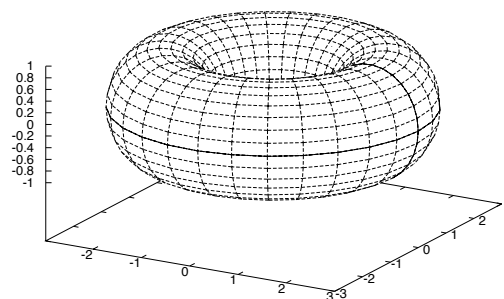


Figura 11.7 Toro

```
(%i60) 4*pi*integrate(x*sqrt(R^2-(x-a)^2),x,a-R,a+R);
```

$$2\pi^2 a R^2$$

Observa que, aunque aquí hemos escrito la salida automáticamente, sin embargo, *wxMaxima* hace varias preguntas sobre los valores de las constantes a y R para poder calcular la integral.

Gráficamente, podemos conseguir este efecto girando la circunferencia de radio 1 que podemos parametrizar de la forma $(\cos(t), \sin(t))$, pero que tenemos que trasladar dos unidades. Ése es el motivo de sumar 2 en la siguiente representación en coordenadas paramétricas.

```
(%i61) with_slider_draw3d(
n,makelist(0.1*%pi*i,i,1,20),
surface_hide=true,
xrange=[-3,3],
yrange=[-3,3],
parametric_surface(cos(u)*(2+cos(v)),sin(u)*(2+cos(v)),sin(v),
u,0,n,v,0,2*%pi)
)
```

En la Figura 11.8 tienes algunos de los pasos intermedios de esta animación.

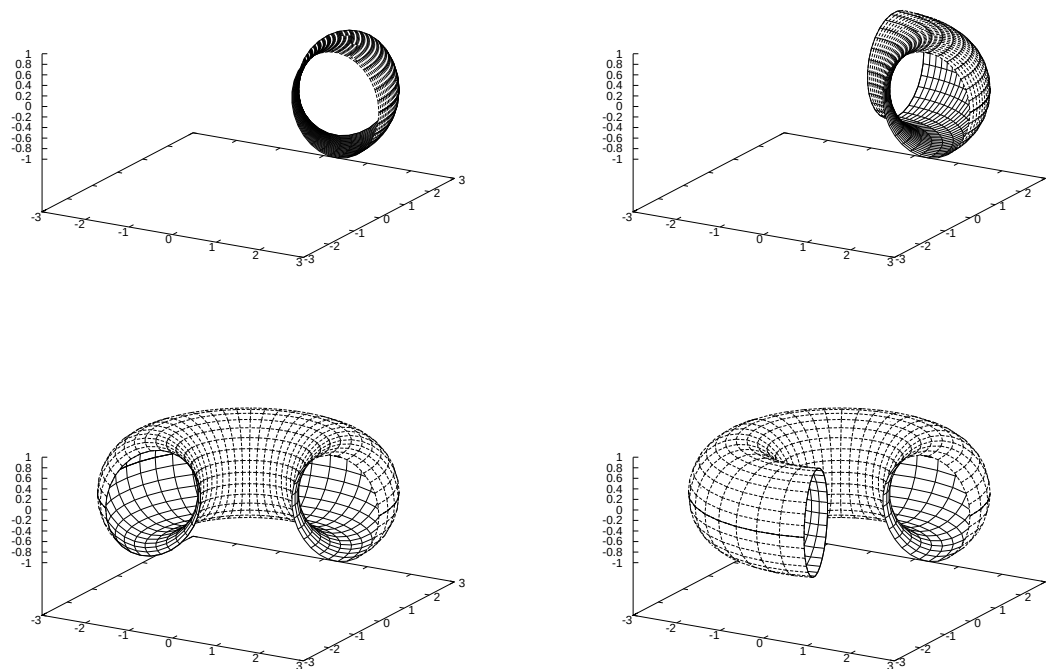


Figura 11.8 Cómo conseguir un toro girando una circunferencia

11.3.4 Área de superficies de revolución

Igual que hemos visto cómo podemos calcular el volumen de una figura obtenida girando una función respecto a alguno de los ejes, también podemos calcular el área de la superficie Ω obtenida al girar respecto al eje OX una función f . El área al girar f en el intervalo $[a, b]$ es

$$\text{área}(\Omega) = 2\pi \int_a^4 f(x) \sqrt{1 + f'(x)^2} dx$$

Por ejemplo, una esfera de radio 1 la podemos obtener girando respecto del eje OX la función $\sqrt{1 - x^2}$.

```
(%i62) integrate(f(x)*sqrt(1+diff(f(x),x)^2), x, -1, 1);
(%o62) 2 * pi^2 * a * R^2
```

11.4 Ejercicios

Integración

Ejercicio 11.1. Calcula una primitiva de las siguientes funciones.

- a) $f(x) = \cos^5(x)$,
- b) $f(x) = 1/(1 + x^4)$,
- c) $f(x) = \sqrt{1 - x^2}$,

¿Sabes calcularlas sin usar *Maxima*?

Ejercicio 11.2. Calcula numéricamente las integrales en el intervalo $[0, 2\pi]$ de los primeros 20 polinomios de Taylor centrados en cero de la función coseno.

Teorema fundamental del Cálculo

Ejercicio 11.3. Calcula la derivada de la función $f(x) = \int_{\sqrt{x}}^{x^2+1} \sin(t) dt$.

Ejercicio 11.4. Estudia los extremos relativos de la función $f(x) = \int_0^{(2x-7)^2} t^3 - 2t dt$.

Área entre curvas

Ejercicio 11.5. Calcula:

- a) área limitada por $y = xe^{-x^2}$, el eje OX , la ordenada en el punto $x = 0$ y la ordenada en el máximo.
- b) área de la figura limitada por la curva $y = x^3 - x^2$ y el eje OX .
- c) área comprendida entre la curva $y = \tan(x)$, el eje OX y la recta $x = \pi/3$.
- d) área del recinto limitado por las rectas $x = 0$, $x = 1$, $y = 0$ y la gráfica de la función $f: \mathbb{R} \rightarrow \mathbb{R}$ definida por $f(x) = \frac{1}{(1+x^2)^2}$.
- e) las dos áreas en los que la función $f(x) = |x| - x \sin(x)e^x$ divide al círculo $x^2 + y^2 \leq 1$.

Ejercicio 11.6. Calcula el área entre las curvas:

- a) $y = -x^2 - 2x$, $y = x^2 - 4$, para $-3 \leq x \leq 1$.
- b) $y^2 = x$, $x^2 + y^2 = 8$.
- c) $x = 12y^2 - 12y^3$, $x = 2y^2 - 2y$.
- d) $y = \sec^2(x)$, $y = \tan^2(x)$, $-\pi/4 \leq x \leq \pi/4$.
- e) $(y - x)^2 = x - 3$, $x = 7$.
- f) $y = x^4 + x^3 + 16x - 4$, $y = x^4 + 6x^2 + 8x - 4$.
- g) $y = xe^{-x}$, $y = x^2e^{-x}$, $x \geq 0$.

Ejercicio 11.7. Divide un círculo de radio 1 en tres trozos del mismo área usando dos líneas verticales, simétricas respecto del eje OY .

Ejercicio 11.8. Una cabra está atada en el borde de un prado circular de 10 metros de diámetro. Calcula la longitud mínima de cuerda para que la cabra tenga acceso a la mitad de la superficie.

Ejercicio 11.9. Calcula el rectángulo de área máxima incluido en la elipse centrada en el origen con semiejes 3 y 4. ¿Cuál es el área entre ambos? Haz la representación gráfica.

Ejercicio 11.10. Se considera la función $f(x) = x \sin(x) \cos(x)$.

- a) Calcula dos puntos de su gráfica, $(a, f(a))$ y $(b, f(b))$, por la condición de que la tangente en cada uno de ellos pase por $(4, 0)$.
- b) Representa juntas gráficamente a la función y las dos rectas tangentes antes calculadas. Debes elegir un intervalo y opciones de color y grosor de línea apropiados.
- c) Calcula el área de la región limitada superiormente por la gráfica de f e inferiormente por las rectas tangentes cuando $a \leq x \leq b$, donde a y b son los valores calculados en el primer apartado.

Longitud de curvas

Ejercicio 11.11.

- a) Calcula la longitud del arco de la cicloide $x = t - \sin(t)$, $y = 1 - \cos(t)$, $(0 \leq t \leq 2\pi)$.
- b) Calcula la longitud del arco de curva $y = x^2 + 4$ entre $x = 0$ y $x = 3$.

Ejercicio 11.12. Calcula la longitud de una circunferencia de radio arbitrario R .

Ejercicio 11.13. Se puede aproximar la longitud de la gráfica de una función f en un intervalo $[a, b]$ mediante el siguiente proceso: se divide el intervalo $[a, b]$ en n subintervalos de la misma longitud, $(b - a)/n$, y se aproxima la longitud de la gráfica en cada intervalo por la longitud del segmento que une las imágenes de los extremos del intervalo. Sumando los resultados en cada uno de los subintervalos se obtiene una aproximación por defecto de la longitud buscada.

- a) Haz un programa usando un bloque en el que las entradas sean la función, el intervalo y el número de subintervalos y que dé como salida la aproximación.

b) La fórmula para calcular la longitud de la gráfica de una función f en un intervalo $[a, b]$ es $\int_a^b \sqrt{1 + f'(x)^2} dx$. Aplica tu método a la función $f(x) = \sin(x)$ en el intervalo $[0, \pi]$ y compara el resultado obtenido con la fórmula anterior.

Ejercicio 11.14. Sea $f(x) = \cos(x) + e^x$ y P su polinomio de Taylor de orden 5 centrado en el origen. ¿Cuál es la diferencia entre las longitudes de las gráficas de f y de P en el intervalo $[0, 3]$?

E **Ejercicio 11.15.** La gráfica de la función $y = x^3 + 1/e$ divide el círculo centrado en el origen de radio el número de tu DNI en dos partes. Calcula el área de cada una de ellas y la longitud de la gráfica incluida en la circunferencia.

Ejercicio 11.16. Consideremos la función $f(x) = x^3 - x - 1$ en el intervalo $[0, 2]$

- Calcula la longitud de la gráfica de la función.
- Calcula 3 puntos aleatorios de dicho intervalo.
- ¿Cuál es la longitud de la curva formada por segmentos de recta que unen los puntos anteriores?

Volumen de cuerpos de revolución

Ejercicio 11.17. Calcular el volumen del sólido generado al rotar respecto al eje OX las siguiente curvas:

- $y = \sec(x)$, $x \in [0, \frac{\pi}{3}]$
- $y = \sqrt{\cos(x)}$, $x \in [0, \frac{\pi}{2}]$
- $y = 9 - x^2$
- $y = e^x$, $x \in [0, \log(3)]$

Ejercicio 11.18. Calcular el volumen del sólido generado al rotar respecto al eje OY las siguiente curvas:

- $y = 1/x$, $x \in [1, 3]$
- $y = \frac{1}{1+x^2}$, $x \in [0, 1]$
- $y = e^{x^2}$, $x \in [1, \sqrt{3}]$

Ejercicio 11.19.

- Calcular la superficie de una esfera de radio R .
- Calcular la superficie de la figura que se obtiene al girar la función $y = \tan(x)$, $x \in [0, \pi/4]$ alrededor del eje OX .

Ejercicio 11.20.

- Calcular el volumen del toro engendrado al girar el círculo de centro $(2, 0)$ y radio 3 alrededor del eje OY .

- b) Calcular el volumen del sólido engendrado al girar alrededor del eje OY la región limitada por la parábolas $y^2 = x$, $x^2 = y$.

Ejercicio 11.21. Sea $f(x) = x^5 + 4x^3 + 2x^2 + 8$. Calcula el volumen al girar dicha función alrededor del eje OX entre los valores donde f alcanza su máximo y su mínimo relativos.

Área de superficies de revolución

Ejercicio 11.22.

- a) Calcula el área de una esfera de radio R .
- b) Calcula el área de la figura obtenida al girar la parábola $y^2 = 4x$ y la recta $x = 5$ alrededor del eje OX .
- c) Calcula el área lateral de un cono de radio R y altura h .

Ejercicio 11.23. Calcular:

- a) La integral de $f(x) = \frac{1}{x^2}$ con $x \in [1, +\infty]$.
- b) El volumen y la superficie lateral del sólido obtenido al girar la gráfica de la anterior función respecto del eje OX .
- c) Ídem a los dos anteriores con $g(x) = \frac{1}{x}$ con $x \in [1, +\infty]$.

Series numéricas y series de Taylor

12

12.1 Series numéricas 183 12.2 Desarrollo de Taylor 185 12.3 Ejercicios 186

12.1 Series numéricas

El estudio de la convergencia de una serie numérica no siempre es fácil y mucho menos el cálculo del valor de dicha suma. *Maxima* sabe calcular la suma de algunas series y siempre tenemos la posibilidad de aplicar algún criterio de convergencia.

<code>sum(expr, n, a, b)</code>	$\sum_{n=a}^b expr$
<code>sum(expr, n, a, b), simpsum</code>	simplifica la suma
<code>nusum (expr, x, a, plus)</code>	$\sum_{n=a}^b expr$

La orden genérica para calcular una suma, finita o infinita, es `sum`. Hace falta indicar qué sumamos, `sum` la variable que hace de índice y entre qué valores varía dicha variable.

```
(%i63) sum(i,i,0,100);
(%o63) 5050
```

Además hace falta indicarle que desarrolle y simplifique el sumatorio si queremos obtener un resultado más razonable. Esto se consigue con el operador `simpsum` que aparece escrito automáticamente si utilizamos el menú de *wxMaxima*.

```
(%i64) sum(i,i,0,n);
(%o64)  $\sum_{i=0}^n i$ 
(%i65) sum(i,i,0,n),simpsum;
(%o65)  $\frac{n^2+n}{2}$ 
```

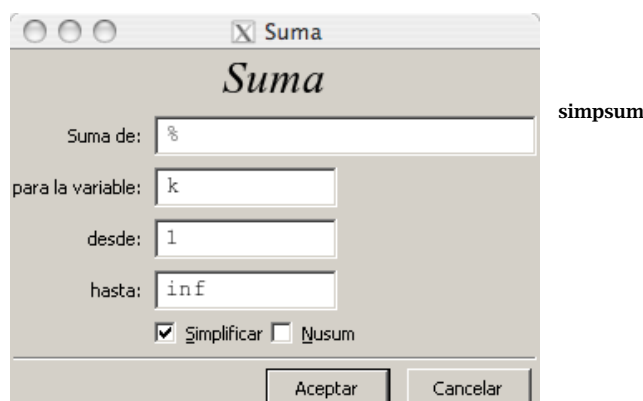


Figura 12.1 Ventana para calcular sumas

La segunda posibilidad es usar la orden `nusum` que utiliza reglas de sumación para obtener en `nusum` algunos casos (funciones hipergeométricas principalmente) una expresión mejor del resultado.

```
(%i66) nusum(i,i,0,n);
(%o66)  $\frac{n(n+1)}{2}$ 
```

A cualquiera de estas dos órdenes se accede desde el menú **Análisis—Calcular suma**. Aparece la ventana de la Figura 12.1 que tienes que rellenar con la expresión a sumar, variable y extremos. En la parte inferior de dicha ventana, puedes marcar **Simplificar** o **Nusum** para escoger entre las órdenes que hemos comentado.

El conjunto de series que sabe sumar *Maxima* no es demasiado grande, ya hemos dicho que es un problema difícil, pero tiene respuesta para algunas series clásicas. Por ejemplo,

```
(%i67) sum(1/n^2,n,1,inf),simpsum;
(%o67)  $\frac{\pi^2}{6}$ 
```

o la serie armónica

```
(%i68) sum(1/n,n,1,inf),simpsum;
(%o68)  $\infty$ 
```

que sabemos que no es convergente. Si intentamos sumar una progresión geométrica cualquiera nos pregunta por la razón y la suma sin mayores problemas

```
(%i69) sum(x^n,n,1,inf),simpsum;
Is |x|-1 positive, negative or zero? negative;
(%o69)  $\frac{x}{1-x}$ 
```

En cambio no suma algunas otras como

```
(%i70) sum(1/n!,n,0,inf),simpsum;
(%o70)  $\sum_{n=0}^{\infty} \frac{1}{n!}$ 
```

Como hemos dicho, calcular la suma de una serie es un problema difícil y lo que hemos hecho en clase es estudiar cuándo son convergentes sin calcular su suma. Aplicar el criterio de la raíz o del cociente para decidir la convergencia de una serie, reduce el problema al cálculo de un límite, que en el caso anterior es trivial, en el que *Maxima* sí puede ser útil. Por ejemplo, la serie $\sum_{n \geq 1} \frac{1}{n^n}$

```
(%i71) a[n]:=1/n^n;
(%o71)  $a_n := \frac{1}{n^n}$ 
(%i72) sum(a[n],n,1,inf),simpsum;
(%o72)  $\sum_{n=1}^{\infty} \frac{1}{n^n}$ 
```

no sabemos sumarla, pero el criterio del cociente

```
(%i73) limit(a[n+1]/a[n],n,inf);
(%o73) 0
```

nos dice que es convergente.

12.2 Desarrollo de Taylor

El desarrollo en serie de Taylor de una función parte de una idea sencilla: si el polinomio de Taylor de la función $\cos(x)$ se parece cada vez más a la función, ¿por qué parar?

```
(%i74) taylor(cos(x),x,0,4);
(%o74) 1 - x^2/2 + x^4/24 + ...
```

¿Qué es eso de un polinomio con infinitos sumandos? Para eso acabamos de ver series de números. Ya sabemos en qué sentido podemos hablar de sumas infinitas.

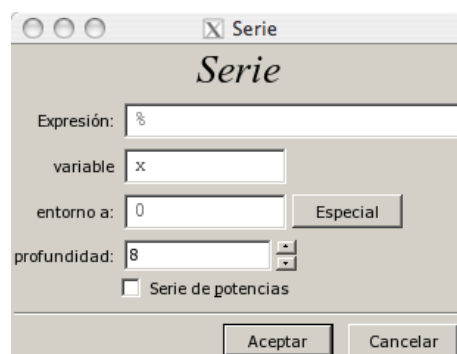


Figura 12.2 Ventana para el cálculo de la serie de Taylor

```
niceindices(powerseries(f(x), x, a))
```

desarrollo en serie de potencias
de la función f centrado en el punto a

Si en el menú **Análisis**→**Calcular serie**, marcamos **Serie de potencias** obtenemos el desarrollo en serie de potencias de la función

```
(%i75) niceindices(powerseries(cos(x), x, 0));
(%o75) 
$$\sum_{i=0}^{\infty} \frac{(-1)^i x^{2i}}{(2i)!}$$

```

La orden `powerseries` es la encargada del cálculo de dicho desarrollo y el comando que se antepone, `niceindices`, escribe la serie de Taylor de una función con índices más apropiados. Si no la utilizamos, simplemente el resultado aparece un poco más “enmarañado”:

```
(%i76) powerseries(cos(x),x,0);
(%o76) 
$$\sum_{i16=0}^{\infty} \frac{(-1)^{i16} x^{2 i16}}{(2 i16)!}$$

```

pero es esencialmente el mismo. Funciona bastante bien para las funciones con las que solemos trabajar en clase:

```
(%i77) niceindices(powerseries(atan(x),x,0));
```

```
(%o77) 
$$\sum_{i=0}^{\infty} \frac{(-1)^i x^{2i+1}}{2i+1}$$

```

Ahora podemos dar una respuesta al problema de en dónde se parecen una función y su polinomio: la primera condición es que la serie de Taylor tiene que ser convergente. El conjunto de valores x en los que es convergente, hemos visto en clase que, depende del radio de convergencia de la serie. Vamos a calcularlo:

```
(%i78) a[i]:=(-1)^i/(2*i+1);
```

```
(%o78) 
$$a_i = \frac{(-1)^i}{2i+1}$$

```

```
(%i79) limit(abs(a[i+1]/a[i]),i,inf);
```

```
(%o79) 1
```

Perfecto. Ya sabemos que la serie de Taylor de la función arcotangente converge para $x \in]-1, 1[$ y no es convergente en $\mathbb{R} \setminus [-1, 1]$ y el dibujo que vimos de su polinomio de Taylor tiene ahora sentido. Queda por saber qué ocurre para $x = \pm 1$. Eso te toca a tí.

Observación 12.1. Se pueden realizar algunas operaciones con series de potencias. Por ejemplo, intenta derivarlas:

```
(%i80) f(x):=niceindices(powerseries(log(1+x^2),x,0))$
```

```
(%i81) diff(f(x),x);
```

```
(%o81) 
$$-2 \sum_{i=1}^{\infty} (-1)^i x^{2i-1}$$

```

12.3 Ejercicios

Ejercicio 12.1. Estudiar la convergencia de las series que tienen el siguiente término general.

a) $a_n = \frac{n^2 - n + 3}{3^n + 2^{-n} + n - 1}$, c) $a_n = \frac{1}{2^n - n}$, e) $a_n = \frac{1}{\log(n)^2}$.
 b) $a_n = \log\left(\frac{n^3 + 1}{n^3 + 2n^2 - 3}\right)$, d) $a_n = \frac{n!}{n^n}$,

Ejercicio 12.2. La serie $\sum_{n \geq 1} \frac{1}{n^2}$ es convergente y su suma es $\frac{\pi^2}{6}$. ¿Cuántos términos hay que sumar para que el error sea menor que 10^{-3} .

Ejercicio 12.3. ¿Cuántos términos de la serie

$$\sum_{n \geq 2} \frac{1}{n \log(n)}$$

hay que sumar para que la suma parcial sea mayor que 3? ¿Es convergente?

Ejercicio 12.4. Se deja caer una pelota elástica desde una altura h . Cada vez que toca el suelo bota hasta una altura que es el 75% de la altura desde la que cayó. Hállese h sabiendo que la pelota recorre una distancia total de 21 m. hasta detenerse.

Ejercicio 12.5. Calcula el desarrollo en serie de potencias centrado en a de las siguientes funciones, realiza una animación de la función y los primeros 20 polinomios de Taylor y, por último, estudia la convergencia de la serie de Taylor.

a) $f(x) = \cos(x^2)$, $a = 0$,

b) $f(x) = \sin^2(x)$, $a = 0$,

c) $f(x) = \log(1 + x^2)$, $a = 0$,

d) $f(x) = \log(5 + x^2)$, $a = 0$,

e) $f(x) = \log(5 + x^2)$, $a = 1$,

f) $f(x) = x^4 + 3x^2 - 2x + 3$, $a = 1$,

g) $f(x) = |x|$, $a = 1$,

h) $f(x) = |x|$, $a = 5$.

Interpolación

Interpolación polinómica

13

13.1 Interpolación lineal 191 13.2 Interpolación de Lagrange 192 13.3 Polinomio de Taylor 196

El problema general es cómo se busca una función que tome unos valores en unos puntos concretos. También se puede exigir que las derivadas de algún orden tengan un valor predeterminado.

13.1 Interpolación lineal

Dados dos puntos del plano (x_1, y_1) , (x_2, y_2) , sabemos que hay una recta que pasa por ellos. Dicha recta es la gráfica de un polinomio de grado 1, en este caso la función es

$$f(x) = y_2 + \frac{y_2 - y_1}{x_2 - x_1} (x - x_2).$$

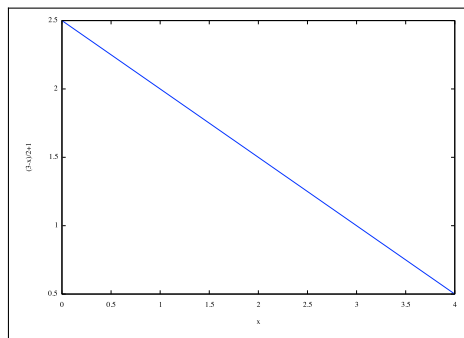
Por ejemplo, la recta que pasa por los puntos $(1, 2)$ y $(3, 1)$ es

```
(%i1) recta(x1,y1,x2,y2):=y2+(x-x2)*(y2-y1)/(x2-x1)$
(%i2) recta(1,2,3,1);
(%o2)  $\frac{3-x}{2}+1$ 
```

y la gráfica

```
(%i3) wxplot2d(recta(1,2,3,1),[x,0,4]);
```

```
(%t3)
```



Podemos hacer lo mismo si tenemos varios puntos. Podemos calcular la *poligonal* que los une, esto es, una función a trozos une dos puntos consecutivos por el correspondiente segmento. El paquete `interp` tiene nos va a ahorrar muchos de los cálculos.

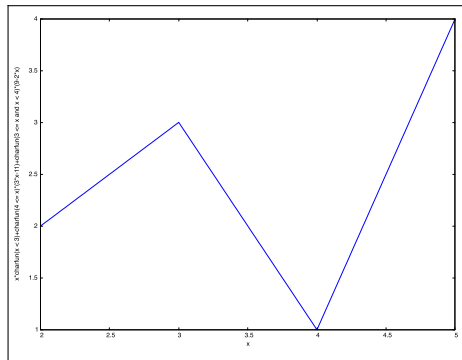
```
(%i4) load(interp);
```

En este caso

<code>linearinterp(lista de puntos)</code>	poligonal que une la lista de puntos
<code>charfun(prop)</code>	función característica
<code>charfun2(x, a, b)</code>	función característica de $[a, b]$

```
(%i5) define(g(x),linearinterp([[2,2],[3,3],[4,1],[5,4]]));
(%o5) g(x):=x*charfun2(x,-inf,3)+(3*x-11)*charfun2(x,4,inf)
      +(9-2*x)*charfun2(x,3,4)
(%i6) wxplot2d(g(x),[x,2,5]);
```

```
(%t6)
```



```
jj
```

13.2 Interpolación de Lagrange

El problema más clásico de interpolación es la *interpolación de Lagrange*:

Dados $n + 1$ pares de puntos $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$, encuentrese el polinomio P de grado menor o igual que n tal que $P(x_i) = y_i$, $i = 0, 1, \dots, n$.

Los puntos $x_0, x_1, \dots, x_n$ se llaman *nodos de interpolación*.

13.2.1 Dos o tres nodos

Comencemos con un caso sencillo. Dada una lista de un par de puntos y un par de valores, ¿cuál es el polinomio que pasa por esos puntos?

```
(%i7) nodos:[1,2];
(%o7) [1,2]
(%i8) valor:[3,7];
(%o8) [3,7]
```

Necesitamos un polinomio de grado uno:

```
(%i9) define(f(x),a*x+b);
```

```
(%o9) f(x):=ax+b
```

que debe verificar que $f(1) = 3$, $f(2) = 7$. Con estas dos condiciones planteamos un sistema de ecuaciones que nos permite calcular a y b :

```
(%i10) solve([f(nodos[1])=valor[1],f(nodos[2])=valor[2]],[a,b]);
(%o10) [[a=4,b=-1]]
```

Podemos aplicar la misma técnica para encontrar el polinomio, de grado 2 en este caso, que pasa por los puntos $(1, 3)$, $(2, 7)$ y $(3, 1)$:

```
(%i11) nodos:[1,2,3]$
(%i12) valor:[3,7,1]$
(%i13) define(f(x),a*x^2+b*x+c)$
(%i14) solve([f(nodos[1])=valor[1],f(nodos[2])=valor[2],
              f(nodos[3])=valor[3]],[a,b,c]);
(%o14) [[a=-5,b=19,c=-11]]
```

Vamos a resolver este mismo problema de otra forma. Busquemos tres polinomios de orden 2, L_1 , L_2 y L_3 verificando que valen 1 en uno de los nodos y cero en el resto. En concreto, buscamos L_1 , L_2 y L_3 tales que $L_1(1)=1$, $L_2(2)=1$, $L_3(3)=1$ y que valen cero en los otros puntos. Si encontramos estos polinomios, entonces la solución a nuestro problema es $3L_1 + 7L_2 + L_3$.

Comencemos con L_1 : el polinomio $(x-2)(x-3)$ se anula en 2 y en 3, pero su valor en 1, $(1-2)(1-3)$, no es 1. Si dividimos por dicha cantidad

$$L_1(x) = \frac{(x-2)(x-3)}{(1-2)(1-3)}$$

ya hemos encontrado el primer polinomio que estábamos buscando. Análogamente

$$L_2(x) = \frac{(x-1)(x-3)}{(2-1)(2-3)}$$

$$L_3(x) = \frac{(x-1)(x-2)}{(3-1)(3-2)}$$

Con estos tres polinomios ya podemos interpolar cualesquiera tres valores en 1, 2 y 3.

13.2.2 Caso general

Si ahora tenemos $n + 1$ nodos distintos, podemos hacer una construcción análoga.

Definición 13.1. Dados x_i , $i = 0, 1, 2, \dots, n$ números reales distintos, los polinomios

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{(x - x_j)}{(x_i - x_j)}, \quad i = 0, 1, 2, \dots, n$$

se llaman *polinomios de Lagrange* de grado n en los puntos $x_0, x_1, \dots, x_n$.

Polinomio de Lagrange

¿Qué propiedades tienen? Fíjate que son muy fáciles de evaluar en los puntos x_i . Si evaluamos en el mismo índice

$$L_i(x_i) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{(x_i - x_j)}{(x_i - x_j)} = 1,$$

ya que numerador y denominador coinciden. Si evaluamos en x_k con $k \neq i$, entonces

$$L_i(x_k) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{(x_k - x_j)}{(x_i - x_j)} = 0,$$

ya que uno de los factores del numerador, en concreto cuando $j = k$, se anula. Resumiendo, los polinomios de Lagrange valen

$$L_i(x_k) = \begin{cases} 1, & \text{si } i = k, \\ 0, & \text{si } i \neq k. \end{cases}$$

para $i, k = 0, 1, \dots, n$. Observa que hemos dado el valor de los polinomios de Lagrange en $n + 1$ puntos diferentes y, por tanto, estos valores los determinan completamente.

Con los polinomios L_i ahora podemos calcular fácilmente un polinomio que tome valores y_i en los nodos x_i . En efecto, el polinomio

$$P(x) = y_0 L_0(x) + y_1 L_1(x) + \dots + y_n L_n(x)$$

cumple que $P(x_i) = y_i$ para $i = 0, 1, \dots, n$. El siguiente teorema recoge toda la información que hemos presentado.

Teorema 13.2. Sean $x_0, x_1, \dots, x_n$ números reales distintos. Entonces dados $y_0, y_1, \dots, y_n$ existe un único polinomio $P_n(x)$ de grado menor o igual que n verificando que

$$P_n(x_i) = y_i, \quad i = 0, 1, \dots, n.$$

Dicho polinomio viene dado por

$$P(x) = y_0 L_0(x) + y_1 L_1(x) + \dots + y_n L_n(x), \quad (13.1)$$

donde

$$L_i(x) = \frac{(x - x_0)(x - x_1) \dots (x - x_{i-1})(x - x_{i+1}) \dots (x - x_n)}{(x_i - x_0)(x_i - x_1) \dots (x_i - x_{i-1})(x_i - x_{i+1}) \dots (x_i - x_n)}$$

para $i = 0, 1, \dots, n$.

La identidad (13.1) se llama *fórmula de Lagrange del polinomio de interpolación*.

Ventajas e inconvenientes

Los polinomios de Lagrange son muy fáciles de calcular. Es por ello que se utilizan como uno de los primeros ejemplos de polinomios interpoladores. Su interés práctico es limitado y suelen presentarse más bien como ejemplo teórico de interpolación.

Su principal inconveniente se presenta cuando el conjunto de nodos es muy grande. En ese caso el grado del polinomio también es muy grande. Esto implica dificultades para el cálculo y, además, hay una alta tendencia a que el polinomio oscile mucho entre dos nodos.

Fórmula de Lagrange del polinomio de interpolación

13.2.3 El paquete *interpol*

Puedes pensar en alguna forma de calcular los polinomios de Lagrange en un conjunto de nodos, pero en *Maxima* disponemos del paquete *interpol* que calcula el polinomio interpolador de Lagrange. En primer lugar cargamos el módulo

```
(%i15) load(interpol)$
```

y podemos usar la orden `lagrange` para calcular el polinomio que interpola una lista de pares (nodo, valor)

```
(%i16) lagrange([[1,3],[2,1],[3,4]]);
(%o16) 2(x-2)(x-1)-(x-3)(x-1)- $\frac{3(2-x)(x-3)}{2}$ 
(%i17) expand(%);
(%o17)  $\frac{5x^2}{2} - \frac{19x}{2} + 10$ 
```

o, en el caso de que los nodos sean 1, 2, 3, 4, etc., simplemente dando la lista de valores

```
(%i18) expand(lagrange([3,1,4]));
(%o18)  $\frac{5x^2}{2} - \frac{19x}{2} + 10$ 
```

<code>lagrange([[nodo1,valor1],[nodo2,valor2],...])</code>	polinomio de Lagrange
<code>lagrange([valor1,valor2,...])</code>	polinomio de Lagrange

13.2.4 Ejercicios

Ejercicio 13.1. ¿Cuál es el error cuando aproximamos $\sqrt{102}$ utilizando el valor de la función raíz cuadrada en 81, 100 y 121? Representa las gráficas de la función raíz cuadrada y compárala con la gráfica del polinomio.

Ejercicio 13.2. Utiliza los valores de la función raíz cuadrada en $n = 2, 3, \dots, 10$ puntos, elegidos por ti, para calcular el polinomio de interpolación y aproximar el valor en 102. Haz una animación que represente la función raíz cuadrada y el polinomio de interpolación de Lagrange en función de su grado.

Ejercicio 13.3. Sea $f(x) = \cos(x)$. Calcula su polinomio de interpolación de Lagrange en los puntos $0, \pm\pi/2$ y $\pm\pi$. Dibuja en un mismo gráfico las dos funciones.

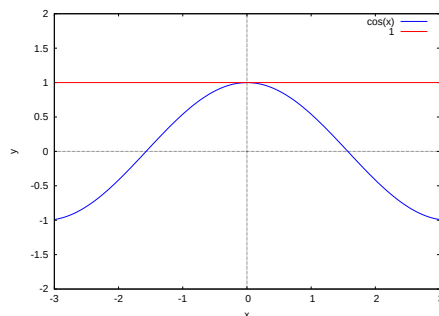
Ejercicio 13.4. Calcula la fórmula de la suma de los cubos de los primeros n naturales sabiendo que es un polinomio de grado cuatro.

Ejercicio 13.5. Se considera la función $f(x) = e^{-x} \sin(2x)$ con $x \in [2, 5]$. Calcula el polinomio de interpolación de Lagrange de la función en los puntos 2, 3 y 4. Calcula los ceros de la función y de dicho polinomio en ese intervalo.

13.3 Polinomio de Taylor

En el Capítulo 10 hemos visto cómo la recta tangente a una función en un punto aproxima localmente a dicha función en ese punto. Es decir, que si sustituimos una función por su recta tangente en un punto, estamos cometiendo un error como se puede ver. En efecto, si dibujamos en una misma gráfica la función $f(x) = \cos(x)$ y su recta tangente en cero, es decir $t(x) = f(0) + f'(0)(x - 0) = 1$ obtenemos

```
(%i19) f(x):=cos(x);
(%o19) f(x):=cos(x)
(%i20) t(x):=1;
(%o20) t(x):=1
(%i21) plot2d([f(x),t(x)], [x,-3,3], [y,-2,2]);
(%o21)
```



En cuanto nos alejamos un poco del punto de tangencia (en este caso el 0), la función coseno y su tangente no se parecen en nada. Una posibilidad de mejorar la exactitud de la aproximación es aumentar el grado del polinomio que usando el valor de las derivadas de orden superior en cero. Este problema es el que resuelve el polinomio de Taylor: encontrar un polinomio que coincida con la función y sus derivadas en un punto a hasta un cierto orden. En general, no es necesario partir de una función y, sencillamente, buscar un polinomio del que conocemos su valor y el de sus derivadas en un punto hasta un orden.

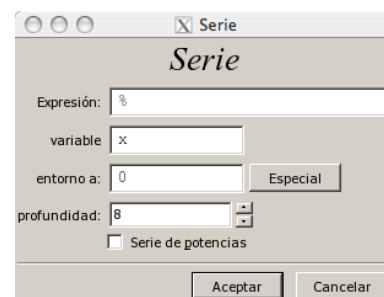


Figura 13.1 Ventana para el cálculo del polinomio de Taylor

Dado un número real a y $n + 1$ valores $y_0, y_1, \dots, y_n$ encuéntrase el polinomio P de grado menor o igual que n tal que $P^{(i)}(a) = y_i$, $i = 0, 1, \dots, n$ o, lo que es lo mismo,

$$P(a) = y_0, P'(a) = y_1, P''(a) = y_2, \dots, P^{(n)}(a) = y_n.$$

Como hemos visto en clase, dicho polinomio es

$$P_n(x) = f(a) + f'(a)(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \frac{f'''(a)}{3!}(x-a)^3 + \dots + \frac{f^{(n)}(a)}{n!}(x-a)^n.$$

Observa que, en el caso $n = 1$, el polinomio de Taylor P_1 es la recta tangente.

Existe una orden para calcular directamente el polinomio de Taylor centrado en un punto a . Se trata del comando `taylor`. En concreto, el comando `taylor(f(x), x, a, n)` nos da el polinomio de Taylor¹ de la función f centrado en a y de grado n . Haciendo uso del menú podemos acceder al comando anterior desde **Análisis** → **Calcular serie**. Entonces se abre una ventana de diálogo en la que, escribiendo la expresión de la función, la variable, el punto en el que desarrollamos y el orden del polinomio de Taylor, obtenemos dicho polinomio. Como en otras ventanas similares, si marcamos la casilla de **Especial**, podemos elegir π o e como centro para el cálculo del desarrollo.

<code>taylor(f(x), x, a, n)</code>	polinomio de Taylor de la función f en el punto a y de orden n
<code>coeff(expr, x, n)</code>	coeficiente de x^n

Veamos un ejemplo.

```
(%i22)  taylor(cos(x), x, 0, 5);
```

```
(%o22)/T/  1 - x^2/2 + x^4/24 + ...
```

```
(%i23)  taylor(log(x), x, 1, 7);
```

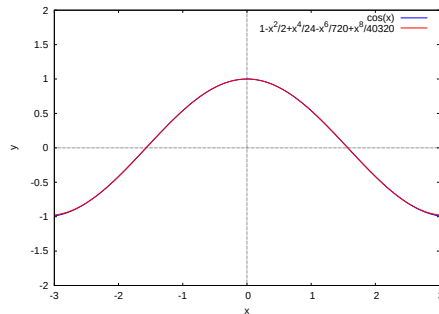
```
(%o23)/T/  x-1 - (x-1)^2/2 + (x-1)^3/3 - (x-1)^4/4 + (x-1)^5/5 - (x-1)^6/6 + (x-1)^7/7 + ...
```

¹ En realidad, este comando calcula el desarrollo de Laurent. Esto es, se permiten potencias enteros (negativas) de $(x - a)$.

En teoría, un polinomio de Taylor de orden más alto debería aproximar mejor a la función. Ya hemos visto cómo aproxima la recta tangente a la función coseno. Vamos ahora a dibujar las gráficas de la función $f(x) = \cos(x)$ y de su polinomio de Taylor de orden 8 en el cero para comprobar que la aproximación es más exacta.

```
(%i24) plot2d([f(x),taylor(f(x),x,0,8)], [x,-4,4], [y,-2,2]);
```

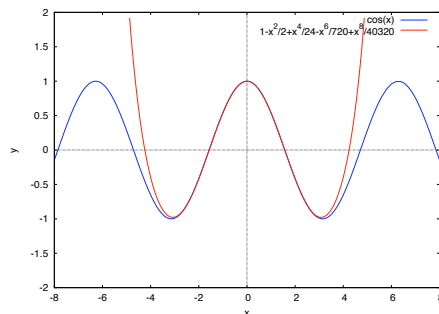
```
(%o24)
```



Pero si aumentamos el dominio podemos ver que el polinomio de Taylor se separa de la función cuando nos alejamos del origen.

```
(%i25) plot2d([f(x),taylor(f(x),x,0,8)], [x,-8,8], [y,-2,2]);
```

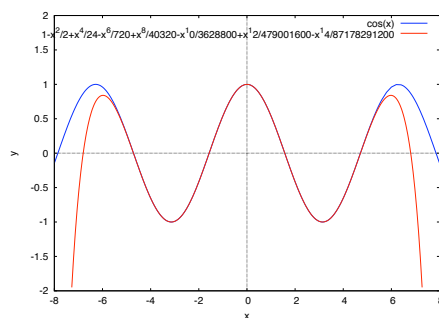
```
(%o25)
```



Esto es lo esperable: la función coseno está acotada y el polinomio de Taylor, como todo polinomio no constante, no lo está. Eso sí, si aumentamos el grado del polinomio de Taylor vuelven a parecerse:

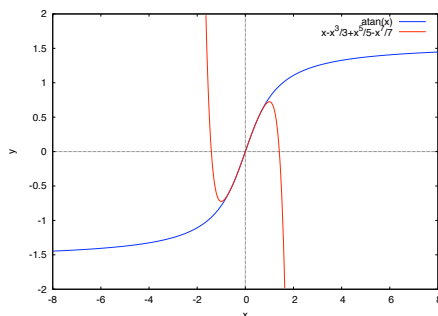
```
(%i26) plot2d([f(x),taylor(f(x),x,0,14)], [x,-8,8], [y,-2,2]);
```

```
(%o26)
```



El hecho de que la función coseno y su polinomio de Taylor se parezcan tanto como se quiera, con sólo aumentar el grado del polinomio lo suficiente, no es algo que le ocurra a todas las funciones. Para la función arcotangente la situación no es tan buena:

```
(%i27) g(x):=atan(x);
(%o27) g(x):=atan(x)
(%i28) plot2d([g(x),taylor(g(x),x,0,8)],[x,-8,8],[y,-2,2]);
(%o28)
```



sólo se parecen, al menos eso se ve en la gráfica, en el intervalo $] -1, 1[$ (a ojo).

Si es necesario, podemos destacar un coeficiente particular, ya sea de un polinomio cualquiera o de un polinomio de Taylor.

```
(%i29) coeff(3*x^4+3*x+1,x,4);
(%o29) 3
(%i30) coeff((taylor(cos(x),x,0,5)),x,2);
(%o30)/R/ -1/2
```

o también se puede escribir

```
(%i31) coeff(3*x^2+2*x+1,x^2);
(%o31) 2
```

Maxima y los polinomios de Taylor

Maxima tiene dos formas de representar internamente los polinomios. Sin entrar en detalles, no se guardan de la misma forma un polinomio de Taylor y un polinomio cualquiera. Esto puede dar lugar a algunas sorpresas. *Maxima* señala esto añadiendo /T/ en la celda correspondiente a un polinomio de Taylor.

<code>ratdisrep(expr)</code>	devuelve el polinomio de Taylor convertido en una expresión racional
<code>taylorp(polimonio)</code>	devuelve true si el polinomio es un polinomio de Taylor

Cuando calculamos el polinomio de Taylor de una función, *Maxima* calcula más términos de los especificados y presenta por pantalla el desarrollo hasta el orden pedido añadiendo unos puntos suspensivos.

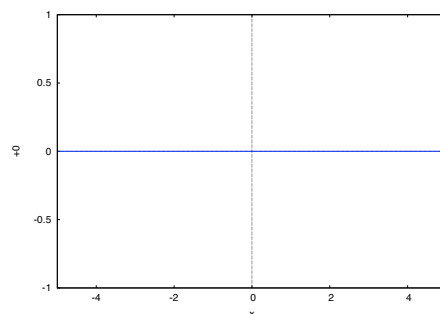
```
(%i32) f(x):=cos(x);
(%o32) f(x):=cos(x)
(%i33) define(g(x),taylor(f(x),x,0,5));
(%o33)/T/ g(x):=1- $\frac{x^2}{2}+\frac{x^4}{24}+\dots$ 
```

Estamos en una situación parecida a cuando mezclamos en una misma expresión números reales de precisión arbitraria y números con una precisión fija. *Maxima* los reduce todos a una forma común, precisión fija en este caso, para poder trabajar con ambos al mismo tiempo. En nuestro caso, *Maxima* convierte la función coseno en un ‘polinomio’ como el de Taylor. Por ejemplo, si calculamos la diferencia,

```
(%i34) f(x)-g(x);
(%o34)/T/ 0+...
```

al convertir $f(x)$ en un polinomio de Taylor, el resultado es cero. Una imagen vale más que mil palabras. y dibujamos la diferencia

```
(%i35) plot2d(f(x)-g(x),[x,-5,5]);
(%o35)
```



Si evaluamos en algún punto, ambos resultados, $f(2)$ y $g(2)$ en este caso, se calculan por separado y obtenemos el resultado correcto.

```
(%i36) f(2)-g(2);
(%o36)  $\frac{3\cos(2)+1}{3}$ 
```

Puedes comprobar que una expresión es un ‘polinomio de Taylor’ usando la orden `taylorp(expr)`

```
(%i37) taylorp(f(x)-g(x));
(%o37) true
```

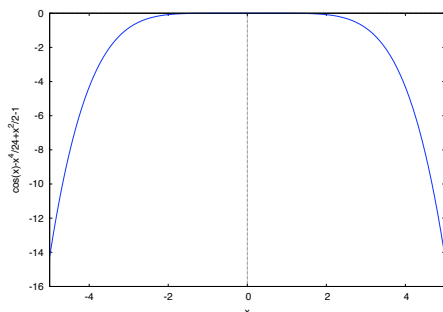
```
(%i38) aylorp(1+x^2);
(%o38)  false
```

La orden `ratdisrep(polinomio de Taylor)` nos permite pasar de polinomio de Taylor a `ratdisrep` polinomio “normal”.

```
(%i39)  f(x)-ratdisrep(g(x));
(%o39)  cos(x)-x^4/24+x^2/2-1
```

Observa como no aparece /T/ en la salida. El gráfico ya es el correcto también.

```
(%i40)  plot2d(f(x)-ratdisrep(g(x)), [x, -5, 5]);
(%o40)
```



Resumiendo, puede ser útil definir la función $g(x)$ como

```
(%i41)  define(g(x), ratdisrep(taylor(cos(x), x, 0, 5)))$
```

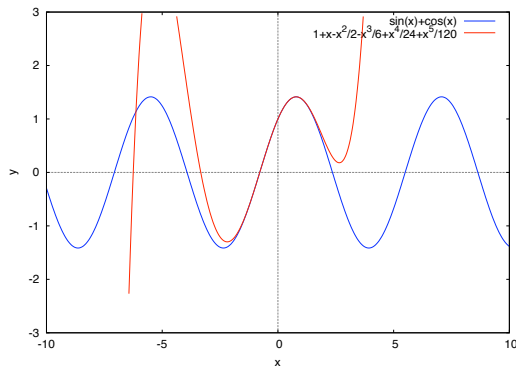
en lugar de usar sólo `taylor`.

Animaciones con polinomios

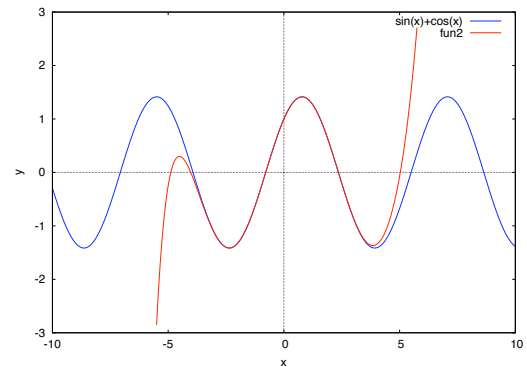
Hasta aquí hemos visto cómo comparar la gráfica de una función con la de su polinomio de Taylor. Ahora bien, en lugar de ir dibujando una función y un polinomio de Taylor, parece más interesante dibujar la función y varios polinomios para ir comprobando si se parecen o no a dicha función cuando aumenta su orden. La orden `with_slider` (que ya conoces) nos va a permitir hacer animaciones de la gráfica de una función y sus polinomios de Taylor. Por ejemplo:

```
(%i42)  f(x):=sin(x)+cos(x);
(%o42)  f(x):=sin(x)+cos(x)
(%i43)  with_slider(
        n,
        makelist(i,i,1,20),
        [f(x), taylor(f(x), x, 0, n)], [x, -10, 10], [y, -3, 3]
    );
```

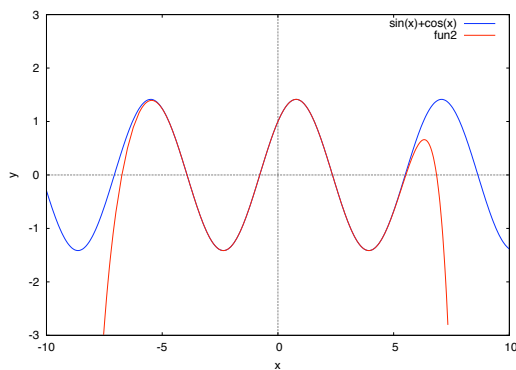
nos permite dibujar los primeros 20 polinomios de Taylor de la función f . En la Figura 13.2 tienes algunos pasos intermedios representados.



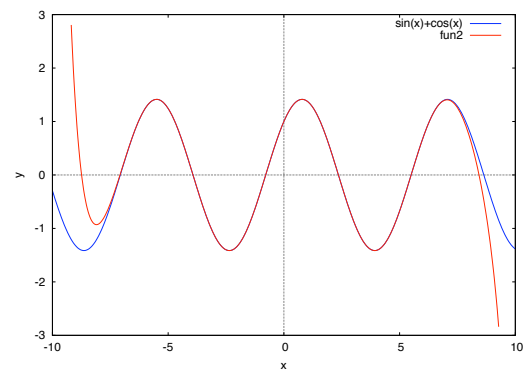
Orden 5



Orden 10



Orden 15



Orden 20

Figura 13.2 Función $\text{sen}(x) + \text{cos}(x)$ y sus polinomios de Taylor

Prueba con otras funciones a ver qué resultados se obtienen. Algunas ideas: prueba con la exponencial y el logaritmo o algunas del resto de funciones trigonométricas o hiperbólicas.

13.3.1 Ejercicios

Ejercicio 13.6. ¿Es cierto o falso que el polinomio de Taylor de una función al cuadrado es el cuadrado del polinomio?

Ejercicio 13.7. Estudia los extremos relativos del polinomio de Taylor de orden 5 centrado en el origen de la función $f(x) = \cos(x) + e^x$.

Derivación e integración numérica

14

14.1 Derivación numérica 203 14.2 Integración numérica 204 14.3 Métodos simples 205 14.4 Métodos de aproximación compuestos 207

El cálculo de la derivada de o de la integral de una función no siempre es fácil. En este capítulo vamos a ver cómo podemos aproximarlos. Una de las herramientas claves es la interpolación. Para polinomios sí es factible calcular derivadas e integrales. Aprovecharemos tanto la interpolación de Lagrange como la de Taylor para aproximar una función por un polinomio.

14.1 Derivación numérica

A veces ocurre que calcular la derivada de una función f en un punto a del interior de dominio no es fácil, ya sea por la complejidad de la función dada, ya sea porque sólo dispongamos de una tabla de valores de f . En esta sección vamos a establecer métodos para calcular $f'(a)$. No es que vayamos a calcular la función derivada primera de f , sino que vamos a aproximar los valores de ésta en un punto dado a .

Hemos visto en este capítulo que la derivada de una función $f: I \rightarrow \mathbb{R}$ en un punto $a \in I$ es

$$f'(a) = \lim_{x \rightarrow a} \frac{f(x) - f(a)}{x - a} = \lim_{h \rightarrow 0} \frac{(a + h) - f(a)}{h}.$$

Si consideramos un valor de h suficientemente pequeño, podemos dar una primera aproximación

$$f'(a) \approx \frac{f(a + h) - f(a)}{h}.$$

14.1.1 Fórmulas de derivación numérica

En las fórmulas que vamos a estudiar en este apartado aparecen dos valores: el que aproxima $f'(a)$ y el error cometido. Aunque este último no se calcula explícitamente, sí se puede dar una acotación del mismo. Notemos que dicho error se obtiene gracias al desarrollo de Taylor de f centrado en a .

En lo que sigue, el parámetro h se suele tomar positivo y “pequeño”.

a) Fórmula de dos puntos:

$$f'(a) = \frac{1}{h} (f(a) - f(a - h)) + \frac{h}{2} f''(\psi), \quad \psi \in]a - h, a[$$

Esta fórmula se llama *regresiva* porque utiliza información de f en a y en $a - h$.

$$f'(a) = \frac{1}{h} (f(a + h) - f(a)) - \frac{h}{2} f''(\psi), \quad \psi \in]a, a + h[$$

Esta fórmula se llama *progresiva* porque utiliza información de f en a y en $a + h$.

El primer sumando de ambas fórmulas nos da una aproximación de $f'(a)$, y el segundo nos indica el error cometido.

b) Fórmula de tres puntos:

$$f'(a) = \frac{1}{2h} (f(a+h) - f(a-h)) - \frac{h^2}{6} f'''(\psi), \quad \psi \in]a-h, a+h[$$

Esta fórmula se llama *central* porque utiliza información de f en $a-h$ y en $a+h$.

$$f'(a) = \frac{1}{2h} (-3f(a) + 4f(a+h) - f(a+2h)) + \frac{h^2}{3} f'''(\psi), \quad \psi \in]a, a+2h[$$

Esta fórmula es progresiva.

c) Fórmula de cinco puntos:

$$f'(a) = \frac{1}{12h} (f(a-2h) - 8f(a-h) + 8f(a+h) - f(a+2h)) - \frac{h^4}{30} f^{(5)}(\psi), \quad \psi \in]a-2h, a+2h[$$

Ésta es central.

$$f'(a) = \frac{1}{12h} (-25f(a) + 48f(a+h) - 36f(a+2h) + 16f(a+3h) - 3f(a+4h)) + \frac{h^4}{5} f^{(5)}(\psi), \quad \psi \in]a, a+4h[$$

Esta es progresiva.

Unas observaciones sobre el término del error:

- Cuanto mayor es el exponente de h en la fórmula del error, mejor es la aproximación.
- Cuanto menor es la constante que aparece en la fórmula del error, mejor es la aproximación.
- Cuidado con los errores de redondeo cuando trabajemos con h excesivamente pequeño. Puede ocurrir que por la aritmética del ordenador, la aproximación numérica sea peor cuanto más pequeño sea h .

Ejemplo 14.1. FALTA (PARA PRÁCTICAS)

14.2 Integración numérica

14.2.1 Introducción

El cálculo de la integral de una función en un intervalo $[a, b]$ a través de la regla de Barrow puede ser en algunos casos no sólo complicado, sino prácticamente imposible por varias razones, principalmente dos.

La primera razón es que nos sea imposible calcular una primitiva elemental de la función a integrar, este caso es muy usual. Por ejemplo la función $f(x) = (1+x^2)^{1/3}$ no somos capaces de calcularle una primitiva expresable con funciones elementales. Otro ejemplo, muy usual éste, es la función de densidad $f(x) = e^{-x^2}$ que tantas veces aparece.

La segunda razón para no poder aplicar la regla de Barrow es que no conozcamos la expresión de la función que queremos integrar o simplemente que no conozcamos el valor de la función en todos los puntos del intervalo donde está definida (le llamaremos $[a, b]$) sino solamente en algunos puntos de dicho intervalo.

Por todo lo anterior, a veces hay que recurrir a métodos de aproximación para calcular el valor de la integral de una función en un intervalo. Ya que la propia definición de la integral viene dada por un límite (de sumas superiores y sumas inferiores), una aproximación a dicho límite podría considerarse un primer método de aproximación numérica de la integral.

Los métodos de aproximación del cálculo de la integral que veremos se llaman *métodos o reglas de integración numérica* y consisten, básicamente, en dada $f: [a, b] \rightarrow \mathbb{R}$ continua, aproximamos dicha función por un polinomio adecuado. Así, la integral de dicho polinomio será una aproximación

de la integral de f . Estos métodos son los llamados *métodos simples*, que, si bien son muy sencillos de enunciar, la aproximación que dan deja mucho que desear en muchos casos. Un análisis posterior nos dará los *métodos compuestos* que producen una aproximación más buena. Primero veremos los simples.

14.3 Métodos simples

14.3.1 Método del trapecio simple

El método del trapecio simple consiste en considerar como polinomio de aproximación de la función f en el intervalo $[a, b]$ el polinomio de interpolación de Lagrange de la función f en $x_0 = a$ y $x_1 = b$ y se toma $\int_a^b P_1(x)dx$ como aproximación de $\int_a^b f(x)dx$. El polinomio de interpolación de Lagrange es

$$P_1(x) = f(a) \frac{x-b}{a-b} + f(b) \frac{x-a}{b-a},$$

lo que nos proporciona

$$\int_a^b f(x)dx \approx \int_a^b P_1(x)dx = \frac{b-a}{2}(f(a) + f(b)).$$

El nombre de método o regla del trapecio está claro si observamos en el siguiente dibujo que aproximación estamos dando de la integral: Si tenemos una función $f: [a, b] \rightarrow \mathbb{R}$ y suponemos que es positiva, la aproximación dada por el método del trapecio es el área del trapecio formado por los puntos $(a, 0)$, $(b, 0)$, $(b, f(b))$ y $(a, f(a))$.

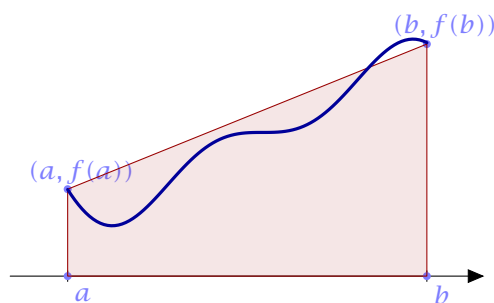


Figura 14.1 Método del trapecio simple

Esta aproximación solo es exacta cuando la función f es un polinomio de grado 1. En cualquier caso tenemos que el error que se comete al aplicar esta regla viene dado por

$$\int_a^b f(x)dx - \frac{b-a}{2}(f(a) + f(b)) = -\frac{1}{12}(b-a)^3 f''(\xi)$$

para conveniente $\xi \in]a, b[$.

14.3.2 Regla del punto medio simple

Para construir esta aproximación de la integral de la función f en el intervalo $[a, b]$ se considera P_0 el polinomio de aproximación de Lagrange de la función en el punto $x_0 = \frac{a+b}{2}$ y se toma $\int_a^b P_0(x)dx$ como aproximación de $\int_a^b f(x)dx$. Como $P_0(x) = f(\frac{a+b}{2})$, entonces la aproximación queda

$$\int_a^b f(x)dx \approx \int_a^b P_0(x)dx = (b-a)f\left(\frac{a+b}{2}\right)$$

que, si f es positiva en el intervalo $[a, b]$, nos da como aproximación del área que queda entre el eje de abscisas y la gráfica de f entre a y b , el área del rectángulo de base $b-a$ (la longitud de intervalo $[a, b]$) y altura $f\left(\frac{a+b}{2}\right)$.

Claramente esta fórmula es exacta cuando la función f es constante en el intervalo $[a, b]$, y, en general, el error cometido al tomar esta aproximación es

$$\int_a^b f(x)dx - (b-a)f\left(\frac{a+b}{2}\right) = \frac{1}{24}(b-a)^3 f''(\xi),$$

para algún $\xi \in]a, b[$.

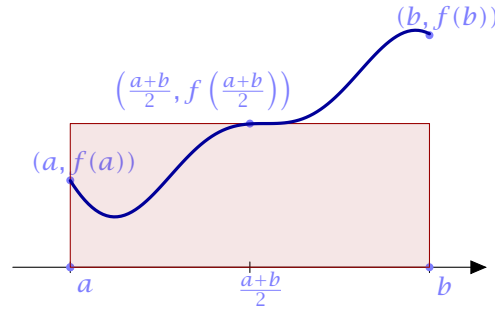


Figura 14.2 Método del punto medio simple

14.3.3 Regla de Simpson simple

En este caso se toma como aproximación de la función P_2 , el polinomio de interpolación de Lagrange de la función f en los puntos $x_0 = a$, $x_1 = \frac{a+b}{2}$ y $x_2 = b$. Como aproximación de $\int_a^b f(x)dx$ se toma $\int_a^b P_2(x)dx$. Así el método de Simpson simple nos da

$$\int_a^b f(x)dx \approx \frac{(b-a)}{6} \left(f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right).$$

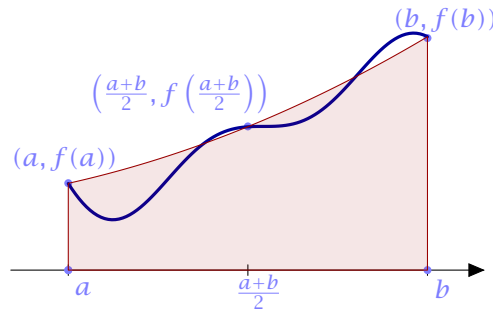


Figura 14.3 Regla de Simpson simple

Este método es exacto cuando la función f es un polinomio de grado menor o igual que 3. El error cometido al sustituir el valor exacto de la integral por la aproximación que nos da la regla de Simpson vale

$$\int_a^b f(x)dx - \frac{(b-a)}{6} \left(f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right) = -\frac{1}{90 \cdot 2^6} (b-a)^5 f^{(4)}(\xi),$$

para algún $\xi \in]a, b[$.

Está claro que, al ser P_2 una mejor aproximación, en general, que P_0 y P_1 , la regla de Simpson es mejor aproximación a la integral que la regla del trapecio y la del punto medio. La siguiente tabla muestra las aproximaciones dadas por las tres reglas que hemos presentado.

ahora viene una tabla que tengo que hacer.

En vista de las fórmulas de los errores cometidos al tomar como aproximación cualquiera de las fórmulas anteriores, estos errores dependen mucho de la longitud del intervalo $[a, b]$. En todos las fórmulas de los errores aparece un término $(b - a)$ elevado a una potencia. Está claro que si $(b - a)$ es muy grande la aproximación no es muy buena. Este hecho además responde a la intuición: no podemos pretender que si el intervalo de definición de la función es muy grande al tomarnos como aproximación de la función un polinomio de grado 0 (e incluso 1 o 2) obtengamos una muy buena aproximación.

Si en el ejemplo de la tabla anterior aumentamos la longitud del intervalo veamos qué ocurre:

Otra tabla con el mismo ejemplo con un intervalo más grande y comentar los errores que se cometen ahora.

14.4 Métodos de aproximación compuestos

Una forma de resolver este problema consiste en dividir el intervalo $[a, b]$ en n subintervalos de idéntica longitud y aplicar un método de aproximación simple en cada uno de los subintervalos obtenidos y después sumar las aproximaciones obtenidas. La aditividad de la integral respecto al intervalo nos garantiza que la suma obtenida es una aproximación de la integral de la función en el intervalo $[a, b]$. Estos métodos de aproximación se llaman métodos de aproximación compuesta y vamos a comentar brevemente los tres correspondientes a los métodos de aproximación simple que acabamos de estudiar.

Lo primero que tenemos que hacer, en los tres métodos, es dividir el intervalo en n subintervalos de igual longitud. Para tal fin consideramos los puntos

$$x_k = a + k \frac{b-a}{n}, \quad k = 0, 1, \dots, n, \quad (14.1)$$

donde n es un número natural. Estos puntos x_k , para $k = 0, 1, \dots, n$ reciben el nombre de nodos. Si ahora aplicamos cada uno de los métodos de aproximación simples a la función en cada uno de los intervalos $[x_k, x_{k+1}]$ obtenemos en cada caso las siguientes fórmulas:

Método del trapecio compuesto

Sean $x_0, x_1, \dots, x_n$ los nodos dados por (14.1). La aproximación de la integral $\int_a^b f(x) dx$ utilizando el método del trapecio compuesto con los subintervalos dados por los anteriores nodos nos proporciona

$$T_n = \frac{(b-a)}{2n} \left(f(a) + 2 \sum_{i=1}^{n-1} f(x_i) + f(b) \right),$$

y el error cometido queda estimado por la fórmula

$$\int_a^b f(x) dx - T_n = -\frac{1}{12} \frac{(b-a)^3}{n^2} f''(\xi)$$

donde ξ es un número del intervalo $]a, b[$.

Método del punto medio compuesto

Considerando los mismos nodos que en el apartado anterior la regla del punto medio compuesto nos da una aproximación

$$M_n = \frac{(b-a)}{n} \sum_{i=0}^{n-1} f\left(a + \frac{(2i+1)(b-a)}{2n}\right).$$

En este caso la expresión del error nos queda

$$\int_a^b f(x)dx - M_n = \frac{(b-a)^3}{24n^2} f''(\xi)$$

para $\xi \in]a, b[$.

Método de Simpson compuesto

En este caso necesitamos que el número n sea par. Usando el método de Simpson en cada intervalo $[x_k, x_{k+1}]$ obtenemos la aproximación

$$S_n = \frac{(b-a)}{3n} [(f(x_0) + 4f(x_1) + f(x_2)) + (f(x_2) + 4f(x_3) + f(x_4)) + \dots + (f(x_{n-2}) + 4f(x_{n-1}) + f(x_n))].$$

Haciendo un fácil cambio de variable obtenemos que la anterior expresión coincide con

$$S_n = \frac{(b-a)}{3n} \sum_{i=1}^{n/2} [f(x_{2i-2}) + 4f(x_{2i-1}) + f(x_{2i})]$$

y el error que se comete viene dado por

$$\int_a^b f(x)dx - S_n = -\frac{1}{180} \frac{(b-a)^5}{n^4} f^{(4)}(\xi)$$

para un $\xi \in]a, b[$.

El error en los tres métodos depende inversamente del número de nodos n . Pero mientras que en el método del trapecio y en el punto medio el error es inversamente proporcional a n^2 en el método de Simpson es inversamente proporcional a n^4 . Está claro por tanto que el método de Simpson es más exacto que los otros dos cuando aumentamos el número de nodos.

Ejemplo 14.2. Consideremos la integral de la función $f(x) = e^x \sin(x)$ en el intervalo $[0, 1]$. A esta función se le puede calcular una primitiva fácilmente integrando por partes (¡hágase!) y evaluando en 0 y en 1 queda

$$\int_0^1 e^x \sin(x) dx = 0.90933067363148.$$

Trapecio	Aproximación	Error
n=20	0.9670887643	5.78×10^{-2}
n=40	0.9380661609	2.87×10^{-2}
n=100	0.9207904171	1.15×10^{-2}
Punto medio	Aproximación	Error
n=20	0.9090435575	2.87×10^{-4}
n=40	0.9092588997	7.18×10^{-5}
n=100	0.9093191900	1.15×10^{-5}
Simpson	Aproximación	Error
n=20	0.9093305474	1.26×10^{-7}
n=40	0.9093306657	7.89×10^{-9}
n=100	0.9093306734	2.02×10^{-10}

Tabla 14.1 Fórmulas de integración compuesta

Ecuaciones diferenciales

Ecuaciones diferenciales ordinarias

15

15.1 Resolución de edos de primer y segundo orden 213 15.2 Resolución de sistemas de ecuaciones lineales 216 15.3 Campos de direcciones y curvas integrales 217 15.4 Ejercicios 220

15.1 Resolución de edos de primer y segundo orden

La posibilidad de resolver una ecuación diferencial que has visto en clase pasa por ser capaz de encuadrarla dentro de un tipo que sepamos resolver: decidir si es lineal, de variables separadas, o cualquier otro. Al igual que `integrate` calcula integrales sin decirnos de qué tipo es la función que estamos integrando, aquí va a pasar lo mismo. Vamos a tener una orden que resuelve ecuaciones diferenciales y punto.

`ode2(ecuación diferencial,y,x)` resuelve la ecuación diferencial

La orden `ode2` resuelve algunas ecuaciones diferenciales ordinarias de primer y segundo orden. `ode2` Para escribir una de estas ecuaciones utilizamos el comando `diff` teniendo la precaución de utilizar la comilla, `'`, para evitar que *Maxima* calcule la derivada. Comencemos por una ecuación de variables separadas: la definimos

```
(%i1) edo:(2*x+3)*'diff(y,x)+y*(x-1)=0;
(%o1) (2x+3)  $\frac{d}{dx}$  y + (x-1)y = 0
```

y la resolvemos

```
(%i2) ode2(edo,y,x);
(%o2) y = %c %e  $\frac{5 \log(2x+3)}{4} - \frac{x}{2}$ 
```

La solución, en este caso dada en forma explícita, depende de un parámetro `%c`. Hay veces en que *Maxima* da la solución en forma implícita. Por ejemplo, la ecuación de variables separadas

```
(%i3) edo1:x*'diff(y,x)+cos(y)*(x-1)=0;
(%o3) x  $\frac{d}{dy}$  y + cos(y)(x-1) = 0
(%i4) ode2(edo1,y,x);
(%o4)  $-\frac{\log(\sin(y)+1)-\log(\sin(y)-1)}{2} = -\log(x)+x+\%c$ 
```

En este caso, *Maxima* no es capaz de despejar y en función de x , pero esto no siempre es así:

```
(%i5) ode2(x*'diff(y,x)+y^2*(x-1)=0,y,x);
```

```
(%o5)  $\frac{1}{y} = -\log(x) + x + \%c$ 
```

```
(%i6) solve(%y)
```

```
(%o6)  $[y = -\frac{1}{\log(x) - x - \%c}]$ 
```

También podemos resolver ecuaciones exactas,

```
(%i7) ode2(x^2*(3*y^2+1)+y*(2*x^3+y)*'diff(y,x),y,x);
```

```
(%o7)  $\frac{y^3+3x^3y^2+x^3}{3} = \%c$ 
```

ecuaciones lineales,

```
(%i8) ode2('diff(y,x)=x+y,y,x);
```

```
(%o8)  $y = ((-x-1) * \%e^{(-x)} + \%c) * \%e^x$ 
```

ecuaciones homogéneas, etc.

```
(%i9) ode2((x^2+y^2)+x*y*'diff(y,x),y,x);
```

```
(%o9)  $\frac{2*x^2*y^2+x^4}{4} = \%c$ 
```

Incluso podemos resolver ecuaciones de segundo orden, ya sean homogéneas,

```
(%i10) ode2('diff(y,x,2)-3*'diff(y,x)+2*y,y,x);
```

```
(%o10)  $y = \%k1 * \%e^{(2 * x)} + \%k2 * \%e^x$ 
```

o no.

```
(%i11) ode2('diff(y,x,2)-3*'diff(y,x)+2*y=2*x,y,x);
```

```
(%o11)  $y = \%k1 * \%e^{(2 * x)} + \%k2 * \%e^x + \frac{2x+3}{2}$ 
```

15.1.1 Condiciones iniciales o de contorno

En todas las soluciones de las ecuaciones diferenciales anteriores aparecen una o dos constantes dependiendo de si se trata de ecuaciones de primer o segundo orden. Volvamos, por ejemplo a la primera ecuación de este tema y consideremos el problema de valores iniciales

$$(2x + 3)y' + y(x - 1) = 0$$

$$y'(0) = 2$$

La solución era

```
(%i12) ode2(edo,y,x);
(%o12) y=%c %e $\frac{5\log(2x+3)}{4}$  -  $\frac{x}{2}$ 
```

<code>ic1(ecuación,x=a,y=b)</code>	resuelve problema de valores iniciales de primer orden
<code>ic2(ecuación,x=a,y=b,diff(y,x)=c)</code>	resuelve problema de valores iniciales de segundo orden
<code>bc2(ecuación,x=a,y=b,x=c,y=d)</code>	resuelve problema de contorno

El comando `ic1` permite ajustar el valor de la constante `%c`:

ic1

```
(%i13) ic1(%,x=0,y=2);
(%o13) y=2*%e $\frac{5\log(2x+3)}{4}$  -  $\frac{x}{2}$  -  $\frac{5\log(3)}{4}$ 
```

y el comando `ic2` juega el mismo papel para ecuaciones de segundo orden.

ic2

Ejemplo 15.1. Vamos a resolver el problema de valores iniciales

$$\begin{aligned} y'' - y' &= e^x, \\ y(0) &= 0, \\ y'(0) &= 1. \end{aligned}$$

```
(%i14) ode2('diff(y,x,2)-'diff(y,x)=e^x,y,x);
(%o14) y=(x-1)%e^x+%k1%e^x+%k2
(%i15) ic2(%,x=0,y=0,diff(y,x)=1);
(%o15) y=(x-1)%e^x+%e^x
```

Los problemas de valores de contorno se resuelven de manera similar pero con `bc2`. Por ejemplo `bc2` la solución del problema

$$\begin{aligned} y'' - y' &= e^x, \\ y(0) &= 0, \\ y(1) &= 0. \end{aligned}$$

la calculamos rápidamente aprovechando el ejemplo anterior:

```
(%i16) bc2(%o14,x=0,y=0,x=1,y=0);
(%o16) y=(x-1)%e^x -  $\frac{\%e^x}{\%e-1}$  +  $\frac{\%e}{\%e-1}$ 
```

15.2 Resolución de sistemas de ecuaciones lineales

desolve La orden **desolve** resuelve ecuaciones o sistemas de ecuaciones diferenciales ordinarias lineales mediante la transformada de Laplace.

desolve(edos, vars)	resuelve la ecuación diferencial lineal
atvalue(f(x), x=a, b)	$f(a) = b$

La forma de escribir las ecuaciones o el sistema de ecuaciones es muy parecida a los comandos que hemos visto, pero hay que indicar explícitamente todas las dependencias entre variables. Por ejemplo, en lugar de escribir `'diff(f,x)` tendremos que escribir `'diff(f(x),x)` en el caso de que f sea función de x . Veamos un ejemplo.

Para resolver la ecuación diferencial $y'' - 2y = 10x$ escribimos

```
(%i17) desolve('diff(y(x),x,2)-2*y(x)=10*x,[y(x)]);
```

```
(%o17) y(x)= $\frac{\sinh\left(2^{\frac{1}{2}}x\right)\left(\frac{d}{dx}y(x)\Big|_{x=0}+5\right)}{\sqrt{2}}+y(0)\cosh\left(2^{\frac{1}{2}}x\right)-5x$ 
```

y la solución del sistema de ecuaciones

$$x'(t) = 3x + 4y$$

$$y'(t) = 4x - y$$

se obtiene mediante

```
(%i18) desolve(['diff(x(t),t)=3*x(t)+4*y(t),'diff(y(t),t)=4*x(t)-y(t)],
               [x(t),y(t)]);
```

```
(%o18) [x(t)=e^t *  $\left(\frac{(2(4y(0)+x(0))+2x(0))\sinh(2\sqrt{5}t)}{4\sqrt{5}}+x(0)\cosh(2\sqrt{5}t)\right)$ ,
        y(t)=e^t *  $\left(\frac{(2y(0)+2(4x(0)-3y(0)))\sinh(2\sqrt{5}t)}{4\sqrt{5}}+y(0)\cosh(2\sqrt{5}t)\right)$ ]
```

Una cuestión importante cuando queramos añadir condiciones iniciales *en el origen* es que éstas deben de escribirse antes de resolver el sistema. Por ejemplo, si queremos resolver

$$x'(t) = 3x + 4y$$

$$y'(t) = 4x - y$$

$$x(0) = 1$$

$$y(0) = 0$$

atvalue primero indicamos el valor de x e y con el comando **atvalue**

```
(%i19) atvalue(x(t),t=0,1);
```

```
(%o19) 1
```

```
(%i20)  atvalue(y(t),t=0,0);
(%o20)  0
```

y, después de esto, resolvemos el sistema

```
(%i21)  desolve(['diff(x(t),t)=3*x(t)+4*y(t)', 'diff(y(t),t)=4*x(t)-y(t)'],
               [x(t),y(t)]);
(%o21)  [x(t)=%e^t * (sinh(2*sqrt(5)*t)/sqrt(5) + cosh(2*sqrt(5)*t)), y(t)=2*%e^t * sinh(2*sqrt(5)*t)/sqrt(5)]
```

15.3 Campos de direcciones y curvas integrales

Hemos visto en clase que podemos hacernos una idea de cuál es la solución de una ecuación diferencial de primer orden $y' = f(x, y)$. Lo que hacemos es dibujar en cada punto del plano (x, y) un segmento que nos indique la pendiente en ese punto, dada por $f(x, y)$.

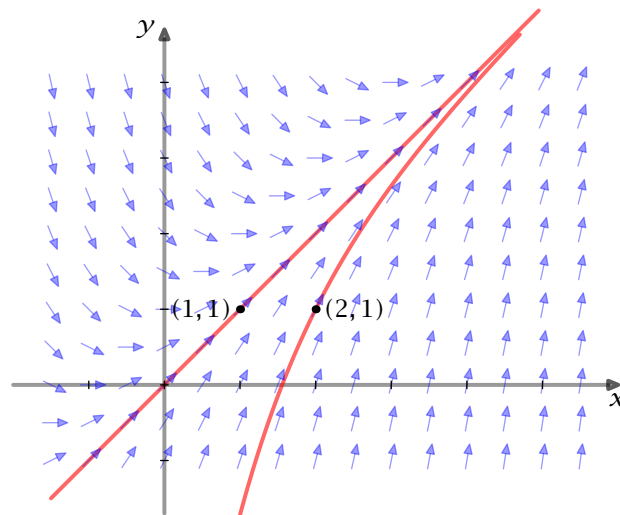


Figura 15.1 Campo de pendientes y curvas integrales

Una vez que tenemos las pendientes dibujadas, una curva es solución de la ecuación diferencial si en cada punto sigue las direcciones marcadas. Este dibujo puede, por tanto, darnos información sobre las soluciones de una ecuación diferencial.

```
plotdf(func, opciones)  dibuja el campo de direcciones dado por func
```

El módulo `plotdf` nos permite realizar este tipo de gráficos. Como todos los módulos adicionales, comenzamos cargándolo.

```
(%i22)  load(plotdf)$
```

En su versión más fácil, podemos dibujar el campo de direcciones de la ecuación $y' = -x/y$ de la siguiente forma:

```
(%i23) plotdf(-x/y);
```

y obtenemos la ventana de la Figura 15.2.

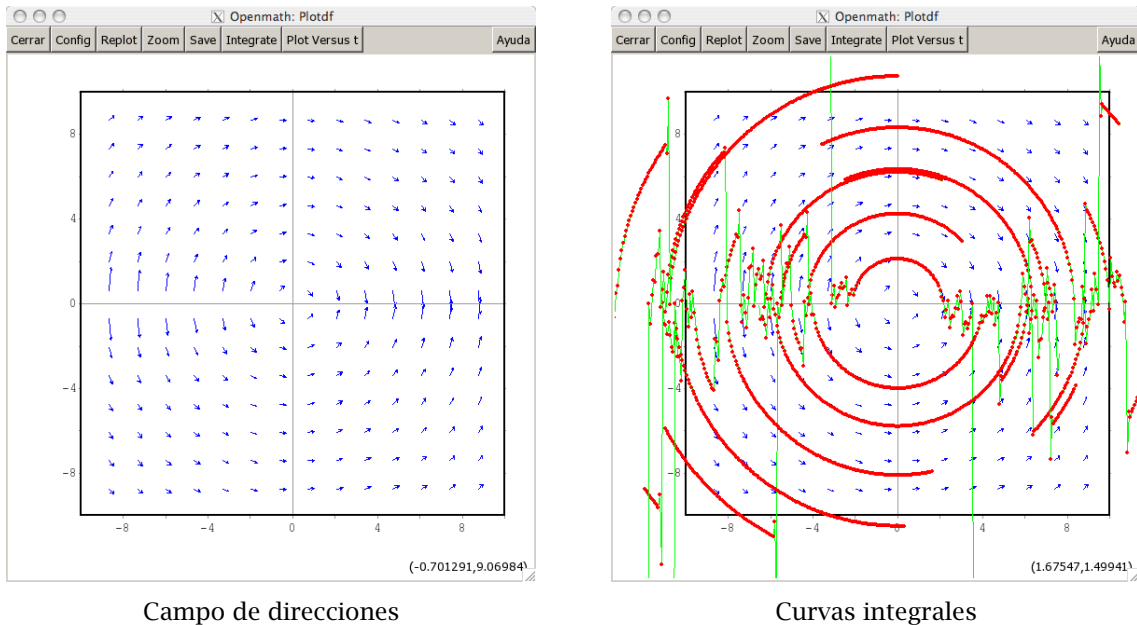


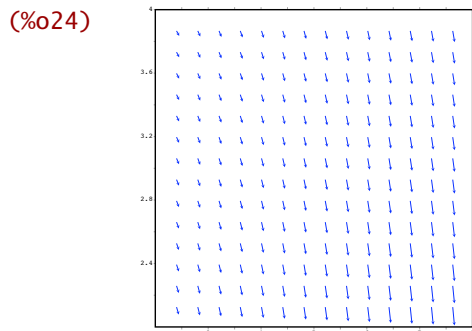
Figura 15.2 Plotdf y Openmath

En primer lugar, observa que estamos utilizando *Openmath* para representar los gráficos y no *Gnuplot* como venía siendo habitual. En segundo lugar, ¿qué pasa cuándo pinchas con el ratón en un punto cualquiera de la gráfica? Lo que ocurre es que se representa la correspondiente curva integral que pasa por dicho punto. Puedes repetir la operación las veces que quieras y vas obteniendo una imagen parecida a la segunda de la Figura 15.2.

Opciones

Hay muchas opciones con las que modificar el comportamiento de la orden `plotdf`. Quizá la primera que alguno se pregunte es cómo variar el rango que se representa de x e y . A diferencia de otros gráficos, aquí se da el centro y el “radio” del gráfico. Por defecto el centro es el origen y el radio en ambos ejes es 10 con lo que las representaciones se hacen en $[-10, 10] \times [-10, 10]$. Las opciones `xcenter`, `xradius`, `ycenter` e `yradius` nos permiten variar esto.

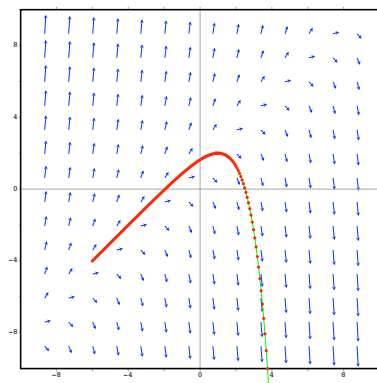
```
(%i24) plotdf(-x/y, [xradius,3], [xcenter,5], [yradius,1], [ycenter,3]);
```



Podemos fijar de partida la curva integral que deseamos dibujar eligiendo el punto por el que pasa con la opción `trajectory_at`. Por ejemplo la curva integral de la ecuación $y' = -x + y - 1$ que pasa por el punto $(1, 2)$ tiene el siguiente aspecto.

```
(%i25) plotdf(-x+y-1,[trajectory_at,1,2]);
```

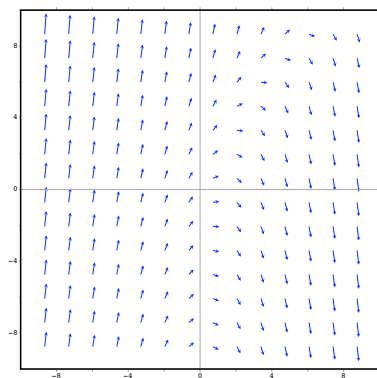
```
(%o25)
```



Es relativamente común encontrar ecuaciones diferenciales que dependen de algún parámetro. La opción `parameters` permite pasar valores de los parámetros a `plotdf`

```
(%i26) plotdf(-x+exp(-k*y),[parameters,"k=-0.2"]);
```

```
(%o26)
```



aunque claro, lo interesante sería ver qué ocurre cuando damos distintos valores al parámetro. Eso es justamente lo que permite la opción `sliders`: dar varios valores al parámetro. Por ejemplo

```
(%i27) plotdf(-x+exp(-k*y),[parameters,"k=-0.2"],[sliders,"k=-3:3"]);
```

nos da como resultado la ventana de la Figura 15.3

Si te fijas, en la parte inferior tienes un botón que te permite cambiar los valores del parámetro k . Como hemos escrito la opción `[parameters,"k=-0.2"]`, la gráfica que presenta *Maxima* es justamente esa y, por supuesto, podemos seguir dibujando curvas integrales que siguen cambiando automáticamente cuando variamos el valor de parámetro.

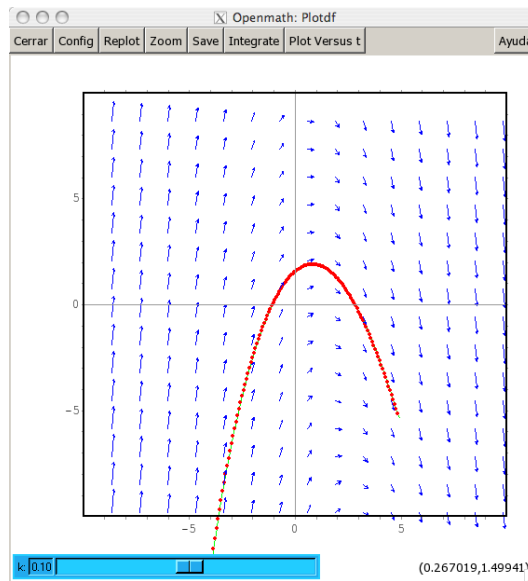


Figura 15.3 Parámetros en plotdf

15.4 Ejercicios

Ejercicio 15.1. Resuelve las ecuaciones diferenciales:

- | | |
|---|--|
| a) $(2x + 3)y' + xy = 0.$ | g) $(x + y + 1)dx - (x - y + 1)dy = 0.$ |
| b) $y' = x + y$ | h) $2x^2 \frac{dy}{dx} = 3xy + y^2; \quad y(1) = -2$ |
| c) $y' - \frac{\cos(x)}{\sin(x)} y = 4 \sin^2(x)$ | i) $y' + 2y = e^{-x}$ |
| d) $y \cos(x)dx + \sin(x)dy = 0,$ | j) $y'' + 3y' - 4y = 0, \quad y(0) = 2, \quad y'(0) = -3$ |
| e) $(x^2 + y^2)dx + xydy = 0.$ | k) $y'' + 4y = 0, \quad y(\pi/6) = 1, \quad y'(\pi/6) = 0$ |
| f) $(x^2 - y^2)dx + x^2dy = 0.$ | l) $y'' - y = xe^{3x}, \quad y(0) = 0, \quad y'(0) = 1$ |

Ejercicio 15.2. Resolver los siguientes sistemas de ecuaciones diferenciales:

a)

$$x' - 4y = 1$$

$$x + y' = 2$$

b)

$$2x' - 5x + y' = e^t$$

$$x' - x + y' = 5e^t$$

Ejercicio 15.3. Hallar las curvas de $\mathbb{R}^2$ que verifican la siguiente propiedad: en cada punto el vector de posición y el vector tangente son ortogonales.

Ejercicio 15.4. Sabiendo que la semivida del Carbono 14 radioactivo (C-14) es aprox. 5600 años, determinar la edad de un fósil que contiene 1/1000 de la cantidad original de C-14.

Ejercicio 15.5. Utiliza el método de las poligonales de Euler para aproximar la solución del problema de valores iniciales

$$y' = x + y, \quad y(0) = 1.$$

Cálculo en varias variables

Diferenciación

16

16.1 Derivadas parciales de una función 225 16.2 Curvas y vectores tangentes 228 16.3 Funciones definidas implícitamente 230 16.4 Extremos relativos 235 16.5 Extremos condicionados 237 16.6 Extremos absolutos 239 16.7 Ejercicios 240

En este capítulo debes aprender a calcular y evaluar derivadas parciales de cualquier orden de una función de varias variables, a aplicar el teorema de la función implícita así como a calcular extremos de funciones reales de varias variables, tanto relativos como condicionados o absolutos. Aprovecharemos la capacidad gráfica de *Maxima* para tener una idea más clara de nociones como el plano tangente.

16.1 Derivadas parciales de una función

El primer objetivo será aprender a calcular derivadas parciales, gradientes, hessianos... de funciones de varias variables, y aplicar todo ello al cálculo de extremos relativos y condicionados.

Recordemos que el comando `diff` nos servía para calcular derivadas de funciones de una variable. Pues bien, también nos va a servir para calcular derivadas parciales de campos escalares de varias variables. Esto es

`diff(f(x1, x2 ...), x1, n1, x2, n2, ...)` derivada parcial de la función $f(x_1, x_2, \dots)$ respecto a x_1 , n_1 veces, respecto a x_2 , n_2 veces,...

Calculemos como ejemplo $\frac{\partial^3 f}{\partial^2 x \partial y}$ y $\frac{\partial^2 f}{\partial x \partial y}$, donde $f(x, y) = \sin(x) \cos(y^2)$:

```
(%i1) f(x,y):= sin(x)*cos(y^2)$
(%i2) diff(f(x,y),x,2,y,1);
(%o2) 2sin(x)y sin(y^2)
(%i3) diff(f(x,y),x,1,y,1);
(%o3) -2cos(x)y sin(y^2)
```

Con las derivadas parciales de primer orden construimos el vector gradiente ($\nabla f(a)$) de un campo escalar en un punto, y la matriz jacobiana ($Jf(a)$) de un campo vectorial en un punto. Para ambos cálculos utilizaremos el mismo comando: `jacobian(f, x)`. Por ejemplo:

jacobian

```
(%i4) jacobian([f(x,y)], [x,y]);
(%o4) [cos(x) cos(y^2) -2sin(x)y sin(y^2)]
```

El resultado es una matriz fila de dos elementos, o lo que es lo mismo, un vector de dos componentes. Mientras que si ahora aplicamos el mismo comando a un campo vectorial, tenemos:

```
(%i5) jacobian([exp(x+y), x^2-y^2], [x,y]);
```

```
(%o5) [ %ey+x  %ey+x
        2x      -2y ]
```

Ahora bien, si queremos calcular la matriz hessiana de un campo escalar, utilizaremos el comando `hessian` de la forma siguiente:

```
(%i6) hessian(f(x,y), [x,y]);
```

```
(%o6) [ -sin(x)cos(y^2)      -2cos(x)y sin(y^2)
        -2cos(x)y sin(y^2)  -2sin(x)sin(y^2)-4sin(x)y^2cos(y^2) ]
```

Recuerda que las derivadas cruzadas de segundo orden coinciden para funciones “suficientemente buenas”. Además, hay que comentar que, si estamos calculando el gradiente (o matriz jacobiana) de un campo escalar, mientras en el comando `hessian` la función se escribe $f(x, y)$, en el comando `jacobian` la función hay que escribirla entre corchetes. La razón es que este comando sirve tanto para campos escalares como vectoriales.

16.1.1 Plano tangente

Ya que sabemos cómo se calculan las derivadas parciales de una función, vamos a intentar representar gráficamente el plano tangente y cómo se obtiene a partir de las derivadas parciales.

Comencemos, por ejemplo, con la función

```
(%i7) f(x,y):=1-(x^2+y^2)$
```

y vamos a calcular en el punto $(1, 1)$ sus derivadas parciales. La definición de derivada parcial respecto a la primera variable era

$$\frac{\partial f}{\partial x}(1, 1) = \lim_{h \rightarrow 0} \frac{f(1+h, 1) - f(1)}{h}$$

Lo que hacemos es trabajar únicamente con la función definida sobre la recta que pasa por el punto $(1, 1)$ y es esa función (de una variable) la que derivamos. La segunda derivada parcial tiene una definición análoga. Dibujemos la gráfica de la función y la imagen de cada una de esas dos rectas. Antes de nada, cargamos el módulo `draw` y definimos las parciales de f ,

```
(%i8) load(draw)$
```

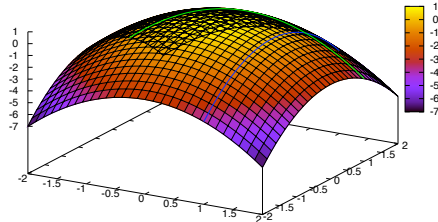
```
(%i9) define(parcialf1(x,y), diff(f(x,y), x, 1))$
```

```
(%i10) define(parcialf2(x,y), diff(f(x,y), y, 1))%
```

dibujamos la función y las curvas cuyas pendientes nos dan las derivadas parciales

```
(%i11) draw3d(enhanced3d=true,
    explicit(f(x,y),x,-2,2,y,-2,2),
    color=blue,
    line_width=2,
    parametric(1,t,f(1,t),t,-2,2),
    color=green,
    line_width=2,
    parametric(t,1,f(t,1),t,-2,2))$
```

(%o11)



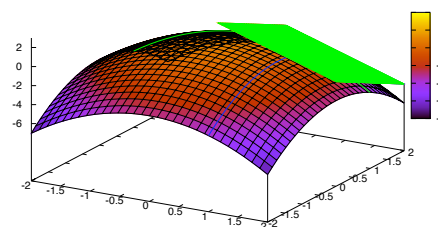
Los vectores tangentes a las dos curvas que hemos dibujado son los dos vectores que generan el plano tangente. Por tanto su ecuación será

$$\begin{aligned} z &= f(1,1) + \frac{\partial f}{\partial x}(1,1)(x-1) + \frac{\partial f}{\partial y}(1,1)(y-1) \\ &= f(1,1) + \langle \nabla f(1,1), (x-1, y-1) \rangle \end{aligned}$$

Para acabar nos falta añadir el plano tangente a la gráfica anterior:

```
(%i12) draw3d(enhanced3d=true,
    explicit(f(x,y),x,-2,2,y,-2,2),
    color=blue,
    line_width=2,
    parametric(1,t,f(1,t),t,-2,2),
    color=green,
    line_width=2,
    parametric(t,1,f(t,1),t,-2,2)),
    enhanced3d=true,
    parametric_surface(x,y,f(1,1)+parcialf1(1,1)*(x-1)
        +parcialf2(1,1)*(y-1),x,0,2,y,0,2))$
```

(%o12)



¿Cómo se podría hacer esto con derivadas direccionales en lugar de derivadas parciales? Más concretamente, dibuja la superficie y la recta que pasa por un punto (a,b) con dirección $\vec{v}$ y su

imagen, la curva cuya pendiente nos da la derivada direccional. Para hacer todo esto, elige tú el punto y la dirección.

16.2 Curvas y vectores tangentes

El paso a funciones de varias variables, ya sean escalares o vectoriales, puede dar lugar a errores por intentar trasladar literalmente nuestros conocimientos de funciones de una variable.

Hasta ahora hemos dibujado el plano tangente de una función de dos variables con valores escalares. El otro ejemplo sencillo de función de varias variables es una función de una variable pero con valores vectoriales. Por ejemplo, la función $t \mapsto (\cos(t), \sin(t))$. Para su representación necesitaríamos 3 dimensiones, una para el dominio y dos para la imagen, pero se suele representar en dos como una curva en coordenadas paramétricas: pensamos en la variable t como tiempo y $(\cos(t), \sin(t))$ representa la posición de una partícula en un momento t . En este caso su derivada es $t \mapsto (-\sin(t), \cos(t))$. Para hacernos una idea, vamos a representar en cada punto el correspondiente vector tangente.

En primer lugar definimos las funciones y el vector tangente (no te olvides de cargar el módulo draw:

```
(%i13) load(draw)$
(%i14) c(t):=[cos(t),sin(t)];
(%o14) c(t):=[cos(t),sin(t)]
(%i15) define(tangente(t),[diff(cos(t),t),diff(sin(t),t)]);
(%o15) tangente(t):=[-sin(t),cos(t)]
```

y ahora realizamos la animación.

```
(%i16) with_slider_draw(
      a,makelist(2*%pi*i/20,i,1,20),
      color=blue,
      line_width=2,
      parametric(cos(t),sin(t),t,0,2*%pi),
      color=red,
      head_length=0.2,
      head_type='empty,
      vector(c(a),tangente(a)),
      xrange=[-2,2],
      yrange=[-2,2])$
```

En la Figura 16.1 puedes ver algunos pasos intermedios de la representación

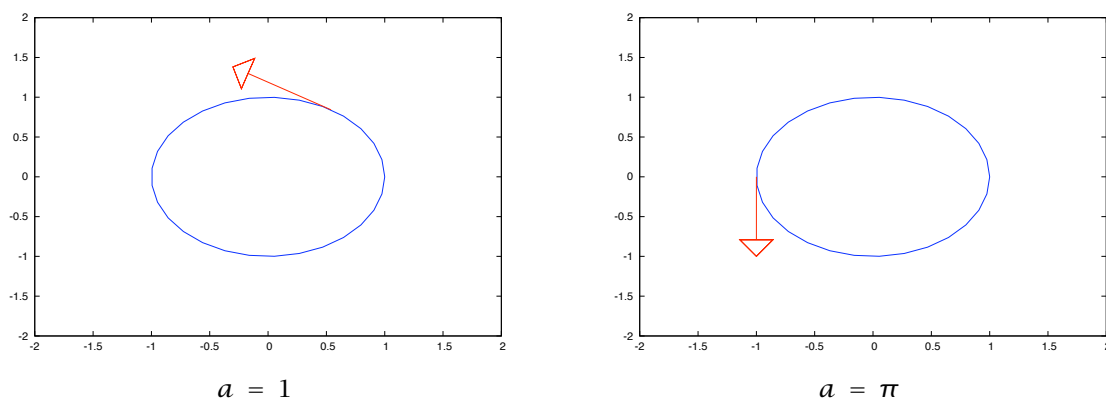


Figura 16.1 Curva y vectores tangentes

La función $t \mapsto (\cos(t), \sin(t))$ es continua y es diferenciable y el rastro que deja cuando variamos t en cualquier intervalo de longitud mayor o igual que 2π es una circunferencia.

Observa la gráfica de la curva que aparece en la Figura 16.2 de al lado. ¿Dirías que es “derivable”? Si has respondido que no, supongo que la justificación usual es que “tiene un pico”. Pero, ¿cómo sabes que es continua?. Tendemos a pensar en la figura como la gráfica de una función de una variable pero ¿cuál es la función de la que estamos afirmando (o no) que es continua o derivable?

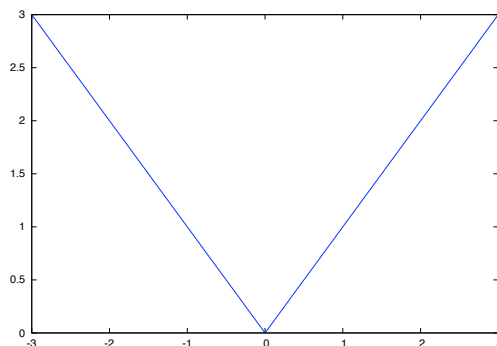


Figura 16.2 ¿Es diferenciable?

Si estás pensando que lo que hemos representado es la función valor absoluto, de acuerdo. Ahora bien, piensa por un momento que estamos ante una curva en coordenadas paramétricas. ¿Qué es ser continua en este caso? Hablando muy a la ligera (y bastante mal) que para valores cercanos sus imágenes también lo estén. Para intentar visualizarlo, vamos a dibujar la curva y un punto que nos vaya indicando en cada momento por dónde vamos.

```
(%i17) with_slider_draw(
        n,makelist(6*i/20-3,i,1,20),
        color=blue,
        nticks=250,
        parametric(t,abs(t),t,-27,27),
        color=red,
        point_type=filled_circle,
        point_size=2,
        points([[n^3,n^2*abs(n)]]),
        xrange=[-27,27],yrange=[0,27]
    )$
```

La continuidad de la función se traduce en que el punto no vaya “dando saltos” como puedes ver en la animación anterior. ¿Y que ocurre con la diferenciabilidad? En este caso tenemos que fijarnos en cómo se mueve el vector tangente. Hay una gran diferencia entre recorrer la figura así:

```
(%i18) with_slider_draw(
      n,makelist(6*i/20-3.05,i,1,20),
      color=blue,nticks=250,
      parametric(t,abs(t),t,-27,27),
      color=red,
      head_length=0.2,head_type='empty,
      vector([n,abs(n)], [1,signum(n)]),
      xrange=[-27,27],yrange=[-1,27]
    )$
```

que vemos cómo el vector tangente cambia bruscamente al llegar al origen y recorrer la curva de la forma $t \mapsto (t^3, t^2 | t|)$:

```
(%i19) with_slider_draw(
      n,makelist(6*i/20-3,i,1,20),
      color=blue,nticks=250,
      parametric(t,abs(t),t,-27,27),
      color=red,
      head_length=0.2,head_type='empty,
      vector([n^3,n^2*abs(n)], [3*n^2,3*n*abs(n)]),
      xrange=[-27,27],yrange=[-1,27]
    )$
```

en la que podemos ver cómo podemos evitar ese problema frenando al llegar al origen (la derivada en ese punto vale cero) para volver acelerar al “doblar la esquina”.

Si hay algo que debería quedar claro después de esto es que lo importante es cómo se recorre la curva y no la gráfica de esta.

16.3 Funciones definidas implícitamente

La orden `diff` deriva considerando todas las variables independientes. Por ejemplo,

```
(%i20) diff(2*x*y+cos(x*y)+y,x);
(%o20) 2y-y sin(xy)
(%i21) diff(2*x*y+cos(x*y)+y,y);
(%o21) -x sin(xy)+2x+1
```

`depends(var1, var2)` *var1* es función de *var2*

`dependencies` lista de variables dependientes

`remove(var,dependency)` hacer *var* una variable independiente

pero, en ocasiones, alguna de las variables no es independiente. En *Maxima* podemos imponer este tipo de condiciones con la orden `depends`. Con ella podemos asumir que una variable o una lista de variables depende de otra o de otras. En el caso anterior, podemos por ejemplo suponer que *y* es función de *x*:


```
(%i22) depends(y,x);
```

```
(%o22) [y(x)]
```

con lo que al derivar respecto de x nos quedaría

```
(%i23) diff(2*x*y+cos(x*y)+y,x);
```

```
(%o23) -(x(d/dx y)+y)sin(xy)+2x(d/dx y)+(d/dx y)+2y
```

La variable `dependencies` lleva la cuenta de todas las dependencias que hayamos definido. Hasta `dependencies` ahora sólo una:

```
(%i24) dependencies;
```

```
(%o24) [y(x)]
```

y, para que no haya problemas posteriormente, podemos anular esta dependencia con la orden `remove`:

`remove`

```
(%i25) remove(y,dependency);
```

```
(%o25) done
```

con lo que ahora ya no tenemos ninguna dependencia definida.

```
(%i26) dependencies;
```

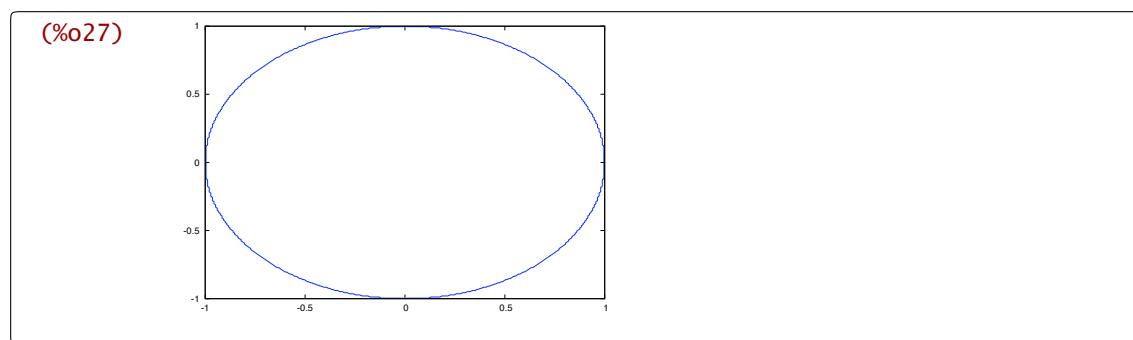
```
(%o26) [ ]
```

Teorema de la función implícita

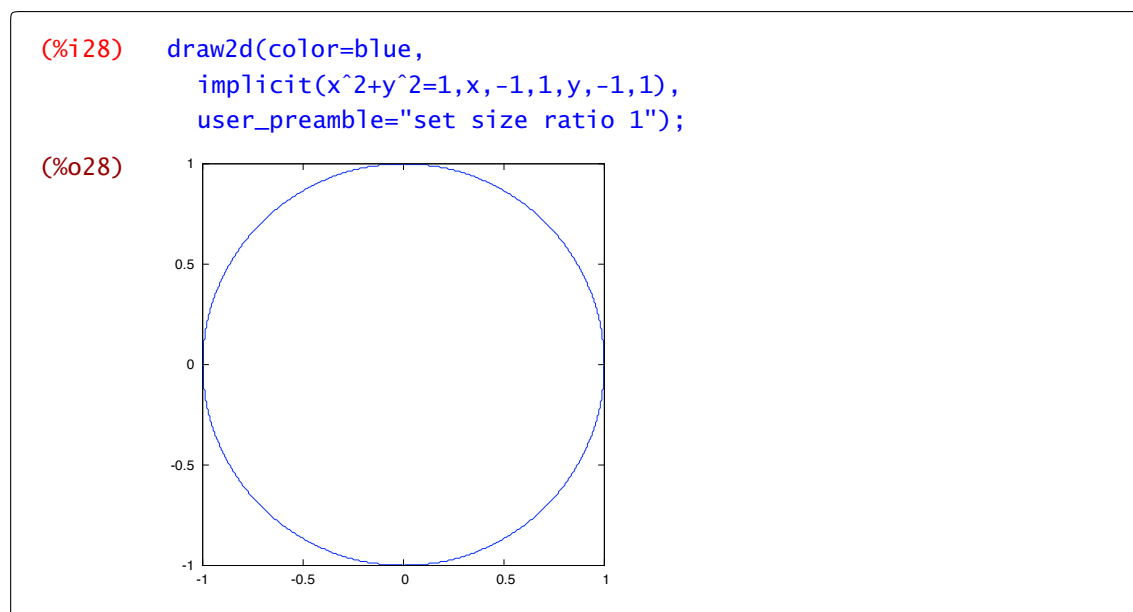
El Teorema de la función implícita nos dice cuándo podemos despejar unas variables en función de otras y qué propiedades verifica la función que nos da ese “despeje”.

Consideremos la ecuación $x^2 + y^2 = 1$. ¿Podemos despejar y en función de x o x en función de y ? La respuesta es que depende. Aunque estoy seguro de que sabes que los puntos que verifican $x^2 + y^2 = 1$ representan una circunferencia de radio uno centrada en el origen, hagamos como que no sabemos nada y dibujemos dicho conjunto.

```
(%i27) draw2d(color=blue,
            implicit(x^2+y^2=1,x,-1,1,y,-1,1));
```



Mejor un poco más redondo,



Es claro que no hay ninguna función ni de x ni de y cuya gráfica puede ser la circunferencia. Ahora bien, para algunos trozos de la circunferencia sí es cierto. Por ejemplo si sólo nos interesa qué pasa en el punto $(0, 1)$ y sus alrededores entonces seguro que sí sabes despejar y en función de x :

$$y(x) = \sqrt{1 - x^2}.$$

Esta fórmula no nos vale para el punto $(0, -1)$, habría que añadir un signo menos, pero también se puede despejar y en función de x . ¿Qué ocurre con el punto $(1, 0)$? Pues que no se puede, cualquier trozo de circunferencia que contenga al punto $(1, 0)$ (sin ser un extremo) no puede ser la gráfica de una función dependiente de x . Ahora bien, sí que podemos despejar x en función de y :

$$x(y) = \sqrt{1 - y^2}.$$

Consideremos la función $f(x, y) = x^2 + y^2 - 1$. Para poder despejar y en función de x en un entorno de un punto (a, b) hace falta que

a) $f(a, b) = 0$,

b) $\frac{\partial f}{\partial y}(a, b) \neq 0$,

y que f sea, al menos, de clase uno. En ese caso

$$y'(x) = -\frac{\frac{\partial f}{\partial x}(a,b)}{\frac{\partial f}{\partial y}(a,b)}$$

¿De dónde sale esta fórmula? Es sencillo, de derivar la ecuación y despejar:

$$f(x, y(x)) = 0 \implies \frac{\partial f}{\partial x} + \frac{\partial f}{\partial y} \frac{\partial y}{\partial x} = 0.$$

Esto mismo lo podemos hacer con *Maxima*: definimos la ecuación,

```
(%i29) eq:x^2+y^2=1;
(%o29) x^2+y^2=1
```

decimos que y depende de x ,

```
(%i30) depends(y,x);
(%o30) [y(x)]
```

derivamos,

```
(%i31) diff(eq,x);
(%o31) 2y(d/dx y)+2x=0
```

y despejamos $y'(x)$,

```
(%i32) solve(%,diff(y,x));
(%o32) [d/dx y=-x/y]
```

En el siguiente ejemplo se puede ver cómo podemos aplicar esto a una ecuación con 3 variables.

Ejemplo 16.1. Comprobar que la ecuación

$$z \arctan(1 - z^2) + 3x + 5z - 8y^3 = 0, (1, 1, 1)$$

verifica las condiciones del Teorema de la función implícita en el punto indicado y, por tanto, define a z como función de x e y en un entorno del punto $(1, 1)$. Obtener $\frac{\partial z}{\partial x}$, $\frac{\partial z}{\partial y}$.

Antes de empezar, borremos variables y posibles dependencias que tenemos de la explicación anterior.

```
(%i33) remvalue(all);
(%o33) [eq]
(%i34) dependencies;
(%o34) [y(x)]
(%i35) remove(y,dependency);
```

```
(%o35) done
```

Definimos la ecuación,

```
(%i36) eq:z*atan(1-z^2)+3*x+5*z-8*y^3;
(%o36) -z atan(z^2-1)+5z-8y^3+3x
```

comprobamos que el punto verifica la ecuación,

```
(%i37) eq,x=1,y=1,z=1;
(%o37) 0
```

y que la derivada respecto de z no se anula,

```
(%i38) diff(eq,z);
(%o38) -atan(z^2-1) - \frac{2z^2}{(z^2-1)^2+1}+5
(%i39) %,x=1,y=1,z=1;
(%o39) 3
```

con lo que el teorema de la función implícita nos da que z depende de x e y en un entorno de $(1, 1)$,

```
(%i40) depends(z,[x,y]);
(%o40) [z(x,y)]
```

derivamos,

```
(%i41) diff(eq,x);
(%o41) -\left(\frac{d}{dx} z\right) atan(z^2-1) - \frac{2z^2 \left(\frac{d}{dx} z\right)}{(z^2-1)^2+1} + 5 \left(\frac{d}{dx} z\right) + 3
```

y despejamos $\frac{\partial z}{\partial x}$,

```
(%i42) solve(%,diff(z,x));
(%o42) [\frac{d}{dx} z = \frac{3z^4-6z^2+6}{(z^4-2z^2+2)atan(z^2-1)-5z^4+12z^2-10}]
```

El cálculo de la derivada respecto de y se hace de manera similar y evaluar en un punto tampoco debería ofrecerte ninguna dificultad.

16.4 Extremos relativos

El método para encontrar extremos relativos de funciones de varias variables suficientemente derivables consiste en buscar primero los puntos críticos, es decir, puntos donde se anula el gradiente, y después estudiar la matriz hessiana en esos puntos. Los resultados que conocemos nos aseguran que todos los puntos extremos de una función están entre los puntos críticos, con lo que una vez calculados éstos nos dedicaremos a estudiar la matriz hessiana en ellos, viendo si es definida, indefinida, semidefinida... Para ello podemos usar el criterio de los valores propios: si todos son del mismo signo, la matriz es definida y hay extremo; si aparecen valores propios de distinto signo es indefinida y hay punto de silla; en otro caso, la matriz es semidefinida y el criterio no decide.

Ejemplo 16.2. Calculemos los extremos relativos de la función $f(x, y) = x^3 + 3xy^2 - 15x - 12y$. Primero definimos la función, y calculamos su gradiente y su matriz hessiana.

```
(%i43) f(x,y):= x^3+3*x*y^2-15*x-12*y;
      j:jacobian([f(x,y)], [x,y]);
      define(h(x,y),hessian(f(x,y), [x,y]));

(%o43) f(x,y):=x^3+3xy^2+(-15)x+(-12)y
(%o44) [3x^2+3y^2-15    6xy-12]
(%o45) [6x    6y]
      [6y   -6x]
```

Para calcular los puntos críticos podemos irnos a **Ecuaciones**→**Resolver sistema algebraico** e ir rellenando los datos que nos van pidiendo. De esta forma entra en acción el comando `algsys` para resolver sistemas de ecuaciones. A la hora de llamar a las ecuaciones, lo haremos de la siguiente forma:

```
(%i46) pcrit:algsys([j[1,1],j[1,2]], [x,y]);
(%o46) [[x=2,y=1],[x=1,y=2],[x=-1,y=-2],[x=-2,y=-1]]
```

donde $j[1,1]$ es el primer elemento del gradiente, es decir $\frac{\partial f}{\partial x}$ y $j[1,2]$ es el segundo, es decir $\frac{\partial f}{\partial y}$. Observad que el resultado es una lista de listas de puntos; lista que además hemos llamado `pcrit` para luego poder acudir a ella a la hora de evaluar la matriz hessiana en los puntos críticos obtenidos. Calculamos entonces la matriz hessiana, la evaluaremos en cada uno de los puntos críticos, y la clasificaremos haciendo uso del comando `eigenvalues` que nos da la lista de valores propios, seguidos de la lista de sus multiplicidades.

```
(%i47) hessian(f(x,y), [x,y]);
(%o47) [6x    6y]
      [6y   6x]
```

Entonces, ahora, vamos evaluando la matriz $h(x,y)$ en cada punto crítico y calculamos los valores propios en cada una de ellas. Lo hacemos en uno de los puntos, y de forma análoga se haría en los restantes.

```
(%i48)  eigenvalues(h(2,1));
(%o48)  [[6,18],[1,1]]
```

Por tanto, en el punto $(2, 1)$ la función f presenta un mínimo relativo al ser sus dos valores propios positivos, esto es, la forma cuadrática asociada a la matriz hessiana de f en dicho punto es definida positiva. Así mismo, en $(-2, -1)$ hay un máximo relativo, y en los puntos $(1, 2)$ y $(-1, -2)$ se tienen puntos de silla.

Ejemplo 16.3. Calculemos los extremos relativos de $g(x, y) = (x^2 + 3y^2)e^{1-x^2-y^2}$. Comenzamos definiendo g :

```
(%i49)  g(x,y):=(x^2+3*y^2)*exp(1-x^2-y^2)$
```

Calculamos ahora los puntos críticos como en el ejemplo anterior y tenemos:

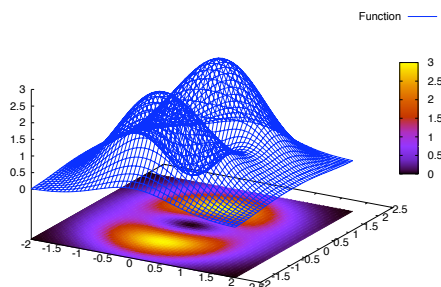
```
(%i50)  j:jacobian([g(x,y)],[x,y])$
(%i51)  pcrit:algsys([j[1,1],j[1,2]],[x,y]);
(%o51)  [[x=0,y=0],[x=-1,y=0],[x=1,y=0],[x=0,y=-1],[x=0,y=1]]
```

y vamos a calcular la matriz hessiana y a clasificarla en cada uno de los puntos críticos encontrados:

```
(%i52)  define(h(x,y),hessian(g(x,y)],[x,y])$
(%i53)  eigenvalues(h(0,0));
(%o53)  [[2%e,6%e],[1,1]]
```

Así, en el punto $(0,0)$ tenemos un mínimo y, razonando de la misma forma en los otros cuatro puntos, dos puntos de máximo en $(0, -1)$ y $(0, 1)$, así como dos puntos de silla en $(-1, 0)$ y $(1, 0)$. Podemos observar esto en la gráfica de la función, mejor aún si a la vez dibujamos el mapa de alturas:

```
(%i54)  plot3d((x^2+3*y^2)*exp(1-x^2-y^2), [x,-2,2], [y,-2,2],
               [plot_format,gnuplot],[grid,50,50],
               [gnuplot_preamble, "set pm3d at b"])$
(%o54)
```



16.5 Extremos condicionados

Usaremos el método de los multiplicadores de Lagrange para calcular extremos condicionados. Se trata de optimizar una función de varias variables en el conjunto de puntos que verifiquen una cierta ecuación o ecuaciones. Dada una función suficientemente diferenciable, y una curva o superficie en forma implícita, el método consiste en encontrar los puntos de dicha curva o superficie que verifican un sistema auxiliar, llamado “sistema de Lagrange”, que dará el equivalente a los puntos críticos; después habrá una condición sobre el hessiano equivalente a la condición para extremos relativos. Hay que notar que no todas las condiciones son válidas, sino que se ha de imponer una hipótesis técnica, pero en la práctica supondremos hecha la comprobación.

Ejemplo 16.4. Calculemos los extremos de la función $f(x, y) = x^2 - y^2$ condicionados a la ecuación $g(x, y) = x^2 + y^2 - 1 = 0$. Comenzamos definiendo la función f , la condición, y la función auxiliar de Lagrange:

```
(%i55)  f(x,y):=x^2-y^2$
        g(x,y):=x^2+y^2-1$ F(x,y,a):= f(x,y)-a*g(x,y)$
(%i56)  z:F(x,y,a);
(%o56)  -a(y^2+x^2-1)-y^2+x^2
```

Definimos el gradiente y planteamos el sistema de Lagrange:

```
(%i57)  j:jacobian([z],[x,y])
(%i58)  pcrit:algsys([j[1,1],j[1,2],g(x,y)],[x,y,a])
(%o58)  [[x=1,y=0,a=1],[x=-1,y=0,a=1],[x=0,y=-1,a=-1],[x=0,y=1,a=-1]]
```

Obtenemos 4 puntos críticos, y para ver si son o no extremos, estudiamos la matriz hessiana auxiliar $h(x, y, a)$ restringida al espacio tangente a la curva en los puntos críticos. Desafortunadamente, no podemos aplicar de una vez la asignación de valores, sino que deberemos ir punto a punto. El hessiano es

```
(%i59)  define(h(x,y,a),hessian(F(x,y,a),[x,y]));
(%o59)  h(x,y,a):=[ 2-2 a    0
                   0    -2 a-2 ]
```

En el primer punto, el hessiano es

```
(%i60)  h(x,y,a),pcrit[1];
(%o60)  [ 0    0
         0   -4 ]
```

que es semidefinido negativo. Tenemos que restringirnos al núcleo de la derivada de g en dicho punto crítico. Veamos, en primer lugar calculamos el jacobiano de g ,

```
(%i61) jacobian([g(x,y)], [x,y]);
(%o61) [ 2x  2y ]
```

evaluamos en el primer punto crítico,

```
(%i62) ev(%,pcrit[1]);
(%o62) [ 2  0 ]
```

y le calculamos el núcleo

```
(%i63) nullspace(%);
(%o63) span([ 0 ]
            [ 2 ])
```

La matriz hessiana restringida al núcleo tiene como única entrada

```
(%i64) [0,2].(ev(h(x,y,a),pcrit[1])).[0,2];
(%o64) -16
```

o sea que es definida negativa por lo que hay un máximo condicionado en el punto (1, 0). Para terminar el problema, repite el procedimiento con el resto de los puntos críticos.

Ejemplo 16.5. Calcular los extremos relativos de $f(x, y, z) = x^2 + y^2 + z^2$ condicionados a la superficie $g(x, y, z) = 2x + 2y - z + 3 = 0$. Comenzamos con las definiciones:

```
(%i65) f(x,y,z):=x^2+y^2+z^2$ g(x,y,z):=2*x+2*y-z+3$
      F(x,y,z,a):= f(x,y,z)-a*g(x,y,z)$
(%i66) t:F(x,y,z,a)$
(%o66) z^2-a(-z+2y+2x+3)+y^2+x^2
```

Definimos el gradiente de t y resolvemos el sistema de Lagrange:

```
(%i67) j:jacobian([t], [x,y,z])$
(%i68) pcrit:algsys([j[1,1],j[1,2],j[1,3],g(x,y,z)], [x,y,z,a]);
(%o68) [[x=2/3,y=-2/3,z=1/3,a=-2/3]]
```

y estudiemos el hessiano de la función auxiliar de Lagrange. En este caso

```
(%i69) hessian(t, [x,y,z]);
```


$$(\%o69) \quad \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{bmatrix}$$

la matriz hessiana es definida positiva y, por tanto, f tiene un mínimo condicionado en el punto $(\frac{2}{3}, \frac{2}{3}, \frac{1}{3})$.

16.6 Extremos absolutos

En conjuntos compactos tenemos garantizada la existencia de extremos absolutos para las funciones continuas. Esto nos simplifica su cálculo: los extremos absolutos son extremos relativos, si se encuentran en el interior del dominio, o extremos condicionados, si se encuentran en la frontera. Vamos a buscar posibles candidatos a extremos relativos o a extremos condicionados y entre ellos tienen que estar los máximos y mínimos absolutos.

Dependiendo del aspecto de la frontera utilizaremos el método de los multiplicadores de Lagrange o trabajaremos directamente con la función. Comencemos con un ejemplo de este último caso.

Ejemplo 16.6. Vamos a calcular los extremos absolutos de la función $f(x, y) = x^2(y - x)$ en el triángulo de vértices $(0, 0)$, $(1, 0)$ y $(1, 1)$.

En el interior, buscamos puntos críticos de la función:

```
(%i70) f(x,y):=x^2*(x-y)$
(%i71) j:jacobian([f(x,y)], [x,y])$
(%i72) algsys([j[1,1], j[1,2]], [x,y]);
(%o72) [[x=0, y=%r1]]
```

La interpretación que damos de las soluciones es que las únicas soluciones son los puntos de la forma $(0, y)$ que, evidentemente, no pertenecen al interior del dominio.

La frontera del conjunto esta formada por tres segmentos. Vamos a estudiarlos uno a uno. En primer lugar el segmento que une el origen y el punto $(1, 0)$. En dicho segmento $y = 0$ y $x \in [0, 1]$. Los posibles extremos son

```
(%i73) f1(x):=f(x,0)$
(%i74) solve(diff(f1(x), x), x);
(%o74) [[x=0]]
```

Nos sale el origen de coordenadas. En segundo lugar, el segmento que une $(1, 0)$ y $(1, 1)$. En este caso $x = 1$ e y varía entre 0 y 1:

```
(%i75) f2(y):=f(1,y)$
(%i76) solve(diff(f2(y), y), y);
(%o76) [ ]
```

No hay solución. El último segmento une el origen y el punto $(1, 1)$. En este caso:

```
(%i77) f3(x):=f(x,x);
(%i78) solve(diff(f3(x),x),x);
(%o78) all
```

Todo el segmento está formado por puntos críticos. A estos puntos tenemos que añadir los extremos del intervalo (los vértices del triángulo): siempre cabe la posibilidad de que una función de una variable alcance un extremo absoluto en uno de ellos.

Para terminar sólo te queda evaluar. Calcular $f(0,0)$, $f(1,0)$, $f(1,1)$ y $f(x,x)$. ¿Cuáles son los extremos absolutos de la función?

Ejemplo 16.7. Calculemos los extremos absolutos de la función $f(x,y) = x^2 - y^2$ en el círculo de centro el origen y radio uno.

De nuevo estamos ante una función continua en un dominio cerrado y acotado por lo que tenemos garantizada la existencia de extremos absolutos. Los puntos críticos en el interior son

```
(%i79) f(x,y):=x^2-y^2$
(%i80) j:jacobian([f(x,y)],[x,y])$
(%i81) algsys([j[1,1],j[1,2]],[x,y]);
(%o81) [[x=0,y=0]]
```

Bien, ya tenemos un punto. ¿Que hacemos con la frontera? Calculamos los puntos críticos de la función de Lagrange

```
(%i82) F(x,y,a):=f(x,y)+a*(x^2+y^2-1)$
(%i83) J:jacobian([F(x,y,a)],[x,y])$
(%i84) algsys([J[1,1],J[1,2],x^2+y^2-1],[x,y,a]);
(%o84) [[x=1,y=0,a=-1],[x=-1,y=0,a=-1],[x=0,y=-1,a=1],[x=0,y=1,a=1]]
```

Evalúa la función f en estos puntos para averiguar dónde alcanza los extremos absolutos.

Observación 16.8. En la función auxiliar de Lagrange, unas veces hemos escrito $f - ag$ y otras, como en el ejemplo anterior, hemos escrito $f + ag$. ¿Cuál es la versión correcta? ¿Hay alguna diferencia?

16.7 Ejercicios

Derivadas parciales. Plano tangente.

Ejercicio 16.1. Sea $f(x,y) = \log(1 + x^2 + 2x + y^2)$. Calcula el gradiente, la matriz hessiana de f y comprueba que es armónica, esto es, que

$$\frac{\partial^2 f}{\partial x^2}(x,y) + \frac{\partial^2 f}{\partial y^2}(x,y) = 0.$$

Ejercicio 16.2. Comprueba que las funciones

$$f(x, y) = 2 \arctan \left(\frac{y}{x + \sqrt{x^2 + y^2}} \right) \text{ y } g(x, y) = \arctan \left(\frac{y}{x} \right)$$

tienen las mismas derivadas parciales.

Ejercicio 16.3. Calcula el plano tangente y la recta normal a cada una de las superficies en el punto P :

a) $z^2 - 2x^2 - 2y^2 - 12 = 0, P = (1, -1, 4).$

b) $z = \log(x^2 + y^2), P = (1, 0, 0).$

c) $z + e^z + 2x + 2y - x^2 - y^2 = 3, P = (1, 1 + \sqrt{e}, 1).$

Ejercicio 16.4. Calcular las derivadas parciales de:

a) $f(x, y, z) = x^{y+z}, \forall x \in \mathbb{R}^+, y, z \in \mathbb{R}$ c) $f(x, y) = \sin(x \sin(y)), \forall x, y \in \mathbb{R}$

b) $f(x, y, z) = (x + y)^z, \forall x, y \in \mathbb{R}^+, z \in \mathbb{R}$

Ejercicio 16.5. Sea $f: \mathbb{R}^2 \setminus \{(0, 0)\} \rightarrow \mathbb{R}$ dada por $f(x, y) = \log(x^2 + y^2)$ para todo $(x, y) \neq (0, 0)$. Se pide:

a) Calcúlese el gradiente de f en todo punto así como la matriz hessiana.

b) Compruébese que

$$\frac{\partial^2 f}{\partial x^2}(x, y) + \frac{\partial^2 f}{\partial y^2}(x, y) = 0 \quad \forall (x, y) \in \mathbb{R}^2 - \{(0, 0)\}.$$

Extremos relativos

Ejercicio 16.6. Calcular los extremos relativos de las siguientes funciones.

a) $f(x, y) = x^2 + y^2 - 2x + 4y + 20.$

d) $f(x, y, z) = x^2 + y^2 + z^2 - 2x.$

b) $f(x, y) = x^3 + x^2y + y^2 + 2y + 5.$

e) $f(x, y) = x^3y^3 - y^4 - x^4 + xy.$

c) $f(x, y) = (x^2 + y^2)e^{x^2 - y^2}.$

Extremos condicionados y absolutos

Ejercicio 16.7. Calcular los extremos condicionados en los siguientes casos:

a) $f(x, y) = x^2 - xy + y^2$ condicionados a $x^2 + y^2 - 4 = 0.$

b) $f(x, y) = x^3 + xy^2$ condicionados a $xy = 1.$

c) $f(x, y, z) = xyz$ condicionados a $x^2 + y^2 + z^2 = 1.$

Ejercicio 16.8.

a) Hallar la mínima distancia de $(0, 0)$ a $x^2 - y^2 = 1.$

b) Entre todos los ortoedros de volumen 1, determinar el que tiene superficie lateral mínima.

- c) Calcular las dimensiones de un ortoedro de superficie lateral 2, para que su volumen sea máximo
- d) Se quiere construir un canal cuya sección sea un trapecio isósceles de área dada S . Calcular la profundidad del canal y el ángulo que deben formar las paredes con la horizontal para que la superficie mojada sea mínima. (Solución: se trata de la mitad de un hexágono regular)

Ejercicio 16.9. Calcula los extremos absolutos de

- a) $f(x, y) = x^2 - xy + y^2$ en el conjunto $\{(x, y); x^2 + y^2 = 4\}$,
- b) $f(x, y) = xyz$, en el conjunto $x^2 + y^2 + z^2 \leq 1$.

Integrales múltiples

17

17.1 Integrales iteradas 243 17.2 Integrales dobles 243 17.3 Ejercicios 246

17.1 Integrales iteradas

En el Capítulo 11 hemos visto cómo podemos utilizar *Maxima* para resolver integrales de funciones de una variable. ¿Qué necesitamos para calcular integrales de funciones de varias variables? Bueno, tenemos el siguiente teorema:

Teorema 17.1. Sea $f: [a, b] \times [c, d] \rightarrow \mathbb{R}$ integrable. Entonces

$$\int_{[a,b] \times [c,d]} f(x, y) d(x, y) = \int_a^b \left(\int_c^d f(x, y) dy \right) dx = \int_c^d \left(\int_a^b f(x, y) dx \right) dy.$$

<code>integrate(f(x), x)</code>	primitiva de la función $f(x)$
<code>integrate(f(x), x, a, b)</code>	$\int_a^b f(x) dx$
<code>quad_quags(f(x), x, a, b)</code>	aproximación numérica de $\int_a^b f(x) dx$
<code>romberg(f(x), x, a, b)</code>	aproximación numérica de $\int_a^b f(x) dx$

Esto quiere decir que una integral doble se calcula mediante dos integrales de una variable. Como ya sabemos hacer integrales de una variable, podemos comprobarlo en un ejemplo. La integral de la función $f(x, y) = 3xy$ en $[0, 1] \times [2, 6]$ vale

```
(%i1) integrate(integrate(3*x*y,x,0,1),y,2,6);
(%o1) 24
```

o, cambiando el orden de integración

```
(%i2) integrate(integrate(3*x*y,y,2,6),x,0,1);
(%o2) 24
```

17.2 Integrales dobles

Como recordarás de clase, sabemos calcular la integral de una función $h(x, y)$ en recintos de alguno de los siguientes tipos:

$$A = \{(x, y) \in \mathbb{R}^2 : a \leq x \leq b, f(x) \leq y \leq g(x)\} \text{ o}$$

$$B = \{(x, y) \in \mathbb{R}^2 : a \leq y \leq b, f(y) \leq x \leq g(y)\}$$

Las correspondientes integrales serían, vía el teorema de Fubini,

$$\int_A h(x, y) d(x, y) = \int_a^b \left(\int_{f(x)}^{g(x)} h(x, y) dy \right) dx, y$$

$$\int_B h(x, y) d(x, y) = \int_a^b \left(\int_{f(y)}^{g(y)} h(x, y) dx \right) dy.$$

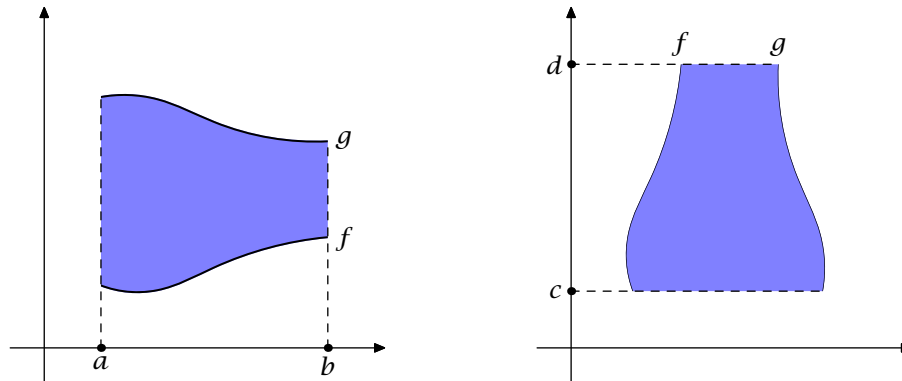


Figura 17.1 Recintos de tipo 1 y 2

El caso más fácil, ya lo hemos visto, se presenta cuando integramos en rectángulos. En este caso las funciones f o g son constantes. Por ejemplo, si queremos calcular la integral de $h(x, y) = x^2 + y^2$ en $[0, 2] \times [1, 5]$ tendríamos que hacer lo siguiente:

```
(%i3) integrate(integrate(x^2 + y^2, x, 0, 2), y, 1, 5);
(%o3) 280/3
```

Pasemos a integrales un poco más complicadas. Supongamos que queremos integrar la función $f(x, y) = x$ en el conjunto

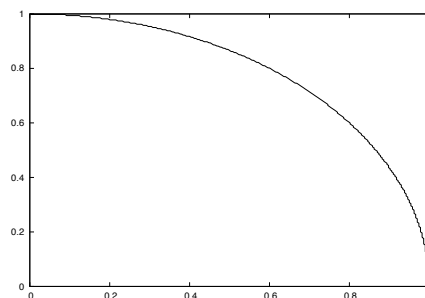
$$A = \{(x, y) \in \mathbb{R}^2: x^2 + y^2 \leq 1, x \geq 0, y \geq 0\}.$$

El primer paso es intentar averiguar cómo es el conjunto, para lo que nos puede ser útil dibujar $x^2 + y^2 = 1$. Este conjunto, la circunferencia de radio uno centrada en el origen, es fácil de identificar, pero la gráfica nos puede ayudar en otros casos. Comenzamos cargando el paquete draw.

```
(%i4) load(draw)$
```

El rango de variación de las variables x e y lo tenemos que decidir nosotros. En nuestro caso, x e y tienen que ser mayores o iguales que cero y menores que 1.

```
(%i5) draw2d(implicit(x^2+y^2=1,x,0,1,y,0,1));
(%o5)
```



Después de ver el conjunto A donde estamos integrando, es fácil ver cómo podemos poner una variable en función de la otra. Por ejemplo, si ponemos y en función de x , para un valor concreto de x entre 0 y 1, y varía entre 0 y $\sqrt{1-x^2}$. Podemos calcular ahora el valor de la integral:

```
(%i6) integrate(integrate(x,y,0,sqrt(1-x^2)),x,0,1);
Is (x-1)*(x+1) positive, negative, or zero? negative;
(%o6) 1/3
```

Ya tenemos resuelta la integral doble.

Observarás que no hemos cambiado a coordenadas polares. Para integrales en varias variables, interesa cambiar de variable cuando el dominio o la función son complicados. En el primer caso, si no podemos escribir el dominio de una de las formas (1) o (2) tendremos que cambiar de variable nosotros (*Maxima* no lo hace automáticamente). El caso de funciones complicadas no nos debe preocupar, pues *Maxima* se encarga de resolver la integral en la inmensa mayoría de los casos, y si no se puede resolver directamente, siempre queda la posibilidad de aproximar el valor de integral.

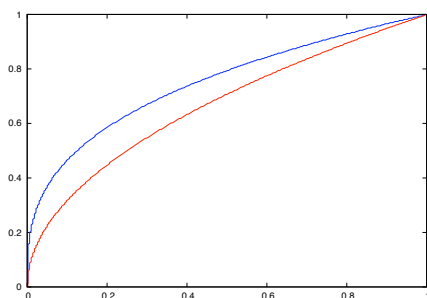
Vamos un par de ejemplos.

Ejemplo 17.2. Vamos a calcular la integral de la función $f(x, y) = e^{x/y}$, en el conjunto

$$A = \{(x, y) \in \mathbb{R}^2 : 0 \leq y^3 \leq x \leq y^2\}.$$

Estamos ante un conjunto del tipo 2. Tenemos la variable x en función de y . Vamos primero a ver los puntos de corte de las ecuaciones $y^3 = x$, $y^2 = x$ y luego las dibujaremos para hacernos una idea de cuál es el dominio de integración:

```
(%i7) solve(y^3=y^2,y);
(%o7) [y=0,y=1]
(%i8) draw2d(color=blue,
            implicit(y^3=x,x,0,1,y,0,1),
            color=red, implicit(y^2=x,x,0,1,y,0,1));
(%o8)
```



De lo hecho se deduce que la integral que queremos calcular es

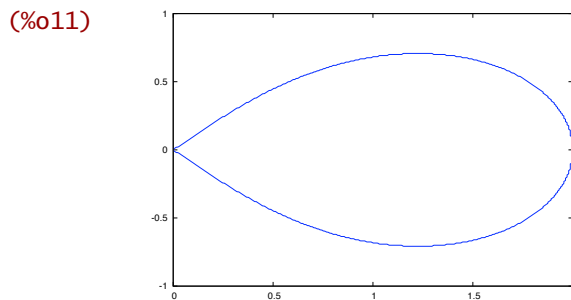
```
(%i9) integrate(integrate(exp(x/y),x,y^3,y^2),y,0,1);
(%o9) %e-3/2
```

Ejemplo 17.3. Vamos a calcular la integral de la función $f(x, y) = x^2 + y^2$ en el conjunto

$$A = \{(x, y) \in \mathbb{R}^2 : (x^2 + y^2)^2 \leq 4(x^2 - y^2), x \geq 0\}.$$

En este caso la función es relativamente sencilla pero el dominio no lo es tanto. No se ve una manera fácil de escribir la integral en términos de x e y . Parece mejor cambiar a coordenadas polares. Antes de hacerlo podemos echar un vistazo al dominio. Recuerda que el rango de valores de las variables x e y es algo que debes decidir por ti mismo.

```
(%i10) load(draw)$
(%i11) draw2d(color=blue,
          implicit((x^2+y^2)^2=4*(x^2-y^2),x,0,2,y,-1,1));
```



Las condiciones del dominio escritas en coordenadas polares son:

$$\rho^4 \leq 4\rho^2(\cos^2(\theta) - \sin^2(\theta)), \rho \cos \theta \geq 0.$$

Después de operar, nos quedaría

$$\rho \leq 2\sqrt{\cos^2(\theta) - \sin^2(\theta)} \text{ y } \theta \in \left[-\frac{\pi}{4}, \frac{\pi}{4}\right],$$

y teniendo en cuenta que $\cos^2(\theta) - \sin^2(\theta) = \cos(2\theta)$, tenemos que resolver la integral

$$\int_{-\pi/4}^{\pi/4} \left(\int_0^{2\sqrt{\cos(2\theta)}} \rho^3 d\rho \right) d\theta$$

```
(%i12) integrate(integrate(r^3,r,0,2*sqrt(cos(2*t))),t,-%pi/4,%pi/4);
Is cos(2*t) positive, negative, or zero? positive;
(%o12) pi
```

17.3 Ejercicios

Ejercicio 17.1. Calcular la integral de las siguientes funciones en los recintos determinados por las siguientes ecuaciones:

- a) $f(x, y) = \sqrt{4x^2 - y^2}$, $x = 1$, $y = 0$, $y = x$
- b) $f(x, y) = xe^{-x^2/y}$, $x = 0$, $y = 1$, $y = x^2$
- c) $f(x, y) = y$, $y \geq 0$, $x^2 + y^2 = a^2$, $y^2 = 2ax$, $x = 0$

Ejercicio 17.2. Calcular el volumen del conjunto A en cada uno de los siguientes casos:

- a) $A = \{(x, y, z) \in \mathbb{R}^3 : x^2 + y^2 \leq z \leq \sqrt{x^2 + y^2}\}$
- b) $A = \{(x, y, z) \in \mathbb{R}^3 : \frac{x^2}{a^2} + \frac{y^2}{b^2} \leq 1, 0 \leq z \leq \sqrt{\frac{x^2}{a^2} + \frac{y^2}{b^2}}\}$
- c) $A = \{(x, y, z) \in \mathbb{R}^3 : 0 \leq z \leq x^2 + y^2, x + y \leq 1, x, y \geq 0\}$
- d) $A = \{(x, y, z) \in \mathbb{R}^3 : 0 \leq z \leq \sqrt{x^2 + y^2}, x^2 + y^2 \leq 2y\}$
- e) $A = \{(x, y, z) \in \mathbb{R}^3 : 0 \leq z \leq 4 - y^2, 0 \leq x \leq 6\}$
- f) $A = \{(x, y, z) \in \mathbb{R}^3 : \sqrt{x} \leq y \leq 2\sqrt{x}, 0 \leq z \leq 9 - x\}$
- g) $A = \{(x, y, z) \in \mathbb{R}^3 : x^2 + y^2 \leq z^2, x^2 + y^2 + z^2 \leq 2z\}$

Ejercicio 17.3. Calcular el volumen limitado superiormente por el cono $4x^2 + 4y^2 - z^2 = 0$, inferiormente por el plano $z = 0$ y lateralmente por el cilindro $x^2 + (y - 2)^2 = 4$.

Ejercicio 17.4. Calcular el volumen de los sólidos siguientes:

- a) Intersección de los cilindros $x^2 + y^2 = r^2$, $y^2 + z^2 = r^2$.
- b) El limitado por el plano $z = 0$, el cilindro $x^2 + y^2 = 2ax$ ($a > 0$) y el cono $x^2 + y^2 = z^2$.

Números complejos

A

Después de calcular la raíz cuadrada de 2, la primera idea que se nos ocurre a todos es probar con números negativos “a ver que pasa”:

```
(%i1)  sqrt(-2);
(%o1)   $\sqrt{2} \%i$ 
```

Correcto. Ya habíamos comentado con anterioridad que $\%i$ representa a la unidad imaginaria. En *Maxima* podemos trabajar sin problemas con números complejos. Se pueden sumar, restar, multiplicar, dividir

```
(%i2)  ((2+3*%i)+(3-%i));
(%o2)  2%i+5
```

<code>rectform(<i>expresión</i>)</code>	<i>expresión</i> en forma cartesiana o binómica
<code>realpart(<i>expresión</i>)</code>	parte real de <i>expresión</i>
<code>imagpart(<i>expresión</i>)</code>	parte imaginaria de <i>expresión</i>
<code>polarform(<i>expresión</i>)</code>	forma polar de <i>expresión</i>
<code>abs(<i>expresión</i>)</code>	módulo o valor absoluto de <i>expresión</i>
<code>cabs(<i>expresión</i>)</code>	módulo de <i>expresión</i> compleja
<code>carg(<i>expresión</i>)</code>	argumento de <i>expresión</i>
<code>conjugate(<i>expresión</i>)</code>	conjugado de <i>expresión</i>
<code>demoivre(<i>expresión</i>)</code>	expresa el número complejo utilizando senos y cosenos
<code>exponentialize(<i>expresión</i>)</code>	expresa el número complejo utilizando exponenciales

Si multiplicamos o dividimos números complejos, hemos visto que *Maxima* no desarrolla completamente el resultado, pero si pedimos que nos lo de en forma cartesiana, por ejemplo,

```
(%i3)  (2+3*%i)/(1-%i);
(%o3)   $\frac{3\%i+2}{1-\%i}$ 
(%i4)  rectform((2+3*%i)/(1-%i));
(%o4)   $\frac{5\%i}{2} - \frac{1}{2}$ 
```

De la misma forma podemos calcular la parte real y la parte imaginaria, el módulo o la forma polar de un número complejo

```
(%i5)  realpart(2-%i);
(%o5)  2
(%i6)  abs(1+3*%i);
(%o6)   $\sqrt{10}$ 
(%i7)  polarform(1+3*%i);
(%o7)   $\sqrt{10}e^{i \operatorname{atan}(3)}$ 
```

No hace falta calcular la forma polar para conocer el argumento principal de un número complejo, `carg` se encarga de ello:

```
(%i8)  carg(1+3*%i);
(%o8)   $\operatorname{atan}(3)$ 
(%i9)  carg(exp(%i));
(%o9)  1
```

Recuerda que muchas funciones reales tienen una extensión al plano complejo. Por ejemplo, `exp` nos da la exponencial compleja,

```
(%i10)  exp(%i*%pi/4);
(%o10)   $\frac{\sqrt{2}i}{2} + \frac{\sqrt{2}}{2}$ 
```

`log` nos da el logaritmo principal

```
(%i11)  log(%i);
(%o11)   $\log(i)$ 
(%i12)  log(-3);
(%o12)   $\log(-3)$ 
```

siempre que se lo pidamos

```
(%i13)  rectform(log(%i));
(%o13)   $\frac{i\pi}{2}$ 
(%i14)  rectform(log(-3));
(%o14)   $\log(3)+i\pi$ 
```

Podemos calcular senos o cosenos,

```
(%i15) cos(1+%i);
(%o15) cos(%i+1)
(%i16) rectform(%);
(%o16) cos(1)cosh(1)-%i sin(1)sinh(1)
```

si preferimos la notación exponencial, `exponentialize` escribe todo en términos de exponenciales

```
(%i17) exponentialize(%);
(%o17) 
$$\frac{(e+e^{-1})(e^{%i}+e^{-%i})}{4} - \frac{(e-e^{-1})(e^{%i}-e^{-%i})}{4}$$

```

y `demoivre` utiliza senos y cosenos en la salida en lugar de las exponenciales:

```
(%i18) demoivre(%);
(%o18) 
$$\frac{(e+e^{-1}) \cos(1)}{2} - \frac{(e-e^{-1}) i \sin(1)}{2}$$

```

Observación A.1. La orden `abs` calcula el módulo de la expresión pedida como ya hemos comentado

```
(%i19) abs(1+%i);
(%o19)  $\sqrt{2}$ 
```

pero en algunas ocasiones el resultado no es el esperado

```
(%i20) abs(log(-3));
(%o20) -log(-3)
```

En este caso, `cabs` nos da el resultado correcto

`cabs`

```
(%i21) cabs(log(-3));
(%o21)  $\sqrt{\log(3)^2 + \pi^2}$ 
```


Avisos y mensajes de error

B

Este apéndice no es, ni pretende ser, un repaso exhaustivo de los errores que podemos cometer utilizando *Maxima*, sino más bien una recopilación de los errores más comunes que hemos cometido mientras aprendíamos a usarlo. La lista no está completa y habría que añadirle la manera más típica de meter la pata: inventarse los comandos. Casi todos utilizamos más de un programa de cálculo, simbólico o numérico, y algún lenguaje de programación. Es muy fácil mezclar los corchetes de *Mathematica* (© by Wolfram Research) y los paréntesis de *Maxima*. ¿Se podía utilizar $\ln$ como logaritmo neperiano o eso era en otro programa? Muchas veces utilizamos lo que creemos que es una función conocida por *Maxima* sin que lo sea. En esos casos un vistazo a la ayuda nos puede sacar de dudas. Por ejemplo, ¿qué sabe *Maxima* sobre $\ln$?

```
(%i1)  ??ln - Función: belln (<n>)
        Representa el n-ésimo número de Bell, de modo que 'belln(n)' es
        el número de particiones de un conjunto de <n> elementos.
        El argumento <n> debe ser un ...
```

No es precisamente lo que esperábamos (al menos yo).

```
(%i2)  2x+1
        Incorrect syntax: X is not an infix operator
        2x+
        ^
```

Es necesario escribir el símbolo de multiplicación, $*$ entre 2 y x .

```
(%i3)  factor((2 x+1)^2)
        Incorrect syntax: X is not an infix operator
        factor((2Spacex+
        ^
```

Es necesario escribir el símbolo de multiplicación, $*$ entre 2 y x . No es suficiente con un espacio en blanco.

```
(%i4)  plot2d(sin(x),[x,0,3]])
        Incorrect syntax: Missing )
        ot2d(sin(x),[x,0,3]]
        ^
```

Repasa paréntesis y corchetes: en este caso hay un corchete de más.

```
(%i5) g(x,y,z):=(2*x,3*cos(x+y))$
(%i6) g(1,%pi);
Too few arguments supplied to g(x,y,z):
[1,π]
- an error. To debug this try debugmode(true);
```

La función tiene tres variables y se la estamos aplicando a un vector con sólo dos componentes.

```
(%i7) f(x):3*x+cos(x)
Improper value assignment:
f(x)
- an error. To debug this try debugmode(true);
```

Para definir funciones se utiliza el signo igual y dos puntos y no sólo dos puntos.

```
(%i8) solve(sin(x)=0,x);
'solve' is using arc-trig functions to
get a solution. Some solutions will be lost.
(%o8) [x=0]
```

Para resolver la ecuación $\sin(x) = 0$ tiene que usar la función arcoseno (su inversa) y *Maxima* avisa de que es posible que falten soluciones y, de hecho, faltan.

```
(%i9) integrate(1/x,x,0,1);
Is x + 1 positive, negative, or zero? positive;
Integral is divergent
- an error. To debug this try debugmode(true);
```

La función $\frac{1}{x}$ no es integrable en el intervalo $[0, 1]$.

```
(%i10) find_root(x^2,-3,3);
function has same sign at endpoints
[f(-3.0)=9.0,f(3.0)=9.0]
- an error. To debug this try debugmode(true);
```

Aunque x^2 se anula entre 3 y -3 (en $x = 0$ obviamente) la orden `find_root` necesita dos puntos en los que la función cambie de signo.

```
(%i11) A:matrix([1,2,3],[2,1,4],[2.1,4]);
All matrix rows are not of the same length.
- an error. To debug this try debugmode(true);
```


En una matriz, todas las filas deben tener el mismo número de columnas: hemos escrito un punto en lugar de una coma en la última fila.

```
(%i12) A:matrix([1,2,3],[2,1,4],[2,1,4])$  
(%i13) B:matrix([1,2],[2,1],[3,1])$  
(%i14) B.A  
incompatible dimensions - cannot multiply  
- an error. To debug this try debugmode(true);
```

No se pueden multiplicar estas matrices. Repasa sus órdenes.

Bibliografía

C

- a) La primera fuente de documentación sobre *Maxima* es el propio programa. La ayuda es muy completa y detallada.
- b) En la página del programa *Maxima*, <http://maxima.sourceforge.net/>, existe una sección dedicada a documentación y enlaces a documentación. El manual de referencia de *Maxima* es una fuente inagotable de sorpresas. Para cualquier otra duda, las listas de correo contienen mucha información y, por supuesto, siempre se puede pedir ayuda.
- c) “*Primeros pasos en Maxima*” de Mario Rodríguez Riotorto es, junto con la siguiente referencia, la base de estas notas. Se puede encontrar en
<http://www.telefonica.net/web2/biomates/>
- d) “*Maxima con wxMaxima: software libre en el aula de matemáticas*” de Rafael Rodríguez Galván es el motivo de que hayamos usado *wxMaxima* y no cualquier otro entorno sobre *Maxima*. Este es un proyecto que se encuentra alojado en el repositorio de software libre de RedIris
<https://forja.rediris.es/projects/guia-wxmaxima/>
- e) Edwin L. Woollett está publicando en su página, capítulo a capítulo unas notas (más bien un libro) sobre *Maxima* y su uso. El título lo dice todo: “*Maxima by example*”. Su página es
<http://www.csulb.edu/~woollett/>

Glosario

' 21
 " 150
 . 83
 ? 29
 ?? 30
 % 12

a

abs 250
 abs_integrate 175
 acos 17
 algsys 109, 235
 allroots 107
 apply 85-86
 asin 17
 atan 17
 atvalue 216

b

base 65
 bc2 215
 bfallroots 107
 bfloat 14
 both 65

c

cabs 251
 carg 250
 ceiling 126
 charpoly 94
 col 91
 color 63
 columns 60
 contour 65
 cos 17, 251
 cosh 251
 cot 17
 csc 17
 cylindrical 58

d

define 34, 150
 demoivre 251
 denom 24

dependencies 231
 depends 230
 desarrollo
 de Laurent 197
 describe 29
 desolve 216
 determinant 90
 diagmatrix 93
 diff 136, 149-150, 152, 156, 213, 225, 230
 discrete 44
 disp 115
 display 115
 do 120
 draw 50, 68, 226
 draw2d 50
 draw3d 50, 68

e

eigenvalues 94, 235
 eigenvectors 95
 ellipse 55
 enhanced3d 61
 entermatrix 92
 erf 164
 error
 absoluto 75
 relativo 75
 ev 26
 evolution 134
 example 30
 exp 15, 250
 expand 24
 explicit 50, 52
 exponentialize 251

f

factor 25
 fill_color 62-63, 174
 filled_func 62, 174
 find_root 111, 254
 first 82
 flatten 83
 float 12, 14
 for 118

forget 20
 fourier_elim 106
 fpprec 14
 fullratsimp 27
 functions 34
 fundef 34

g

genmatrix 93
 go 118
 grid 59

h

hessian 226

i

ic1 215
 ic2 215
 if 121
 implicit 52
 ind 142
 infinity 143
 integrate 163
 interpol 195
 interpolación
 de Lagrange 192
 invert 90

j

jacobian 225

k

key 64
 kill 21-22

l

lagrange 195
 last 82
 length 84
 lhs 99
 limit 141
 lines 46
 line_width 64
 lista 81
 listp 88
 local 114
 log 16, 250
 logexpand 25

m

método
 de la secante 137
 de Steffensen 138
 makelist 56, 67, 84, 170
 map 66, 85, 102
 matrix 87
 matrixp 88
 matriz_size 88
 minor 91
 mnewton 137

método

 del punto medio compuesto 208
 del punto medio simple 205
 del trapecio compuesto 207
 del trapecio simple 205
 de Newton-Raphson 135
 de regula falsi 129
 de Simpson compuesto 208
 de Simpson simple 206
 multiplicities 101

n

niceindices 185
 nodo
 de interpolación 192
 nticks 41, 55, 64
 nullspace 92
 num 24
 numer 13, 125
 nusum 183

o

ode2 213

p

parameters 219
 parametric 50, 57, 69
 parametric_surface 57
 part 82, 102
 partfrac 24
 plot2d 37, 42
 plot3d 47
 plotdf 217
 points 46, 56
 point_size 62
 points_joined 62
 point_type 62
 polar 57

polarform 250
 polinomio
 de Lagrange 193
 powerseries 185
 print 115
 proportional_axes 60

q

quadpack 168
 quad_qagi 168
 quad_quags 168
 quadrilateral 54

r

radcan 27
 radexpand 25
 random 18, 56, 170-172
 rank 91
 ratdisrep 201
 ratsimp 27
 read 118
 realonly 109
 realpart 250
 realroots 107-108
 recta
 normal 155
 tangente 153
 rectangle 54
 rectform 249
 region 53
 remfunction 34
 remove 231
 remvalue 21
 rhs 99, 102
 romberg 168
 row 92

s

sec 17
 second 82
 simpsum 183
 sin 17, 251
 sinh 251
 sliders 219
 solve 103
 solve_rec 144-145
 sort 83
 spherical 58
 sqrt 12

staircase 134
 style 46
 submatrix 91
 subst 105, 128
 sum 171, 183
 surface 65
 surface_hide 65

t

tan 17
 taylor 185, 197
 tenth 82
 teorema
 de Newton-Raphson 135
 title 60
 tolerancia 134
 to_poly_solve 105
 trajectory_at 219
 transform 66
 transpose 90
 triangle 54
 triangularize 91
 trigexpand 27-28
 trigexpandplus 28
 trigexpandtimes 28
 trigreduce 27
 trigsimp 27

u

und 142
 union 105
 unique 83
 unless 120

v

valor
 principal 166
 values 21
 vector 56

w

while 120
 with_slider 67, 201
 with_slider_draw 68, 228-230
 with_slider_draw3d 68, 178
 wxplot2d 38

x

xaxis 61

xcenter 218
xlabel 61
xradius 218
xrange 59
xtics 61

y

yaxis 61
ycenter 218
ylabel 61

yradius 218
yrange 59
ytics 61

z

zaxis 61
zlabel 61
zrange 59
ztics 61

